Data Structure & Algorithms

DSA Roadmap:DSA Rodamap
DSA patterns: DSA pattern breakdown

Chapter 1: Time Complexity

1.1 Why Time Complexity Is Needed
1.2 What Does n Mean?
1.3 Asymptotic Notations
    1.3.1 Big-O — Upper Bound
    1.3.2 Big-Omega — Lower Bound
    1.3.3 Big-Theta — Tight Bound
1.4 Best, Average and Worst Case
1.5 Simplification Rules
    1.5.1 Drop Constants
    1.5.2 Drop Lower-Order Terms
1.6 Sequential Code — Add Complexities
1.7 Nested Independent Loops — Multiply
1.8 Nested Loops
1.9 Sums for Time Complexity
    1.9.1 Constant Sum
    1.9.2 Natural Numbers
    1.9.3 Squares
    1.9.4 Geometric Series
    1.9.5 Harmonic Series
1.10 Common Growth Rates
1.11 Constant Time — O(1)
1.12 Linear Time — O(n)
1.13 Logarithmic Time — O(log n)
1.14 Why Logarithm Base Usually Disappears
1.15 Square-Root Time — O(√n)
1.16 Direct Method for Iterative Code
1.17 Fast Iterative Recognition Rules
1.18 if / else
1.19 Loop Body Is Not Always O(1)
1.20 Recursion — Main Idea
1.21 Recurrence Relation
    1.21.1 Method 1 — Recurrence Expansion / Substitution
    1.21.2 Method 2 — Recursion Tree
1.22 Recursion Tree vs Recursion Depth
1.23 Common Recursive Shapes

CodeWithNishchal

1.24 Fibonacci-Type Recursion
1.25 Quick Interview Map — Fastest Recognition
1.26 The 25 Time-Complexity Patterns

Chapter 2: Space Complexity

2.1 Space Complexity
2.2 Total Space vs Auxiliary Space
2.3 Fixed Space and Variable Space
2.4 Most Important Rule — Peak Live Memory
2.5 What Memory Should We Check?
2.6 Primitive Variables
2.7 Arrays
2.8 Two-Dimensional Arrays
2.9 Three-Dimensional Arrays
2.10 Fixed-Size Arrays
2.11 Range-Based Arrays
2.12 Collections
2.13 Multiple Data Structures
2.14 Nested Loops Do Not Mean Nested Space
2.15 Recursion and Call Stack
2.16 Recursive Space Formula
2.17 Count Active Calls, Not Total Recursive Calls
2.18 Fastest Way to Calculate Space Complexity
2.19 Fast Recognition Table

Chapter 3: Array

3.1 Introduction
3.2 Why Do We Need Arrays?
3.3 Basic Structure of an Array
3.4 Important Terminology
    3.4.1 Element
    3.4.2 Index
    3.4.3 Length
3.5 Characteristics of Arrays
3.6 Array Declaration in Java
3.7 Array Creation
3.8 Array Initialization
    3.8.1 Initialization After Creation
    3.8.2 Direct Initialization
    3.8.3 Using new With Values
3.9 Default Values in Java Arrays
3.10 Accessing Array Elements
3.11 Updating an Array Element

CodeWithNishchal

3.12 Array Index Range
3.13 ArrayIndexOutOfBoundsException
3.14 Memory Representation of an Array
3.15 Traversing an Array
3.16 Enhanced for Loop
3.17 Searching in an Array
3.18 Insertion in an Array
3.19 Insertion at Different Positions
    3.19.1 Insertion at End
    3.19.2 Insertion at Beginning
3.20 Deletion From an Array
3.21 Deletion at Different Positions
3.22 Fixed Size Nature of Arrays
3.23 Copying an Array
3.24 System.arraycopy()
3.25 Arrays.copyOf()
3.26 Comparing Arrays
3.27 Printing Arrays
3.28 Arrays Utility Class
3.29 One-Dimensional Array
3.30 Two-Dimensional Array
3.31 Internal Nature of 2D Arrays in Java
3.32 Jagged Array
3.33 Traversing a 2D Array
3.34 Passing Arrays to Methods
3.35 Important Array Operations and Complexities
3.36 Space Complexity
3.37 Why Is Array Access O(1)?
3.38 Why Are Insertions and Deletions Often O(n)?
3.39 Advantages of Arrays
3.40 Limitations of Arrays
3.41 Array vs ArrayList
3.42 Overall Array Flow
3.43 Final Complexity Summary

Chapter 4: String

4.1 Introduction
4.2 Creating Strings in Java
    4.2.1 Using String Literal
    4.2.2 Using new
4.3 String Pool
4.4 Strings Are Immutable
4.5 Length and Character Access

CodeWithNishchal

4.6 Traversing a String
4.7 Comparing Strings
4.8 Important String Methods
4.9 Substring
4.10 Searching Inside a String
4.11 Concatenation
4.12 StringBuilder
4.13 StringBuffer
4.14 String vs StringBuilder vs StringBuffer
4.15 Converting Between String and Character Array
4.16 Converting Other Types to String
4.17 Empty String vs Null String
4.18 Arrays of Strings
4.19 String Complexity

Chapter 5: Hashing

5.1 Introduction
5.2 Why Hashing Is Needed
5.3 Hash Function
5.4 Hash Table
5.5 Collision
5.6 Separate Chaining
5.7 Open Addressing
5.8 Load Factor
5.9 Rehashing
5.10 Time Complexity of Hashing
5.11 HashMap in Java

Chapter 6: Two Pointer Pattern

6. Two Pointer Pattern
6.1 What is the Two Pointer Technique?
6.2 Why Two Pointers Are Useful
6.3 Core Idea Behind Pointer Movement
6.4 General Two Pointer Flow
6.5 How to Recognize Two Pointer Problems
6.6 Opposite Ends — Left and Right Pointers
6.7 Same Direction — Fast and Slow Pointers
6.8 Remove Duplicates From Sorted Array
6.9 Fast and Slow Does Not Always Mean Different Speeds
6.10 Partition Pattern
6.11 Dutch National Flag Pattern
6.12 Comparison of Two Pointer Variants

CodeWithNishchal

6.13 Why Two Pointer Solutions Are Often O(n)
6.14 Two Pointers vs Nested Loops

Chapter 7: Sliding Window Pattern

7.1 Fixed Size Sliding Window
7.2 Variable Size Sliding Window
7.3 Example: Longest Substring Without Repeating Characters
7.4 Window State
7.5 Monotonic Window
7.6 Sliding Window Maximum
7.7 Fixed vs Variable vs Monotonic Window
7.8 Sliding Window vs Two Pointers
7.9 When Sliding Window Works
7.10 Important Limitation of Variable Sliding Window
7.11 Final Pattern Summary

Chapter 8: More Array Patterns

8.1 Prefix Based Pattern
8.2 Kadane / Subarray Patterns
8.3 Binary Search
8.4 Binary Search on Answer

Chapter 9: Stack

9. Stack
9.1 Core Stack Operations
9.2 Stack Pointer / Top
9.3 Push Operation
9.4 Pop Operation
9.5 Peek Operation
9.6 Stack Underflow and Overflow
    9.6.1 Underflow
    9.6.2 Overflow
9.7 Stack Implementation Using Array
9.8 Stack Implementation Using Linked List
9.9 Stack in Java
9.10 Why ArrayDeque Is Preferred Over Stack
9.11 push, pop, peek vs Deque Methods
9.12 Time Complexity of Stack Operations
9.13 Balanced Parentheses Pattern
9.14 Expression Handling

CodeWithNishchal

9.15 Monotonic Stack Pattern
9.16 Monotonic Increasing vs Decreasing Stack
    9.16.1 Monotonic Increasing Stack
    9.16.2 Monotonic Decreasing Stack
9.17 Next Greater Element
9.18 Next Greater Element — Right-to-Left Approach
9.19 Next Smaller Element
9.20 Previous Greater / Previous Smaller
9.21 Monotonic Stack Reference Table
9.22 Why Monotonic Stack Is O(n)
9.23 Stock Span Pattern
9.24 Overall Complexity Summary

Chapter 10: Queue

10.1 Core Queue Operations
10.2 Array-Based Queue
10.3 Limitation of a Linear Array Queue
10.4 Queue Using Linked List
    10.4.1 Enqueue
    10.4.2 Dequeue
10.5 Queue in Java
10.6 offer() vs add() and poll() vs remove()
10.7 Queue Operation Complexity
10.8 Circular Queue
10.9 Deque
    10.9.1 Core Deque Operations
    10.9.2 Deque in Java
    10.9.3 Java Deque Methods
    10.9.4 Using Deque as a Queue
    10.9.5 Using Deque as a Stack
    10.9.6 Deque Time Complexity
10.10 Queue vs Deque vs Stack
10.11 Monotonic Deque Pattern
10.12 Overall Operation Complexity

Chapter 11: Linked List

11.1 Why Linked Lists Are Used
11.2 Basic Terminology
    11.2.1 Node
    11.2.2 Head
    11.2.3 Tail
11.3 Types of Linked Lists
11.4 Singly Linked List

CodeWithNishchal

11.5 Traversing a Singly Linked List
11.6 Insertion in Singly Linked List
    11.6.1 Insert at Beginning
    11.6.2 Insert at End
    11.6.3 Insert at a Position
11.7 Deletion in Singly Linked List
    11.7.1 Delete First Node
    11.7.2 Delete Last Node
    11.7.3 Delete by Value
    11.7.4 Delete at Position
11.8 Searching in a Linked List
11.9 Accessing an Element by Position
11.10 Updating a Node
11.11 Reversing a Singly Linked List
11.12 Finding the Middle Node
11.13 Detecting a Cycle
11.14 Complete Singly Linked List Complexity
11.15 Doubly Linked List
11.16 Doubly Linked List Structure
11.17 Insert at Beginning — Doubly Linked List
11.18 Insert at End — Doubly Linked List
11.19 Insert Between Two Nodes
11.20 Delete a Node — Doubly Linked List
11.21 Traversing Doubly Linked List
11.22 Reverse Doubly Linked List
11.23 Circular Singly Linked List
11.24 Traversing Circular Singly Linked List
11.25 Circular Singly Insertion
    11.25.1 Insert at Beginning
    11.25.2 Insert at End
11.26 Circular Doubly Linked List
11.27 Circular Doubly Structure
11.28 Circular vs Non-Circular Linked List
11.29 Complete Type Comparison
11.30 Linked List Operation Complexity Summary

Chapter 12: Tree

12.1 Tree Terminology
12.2 Basic Properties
12.3 Important Types of Trees
    12.3.1 General Tree
    12.3.2 Binary Tree
    12.3.3 Full Binary Tree
    12.3.4 Complete Binary Tree

CodeWithNishchal

12.3.5 Perfect Binary Tree
12.3.6 Balanced Binary Tree
12.3.7 Skewed Tree
12.4 Binary Tree Representation in Java
12.5 Tree Traversal
12.6 DFS — Depth First Search
12.6.1 Preorder Traversal
12.6.2 Inorder Traversal
12.6.3 Postorder Traversal
12.6.4 DFS Order Comparison
12.7 DFS Complexity
12.8 Iterative DFS
12.9 BFS — Breadth First Search / Level Order
12.10 Zigzag Level Order
12.11 Right Side View
12.12 DFS vs BFS
12.13 Top-Down Approach
12.14 Bottom-Up Approach
12.15 Top-Down vs Bottom-Up
12.16 Common Tree Recursive Patterns
12.16.1 Height / Depth
12.16.2 Count Nodes
12.16.3 Count Leaves
12.16.4 Search
12.17 Binary Search Tree — BST
12.18 Why BST Is Useful
12.19 Search in BST
12.20 BST Insertion
12.21 Minimum and Maximum in BST
12.22 BST Deletion
12.22.1 Case 1: Leaf Node
12.22.2 Case 2: Node with One Child
12.22.3 Case 3: Node with Two Children
12.22.4 BST Delete Implementation
12.23 Inorder Traversal of BST
12.24 BST Complexity
12.25 Balanced vs Skewed BST
12.26 Binary Tree vs BST
12.27 Tree Operation Complexity Overview

Chapter 13: Recursion

13.1 Base Case and Recursive Case
13.2 How Recursion Actually Works
13.3 Call Stack Visual

CodeWithNishchal

13.4 Recursion Flow
13.5 Work Before and After Recursive Call
13.6 Recursive Function With Return Value
13.7 Three Steps for Writing Recursive Solutions
13.8 Direct Recursion
13.9 Indirect Recursion
13.10 Head Recursion
13.11 Tail Recursion
13.12 Linear Recursion
13.13 Multiple / Tree Recursion
13.14 Recursion Tree
13.15 Recursion on Arrays
13.16 Recursion on Strings
13.17 Top-Down Recursion
13.18 Bottom-Up Recursion
13.19 Binary Search Using Recursion
13.20 Recurrence Relation
13.21 Time Complexity of Recursive Functions
13.22 Space Complexity of Recursion
13.23 Infinite Recursion
13.24 Recursion vs Iteration

Chapter 14: Backtracking

14.1 Decision Tree Idea
14.2 Core Backtracking Structure
14.3 Why Undo Is Necessary
14.4 Backtracking vs Normal Recursion
    14.4.1 Normal Recursion
    14.4.2 Backtracking
14.5 Backtracking vs Brute Force
14.6 Pruning
14.7 State in Backtracking
14.8 Subsets Pattern
14.9 Permutation Pattern
14.10 Combination Pattern
14.11 Combination Sum Pattern
14.12 Subset vs Combination vs Permutation
14.13 Handling Duplicates
14.14 Grid Backtracking
14.15 Word Search Pattern
14.16 N-Queens Pattern
14.17 Backtracking With In-Place Swap
14.18 Backtracking Complexity

CodeWithNishchal

14.19 Output Complexity
14.20 Backtracking vs DFS
    14.20.1 DFS
    14.20.2 Backtracking
14.21 Backtracking vs Dynamic Programming
    14.21.1 Backtracking
    14.21.2 Dynamic Programming
14.22 Backtracking vs Greedy
14.23 Standard Backtracking Template

Chapter 15: Divide and Conquer

15.1 General Structure
15.2 Base Case
15.3 Divide Phase
15.4 Conquer Phase
15.5 Combine Phase
15.6 Complete Divide and Conquer Flow
15.7 Divide and Conquer vs Recursion
15.8 Binary Search — Divide and Conquer
    15.8.1 Recursive Implementation
    15.8.2 Divide and Conquer View
15.9 Merge Sort
15.10 Quick Sort
15.11 Finding Maximum Using Divide and Conquer
15.12 Recurrence Relations
15.13 Recursion Tree Analysis
15.14 Divide and Conquer vs Dynamic Programming
    15.14.1 Divide and Conquer
    15.14.2 Dynamic Programming
15.15 Divide and Conquer vs Backtracking
    15.15.1 Divide and Conquer
    15.15.2 Backtracking
15.16 Divide and Conquer vs Greedy
    15.16.1 Divide and Conquer
    15.16.2 Greedy
15.17 Complexity Comparison
15.18 Final Pattern Map

Chapter 16: Heap

16.1 Complete Binary Tree Property
16.2 Array Representation of Heap
16.3 Min Heap vs Max Heap
16.4 Core Heap Operations

CodeWithNishchal

16.5 Insertion — Heapify Up
16.6 Remove Root — Heapify Down
16.7 Heapify
16.8 Building a Heap
16.9 Manual Min Heap Implementation
16.10 PriorityQueue in Java
16.11 Max Heap in Java
16.12 Heap Sort
16.13 Heap Pattern Recognition
16.14 Top K Pattern
16.15 K Largest Elements
16.16 Kth Largest Element
16.17 Kth Smallest Element
16.18 K Closest Points
16.19 Top-K Pattern Summary
16.20 Greedy + Heap Pattern
16.21 Task Scheduler
16.22 Meeting Rooms
16.23 Reorganize String
16.24 Huffman Encoding
16.25 Greedy + Heap Summary
16.26 K-Way Merge
16.27 K-Way Merge Structure
16.28 Merge K Sorted Arrays
16.29 Merge K Sorted Linked Lists
16.30 Heap vs Balanced BST
16.31 Overall Complexity Table

Chapter 17: Graphs

17.1 Important Graph Terminology
17.2 Types of Graphs
    17.2.1 Undirected Graph
    17.2.2 Directed Graph
    17.2.3 Weighted Graph
    17.2.4 Unweighted Graph
17.3 Graph Representation
    17.3.1 Adjacency Matrix
    17.3.2 Adjacency List
17.4 Graph Traversal
17.5 BFS — Breadth First Search
17.6 DFS — Depth First Search
    17.6.1 Recursive DFS
    17.6.2 BFS vs DFS
17.7 Disconnected Graphs

CodeWithNishchal

17.8 Cycle Detection in Undirected Graph
    17.8.1 DFS Cycle Detection — Undirected
    17.8.2 BFS Cycle Detection — Undirected
17.9 Cycle Detection in Directed Graph
    17.9.1 DFS Cycle Detection — Directed
17.10 Topological Sort
17.11 Kahn's Algorithm — BFS Topological Sort
17.12 Cycle Detection Using Kahn's Algorithm
17.13 DFS-Based Topological Sort
17.14 Kahn vs DFS Topological Sort
17.15 Shortest Path Overview
17.16 Unweighted Shortest Path — BFS
17.17 Weighted Graph Representation
17.18 Dijkstra's Algorithm
    17.18.1 Relaxation
    17.18.2 Dijkstra Flow
17.19 Why Dijkstra Fails With Negative Edges
17.20 Bellman-Ford Algorithm
    17.20.1 Bellman-Ford Code
    17.20.2 Algorithm — Bellman-Ford
17.21 Negative Cycle Detection
17.22 Floyd-Warshall Algorithm
17.23 Dijkstra vs Bellman-Ford vs Floyd-Warshall
17.24 Minimum Spanning Tree
17.25 Kruskal's Algorithm
17.26 Prim's Algorithm
17.27 Kruskal vs Prim
17.28 Union-Find / Disjoint Set Union
17.29 DSU Internal Structure
17.30 Path Compression
17.31 Union by Rank / Size
    17.31.1 DSU Implementation
17.32 Cycle Detection in Undirected Graph Using DSU
17.33 Bipartite Graph
17.34 Bipartite Check Using BFS
17.35 Multi-Source BFS
17.36 0-1 BFS
17.37 Shortest Path in a DAG
17.38 Strongly Connected Components
17.39 Bridges and Articulation Points
    17.39.1 Bridge
    17.39.2 Articulation Point
17.40 Graph Algorithm Selection Guide
17.41 Important Complexity Table

CodeWithNishchal

Chapter 18: Trie

18.1 Why Trie Is Useful
18.2 Trie Node Structure
18.3 Why isEndOfWord Is Necessary
18.4 Trie Root
18.5 Insert Operation
18.6 Search Complete Word
18.7 Prefix Search
18.8 Search vs Prefix Search
18.9 Complete Basic Trie Implementation
18.10 Time Complexity
18.11 Space Complexity
18.12 Array Children vs HashMap Children
    18.12.1 Fixed Array
    18.12.2 HashMap
18.13 Deleting a Word
18.14 When Can a Trie Node Be Removed?
18.15 Counting Words With a Prefix
18.16 Count Words Starting With Prefix
18.17 Autocomplete / Search Suggestions
18.18 Longest Common Prefix Using Trie
18.19 Trie vs HashSet
18.20 Trie vs BST
18.21 Trie Traversal
18.22 Word Dictionary With Wildcard
18.23 Word Search + Trie
18.24 Binary Trie / Bitwise Trie
18.25 Prefix Matching Pattern
18.26 Trie Complexity Summary
18.27 Common Trie Patterns
18.28 Common Mistakes

Chapter 19: Dynamic Programming

19.1 Why Dynamic Programming Is Needed
19.2 Two Properties Behind DP
    19.2.1 Overlapping Subproblems
    19.2.2 Optimal Substructure
19.3 The Most Important DP Concept — State
19.4 State Dimensions
    19.4.1 1D State dp[i]
    19.4.2 2D State
    19.4.3 3D or Higher
19.5 Transition

CodeWithNishchal

19.6 Base Cases
19.7 DP Workflow
19.8 Memoization — Top-Down DP
19.9 Tabulation — Bottom-Up DP
19.10 Memoization vs Tabulation
19.11 Space Optimization
19.12 1D Dynamic Programming
19.13 Linear DP — Climbing Stairs
19.14 Decision DP — House Robber
19.15 Decision DP Pattern
19.16 2D Dynamic Programming
19.17 Grid DP
19.18 Unique Paths
19.19 Minimum Path Sum
19.20 Grid DP Recognition
19.21 Knapsack Pattern
19.22 0/1 Knapsack
19.23 0/1 Knapsack — Tabulation
19.24 Knapsack Variants
19.25 Subset Sum
19.26 Equal Sum Partition
19.27 Unbounded Knapsack
19.28 Coin Change
19.29 Sequence DP
19.30 Longest Common Subsequence — LCS
19.31 Longest Increasing Subsequence — LIS
19.32 Edit Distance
19.33 Longest Palindromic Subsequence
19.34 Partition DP
19.35 Matrix Chain Multiplication
19.36 Interval DP
19.37 Interval DP Filling Order
19.38 DP on Stocks / State Machine DP
19.39 State Transition View
19.40 DP as a DAG
19.41 Bitmask DP
19.42 Bitmask DP State
19.43 Travelling Salesman DP
19.44 Digit DP
19.45 Digit DP Core State
19.46 Digit DP Range Trick
19.47 DP on Trees
19.48 Simple Tree DP — Subtree Size
19.49 Include / Exclude Tree DP

CodeWithNishchal

19.50 Rerooting DP
19.51 Memoization Complexity
19.52 How to Count DP States
19.53 Time Complexity Examples
19.54 Space Optimization in 2D DP
19.55 DP Table Filling Order
19.56 Pull DP vs Push DP
    19.56.1 Pull DP
    19.56.2 Push DP
19.57 DP Pattern Classification
19.58 How to Recognize a DP Problem
19.59 DP and Greedy
19.60 DP and Backtracking
19.61 DP and Divide & Conquer
    19.61.1 Divide & Conquer
    19.61.2 Dynamic Programming
19.62 Common DP Mistakes
19.63 DP Complexity Rule
19.64 Core Pattern Summary Table

Chapter 20: Greedy Algorithms

20.1 Greedy Thinking
20.2 Greedy vs Dynamic Programming
    20.2.1 Greedy
    20.2.2 Dynamic Programming
20.3 When Greedy Works
    20.3.1 Greedy Choice Property
    20.3.2 Optimal Substructure
20.4 How Greedy Correctness Is Usually Proved
    20.4.1 Exchange Argument
    20.4.2 Staying Ahead
    20.4.3 Structural Argument
20.5 Sorting Is Central to Greedy
20.6 Interval Greedy Pattern
20.7 Activity Selection
20.8 Why Earliest Finish Works
20.9 Non-Overlapping Intervals
20.10 Minimum Removals to Make Intervals Non-Overlapping
20.11 Alternative Strategy — Keep Smaller End
20.12 Interval Greedy Recognition
20.13 Scheduling Greedy
20.14 Job Sequencing with Deadlines and Profit
20.15 Deadline Scheduling Insight

CodeWithNishchal

20.16 Resource Allocation Greedy
20.17 Meeting Rooms
20.18 Why Meeting Rooms Is Greedy + Heap
20.19 Minimum Platforms
20.20 Meeting Rooms vs Activity Selection
    20.20.1 Activity Selection
    20.20.2 Meeting Rooms
20.21 Jump Game Pattern
20.22 Jump Game I
20.23 Jump Game I Visual
20.24 Jump Game II
20.25 Huffman Encoding
20.26 Huffman Algorithm
20.27 Minimum Merge Cost Pattern
20.28 Minimum Merge Cost Code
20.29 Fractional Knapsack — Important Greedy Pattern
20.30 Fractional Knapsack vs 0/1 Knapsack|
    20.30.1 Fractional Knapsack
    20.30.2 0/1 Knapsack
20.31 Gas Station Pattern
20.32 Greedy + Heap
20.33 Greedy + Sorting
20.34 Greedy + Two Pointers
20.35 Common Greedy Patterns
20.36 Choosing the Correct Greedy Rule
20.37 How to Recognize Greedy Problems
20.38 When Greedy Fails
20.39 Common Mistakes
20.40 Complexity Summary
20.41 Greedy vs Other Techniques

Chapter 21: Bit Manipulations

21.1 Binary Basics
21.2 Bit Indexing
21.3 Bitwise Operators in Java
21.4 Bitwise AND — &
21.5 Bitwise OR — |
21.6 Bitwise XOR — ^
21.7 Important XOR Properties
21.8 Bitwise NOT — ~
21.9 Left Shift — <<
21.10 Signed Right Shift — >>
21.11 Unsigned Right Shift — >>>

CodeWithNishchal

21.12 Java Shift Important Detail
21.13 Bit Mask
21.14 Check Whether the ith Bit Is Set
21.15 Set the ith Bit
21.16 Clear the ith Bit
21.17 Toggle the ith Bit
21.18 Update a Bit
21.19 Basic Bit Operations Summary
21.20 Odd or Even Using Bits
21.21 Multiply / Divide by Powers of Two
21.22 Power of Two
21.23 Why n & (n - 1) Works
21.24 Remove Rightmost Set Bit
21.25 Count Set Bits — Simple Method
21.26 Brian Kernighan's Algorithm
21.27 Isolate the Rightmost Set Bit
21.28 Why n & -n Works
21.29 Single Number Pattern
21.30 Missing Number Using XOR
21.31 Two Unique Numbers
21.32 Number Appearing Once When Others Appear Three Times
21.33 Generate All Subsets Using Bitmask
    21.33.1 Subset Code
21.34 Bitmask as a Set
    21.34.1 Add element i
    21.34.2 Remove element i
    21.34.3 Check element
    21.34.4 Toggle
21.35 Union, Intersection, Difference With Masks
    21.35.1 Union
    21.35.2 Intersection
    21.35.3 Symmetric Difference
    21.35.4 Remove all elements of B from A
21.36 Check Whether One Mask Is a Subset of Another
21.37 Enumerating Submasks
21.38 Count Set Bits for All Numbers
21.39 Hamming Distance
21.40 Hamming Weight
21.41 Clear the Rightmost Set Bit
21.42 Set the Rightmost Unset Bit
21.43 Clear All Bits From LSB Through Position i
21.44 Extract Lower k Bits
21.45 Range Bitwise AND
21.46 XOR From 1 to n
21.47 Java Signed Integer Representation

CodeWithNishchal

21.48 Operator Precedence and Parentheses
21.49 Important Built-In Java Methods
21.50 Operation Complexity
21.51 Core Formula Table

Chapter 22: Range Structures

22.1 Why Range Structures Are Needed
22.2 Prefix Sum vs Range Structures
22.3 Segment Tree
22.4 Fenwick Tree
22.5 Segment Tree vs Fenwick Tree
22.6 When to Use Fenwick Tree
22.7 When to Use Segment Tree
22.8 Segment Tree Node as Information
22.9 Complexity Comparison

Chapter 23: Sorting Algorithms

23.1 Main Classification
    23.1.1 Comparison-Based Sorting
    23.1.2 Non-Comparison Sorting
23.2 Important Sorting Properties
    23.2.1 Stable Sorting
    23.2.2 In-Place Sorting
    23.2.3 Adaptive Sorting
23.3 Bubble Sort
23.4 Selection Sort
23.5 Insertion Sort
23.6 Merge Sort
23.7 Quick Sort
23.8 Heap Sort
23.9 Counting Sort
23.10 Radix Sort
23.11 Bucket Sort
23.12 Bubble vs Selection vs Insertion
23.13 Merge vs Quick vs Heap
23.14 Complexity Summary

CodeWithNishchal

1. Time Complexity

Time Complexity describes how the amount of work performed by an algorithm grows as the input size grows. It does **not** mean: "This program takes exactly 2 seconds."

It answers: "If input size n becomes very large, how does the number of operations grow?"

Example:

```
for (int i = 0; i < n; i++) {
    System.out.println(i);
}
```

The loop executes approximately n times.

Therefore: Time Complexity = O(n)

The important quantity is the **growth rate**, not the exact execution time.

1.1 Why Time Complexity Is Needed

Suppose two algorithms solve the same problem.

Algorithm A: $T(n) = n$

Algorithm B: $T(n) = n^2$

For: n = 10

the difference may not seem large.

But for:

n = 1,000,000

approximately: O(n) → 1,000,000 operations

$O(n^2)$ → 1,000,000,000,000 operations

So complexity tells us whether an algorithm will scale.

1.2 What Does n Mean?

Usually: n = input size

Examples:

Array → n = number of elements

String → n = number of characters

Matrix → rows × columns

Graph → V vertices and E edges

Sometimes there are multiple independent input sizes:

n = length of array A , m = length of array B

Then complexity may be: O(n + m), O(nm), O(n log m)

Do **not** automatically replace every input variable with n.

CodeWithNishchal

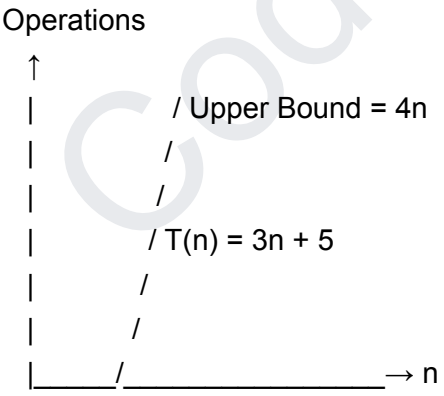
1.3. Asymptotic Notations

Asymptotic notations describe how the running time of an algorithm grows as input size n becomes very large. The three important notations are:
O → Big-O → Upper Bound
Ω → Big-Omega → Lower Bound
Θ → Big-Theta → Tight Bound
Easy way to remember:
O → Ceiling
Ω → Floor
Θ → Tight/Sandwiched Bound
We will use the same example for all three:
T(n) = 3n + 5

1.3.1 Big-O — Upper Bound

Big-O represents an asymptotic upper bound. It tells us that the running time will not grow faster than a particular growth rate after n becomes sufficiently large.
Example: T(n) = 3n + 5
Take an upper function: 4n
For n ≥ 5:
3n + 5 ≤ 4n

Example for n = 10:
T(10) = 3(10) + 5 = 35
4(10) = 40
Therefore:
35 ≤ 40
So:
T(n) = O(n)
Visual:



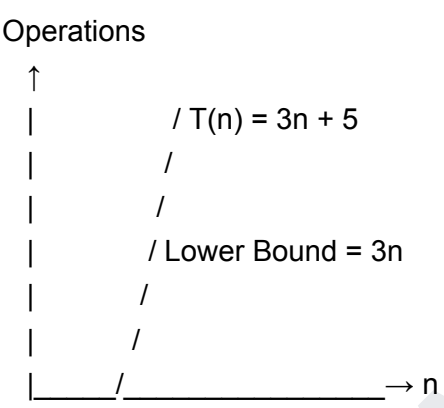
Meaning: O(n) → The algorithm grows at most linearly asymptotically.
Important: Big-O means an upper bound. It does not itself mean "worst case." However, worst-case complexity is commonly expressed using Big-O in interviews.

CodeWithNishchal

1.3.2 Big-Omega — Lower Bound

Big-Omega represents an asymptotic lower bound. It tells us that the running time grows at least as quickly as a particular growth rate.
Again: T(n) = 3n + 5
Take: 3n
We have:
3n + 5 ≥ 3n

Example for n = 10:
T(10) = 35
3(10) = 30
Therefore:
35 ≥ 30
So: T(n) = Ω(n)
Visual:

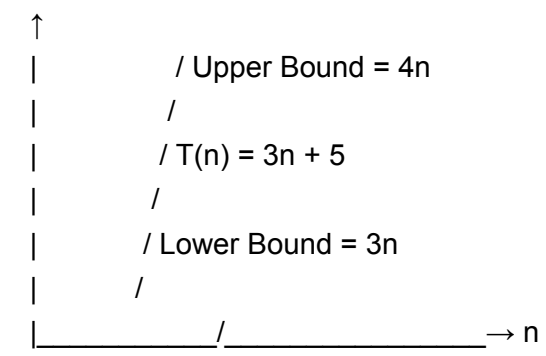


Meaning:
Ω(n) → The algorithm grows at least linearly asymptotically.
Easy difference:
O(n) → No faster than linear growth.
Ω(n) → No slower than linear growth.

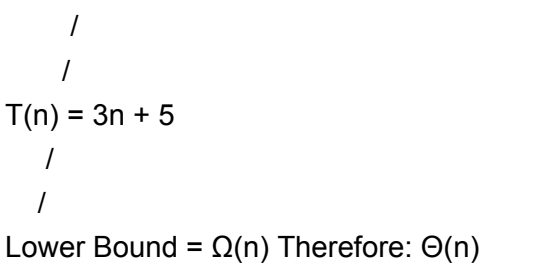
1.3.3 Big-Theta — Tight Bound

Big-Theta represents a tight asymptotic bound. It means both the upper and lower bounds have the same growth rate.
Again:
T(n) = 3n + 5
We have:
3n ≤ 3n + 5 ≤ 4n for sufficiently large n.
Lower bound: Ω(n)
Upper bound: O(n)
Since both have the same linear growth:
T(n) = Θ(n)
Visual:

CodeWithNishchal



Think of Theta as the actual function being trapped between two functions having the same growth rate:



1.4. Best, Average and Worst Case

An algorithm can behave differently for different inputs of the same size.
Example: Linear Search.
[10, 20, 30, 40, 50]
Search for 10:
Best Case → first comparison → O(1)

Search for 50:
Worst Case → inspect all elements → O(n)
Average case: roughly inspect a fraction of n → O(n)
Since constants disappear asymptotically: n/2 → O(n)
When an interview simply asks:
"What is the time complexity?"
the expected answer is usually the **worst-case Big-O**, unless another case is explicitly requested.

1.5. Simplification Rules

Time-complexity expressions are simplified by keeping the dominant growth.

Drop Constants

O(2n) → O(n)
O(n/5) → O(n)
O(100) → O(1)

Drop Lower-Order Terms

O(n² + n + 10) → O(n²) because as n grows: n² dominates.

CodeWithKishchal

1.6. Sequential Code — Add Complexities

Suppose:

```
for (int i = 0; i < n; i++) {
    // O(1)
}
for (int j = 0; j < n * n; j++) {
    // O(1)
}
```

First loop: O(n)

Second: O(n²)

Total: O(n + n²)

Keep dominant term: O(n²)

So for **sequential blocks**: ADD their work

↓

then keep dominant term

1.7. Nested Independent Loops — Multiply

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {

        // O(1)
    }
}
```

Outer: n

For every outer iteration: inner = n Therefore: n × n = n² Complexity: O(n²)

Your quick guide uses exactly this recognition rule for single, nested, and triple loops.

1.8. Important Rule — Do Not Blindly Multiply Nested Loops

Nested loops are **not automatically**: O(n²)

Example:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        // work
    }
}
```

Inner loop depends on i.

Total work: 1 + 2 + 3 + ... + n not: n × n

although both eventually simplify to O(n²) here.

For more complex loops, this distinction becomes critical.

CodeWithNishchal

1.9. Important Sums for Time Complexity

You do not need advanced mathematics, but these few sums are extremely useful.

Constant Sum

1 + 1 + 1 + ... n times = n

So: O(n)

Natural Numbers

1 + 2 + 3 + ... + n = n(n+1)/2

Dominant term: $n^2/2$ Therefore: $O(n^2)$

Your Pattern 5 uses exactly this triangular-loop calculation.

Squares

$1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6$ Dominant term: n^3 Therefore: $O(n^3)$

Geometric Series

n + n/2 + n/4 + n/8 + ...

is bounded by: 2n

Therefore: O(n) . This is extremely important for shrinking nested loops.

Harmonic Series

1 + 1/2 + 1/3 + ... + 1/n

grows as: O(log n) Therefore: n(1 + 1/2 + ... + 1/n) = O(n log n)

This is exactly the idea used by Pattern 19.

1.10. Common Growth Rates

From fastest-growing efficiency to slowest:

Complexity	Common Meaning
O(1)	Constant
O(log n)	Logarithmic
O(√n)	Square-root
O(n)	Linear
O(n log n)	Linearithmic
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
O(2^n)	Exponential
O(n!)	Factorial

Order: O(1) < O(log n) < O(√n) < O(n) < O(n log n) < $O(n^2)$ < $O(n^3)$ < O(2^n) < O(n!)

CodeWithNishchal

1.11. Constant Time — O(1)

Example:

int x = arr[5];

Regardless of whether the array contains: 10 elements or 10 million elements

one direct array access is performed.

Therefore: O(1)

Other common examples: Arithmetic on fixed-width primitive values, Comparison, Assignment , Array indexing, HashMap lookup — average case, Stack push/pop

1.10. Linear Time — O(n)

Example:

```
for (int i = 0; i < n; i++) {
    process(arr[i]);
}
```

Iterations: n Therefore: O(n)

Changing the step by a constant:

for (int i = 0; i < n; i += 5)

gives:n/5 iterations.

Drop constant: O(n)

1.11. Logarithmic Time — O(log n)

Whenever the problem size is repeatedly:Multiplied or divided

by a constant factor, think logarithmic.

Example:

```
for (int i = 1; i <= n; i *= 2) {
}
```

Values:

1

2

4

8

...

After k iterations: 2^k ≈ n Therefore: k ≈ $\log_2 n$ Complexity: O(log n)

1.12. Why Logarithm Base Usually Disappears

Suppose one loop doubles: $O(\log_2 n)$. another triples: $O(\log_3 n)$

Using change of base: $\log_3 n = \log_2 n / \log_2 3$, $1/\log_2 3$ is constant.

Therefore asymptotically: $O(\log_2 n) = O(\log_3 n) = O(\log n)$

The base usually does not matter in Big-O.

1.13. Square-Root Time — O(√n)

```
for (int i = 1; i * i <= n; i++) {
}
```

Loop stops when: $i^2 > n$ Therefore: $i \approx \sqrt{n}$ Iterations: √n Complexity: O(√n)

CodeWithLishchal

1.14. Direct Method for Iterative Code

1. Identify each loop variable.
2. Write its starting value.
3. Write how it changes.
4. Find how many updates are required before the stopping condition.
5. Independent nested loops → multiply.
6. Dependent loops → write a summation.
7. Sequential blocks → add.
8. Remove constants and lower terms.

This is usually the fastest method in interviews.

1.15. Fast Iterative Recognition Rules

Addition/Subtraction by Constant

i++

i += 2

i += 100

over a range proportional to n: O(n)

Multiplication by Constant

i *= 2

i *= 3

until n: O(log n)

Division by Constant

i /= 2

i /= 3

until 1: O(log n)

Condition i * i <= n

O(√n)

---

j += i Inside i = 1...n

Inner work: n/i, Total: n(1 + 1/2 + ... + 1/n) = O(n log n)

1.16. if / else

```
if (condition) {
    linearWork();     // O(n)
} else {
    quadraticWork();  // O(n²)
}
```

For worst-case complexity: max(O(n), O(n²)) = O(n²). Only one branch executes.

So: if/else → do not add mutually exclusive branches → take the relevant/worst branch

CodeWithNishchal

1.17. Loop Body Is Not Always O(1)

```
for (int i = 0; i < n; i++) {
    binarySearch(arr, target);
}
```

Loop: n times Body: O(log n). Total: O(n log n). Always include the complexity of the loop body.

1.18. Recursion — Main Idea

For recursive code, do **not** analyze only one function invocation.

You must analyze:

1. How many recursive calls are created?
2. How large is each new input?
3. How much non-recursive work
   happens inside each call?
4. How many total calls/states occur?

1.19. Recurrence Relation

A recurrence describes recursive running time.

Example:

```
void solve(int n) {
    if (n == 0) {
        return;
    }

    solve(n - 1);
}
```

Each call performs:

one recursive call on n-1

+

constant work

So:

T(n)

=

T(n - 1) + O(1)

1.19.1. Method 1 — Recurrence Expansion / Substitution

Consider: T(n) = T(n-1) + O(1)

Expand: T(n) = T(n-2) + O(1) + O(1) = T(n-2) + 2O(1)

Again: T(n) = T(n-3) + 3O(1)

After k expansions: T(n) = T(n-k) + kO(1)

Base case occurs when: n-k = 0, so: k = n Therefore: T(n) = O(n)

CodeWithNishchal

1.19.2 Method 2 — Recursion Tree

A recursion tree visually represents all recursive calls.

$T(n) = 2T(n/2) + O(n)$

Tree:

```
            n
          work = n
         /        \
       n/2        n/2
      work        work
       ↓           ↓
    n/4 n/4     n/4 n/4
```

At each level:

Level 0: n work

Level 1: n/2 + n/2 = n

Level 2: 4 × n/4 = n

Every level: O(n), Number of levels: O(log n) Therefore: O(n) × O(log n)= O(n log n)

This is the Merge Sort pattern used in the attached sheet.

1.20. Recursion Tree vs Recursion Depth

Example Fibonacci:

```
          f(n)
          /   \
      f(n-1)  f(n-2)
```

Depth is approximately: O(n), but there are many branches. So total running time is **not** O(n). For time complexity: count total work / total calls not merely the longest path.

1.21. Common Recursive Shapes

One Call, Input n-1

$T(n) = T(n-1) + O(1)$

n

↓

n-1

↓

n-2

↓

...

↓

1

Complexity: O(n)

CodeWithNishchal

One Call, Input n/2

T(n) = T(n/2) + O(1)

n

↓

n/2

↓

n/4

↓

...

↓

1 Complexity: O(log n)

Two Calls on Halves + Constant Work

T(n) = 2T(n/2) + O(1) Tree nodes: 1 + 2 + 4 + ... + n Total: O(n)

Two Calls on Halves + Linear Work

T(n) = 2T(n/2) + O(n)

Each level: O(n) , Number of levels: O(log n) Total: O(n log n)

One Call on n-1 + Linear Work

T(n) = T(n-1) + O(n)

Expansion: n + (n-1) + (n-2) + ... + 1 Therefore: O(n²)

This describes badly unbalanced Quick Sort when partitioning repeatedly leaves one side of size n-1.

1.22. Fibonacci-Type Recursion

```
int fib(int n) {
   if (n <= 1) {
      return n;
   }
   return fib(n - 1)  + fib(n - 2);
}
```

Recurrence: T(n) = T(n-1) + T(n-2) + O(1)

```
              f(n)
             /    \
        f(n-1)     f(n-2)
        /   \       /   \
      ...   ...   ...   ...
```

The tree grows exponentially. Common interview bound: O(2^n)

A tighter mathematical bound is: Θ(φ^n) where φ ≈ 1.618, but: O(2^n) is the normal DSA/interview shorthand.

CodeWithNishchal

1.23. Quick Interview Map — Fastest Recognition

Code/Pattern	Think Immediately	Complexity
One direct operation	Constant	O(1)
i++ until n	Linear steps	O(n)
i += c until n	n/c	O(n)
Two independent n loops nested	n × n	$O(n^2)$
Three independent n loops	n^3	$O(n^3)$
i *= 2	Multiplicative growth	O(log n)
i /= 2	Repeated halving	O(log n)
i*i <= n	Stop near √n	O(√n)
Outer n, inner log n	Multiply	O(n log n)
Two nested logarithmic loops	Multiply	$O(\log^2 n)$
j <= i	Sum 1+2+...+n	$O(n^2)$
j += i	Harmonic sum	O(n log n)
n+n/2+n/4+...	Geometric sum	O(n)
T(n)=T(n-1)+1	Linear recursion	O(n)
T(n)=T(n/2)+1	Halving recursion	O(log n)
T(n)=2T(n/2)+1	Number of tree nodes	O(n)
T(n)=2T(n/2)+n	n per level × log n	O(n log n)
T(n)=T(n-1)+n	1+...+n	$O(n^2)$
Fibonacci recursion	Exponential tree	O(2^n)
Generate all subsets	2^n subsets	O(2^n) minimum
Generate all permutations	n! arrangements	O(n!) branching
Scan graph adjacency lists	vertices + edges	O(V+E)

This follows the same quick-recognition philosophy as your attached guide.

CodeWithNishchal

1.24. The 25 Time-Complexity Patterns

Now apply the rules directly to your 25-pattern set.

Pattern 1 — Single Linear Loop

```
for (int i = 1; i <= n; i++) {
   work();
}
```

Iterations: n

Work each iteration: O(1) , Total: n × O(1) = O(n) ⇒ O(n)

Pattern 2 — Two Independent Nested Linear Loops

```
for (int i = 1; i <= n; i++) {
   for (int j = 1; j <= n; j++) {
      work();
   }
}
```

Outer: n, Inner per outer: n, Total: n × n = n^2 Answer: $O(n^2)$

Pattern 3 — Triple Nested Linear Loops

```
for (int i = 1; i <= n; i++) {
   for (int j = 1; j <= n; j++) {

      for (int k = 1; k <= n; k++) {
         work();
      }
   }
}
```

Total: n × n × n = n^3, Answer: $O(n^3)$

The source similarly analyzes the three independent loops as $O(n^3)$.

Pattern 4 — Multiplicative Growth

```
for (int i = 1; i <= n; i *= 2) {
   work();
}
```

Values:

1, 2, 4, 8, ..., 2^k

Stop when: 2^k > n

Therefore: $k \approx \log_2 n$

Answer: O(log n)

CodeWithNishchal

Pattern 5 — Triangular Nested Loop

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        work();
    }
}
```

Work:
i=1 → 1
i=2 → 2
i=3 → 3
...
i=n → n
Total:
1 + 2 + ... + n = n(n+1)/2
Dominant term: n^2
Answer: $O(n^2)$

Pattern 6 — Repeated Division

```
for (int i = n; i > 0; i /= 2) {
    work();
}
```

Values:
n
n/2
n/4
n/8
...
1
After k iterations: n / 2^k ≈ 1
So: 2^k ≈ n
Therefore: k ≈ $\log_2 n$
Answer: O(log n)

Pattern 7 — Constant Step

```
for (int i = 1; i <= n; i += 3) {
    work();
}
```

Iterations: n/3
Ignore constant: O(n)
General rule: i += constant → linear

CodeWithNishchal

Pattern 8 — Nested Logarithmic Loops

```
for (int i = 1; i <= n; i *= 2) {
    for (int j = n; j > 0; j /= 2) {
        work();
    }
}
```

Outer: O(log n)

Inner: O(log n)

Independent nesting: O(log n) × O(log n) = O(log² n)

Pattern 9 — Linear × Logarithmic

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j *= 2) {
        work();
    }
}
```

Outer: O(n), Inner: O(log n), Total: O(n log n)

Pattern 10 — Shrinking Inner Work

```
for (int i = n; i > 0; i /= 2) {
    for (int j = 1; j <= i; j++) {
        work();
    }
}
```

A common mistake: outer = log n, inner = n

therefore n log n

That is incorrect because the inner loop is **not n every time**.

Actual work:

n

+

n/2

+

n/4

+

n/8

+

...

Geometric series: < 2n

Therefore: O(n)

This is an extremely important dependent-loop pattern.

CodeWithNishchal

Pattern 11 — Three Linear Loops + Logarithmic Work

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        for (int k = 1; k <= n; k++) {
            int x = 1;
            while (x < k) {
                x *= 2;
            }
        }
    }
}
```

For each fixed k:
while ≈ O(log k)

Summed over: k = 1...n
this is: Θ(n log n)
The outer two loops contribute: n^2
Therefore: $O(n^2 \times n \log n) = O(n^3 \log n)$

Pattern 12 — Cumulative Growth Gives √n

```
int i = 1;
int sum = 1;
while (sum <= n) {
    i++;
    sum += i;
}
```

After k iterations:
sum ≈ 1 + 2 + 3 + ... + k
Use: k(k+1)/2 ≈ n
Ignore lower terms: $k^2 \approx 2n$
Therefore: $k \approx \sqrt{2n}$
Constants disappear: $O(\sqrt{n})$
The attached pattern derives the same result through the cumulative natural-number sum.

Pattern 13 — i² <= n

```
for (int i = 1; i * i <= n; i++) {
    work();
}
```

Stopping point: $i^2 \approx n$
Therefore: $i \approx \sqrt{n}$
Answer: $O(\sqrt{n})$

CodeWithVishchal

Pattern 14 — Dependent Loop × Constant Loop

```
for (int i = 1; i <= n; i++) {
   for (int j = 1; j <= i; j++) {
      for (int k = 1; k <= 11; k++) {
         work();
      }
   }
}
```

Innermost loop: 11 = O(1)
For each i: middle = i
Total: 11(1 + 2 + 3 + ... + n)
Ignore constant 11: $O(n^2)$

Pattern 15 — Sum of Squares × Linear Work

```
for (int i = q; i <= n; i++) {
   for (int j = 1; j <= i * i; j++) {

      for (int k = 1; k <= n / 2; k++) {
         work();
      }
   }
}
```

For fixed i: j iterations = i^2
For every j: k iterations = n/2 = O(n)
Total: $n \times \Sigma i^2$ And: $\Sigma i^2 = O(n^3)$
Therefore: $O(n \times n^3) = O(n^4)$

Pattern 16 — Linear × Linear × Logarithmic

```
for (int i = n / 2; i <= n; i++) {
   for (int j = 1; j <= n / 2; j++) {
      for (int k = 1; k <= n; k *= 2) {
         work();
      }
   }
}
```

Outer: n/2 → O(n)
Middle: n/2 → O(n)
Inner: O(log n)

Total: $O(n^2 \log n)$

CodeWithNishchal

Pattern 17 — Loop That Looks Large but Executes Once

```
for (int i = n / 2; i <= n; i += 2 * i) {
    ...
}
```

First value: n/2, Next: n/2 + 2(n/2) = 3n/2

But: 3n/2 > n, So the outer loop executes only once: O(1)

If inside it we have: O(n) × O(log n)

then total: O(1 × n × log n) = O(n log n)

Pattern 18 — While Loop Halving Input

```
while (n > 1) {
    n /= 2;
}
```

Sequence:

n

n/2

n/4

n/8

...

1

At iteration k: n / 2^k ≈ 1, Therefore: 2^k ≈ n and: k ≈ $\log_2 n$ Answer: O(log n)

Pattern 19 — Harmonic Nested Loop

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j += i) {
        work();
    }
}
```

For fixed i: j executes approximately n/i times

Total: n/1 + n/2 + n/3 + ... + n/n

Factor out n: n(1 + 1/2 + 1/3 + ... + 1/n)

Harmonic series: ≈ log n Therefore: O(n log n)

Pattern 20 — Fibonacci Recursion

```
int fibonacci(int n) {

    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1)
         + fibonacci(n - 2);
}
```

CodeWithNishchal

T(n) = T(n-1) + T(n-2) + O(1)
Recursion tree:

```
            f(n)
           /    \
       f(n-1)    f(n-2)
       /   \      /   \
     ...   ...  ...   ...
```

There are massive overlapping calls. Interview upper bound: O(2^n). Tighter bound: Θ(φ^n)
The pattern sheet classifies it as exponential due to recursive-call-tree growth.

Pattern 21 — Generate All Permutations

For n distinct items:
First position: n choices
Second: n-1, Third: n-2 ... , Last: 1
Leaves: n × (n-1) × ... × 1 = n! So the **number of generated permutations** is: n!
Your sheet records the branching complexity as O(n!).

Pattern 22 — Recursive Sum

```
int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum(n - 1);
}
```

Recurrence: T(n) = T(n-1) + O(1)
Expansion: T(n) = T(n-2) + 2O(1)
= T(n-3) + 3O(1)
...
= T(0) + nO(1) Therefore:O(n)

Pattern 23 — Factorial Recursion

```
long factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Assuming fixed-width arithmetic for complexity analysis:
one recursive call + O(1) work
Recurrence: T(n) = T(n-1) + O(1)
Chain:
n → n-1 → n-2 → ... → 1
Number of calls: O(n) Answer: O(n)

CodeWithNishchal

Pattern 24 — Merge Sort Recursion Tree

Recurrence:

T(n) = 2T(n/2) + O(n)

Recursion tree:

Level 0:

n

work n

Level 1:

n/2 n/2

total work = n

Level 2:

n/4 n/4 n/4 n/4

total work = n

Levels: log n

Work each level: n

Total: n × log n = O(n log n)

Pattern 25 — Quick Sort

Quick Sort performs:

partition + recursive left side + recursive right side

Partitioning costs: O(n)

Balanced / Average-Style Case

If the pivot divides the input approximately equally:

T(n) = 2T(n/2) + O(n)

Using a recursion tree:

work per level = O(n)

levels = O(log n),Therefore: O(n log n)

Worst Case

If every pivot leaves:

0 elements on one side

n-1 on the other

then: T(n) = T(n-1) + O(n)

Expansion: n + (n-1)+ (n-2) + ... + 1

Therefore: $O(n^2)$

This can happen with a consistently poor pivot choice—for example, choosing the last element on already sorted data.

CodeWithNishchal

2. SPACE COMPLEXITY

2.1 Space Complexity

Space Complexity describes how much memory an algorithm requires as the input size grows.
If input size is n, we ask:
How does the memory requirement grow when n grows?
Example:

```
static int sum(int[] arr) {
    int sum = 0;

    for (int value : arr) {
        sum += value;
    }
    return sum;
}
```

The input array arr already exists. The algorithm creates only sum and value. Their number does not increase with n.
Therefore: Auxiliary Space = O(1)
Main difference: Time Complexity → How much work happens?
Space Complexity → How much memory is alive at the same time?

2.2 Total Space vs Auxiliary Space

Total space can be represented as: **Total Space = Input Space + Auxiliary Space**
Example:

```
static int sum(int[] arr) {
    int sum = 0;
    for (int x : arr) {
        sum += x;
    }
    return sum;
}
```

If arr has n elements: Input Space = O(n)
Extra variables: sum, x → O(1)
Therefore: Total Space = O(n)
Auxiliary Space = O(1)

In most DSA interview questions, when we say the algorithm uses O(1) or O(n) extra space, we are usually talking about auxiliary space.

Main formula:
Auxiliary Space = Peak extra memory alive at the same time, excluding supplied input.

CodeWithNishchal

2.3 Fixed Space and Variable Space

Fixed Space

Fixed space does not depend on input size.

int a = 10;

int b = 20;

boolean found = false;

No matter how large n becomes, there are still only three variables.
Space = O(1)

Variable Space

Variable space grows with input.

int[] arr = new int[n]; Space = O(n)

int[][] matrix = new int[n][n]; Space = $O(n^2)$

boolean[] visited = new boolean[V]; Space = O(V)

2.4 Most Important Rule — Peak Live Memory

Space complexity normally measures the maximum amount of memory that must exist at one time.
Consider:

```
for (int i = 0; i < n; i++) {
    int[] temp = new int[n];
}
```

The loop executes n times, so n arrays may be created during the entire execution.
But the previous array is not retained for every future iteration.
At peak: One array of size n is alive. Therefore: Space = O(n) NOT: $O(n^2)$
This is the key rule: **SPACE COMPLEXITY = PEAK MEMORY ALIVE AT THE SAME TIME**

2.5 What Memory Should We Check?

When calculating space complexity, check:

1. Primitive/local variables
2. Extra arrays
3. Strings/StringBuilder
4. ArrayList/List
5. HashSet
6. HashMap
7. Stack
8. Queue
9. Trees created by the algorithm
10. Graph structures
11. Recursion call stack
12. Temporary buffers
13. Stored output

CodeWithNishchal

2.6 Primitive Variables

int i = 0;

int j = 0;

long sum = 0;

boolean found = false;

These are a fixed number of variables. Space = O(1)
Even:

for (int i = 0; i < n; i++) {

}

uses:
O(1) auxiliary space.
The loop runs n times, but the same variable i is reused.
Important:
Many iterations affect time complexity, not necessarily space complexity.

2.7 Arrays

int[] arr = new int[n]; Space = O(n)
Two arrays:

int[] a = new int[n];

int[] b = new int[n]; **Memory: O(n) + O(n) = O(2n) = O(n)**
We add their sizes because both arrays exist simultaneously.

2.8 Two-Dimensional Arrays

int[][] matrix = new int[n][m];

Stored cells: n × m
Therefore: Space = O(nm)
If both dimensions are n:

int[][] matrix = new int[n][n];Space = O(n^2)

2.9 Three-Dimensional Arrays

int[][][] cube = new int[n][n][n];

Stored elements: n × n × n = n^3 Therefore: Space = O(n^3)

2.10 Fixed-Size Arrays

int[] frequency = new int[26];

The array always contains exactly 26 elements.
Its size does not depend on n.
Therefore: O(26) = O(1)

Rule: Fixed-size structure → O(1)

CodeWithNishchal

2.11 Range-Based Arrays

int[] count = new int[maxValue + 1];

If: maxValue = k then: Space = O(k)

Do not automatically call this O(n) because k and n can represent different quantities.

Your range-frequency pattern makes this distinction explicitly.

2.12 Collections

If: HashSet<Integer> set = new HashSet<>();

stores up to n elements:

Space = O(n)

If only d distinct elements are stored:

Space = O(d), where: d ≤ n

Worst case: O(n)

Similarly:

ArrayList with n elements → O(n)

HashMap with n entries → O(n)

Stack with n elements → O(n)

Queue with n elements → O(n)

2.13 Multiple Data Structures

Suppose:

HashSet<Integer> seen = new HashSet<>();

ArrayList<Integer> answer = new ArrayList<>();

If both can contain n elements:

O(n) + O(n) = O(2n) = O(n)

Do NOT write:

O(n²)

Rule:

Independent structures alive together → Add their memory.

Your Pattern 8 applies this same rule.

2.14 Nested Loops Do Not Mean Nested Space

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.println(arr[i] + arr[j]);
    }
}
```

Time: O(n²)

Extra memory: i → O(1), j → O(1)

Therefore: Space = O(1)

Remember:

Loops tell us how many times work occurs.

Stored information tells us how much space is required.

CodeWithNishchal

2.15 Recursion and Call Stack

Every recursive call creates a stack frame containing information such as:
Parameters, Local variables, Return address, Execution state

Example:

```
static int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

Active stack:

factorial(n)

factorial(n-1)

factorial(n-2)

...

factorial(1)

Maximum depth: n, Each frame uses constant space. Therefore: Space = O(n)
Your guide also calculates recursive space using maximum recursion depth.

2.16 Recursive Space Formula

General rule:
Recursive Space = Maximum Active Depth × Space Per Active Frame
If each frame uses O(1): Depth n → O(n), Depth log n → O(log n)
If each recursive frame creates an input-dependent structure, then we must add the memory of all simultaneously active frames.

2.17 Count Active Calls, Not Total Recursive Calls

Consider Fibonacci:

```
static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

There are exponentially many calls over the entire execution. So time is approximately: O(2^n)
But space does not count every call ever made. At one moment, the deepest active path is approximately:

fib(n)

→ fib(n-1)

→ fib(n-2)

→ ...

→ fib(1)

Depth: O(n)
Therefore: Space = O(n)

CodeWithNishchal

2.18 Fastest Way to Calculate Space Complexity

1. Identify what is supplied as input. Ignore supplied input if auxiliary space is asked.
2. Count constant-size variables as O(1).
3. Find every extra array/list/set/map/stack/queue.
4. Write the maximum size of each structure.
5. Check which structures are alive simultaneously.
6. If recursion exists, calculate maximum active recursion depth.
7. Check how much memory each recursive frame creates.
8. Add simultaneously alive memory and keep the dominant term.

2.19 Fast Recognition Table

Pattern	Fast Calculation	Space
Few primitive variables	Fixed count	O(1)
Loop only	Variables reused	O(1)
Nested loops only	No growing storage	O(1)
new int[n]	n cells	O(n)
Two arrays of n	n+n	O(n)
new int[n][m]	n×m	O(nm)
new int[n][n]	n^2	$O(n^2)$
new int[n][n][n]	n^3	$O(n^3)$
new int[26]	Fixed size	O(1)
HashSet with d values	d entries	O(d)
HashMap with n entries	n entries	O(n)
Linear recursion	n frames	O(n)
Halving recursion	log n frames	O(log n)
Tree DFS	height	O(h)
Tree BFS	maximum width	O(w)
Graph DFS	visited + stack	O(V)
Adjacency List	V + E	O(V+E)
Adjacency Matrix	V×V	$O(V^2)$
Store all subsets	n×2^n	O(n2^n)
Store all permutations	n×n!	O(n·n!)

CodeWithNishchal

DSA Notes X CodeWithNishchal

3. Array

3.1 Introduction

An **array** is a linear data structure used to store multiple values of the **same data type** under a single variable name.

Each value stored inside an array is called an **element**, and every element is accessed using its **index**. In Java, array indexing starts from 0.

For example, suppose we want to store marks of five students.

Without an array:

int mark1 = 80;

int mark2 = 75;

int mark3 = 90;

int mark4 = 88;

int mark5 = 70;

Using an array: int[] marks = {80, 75, 90, 88, 70};

The array allows all related values to be managed using one variable.

3.2 Why Do We Need Arrays?

Suppose we need to store 100 integers. Creating 100 separate variables is difficult:

int n1;

int n2;

int n3;

...

int n100;

This causes several problems:

- too many variables
- difficult traversal
- difficult searching
- difficult sorting
- difficult processing
- difficult maintenance

Instead, we can create one array: int[] numbers = new int[100];

Now the complete collection can be accessed using indexes.

numbers[0]

numbers[1]

numbers[2]

...

numbers[99]

Arrays therefore provide an organized way to store and process a collection of related data.

CodeWithNishchal

3.3. Basic Structure of an Array

Consider: int[] arr = {10, 20, 30, 40, 50};

Visual representation:

```
Index:     0      1      2      3      4
         +------+------+------+------+------+
Value:   |  10  |  20  |  30  |  40  |  50  |
         +------+------+------+------+------+
           ↑
         arr[0]
```

Here:

- arr[0] = 10
- arr[1] = 20
- arr[2] = 30
- arr[3] = 40
- arr[4] = 50

The first element is stored at index 0.

The last element is stored at: length - 1

For this array: length = 5

Last index = 5 - 1 = 4

3.4. Important Terminology

3.4.1 Element

A value stored inside an array.

Example: int[] arr = {10, 20, 30};

10, 20, and 30 are elements.

3.4.2 Index

The position used to access an element.

```
Index:   0   1   2
Value:  10  20  30
```

3.4.3 Length

The total number of elements that an array can contain.

In Java:

arr.length

Example: int[] arr = {10, 20, 30, 40};

System.out.println(arr.length);

Output: 4

Important: arr.length is a **property**, not a method.

Therefore: arr.length is correct. arr.length() is incorrect.

CodeWithNishchal

3.5 Characteristics of Arrays

The main characteristics of arrays are:

1. Arrays store multiple values using one variable.
2. All elements belong to the same declared data type.
3. Arrays use zero-based indexing.
4. Array size is fixed after creation.
5. Elements can be directly accessed using their index.
6. Arrays can store primitive values as well as object references.
7. Arrays themselves are objects in Java.
8. Java automatically performs bounds checking when accessing an array.

3.6. Array Declaration in Java

General syntax: dataType[] arrayName;
Example: int[] arr;
Another valid Java syntax is: int arr[];
However, the preferred style is: int[] arr;
because it clearly indicates that the variable is an array of integers.

3.7. Array Creation

An array can be created using the new keyword.
Syntax: arrayName = new dataType[size];
Example:

```
int[] arr;
arr = new int[5];
```

It can also be written in one statement:
int[] arr = new int[5];

This creates an integer array capable of storing five elements.
Initially:

```
Index:    0     1     2     3     4
        +------+------+------+------+------+
Value:  | 0  | 0  | 0  | 0  | 0  |
        +------+------+------+------+------+
```

Because Java automatically initializes array elements with default values.

3.8 Array Initialization

Initialization means assigning values to array elements.
There are multiple ways to initialize an array.

CodeWithNishchal

3.8.1 Initialization After Creation

```
int[] arr = new int[5];
arr[0] = 10;
arr[1] = 20;
arr[2] = 30;
arr[3] = 40;
arr[4] = 50;
```

3.8.2 Direct Initialization

```
int[] arr = {10, 20, 30, 40, 50};
```

Java automatically determines the array size. Size = 5

3.8.3 Using new With Values

```
int[] arr = new int[]{10, 20, 30, 40, 50};
```

This is also valid.

3.9 Default Values in Java Arrays

When an array is created using new, Java automatically initializes every element.

Data Type	Default Value
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
char	'\u0000'
boolean	false
Object reference	null

3.10. Accessing Array Elements

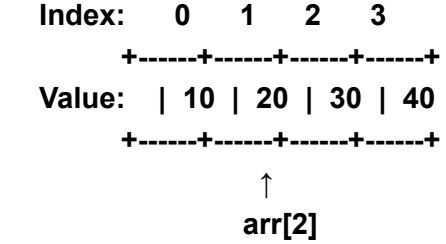
Array elements are accessed using their index.

Syntax: arrayName[index]

Example:

```
int[] arr = {10, 20, 30, 40}; System.out.println(arr[2]);
```

Output: 30



CodeWithNishchal

3.11. Updating an Array Element

An existing value can be replaced using its index.
Example:
int[] arr = {10, 20, 30};
arr[1] = 100;

Before: [10, 20, 30]
After: [10, 100, 30]
Updating an element takes constant time because the index is directly known.
Time Complexity = O(1)

3.12. Array Index Range

For an array of size n, valid indexes are: 0 to n - 1
Example: int[] arr = new int[5];
Valid indexes: 0,1,2,3,4
Invalid: -1,5,6,....

3.13. ArrayIndexOutOfBoundsException

Java checks whether an index is valid before accessing an element.
Example:
int[] arr = {10, 20, 30};

System.out.println(arr[3]);
The valid indexes are:
0, 1, 2
Therefore accessing index 3 causes:
ArrayIndexOutOfBoundsException

Visual:
Array size = 3

Index: 0 1 2

```
+------+------+------+
| 10 | 20 | 30 |
+------+------+------+
```

Valid ✓ ✓ ✓

Index 3 ✗

CodeWithNishchal

3.14. Memory Representation of an Array

Conceptually, an array provides indexed storage where elements are placed in an ordered sequence. For DSA understanding, we often visualize an array as consecutive slots:

arr

```
    0      1      2      3
  +-------+-------+-------+-------+
  | 10  | 20  | 30  | 40  |
  +-------+-------+-------+-------+
```

This indexed organization allows direct access to any element.

For example: arr[3]

can be reached without traversing: arr[0] → arr[1] → arr[2]

Therefore access is: O(1)

3.15. Traversing an Array

Traversal means visiting every element of the array.

Using a for Loop

```
int[] arr = {10, 20, 30, 40};
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Flow:

```
Start
  |
  v
i = 0
  |
  v
Is i < arr.length?
  |
  +---- No ----> Stop
  |
 Yes
  |
  v
Process arr[i]
  |
  v
i++
  |
  +-------------> Check condition again
```

Since every element is visited once:

Time Complexity = O(n)

CodeWithNishchal

3.16. Enhanced for Loop

Java also provides the enhanced for loop.

```
int[] arr = {10, 20, 30};

for (int value : arr) {
    System.out.println(value);
}
```

This is useful when we only need element values.
Conceptually:
10 → process
20 → process
30 → process

However, the enhanced for loop does not directly provide the index. Therefore use a normal for loop when indexes are required.

3.17 Searching in an Array

Suppose:

```
int[] arr = {10, 20, 30, 40, 50};
```

To search for 40, we may inspect elements sequentially:
10 → not found
20 → not found
30 → not found
40 → found

Example:

```
int target = 40;

for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) {
        System.out.println("Found at index " + i);
        break;
    }
}
```

In the worst case, all elements may need to be checked.
Time Complexity = O(n)

This type of direct sequential searching is called **linear search**.

CodeWithNishchal

3.18. Insertion in an Array

Arrays have fixed size.
Therefore inserting an element means placing it into an available position or shifting existing elements when maintaining order.
Suppose: [10, 20, 30, 40, _]
We want to insert 25 at index 2.
Before:
Index: 0 1 2 3 4

10 20 30 40 _
Shift elements toward the right: 40 → index 4, 30 → index 3
Then insert 25: [10, 20, 25, 30, 40]

Before

```
+----+----+----+----+----+
| 10 | 20 | 30 | 40 |    |
+----+----+----+----+----+
```

Shift:

```
30 -------->
40 -------->
```

After

```
+----+----+----+----+----+
| 10 | 20 | 25 | 30 | 40 |
+----+----+----+----+----+
```

Worst-case insertion may require shifting many elements.
Therefore: Time Complexity = O(n)

3.19. Insertion at Different Positions

Insertion at End

If free capacity already exists:
[10, 20, 30, _, _]

Insert 40:
[10, 20, 30, 40, _]

No shifting is needed.
Conceptually:
O(1)

provided the location is available.

CodeWithNishchal

Insertion at Beginning

Before:
[10, 20, 30, 40, _]

Insert 5:
Step 1: Shift 40
Step 2: Shift 30
Step 3: Shift 20
Step 4: Shift 10
Step 5: Insert 5

After: [5, 10, 20, 30, 40]
Time complexity: O(n)

3.20. Deletion From an Array

Arrays do not automatically shrink when an element is removed. Instead, remaining elements may need to be shifted to fill the gap.

Example:
[10, 20, 30, 40, 50]
Delete element at index 2.
Value: 30
Shift: 40 → index 2 50 → index 3
Result logically becomes: [10, 20, 40, 50]
Within the original physical array capacity, one extra slot still exists.

Before

```
+----+----+----+----+----+
| 10 | 20 | 30 | 40 | 50 |
+----+----+----+----+----+
```

Delete 30

```
     X
```

Shift left

```
40 <----
50 <----
```

Logical result

```
+----+----+----+----+
| 10 | 20 | 40 | 50 |
+----+----+----+----+
```

Worst-case time complexity:
O(n)

CodeWithNishchal

3.21 Deletion at Different Positions

Operation	Reason	Complexity
Delete first element	All remaining elements shift left	O(n)
Delete middle element	Some elements shift left	O(n)
Delete last logical element	No shifting needed	O(1)

The O(1) case assumes that the position of the last logical element is already known.

3.22. Fixed Size Nature of Arrays

One of the most important characteristics of a Java array is that its length cannot be changed after creation.
Example:
int[] arr = new int[5];

The size will always remain: 5
You cannot later convert the same array object into:
size 10

To obtain a larger array, a new array must be created.
Conceptually:
Old Array

[10][20][30]

↓ Copy
New Array
[10][20][30][_][_][_]

This limitation is one reason dynamic data structures such as ArrayList exist.

3.23 Copying an Array

Manual Copy

```
int[] oldArray = {10, 20, 30};
int[] newArray = new int[5];

for (int i = 0; i < oldArray.length; i++) {
    newArray[i] = oldArray[i];
}
```

Since each existing element is copied: Time Complexity = O(n)

CodeWithNishchal

3.24. System.arraycopy()

Java provides:
System.arraycopy()

Example:
```
int[] source = {10, 20, 30};
int[] destination = new int[5];

System.arraycopy(source, 0, destination, 0, source.length);
```

General form:
```
System.arraycopy(
    sourceArray,
    sourceStartIndex,
    destinationArray,
    destinationStartIndex,
    numberOfElements
);
```

3.25. Arrays.copyOf()

Java's Arrays class provides:
Arrays.copyOf()

Example:
```
import java.util.Arrays;
int[] arr = {10, 20, 30};
int[] copy = Arrays.copyOf(arr, arr.length);
```

To create a larger array:
```
int[] larger = Arrays.copyOf(arr, 5);
```

Result: [10, 20, 30, 0, 0]

3.26. Comparing Arrays

```
int[] a = {10, 20, 30};
int[] b = {10, 20, 30};
```
Using: a == b

does not compare the contents. It checks whether both references point to the same array object.
To compare contents: **Arrays.equals(a, b);**

CodeWithNishchal

3.27. Printing Arrays

This does not properly print array elements:
System.out.println(arr);

For readable output use: Arrays.toString(arr);

Example:
int[] arr = {10, 20, 30};
System.out.println(Arrays.toString(arr));

Output: [10, 20, 30]

3.28. Arrays Utility Class

Java provides the java.util.Arrays class containing several useful methods.
Import:
import java.util.Arrays;

Common methods include:

Method	Purpose
Arrays.toString()	Print array contents
Arrays.sort()	Sort an array
Arrays.equals()	Compare contents
Arrays.copyOf()	Copy an array
Arrays.fill()	Fill elements with a value
Arrays.binarySearch()	Search a sorted array

3.29 One-Dimensional Array

A one-dimensional array is the simplest form of array.
Example:
int[] arr = {10, 20, 30, 40};

Visual:

```
+----+----+----+----+
| 10 | 20 | 30 | 40 |
+----+----+----+----+
  0    1    2    3
```

Only one index is required: arr[i]

CodeWithNishchal

3.30. Two-Dimensional Array

A two-dimensional array can be visualized like rows and columns.

Example:

```
int[][] matrix = {
   {1, 2, 3},
   {4, 5, 6}
};
```

Visual:

```
          Column
          0   1   2
```

Row 0   [1] [2] [3]
Row 1   [4] [5] [6]

Access: matrix[0][1]
returns: 2

3.31. Internal Nature of 2D Arrays in Java

A Java two-dimensional array is actually an **array of arrays**.

Conceptually:

```
matrix
  |
  v
+------+------+
| ref  | ref  |
+--|---+--|---+
  |      |
  v      v
[1][2][3]

[4][5][6]
```

Because each row is itself an array, different rows may even have different lengths.

3.32. Jagged Array

A jagged array is a multidimensional array in which rows have different lengths.

Example:

```
int[][] arr = {
   {1, 2},
   {3, 4, 5},
   {6}
};
```

Row 0 → [1] [2]
Row 1 → [3] [4] [5]
Row 2 → [6]

CodeWithNishchal

3.33. Traversing a 2D Array

Example:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

For a matrix having:
rows = r
columns = c

Traversal complexity is: O(r × c) because every element is visited.

3.34. Passing Arrays to Methods

Arrays can be passed to methods.

Example:

```
static void printArray(int[] arr) {
    for (int value : arr) {
        System.out.println(value);
    }
}
```

Calling: int[] numbers = {10, 20, 30};
printArray(numbers);
When an array is passed to a method, Java passes the value of the reference.
Both the calling code and the method can therefore refer to the same array object.
Example:

```
static void change(int[] arr) {
    arr[0] = 100;
}
```

Then:

```
int[] numbers = {10, 20, 30};
change(numbers);
```

Result: [100, 20, 30]
The array contents were modified.

CodeWithNishchal

3.35. Important Array Operations and Complexities

For an array containing n elements:

Operation	Time Complexity
Access by index	O(1)
Update by index	O(1)
Traverse	O(n)
Search in unsorted array	O(n)
Insert at beginning	O(n)
Insert in middle	O(n)
Insert at available end position	O(1)
Delete from beginning	O(n)
Delete from middle	O(n)
Delete last logical element	O(1)
Copy complete array	O(n)

3.36. Space Complexity

If an array stores n elements: Space Complexity = O(n)
because storage grows linearly with the number of elements.

Example:

n elements
↓
n storage locations

3.37. Why Is Array Access O(1)?

Consider:
int[] arr = {10, 20, 30, 40, 50};

Suppose we want: arr[4]

The program does not need to inspect:

arr[0]
arr[1]
arr[2]
arr[3]

First, the index directly identifies the required slot. Conceptually:

Requested index = 4

```
          Direct access
               |
               v
+----+----+----+----+----+
| 10 | 20 | 30 | 40 | 50 |
+----+----+----+----+----+
  0    1    2    3    4
```

Therefore the number of steps does not grow with n. Hence: O(1)

CodeWithNishchal

3.38. Why Are Insertions and Deletions Often O(n)?

The problem is not finding the index.
The problem is **shifting elements**.
Suppose:
[10, 20, 30, 40, 50]

Insert at index 0.
Existing elements must move:
50 →
40 →
30 →
20 →
10 →

As the array grows, the number of shifts can grow proportionally to n.
Therefore: O(n)
The same reasoning applies to deletion near the beginning.

3.39. Advantages of Arrays

1. Fast Random Access

Elements can be accessed directly using an index. O(1)

---

2. Simple Structure

Arrays are easy to understand and use.

---

3. Efficient Traversal

Elements can be processed sequentially using loops.

---

4. Low Structural Overhead

Unlike linked structures, arrays do not require an additional pointer/reference field for every element.

---

5. Useful Foundation

Many important concepts and structures are built using arrays, including:

- matrices
- heaps
- hash table implementations
- dynamic arrays
- many sorting algorithms
- many searching algorithms

CodeWithNishchal

3.40. Limitations of Arrays

1. Fixed Size

Once created: new int[5]
its length cannot become 10.

---

2. Expensive Middle Insertion

Elements may need to be shifted. O(n)

---

3. Expensive Middle Deletion

Elements may need to be shifted. O(n)

---

4. Capacity Must Often Be Known

If too little space is allocated, a new larger array must be created.
If too much space is allocated, part of the capacity may remain unused.

---

5. Same Declared Type

An array is designed to contain values compatible with its declared element type.
Example: int[] arr
stores integer values.

3.41. Array vs ArrayList

This comparison is useful because both provide indexed collections in Java.

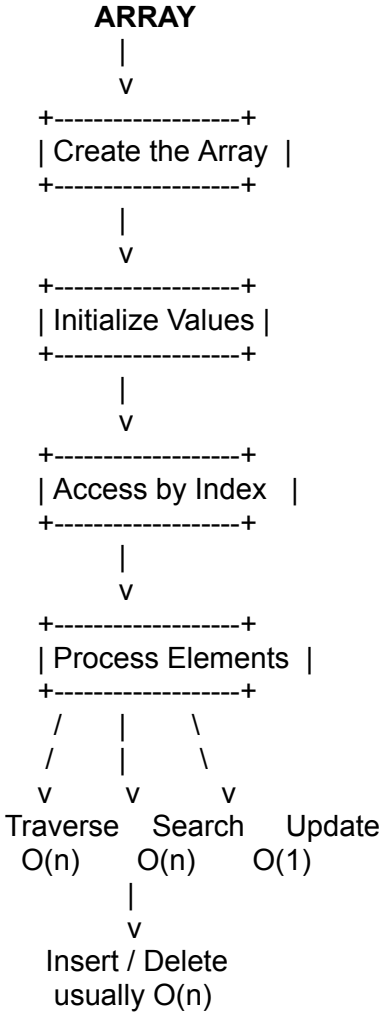
Feature	Array	ArrayList
Size	Fixed	Dynamic
Primitive values	Directly supported	Uses wrapper classes
Indexed access	O(1)	O(1)
Resize automatically	No	Yes
Syntax	Simple	Uses collection API
Length/size	arr.length	list.size()
Generic support	No generics on primitive arrays	Supports generics
Structural flexibility	Lower	Higher

Example array: int[] arr = new int[5];
Example ArrayList: ArrayList<Integer> list = new ArrayList<>();

An ArrayList internally uses a resizable array concept, but it provides additional functionality for dynamic growth.

CodeWithNishchal

3.42. Overall Array Flow



3.43. Final Complexity Summary

Operation	Best / Typical Relevant Cost	Worst Case
Access	O(1)	O(1)
Update	O(1)	O(1)
Traversal	O(n)	O(n)
Linear search	O(1) if first match	O(n)
Insert at beginning	O(n)	O(n)
Insert in middle	O(n)	O(n)
Insert at available end	O(1)	O(1)
Delete at beginning	O(n)	O(n)
Delete in middle	O(n)	O(n)
Delete last logical element	O(1)	O(1)
Copy array	O(n)	O(n)

CodeWithNishchal

4. String

4.1 Introduction

A **String** is a sequence of characters used to represent textual data. Examples include names, sentences, passwords, URLs, and any other character-based information.

In Java, String is **not a primitive data type**. It is a class available in java.lang, so it can be used directly without importing anything.

String name = "Nishchal";
String message = "Hello World";

Conceptually:

String: "HELLO"

```
Index:     0   1   2   3   4
         +---+---+---+---+---+
Character | H | E | L | L | O |
         +---+---+---+---+---+
```

Like arrays, String indexing starts from 0.

For a String of length n:

First index = 0
Last index = n - 1

4.2 Creating Strings in Java

Strings are mainly created in two ways.

Using String Literal

```
String str = "Hello";
```

Using new

```
String str = new String("Hello");
```

Both contain the same text, but their object creation behavior is different because Java maintains a special **String Pool**.

4.3 String Pool

Java stores String literals in a special area called the **String Pool** so that identical String literals can be reused instead of unnecessarily creating duplicate objects.

```
String s1 = "Java";  String s2 = "Java";
```

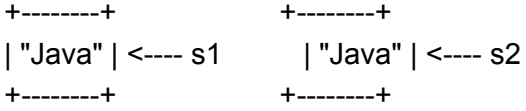
Conceptually:

```
String Pool
        +--------+
s1 ---->| "Java" |
s2 ---->|        |
        +--------+
```

CodeWithNishchal

Both references can point to the same pooled String object.
Now consider:
String s1 = "Java";
String s2 = new String("Java");

Conceptually:
String Pool Heap



new String() explicitly creates a separate String object.
This behavior becomes important when comparing Strings using ==.

4.4 Strings Are Immutable

One of the most important properties of Java Strings is **immutability**.
Immutable means that once a String object is created, its contents cannot be changed.
Example:
String str = "Hello";
str = str + " World";

It may appear that "Hello" was modified, but internally a new String is produced.
Conceptually:
Initially: str ----> "Hello"
After: str ----> "Hello World"
"Hello"
old String remains unchanged

Why Is String Immutable?

Immutability provides several benefits:

- String Pool objects can be safely shared.
- Strings are safer for sensitive values such as file paths, URLs, and class names.
- Strings work reliably as keys in hash-based structures such as HashMap.
- Immutable objects are easier to use safely across multiple threads.

4.5 Length and Character Access

The number of characters in a String is obtained using: **str.length()**

Example:
String str = "Java";
System.out.println(str.length());
Output: 4

CodeWithNishchal

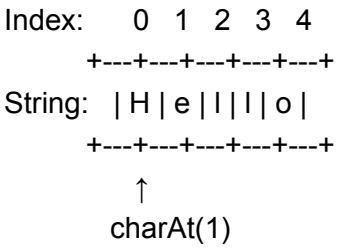
Unlike arrays: arr.length
Strings use a method: str.length()

A character at a particular index can be accessed using:
str.charAt(index)

Example:
String str = "Hello";
System.out.println(str.charAt(1));

Output: e

Visual:



Accessing an invalid index causes: StringIndexOutOfBoundsException

---

4.6 Traversing a String

A String can be traversed character by character.

```
String str = "Java";

for (int i = 0; i < str.length(); i++) {
   System.out.println(str.charAt(i));
}
```

For a String containing n characters, complete traversal takes: O(n)

4.7 Comparing Strings

String comparison is extremely important in Java.

== Operator

== compares whether two references point to the same object.

```
String s1 = new String("Java");
String s2 = new String("Java");
System.out.println(s1 == s2);
```

Output: false

Even though the contents are equal, the objects are different.

CodeWithNishchal

equals()

equals() compares String contents.
System.out.println(s1.equals(s2));
Output: true

Therefore, for normal String value comparison, use:
s1.equals(s2) not: s1 == s2

Case-Insensitive Comparison

String a = "JAVA";
String b = "java";
System.out.println(a.equalsIgnoreCase(b));
Output: true

4.8 Important String Methods

Method	Purpose
length()	Returns number of characters
charAt(i)	Returns character at index i
substring()	Extracts part of a String
equals()	Compares contents
equalsIgnoreCase()	Compares ignoring case
compareTo()	Lexicographical comparison
contains()	Checks whether a substring exists
indexOf()	Finds first occurrence
lastIndexOf()	Finds last occurrence
startsWith()	Checks prefix
endsWith()	Checks suffix
toLowerCase()	Converts to lowercase
toUpperCase()	Converts to uppercase
trim()	Removes leading/trailing basic whitespace
replace()	Replaces characters or substrings
split()	Splits String using a delimiter
isEmpty()	Checks whether length is 0

CodeWithNishchal

4.9 Substring

A substring is a smaller portion of an existing String.

substring(start)

String str = "Programming";
System.out.println(str.substring(3));

Output: gramming

substring(start, end)

System.out.println(str.substring(0, 4));

Output: Prog

Important rule:
Start index → Included
End index → Excluded

Visual:
String: "PROGRAM"

Index: 0 1 2 3 4 5 6
P R O G R A M

substring(1, 4)

Indexes: 1 2 3
Result: "ROG"

4.10 Searching Inside a String

indexOf()

Returns the first index of a character or substring.
String str = "banana";
System.out.println(str.indexOf('a'));

Output: 1

lastIndexOf()

System.out.println(str.lastIndexOf('a'));
Output: 5
If the value is not found: -1
Example:
System.out.println(str.indexOf('z'));
Output: -1

CodeWithNishchal

4.11 Concatenation

Concatenation means joining Strings.

Using +:

```
String first = "Hello";
String second = "World";

String result = first + " " + second;
```

Result: Hello World

Using: first.concat(second);

Example: String result = "Hello".concat(" World");

Because Strings are immutable, repeated concatenation can create many intermediate String objects.

Example:

```
String str = "";
for (int i = 0; i < 1000; i++) {
    str = str + i;
}
```

For repeated modifications, StringBuilder is usually more appropriate.

4.12 StringBuilder

StringBuilder represents a **mutable sequence of characters**. Unlike String, its contents can be modified without creating a completely new object after every change.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb);
```

Output: Hello World

Important methods:

Method	Purpose
append()	Adds content at the end
insert()	Inserts content
delete()	Deletes characters
replace()	Replaces a range
reverse()	Reverses characters
charAt()	Accesses character
setCharAt()	Changes character
length()	Returns length
toString()	Converts builder to String

CodeWithNishchal

4.13 StringBuffer

StringBuffer is similar to StringBuilder, but its methods are synchronized, making it suitable when the same mutable character sequence must be safely accessed by multiple threads.

```
StringBuffer buffer = new StringBuffer("Hello");

buffer.append(" Java");
```

For most single-threaded DSA problems:
StringBuilder

is generally preferred because synchronization overhead is unnecessary.

4.14 String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe mutable operations	Not applicable as mutable structure	No	Yes
Modification speed	Slower for repeated changes	Fast	Slightly slower than StringBuilder
String Pool	Literals may use pool	No	No
Common use	Fixed text	Repeated modifications	Thread-safe modifications

For DSA problems involving frequent character updates or building results:
StringBuilder sb = new StringBuilder(); is commonly used.

4.15 Converting Between String and Character Array

A String can be converted into a char[].

```
String str = "Java";
char[] arr = str.toCharArray();
```

Result:

```
String
"Java"
  ↓ toCharArray()

+---+---+---+---+
| J | a | v | a |
+---+---+---+---+
```

A character array can be converted back:

```
char[] arr = {'J', 'a', 'v', 'a'};
String str = new String(arr);
```

This conversion is useful when characters need to be modified directly.

CodeWithNishchal

4.16 Converting Other Types to String

Integer to String

int num = 123;

String str = String.valueOf(num);
or:
String str = Integer.toString(num);

String to Integer

String str = "123";
int num = Integer.parseInt(str);

Similarly:
Double.parseDouble()
Long.parseLong()
Boolean.parseBoolean()
are available for other data types.

4.17 Empty String vs Null String

These are different concepts.

Empty String

String str = "";

A valid String object exists, but it contains zero characters.
str.length(); // 0

Null Reference

String str = null;

Here, str does not refer to any String object.
Calling:
str.length();

causes:
NullPointerException

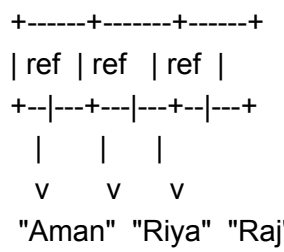
Comparison:

Value	Meaning
""	Existing String with length 0
" "	String containing one space
null	No String object referenced

CodeWithNishchal

4.18 Arrays of Strings

Java allows arrays containing String references.
String[] names = {"Aman", "Riya", "Raj"};
Conceptually:
names



Traversal:

```
for (String name : names) {
    System.out.println(name);
}
```

4.19 String Complexity

For a String containing n characters:

Operation	Typical Time Complexity
length()	O(1)
charAt(index)	O(1)
Complete traversal	O(n)
equals()	O(n) worst case
compareTo()	O(min(n, m))
indexOf()	Up to O(n × m) in general API-level reasoning
substring()	O(k) in modern Java, where k is substring length
toCharArray()	O(n)
toLowerCase() / toUpperCase()	O(n)
String concatenation	Depends on copied characters
StringBuilder.append()	Amortized O(1) for a small append operation

For interview analysis, always consider how many characters must be inspected or copied rather than blindly memorizing one complexity for every method

CodeWithNishchal

5. Hashing

5.1 Introduction

Hashing is a technique used to store and retrieve data efficiently by converting a key into an integer-like value called a **hash value** or **hash code**. That hash value is then used to decide where the data should be stored.

Instead of searching every element one by one, hashing tries to directly locate the required data.

Conceptually:

Key
|
v
Hash Function
|
v
Hash Value
|
v
Index / Bucket
|
v
Stored Data

Example:

Key = "apple"
|
v
hash("apple")
|
v
Hash Value = 93029210
|
v
Bucket Index = 5

The exact hash values and bucket calculations depend on the implementation.

5.2 Why Hashing Is Needed

Suppose we store:

10, 20, 30, 40, 50

If we use a normal unsorted array and search for 40, we may need to scan elements:

10 → 20 → 30 → 40

Worst-case search: O(n)

With hashing, the key is processed through a hash function so that its storage location can be found much faster.

Average lookup in a good hash-based structure is commonly: O(1)

CodeWithNishchal

This makes hashing useful for:

- fast lookup
- frequency counting
- duplicate detection
- membership checking
- grouping data
- mapping keys to values
- caching

5.3 Hash Function

A **hash function** converts a key into a hash value.
Conceptually:
Key ----> Hash Function ----> Hash Value

Example:
"Java" ----> hash function ----> 2301506

A good hash function should distribute keys reasonably evenly so that too many keys do not end up in the same bucket.
For a simple conceptual integer example:
index = key % tableSize
If: tableSize = 10
key = 37
then: index = 37 % 10 = 7

So the value may be mapped to bucket 7.
This simple modulo approach is useful for understanding hashing, although real Java implementations use more involved processing.

5.4 Hash Table

A **hash table** is the data structure that stores keys or key-value pairs using hashing.
Example:
Bucket
Index
0 →
1 → 21
2 →
3 → 43
4 →
5 → 15
6 →
7 → 37
8 →
9 →

CodeWithNishchal

If the hash function is:
index = key % 10

then:
21 % 10 = 1
43 % 10 = 3
15 % 10 = 5
37 % 10 = 7

5.5 Collision

A **collision** happens when two different keys map to the same bucket.
Example:
21 % 10 = 1
31 % 10 = 1

Both keys want bucket 1.

```
21 ----\
       ---> Bucket 1
31 ----/
```

Collisions are normal in hashing and must be handled correctly.
Two common collision-handling ideas are:

1. **Separate Chaining**
2. **Open Addressing**

Java's HashMap conceptually uses bucket-based collision handling; modern implementations may also convert heavily populated buckets into tree-based structures.

5.6 Separate Chaining

In separate chaining, multiple entries that map to the same bucket are stored together.
Example:
Bucket 0 → null
Bucket 1 → 21 → 31 → 41
Bucket 2 → null
Bucket 3 → 13

Here:
21 % 10 = 1
31 % 10 = 1
41 % 10 = 1

All three collide at bucket 1.
Conceptually, each bucket can hold multiple entries instead of only one.

CodeWithNishchal

5.7 Open Addressing

In open addressing, if the calculated location is occupied, another location is searched.
Example:
Key = 21
Index = 1

Bucket 1 → occupied Try next available location
Bucket 2 → free Store at Bucket 2
Different techniques include:

- linear probing
- quadratic probing
- double hashing

These concepts are important in general hashing theory, but Java's standard HashMap is not normally explained as an open-addressing structure.

5.8 Load Factor

The **load factor** measures how full a hash table is.
Formula: **Load Factor = Number of Stored Entries / Number of Buckets**
Example:
Entries = 12
Buckets = 16
Load Factor = 12 / 16 = 0.75

As the table becomes too full, collisions usually increase. Therefore hash tables may resize when a threshold is crossed. Java's HashMap commonly uses a default load factor of: 0.75

5.9 Rehashing

When the table becomes too full, a larger internal table can be created and existing entries redistributed.
Conceptually:
Old Table
Size = 4
[ ][A][B][C]

|
v
Resize
New Table
Size = 8
[ ][ ][A][ ][B][ ][C][ ]

The entries may move because bucket positions depend on the table capacity.
Rehashing itself can take: O(n)
but it does not happen for every insertion, so average insertion remains efficient.

CodeWithNishchal

5.10 Time Complexity of Hashing

For a well-distributed hash table:

Operation	Average	Worst Case
Insert	O(1)	Can degrade
Search	O(1)	Can degrade
Delete	O(1)	Can degrade

The worst case happens when many keys collide badly.

In modern Java HashMap, a heavily collided bucket can become tree-based under suitable conditions, improving lookup in that bucket compared with a simple long linked chain.

5.11. HashMap in Java

5.11.1 Introduction

HashMap is a class in Java that stores data in **key-value pairs**.

Example:

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Aman", 90);
marks.put("Riya", 95);
marks.put("Raj", 80);
```

Conceptually:

"Aman" → 90
"Riya" → 95
"Raj" → 80

A common declaration is: **Map<String, Integer> map = new HashMap<>();**

Using the Map interface on the left side is often preferred because it keeps the code flexible.

5.11.2 Important Properties of HashMap

A HashMap has the following important characteristics:

- stores key-value pairs
- keys are unique
- values may repeat
- allows one null key
- allows multiple null values
- does not guarantee insertion order
- average lookup, insertion, and deletion are usually O(1)
- uses hashing internally
- is not synchronized

CodeWithNishchal

Example:

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);
map.put("A", 50);
```

Final contents logically become: A → 50 B → 20
because inserting the same key again updates its value.

5.11.3 Role of hashCode() and equals()

These two methods are fundamental for custom objects used as keys.

hashCode()

Determines a hash value used to find the likely bucket.

equals()

Determines whether two keys should be considered logically equal.
General rule:
If two objects are equal according to equals(),
they must return the same hashCode().

If equals() and hashCode() are inconsistent, HashMap behavior can become incorrect for logical key lookup.

5.11.4 Core HashMap Operations

Insert or Update

```
map.put(key, value);
```

Example:

```
map.put("A", 10);
map.put("B", 20);
```

Updating: `map.put("A", 100);`

Retrieve

```
map.get(key);
```

Example:

```
Integer value = map.get("A");
```

If the key is absent: null
is returned.

Check Key

```
map.containsKey(key);
```

Example:

```
if (map.containsKey("A")) {
   System.out.println("Present");
}
```

CodeWithNishchal

Check Value

map.containsValue(value);

Example: map.containsValue(100);

Checking a value may require scanning entries, so it is not generally O(1).

Remove

map.remove(key);

Example: map.remove("A");

Size and Empty Check

map.size();

map.isEmpty();

Clear

map.clear();

Removes all entries.

5.11.5 Useful HashMap Methods

Method	Purpose
put(k, v)	Insert or update
get(k)	Get value
getOrDefault(k, d)	Get value or default
containsKey(k)	Check key
containsValue(v)	Check value
remove(k)	Remove entry
size()	Number of entries
isEmpty()	Check whether empty
clear()	Remove all entries
keySet()	Get all keys
values()	Get all values
entrySet()	Get key-value entries
putIfAbsent()	Insert only if key is absent

A particularly useful DSA method is: map.getOrDefault(key, defaultValue);

Example:

map.put(num, map.getOrDefault(num, 0) + 1);

This is commonly used for frequency counting.

CodeWithNishchal

5.11.6 Traversing a HashMap

Using entrySet()

This is commonly the best way when both key and value are required.

```
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

Using keySet()

```
for (String key : map.keySet()) {
    System.out.println(key + " -> " + map.get(key));
}
```

Using values()

```
for (Integer value : map.values()) {
    System.out.println(value);
}
```

5.11.7 HashMap Time Complexity

Operation	Average	Important Note
put()	O(1)	Depends on hashing and collisions
get()	O(1)	Average case
containsKey()	O(1)	Average case
remove()	O(1)	Average case
containsValue()	O(n)	May scan values
Full traversal	O(n)	Visits entries
size()	O(1)	Stored internally

Worst-case behavior depends on collision structure and Java implementation details.

5.11.8 HashMap vs HashSet

Feature	HashMap	HashSet
Stores	Key-value pairs	Unique values
Duplicate keys/elements	Keys unique	Elements unique
Retrieve associated value	Yes	No
Main use	Mapping	Membership / uniqueness

Example:

CodeWithNishchal

Map<String, Integer> map = new HashMap<>();

versus:
Set<Integer> set = new HashSet<>();

Use a HashMap when you need:
key → information

Use a HashSet when you only need:
Does this value exist?

5.11.9 HashMap Patterns

The following are common and useful HashMap-based patterns.

Pattern 1: Frequency Based

Idea

Store:
Element → Frequency

Example:
Array: [1, 2, 2, 3, 1, 2]

Frequency Map:
1 → 2
2 → 3
3 → 1
Typical uses:

- count element frequency
- find duplicates
- find most frequent element
- check whether two collections have matching counts
- character frequency
- anagram-type problems

Standard Template

```
Map<Integer, Integer> freq = new HashMap<>();
for (int num : arr) {
    freq.put(num, freq.getOrDefault(num, 0) + 1);
}
```

Complexity:
Time = O(n) average
Space = O(k)
where k is the number of distinct elements.

CodeWithNishchal

Pattern 2: Lookup Based

Idea

Store previously seen information so that a required value can be found quickly.
Instead of repeatedly searching an array:
O(n) per search

store useful information in a map and perform average:
O(1) lookup

A classic structure is:
Value → Useful Information

Example:
Value → Index

Suppose:
Array = [2, 7, 11, 15]
Target = 9

While processing 7, we need:
9 - 7 = 2

If 2 is already in the map, the required pair exists.

```
Map<Integer, Integer> map = new HashMap<>();

for (int i = 0; i < arr.length; i++) {
    int required = target - arr[i];

    if (map.containsKey(required)) {
        System.out.println(map.get(required) + " " + i);
    }

    map.put(arr[i], i);
}
```

This pattern is useful when a problem contains phrases such as:

- "have we seen this before?"
- "find complement"
- "find matching value"
- "retrieve previous index"
- "check existence quickly"

CodeWithNishchal

Pattern 3: Set Based

Strictly speaking, this pattern is more naturally implemented using HashSet, but it belongs to the same hashing family.

Idea

Store only whether an element has appeared.
Element → Present / Not Present

Example: Input: [4, 2, 7, 4]
Set while traversing:

4
4, 2
4, 2, 7

Next 4 already exists → duplicate found
Example:

```
Set<Integer> seen = new HashSet<>();
for (int num : arr) {
    if (seen.contains(num)) {
        System.out.println("Duplicate: " + num);
    }

    seen.add(num);
}
```

Use this pattern when you do **not** need a value or count, only membership.
Typical uses:

- duplicate detection
- unique elements
- visited values
- membership checking

Pattern 4: Index Mapping

Idea

Store: Value → Index
or: Value → List of Indices
Example: Array: [10, 20, 30, 20]
Map:
10 → 0
20 → 3
30 → 2

If multiple indexes are important:
10 → [0]
20 → [1, 3]
30 → [2]

CodeWithNishchal

Example:

```
Map<Integer, List<Integer>> indexMap = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
   indexMap
      .computeIfAbsent(arr[i], k -> new ArrayList<>())
      .add(i);
}
```

Useful when problems require:

- previous position
- first occurrence
- last occurrence
- all positions
- matching indexes
- restoring positional information

Pattern 5: Grouping Pattern

Idea

Store: Group Key → Collection of Related Elements

Example:Words: ["eat", "tea", "tan", "ate"]
Possible grouping:
"aet" → ["eat", "tea", "ate"]
"ant" → ["tan"]

Here the normalized representation becomes the key.
Structure:

```
Map<String, List<String>> groups = new HashMap<>();
```

General template:

```
for (String item : items) {
   String key = createGroupKey(item);
   groups
      .computeIfAbsent(key, k -> new ArrayList<>())
      .add(item);
}
```

Typical uses:

- group anagrams
- group by category
- group by remainder
- group by frequency signature
- group records sharing the same property

The important idea is:
Create a common key for related elements
|
v
Store all matching elements under that key

CodeWithNishchal

5.11.10 Choosing the Correct Hashing Pattern

Requirement	Best Structure / Pattern
Need count	Frequency Map
Need fast key lookup	Lookup Map
Need only existence	HashSet
Need original position	Index Mapping
Need multiple positions	Map<Value, List>
Need to collect similar elements	Grouping Map
Need key → associated information	HashMap

A useful mental rule is:

Need only presence?

|

Yes

|

HashSet

Need extra information attached to a key?

|

Yes

|

HashMap

5.11.11 Common Mistakes

1. Using get() Without Checking for Missing Key

int count = map.get(key);

If the key is absent and the value type is Integer, map.get(key) returns null, which can cause problems when unboxed.

Safer: int count = map.getOrDefault(key, 0);

---

2. Assuming HashMap Maintains Order

HashMap does not guarantee insertion or sorted order.

If ordering matters, structures such as LinkedHashMap or TreeMap may be more appropriate.

---

3. Using HashMap When Only Membership Is Needed

Instead of:

Map<Integer, Boolean> map = new HashMap<>();

prefer:

Set<Integer> set = new HashSet<>();

when no associated value is required.

CodeWithNishchal

6. Array Patterns



6. Two Pointer Pattern

6.1 What is the Two Pointer Technique?

The **Two Pointer technique** is a problem-solving pattern in which two indexes or references are used to process an array or String instead of relying on nested loops.

The two pointers usually represent positions such as:

- **left and right**
- **fast and slow**
- **write and read**

Depending on the problem, pointers may:

- start from opposite ends,
- start from the same side and move at different speeds,
- divide the array into regions while partitioning elements.

The major goal is usually to process an array or String in a **single or limited number of passes**.

Example: **Array: [1, 2, 3, 4, 5, 6]**

Instead of:

for every element

  check every other element

O(n²)

Two pointers may allow:

left ---------->

<------ right

O(n)

Two pointers are especially useful when the problem involves **pairs, sorted data, reversing, comparison from both ends, removing elements, compacting data, or partitioning elements**.

Code With Nishchal

6.2 Why Two Pointers Are Useful

Consider finding two numbers in a sorted array whose sum equals a target.
Array = [1, 2, 4, 6, 8, 10]
Target = 14

A brute-force solution checks every pair:
1 with 2, 4, 6, 8, 10
2 with 4, 6, 8, 10
...

Time complexity:
$O(n^2)$

With two pointers:

```
L                 R
↓                 ↓
1   2   4   6   8   10
```

1 + 10 = 11
11 < 14
Move L right.

```
   L              R
   ↓              ↓
1   2   4   6   8   10
```

2 + 10 = 12
12 < 14

Move L right.

```
      L           R
      ↓           ↓
1   2   4   6   8   10
```

4 + 10 = 14 Found

Each pointer moves only a limited number of times.
Therefore:
Time = O(n)
Space = O(1)

This reduction from $O(n^2)$ to O(n) is one of the main reasons the pattern is important.

CodeWithNishchal

6.3 Core Idea Behind Pointer Movement

Two pointers work only when we can decide **which pointer should move based on the current state**.

For example, in a sorted array:

currentSum = arr[left] + arr[right]

If:

currentSum < target

we need a larger sum, so move:

left++

because moving the left pointer right gives us a larger value.

If:

currentSum > target

we need a smaller sum, so move: right--

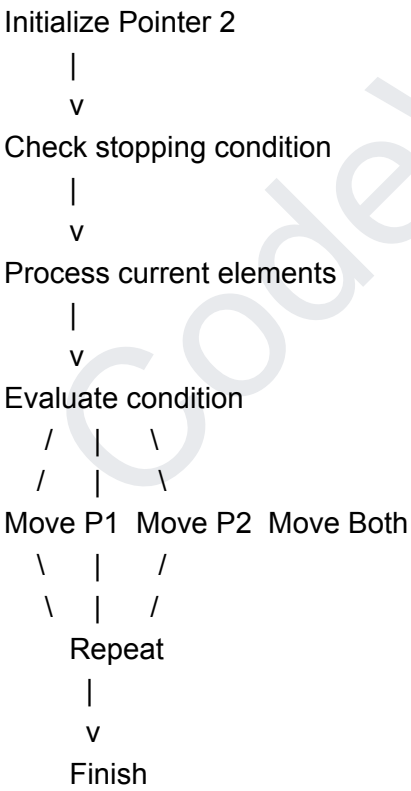
This is the most important reasoning behind the pattern.

Two pointers are not simply:

Create two variables and move them somehow.

There must be a logical reason why moving one pointer safely eliminates unnecessary possibilities.

6.4 General Two Pointer Flow



The exact initialization and movement depend on the type of two-pointer pattern.

CodeWithNishchal

6.5 How to Recognize Two Pointer Problems

Two pointers should come to mind when a problem involves things such as:
Pair of elements
Sorted array
Compare two ends
Palindrome
Reverse array/String
Remove duplicates
Move selected elements
Merge/compact elements
Partition array
Separate elements based on condition

Common phrases include:
"find two elements..."
"pair whose sum..."
"in-place..."
"remove duplicates..."
"move all..."
"reverse..."
"palindrome..."
"sorted array..."
"partition..."

However, seeing these words does not automatically mean two pointers will work. Pointer movement must still allow us to safely eliminate possibilities.

6.6. Opposite Ends — Left and Right Pointers

This is the most recognizable form of two pointers.
Pointers start at opposite ends:
left = 0
right = n - 1
Visual:

```
L                 R
↓                 ↓
[ 1 ][ 2 ][ 3 ][ 4 ][ 5 ][ 6 ]
```

The pointers gradually move toward each other:
L → → → ← ← ← R

The loop normally continues while: left < right
or sometimes: left <= right
depending on whether the middle element must also be processed.

CodeWithNishchal

6.6.1 General Template

```
int left = 0;
int right = arr.length - 1;
while (left < right) {
    // Process arr[left] and arr[right]
    if (condition1) {
        left++;
    } else if (condition2) {
        right--;
    } else {
        left++;
        right--;
    }
}
```

6.6.2 When Opposite-End Pointers Are Useful

Common cases include:

- finding a pair in a sorted array, reversing an array,
- reversing a String or character array, palindrome checking,
- comparing elements from both sides, some container/area problems,
- shrinking a search region.

6.6.3 Example 1 — Pair Sum in Sorted Array

Problem:Given a sorted array and a target value, determine whether there exists a pair of elements whose sum is equal to the target.

Example: Array = [1, 2, 4, 6, 8, 10] Target = 14

Possible valid pair: 4 + 10 = 14 or 6 + 8 = 14

Initialize:

```
L                   R
↓                   ↓
1   2   4   6   8   10
```

Since the array is sorted, place one pointer at the beginning and one at the end:
left = 0
right = arr.length - 1
At every step, calculate: sum = arr[left] + arr[right]
Now there are three cases:

1. If sum == target, the required pair is found.
2. If sum < target, the current sum is too small. Because the array is sorted, move left to the right to get a larger value.
3. If sum > target, the current sum is too large. Move right to the left to get a smaller value.
   Continue until:
   left >= right

CodeWithNishchal

```
static boolean hasPairWithSum(int[] arr, int
target) {
    int left = 0;
    int right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) {
            return true;
        }
        if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
    return false;
}
```

Complexity: Time = O(n) Space = O(1)

6.6.4 Example 2 — Reverse an Array

Given an array, reverse its elements in-place so that the first element becomes the last, the second becomes the second-last, and so on.
Example: Input = [10, 20, 30, 40, 50] Output = [50, 40, 30, 20, 10]

Approach:

left starts at the beginning of the array. right starts at the end of the array. **At every step:**

1. Swap arr[left] and arr[right].
2. Move left one position to the right.
3. Move right one position to the left.
4. Continue until left >= right.
   Why does this work? Each swap places **two elements into their final reversed positions**:
   arr[left] moves to its corresponding position from the end.
   arr[right] moves to its corresponding position from the beginning.
   When the pointers meet or cross, every required pair has already been swapped.

```
int left = 0;
int right = arr.length - 1;
while (left < right) {
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    left++; right--;
}
```

Complexity: Time = O(n) Space = O(1)

CodeWithRishchal

6.7. Same Direction — Fast and Slow Pointers

In this variation, both pointers usually begin near the start of the structure and move in the **same direction**.

S
F
↓
[ ][ ][ ][ ][ ][ ]

Slow →
Fast → → →

The two pointers have different responsibilities rather than necessarily different physical speeds.

A common interpretation is:

Fast / Read Pointer → examines elements

Slow / Write Pointer → represents where valid data should be placed

This pattern is extremely useful for **in-place modification**.

6.7.1 General Idea

Suppose an array contains elements we want to keep and elements we want to remove.

[valid, remove, valid, remove, valid]

The fast pointer examines every element. The slow pointer tracks the next position where a valid element should be stored.

Fast: checks every element Slow: controls valid part of array

---

6.7.2 General Template

```
int slow = 0;
for (int fast = 0; fast < arr.length; fast++) {
    if (shouldKeep(arr[fast])) {
        arr[slow] = arr[fast];
        slow++;
    }
}
```

At the end: slow = size of valid portion

6.7.3 Example — Remove a Given Value

Given an array and a value to remove, remove all occurrences of that value **in-place** and return the number of valid elements remaining at the beginning of the array.

Example:

Array = [3, 2, 2, 3, 4] Remove = 3

After removal, the beginning of the array should contain:

[2, 2, 4, ...]

Valid length: 3

The values after the valid length do not matter.

CodeWithNishchal

Approach — Slow and Fast Pointers

Use two pointers:
fast scans every element in the array.
slow marks the position where the next valid element should be placed. At every step:

1. If arr[fast] == value, skip it. Do not move slow.
2. If arr[fast] != value, copy it to arr[slow].
3. Increment slow because one more valid element has been placed.
4. Continue until fast reaches the end.
   At the end:
   slow = number of valid elements
   This works because slow always points to the next position where a non-removed value should be stored.

```
static int removeElement(int[] arr, int value) {
    int slow = 0;
    for (int fast = 0; fast < arr.length; fast++) {
        if (arr[fast] != value) {
            arr[slow] = arr[fast];
            slow++;
        }
    }
    return slow;
}
```

Complexity: Time = O(n) Space = O(1)

6.8 Remove Duplicates From Sorted Array

Given a **sorted array**, remove duplicate values **in-place** so that every unique value appears only once at the beginning of the array. Return the number of unique elements.

Example: Input = [1, 1, 2, 2, 3, 4, 4] After removing duplicates: [1, 2, 3, 4, ...]

Unique length: 4. The values after the first 4 positions do not matter.

Approach — Slow and Fast Pointers

Because the array is sorted, all duplicate values appear next to each other.
Use:
slow → points to the last unique element already stored.
fast → scans the array to find the next new value.
Start with: slow = 0 and: fast = 1. At every step:

1. Compare arr[fast] with arr[slow].
2. If they are equal, arr[fast] is a duplicate, so skip it.
3. If they are different, a new unique value has been found.
4. Move slow forward.
5. Store the new unique value at arr[slow].
   At the end:
   slow + 1 = number of unique elements

CodeWithNishchal

```
static int removeDuplicates(int[] arr) {
    if (arr.length == 0) {
        return 0;
    }
    int slow = 0;
    for (int fast = 1; fast < arr.length; fast++) {
        if (arr[fast] != arr[slow]) {
            slow++;
            arr[slow] = arr[fast];
        }
    }
    return slow + 1;
}
```

6.9 Fast and Slow Does Not Always Mean Different Speeds

This distinction is important.
In arrays: **fast = scanning pointer, slow = write/valid-region pointer**
They may both move one position at a time depending on conditions.
In structures such as linked lists, fast and slow may literally move at different speeds:
slow → 1 step fast → 2 steps
Therefore **fast and slow** refers to pointer roles as much as literal movement speed.

6.10. Partition Pattern

Partitioning means rearranging elements so that values satisfying different conditions occupy different regions.
Example: Input: [7, 2, 8, 3, 1, 6]
Partition around condition: value < 5
Possible result:
[2, 3, 1 | 7, 8, 6]
 < 5 ≥ 5
The exact order inside each region may not matter unless the problem specifically requires it.

6.10.1 Basic Partition Using Two Pointers

A common partition approach uses: boundary pointer, current pointer
Suppose we want all even numbers before odd numbers. [3, 2, 4, 1, 6]
The current pointer scans the array. The boundary pointer marks the next location where a matching element should be placed.
Conceptually:
Processed matching | Processed non-matching | Unprocessed
Whenever the current element belongs in the first region:
swap arr[boundary] and arr[current]
boundary++

CodewithNishchal

6.10.2 General Partition Template

```
int boundary = 0;
for (int current = 0; current < arr.length;
current++) {
    if (belongsToLeftPartition(arr[current])) {
        int temp = arr[boundary];
        arr[boundary] = arr[current];
        arr[current] = temp;
        boundary++;
    }
}
```

At the end: [left partition | right partition]

Complexity: Time = O(n) Space = O(1)

This idea also appears in algorithms such as **Quick Sort partitioning**.

6.11. Dutch National Flag Pattern

The **Dutch National Flag algorithm** is a specialized partitioning technique used when elements must be divided into **three regions**.

Given an array containing only 0, 1, and 2, sort the array **in-place** so that all 0s come first, followed by all 1s, and then all 2s.

Example: Input = [2, 0, 2, 1, 1, 0] Output = [0, 0, 1, 1, 2, 2]

The goal is to solve the problem in a **single traversal** without using a general sorting algorithm or an extra array.

6.11.1 Meaning of the Three Pointers

During execution:

0 ... low-1 → all 0s

low ... mid-1 → all 1s

mid ... high → unknown / unprocessed

high+1 ... n-1 → all 2s

Visual:

```
+---------+---------+---------------+---------+
|  0s     |  1s     |  Unknown      |  2s     |
+---------+---------+---------------+---------+
          ↑         ↑               ↑
         low       mid             high
```

Initialization:

```
int low = 0;
int mid = 0;
int high = arr.length - 1;
```

CodeWithNishchal

6.11.2 Dutch Flag Rules

There are only three cases.

Case 1: arr[mid] == 0

0 belongs in the left region.
Swap arr[mid] with arr[low]
low++
mid++

Case 2: arr[mid] == 1

1 is already in its correct middle region. mid++

Case 3: arr[mid] == 2

2 belongs in the right region.
Swap arr[mid] with arr[high]
high--
Important: Do NOT increment mid here.
Why? Because the element brought from high into mid has not yet been examined.

6.11.3 Dutch Flag Code

```
static void sortColors(int[] arr) {
    int low = 0;
    int mid = 0;
    int high = arr.length - 1;
    while (mid <= high) {
        if (arr[mid] == 0) {
            int temp = arr[low];
            arr[low] = arr[mid];
            arr[mid] = temp;
            low++;
            mid++;
        } else if (arr[mid] == 1) {
            mid++;
        } else {
            int temp = arr[mid];
            arr[mid] = arr[high];
            arr[high] = temp;
            high--;
        }
    }
}
```

Complexity: Time = O(n) Space = O(1)
Each element is processed only a constant number of times.

nNishchal

6.12. Comparison of Two Pointer Variants

Pattern	Pointer Initialization	Movement	Main Purpose
Opposite Ends	Start and end	Toward each other	Pair search, reversal, palindrome
Same Direction	Usually near start	Both move forward	Remove, compact, unique elements
Partition	Boundary + current	Forward	Divide into regions
Dutch Flag	low, mid, high	Region-based	Three-way partition

6.13. Why Two Pointer Solutions Are Often O(n)

Suppose:

left starts at 0 and right starts at n - 1

Even though there is a loop, neither pointer repeatedly returns to old positions.

left → → → →

right ← ← ← ←

Across the entire algorithm, the total number of pointer movements is proportional to n.

Therefore: O(n)

Similarly, with fast and slow pointers: fast scans at most n elements. slow advances at most n times This is still: O(n + n) = O(2n) = O(n)

Two pointers are therefore often used to replace nested-loop solutions.

6.14. Two Pointers vs Nested Loops

Nested Loop

```
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        // check pair
    }
}
```

Complexity: $O(n^2)$

Two Pointers

```
int left = 0; int right = n - 1;
while (left < right) {
    // process
    left++;
    // or right--
}
```

Typical complexity:O(n)

However, two pointers cannot replace every nested loop. It works only when pointer movement lets us logically discard combinations we no longer need to consider.

CodeWithNishchal

7. Sliding Window Pattern

Sliding Window is used when a problem deals with a **continuous part of an array or String**, such as a:

- subarray,
- substring,
- consecutive sequence,
- contiguous range.

Instead of recomputing information for every possible range, we maintain a **window** and update only what changes when that window moves.

Example:

Array:

[2, 1, 5, 1, 3, 2]

Window size = 3

```
[2, 1, 5] 1  3  2
   ↓ move

 2 [1, 5, 1] 3  2
     ↓ move

 2  1 [5, 1, 3] 2
```

A brute-force approach may repeatedly calculate every subarray: O(n × k) or even: O(n²)
Sliding Window often reduces this to: O(n)

because information from the previous window is reused.

The basic structure is:

right pointer → expands/moves the window
left pointer → removes elements when required

A window represents:

[left ... right]

The technique has two major forms:

```
Sliding Window
    |
    +---- Fixed Size
    |
    +---- Variable Size
          |
          +---- Expand–Shrink
```

A **monotonic deque/window** is an additional technique used when we need to efficiently maintain the minimum or maximum inside a moving window. It can be used with fixed-size or variable-size windows, so it is not strictly a subtype of variable sliding window.

CodeWithNishchal

7.1. Fixed Size Sliding Window

Use this when the window size is already given or must remain exactly k.

Typical problems: Maximum sum subarray of size k, Average of every subarray of size k, Maximum/minimum in every window of size k, Count something in every substring of length k

For example:
Array = [2, 1, 5, 1, 3, 2]
k = 3

Possible windows:
[2, 1, 5] → sum = 8
[1, 5, 1] → sum = 7
[5, 1, 3] → sum = 9
[1, 3, 2] → sum = 6

Instead of calculating each sum from scratch, reuse the previous one.
Current Window Sum
|
+ remove outgoing element
|
+ add incoming element

For example:
First window:
[2, 1, 5] sum = 8
Move one step:
Remove 2
Add 1
newSum = 8 - 2 + 1 = 7
So each new window is updated in constant time.

General Fixed-Window Template

```
int left = 0;
for (int right = 0; right < arr.length; right++) {
    // Add arr[right] to current window
    if (right - left + 1 == k) {
        // Process the complete window
        // Remove arr[left]
        left++;
    }
}
```

The window size is: right - left + 1
This formula is fundamental in sliding-window problems.

---

devWithNishchal

Example: Maximum Sum Subarray of Size K

Given an array and an integer k, find the maximum sum of any contiguous subarray of size k.
Example: Array = [2, 1, 5, 1, 3, 2] k = 3
All size-3 windows are:
[2, 1, 5] → 8
[1, 5, 1] → 7
[5, 1, 3] → 9
[1, 3, 2] → 6
Therefore: Maximum Sum = 9
from: [5, 1, 3]

Approach — Fixed-Size Sliding Window

Use a window of exactly k elements.
Maintain: left → start of the current window
right → expands the window
windowSum → sum of elements inside the current window
maxSum → maximum window sum found so far
At every step:

1. Add arr[right] to windowSum.
2. Check the current window size using: right - left + 1
3. When the window size becomes exactly k, update: maxSum = max(maxSum, windowSum)
4. Before moving to the next window, remove arr[left] from windowSum.
5. Increment left. This avoids recalculating the sum of every window from scratch.
   Instead of: O(k) work for every window, we update the window sum in O(1) time by:
   adding the new element and removing the outgoing element

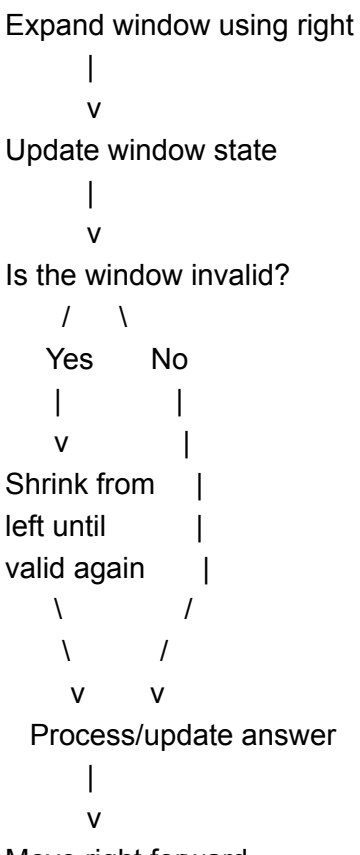
```
static int maxSum(int[] arr, int k) {
    int left = 0;
    int windowSum = 0;
    int maxSum = Integer.MIN_VALUE;
    for (int right = 0; right < arr.length; right++) {
        windowSum += arr[right];
        if (right - left + 1 == k) {
            maxSum = Math.max(maxSum,
windowSum);
            windowSum -= arr[left];
            left++;
        }
    }
    return maxSum;
}
```

For: [2, 1, 5, 1, 3, 2] k = 3
the maximum window is: [5, 1, 3] sum = 9
Complexity: Time = O(n) Space = O(1)
Each element enters the window once and leaves once.

7.2. Variable Size Sliding Window

In a variable-size sliding window, the window size changes dynamically based on a condition.
The right pointer is used to **expand** the window, while the left pointer is used to **shrink** the window whenever the current window becomes invalid.

Typical structure:



Core idea:

right → adds new elements to the window.
left → removes elements when the condition is violated.
The window is continuously adjusted so that it satisfies the required condition.

General template:

```
int left = 0;
for (int right = 0; right < arr.length; right++) {
    // Include arr[right]
    while (windowIsInvalid()) {
        // Remove arr[left]
        left++;
    }
    // Current window is valid here
}
```

The key idea is: right → discovers / expands
left → restores validity / shrinks

CodeWithNishchal

7.2.1 Expand–Shrink Pattern

This is the standard variable-size sliding-window pattern.
We continuously expand the window by moving right.

```
[left ..... right]
           ↑
        expand
```

If some constraint becomes invalid, move left forward until the window becomes valid again.
Before:

```
L                R
↓                ↓
[ ... window violates condition ... ]
```

Shrink:

```
    L            R
    ↓            ↓
    [ valid window ]
```

Typical conditions include:

- sum exceeds a limit,
- too many distinct characters,
- duplicate character appears,
- frequency exceeds allowed count,
- window does not satisfy a requirement.

Example: Longest Subarray With Sum ≤ K

Given an array of **non-negative integers** and a value K, find the length of the **longest contiguous subarray whose sum is less than or equal to K**.
Example: Array = [2, 1, 3, 2, 1] K = 5
A longest valid subarray is: [2, 1], [1, 3], [3, 2], or [2, 1]
So: Maximum Length = 2

Approach — Variable-Size Sliding Window

Use two pointers: left → start of the current window. right → expands the window.
Also maintain: sum → sum of all elements inside the current window.
maxLength → maximum valid window length found so far. At every step:

1. Add arr[right] to sum.
2. If sum <= K, the current window is valid.
3. If sum > K, the window is invalid, so remove elements from the left until the sum becomes valid again.
4. After the window becomes valid, calculate its length: right - left + 1
5. Update maxLength.
   This works because all elements are **non-negative**. When we expand the window, the sum can only stay the same or increase. When the sum becomes too large, removing elements from the left can only decrease it.

CodeWithNishchal

```
static int longestSubarray(int[] arr, int k) {
    int left = 0;
    int sum = 0;
    int maxLength = 0;
    for (int right = 0; right < arr.length; right++) {
        sum += arr[right];
        while (sum > k) {
            sum -= arr[left];
            left++;
        }
        maxLength = Math.max(
            maxLength,
            right - left + 1
        );
    }
    return maxLength;
}
```

Complexity: Time = O(n), Space = O(1)

Important Condition

This approach works here because values are non-negative. Why?

If: sum > k

removing elements from the left cannot increase the sum when all values are non-negative.

With negative numbers: [positive, negative, positive ...]

window expansion and shrinking may no longer behave monotonically, so the normal sliding-window logic may fail. This is an important limitation.

7.2.2 Longest vs Shortest Window

Longest Valid Window

Maintain validity and maximize: right - left + 1

Typical logic:

```
while (window is invalid) {
    shrink();
}
answer = Math.max(answer, right - left + 1);
```

Examples: Longest substring without repeating characters, Longest subarray satisfying a limit

Shortest Valid Window

Expand until the condition becomes valid, then shrink as much as possible while keeping it valid.

Examples: Minimum length subarray with sum >= target, Minimum window containing required characters

This difference is important:

Longest problem → shrink when invalid

Shortest problem → shrink while valid

CodeWithNishchal

7.3 Example: Longest Substring Without Repeating Characters

Given a string, find the length of the **longest contiguous substring containing no repeating characters**.
Example: Input = "abcabcbb"
The longest substring without repeating characters is: "abc" So: Maximum Length = 3

Approach — Variable-Size Sliding Window + HashSet

Use a sliding window with two pointers:
left → start of the current valid substring.
right → expands the window by examining new characters.
Use a HashSet to store the characters currently present inside the window.
At every step:

1. Take the current character s.charAt(right).
2. If the character is not already in the set, add it to the current window.
3. If the character already exists in the set, a duplicate has appeared and the window becomes invalid.
4. Remove characters from the left side until the duplicate character is no longer present.
5. Add the current character to the set.
6. Calculate the current valid window length using: right - left + 1
7. Update maxLength.
   The main idea is: Expand with right → Duplicate found → Shrink from left → Window valid again → Update answer

```
static int longestUniqueSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0;  int maxLength = 0;
    for (int right = 0; right < s.length(); right++) {
        char current = s.charAt(right);
        while (set.contains(current)) {
            set.remove(s.charAt(left));
            left++;
        }
        set.add(current);
        maxLength = Math.max(
            maxLength,
            right - left + 1
        );
    }
    return maxLength;
}
```

Complexity: Time = O(n) average Space = O(k)
where k is the number of distinct characters stored in the window.
Although a while loop exists inside the for loop, this is still O(n) because each character enters the window once and leaves at most once.

7.4. Window State

Most sliding-window problems maintain some information about the current window.

Examples: Current sum, Frequency map, Number of distinct elements, Number of invalid characters, Maximum/minimum, Count of zeros, Count of matching elements
A good sliding-window solution usually avoids recomputing this information from scratch.

Example:
Window Sum
add: sum += arr[right]
remove: sum -= arr[left]
For frequencies: freq.put(ch, freq.getOrDefault(ch, 0) + 1);
When removing: freq.put(ch, freq.get(ch) - 1);
The important idea is: Update only what enters + Update only what leaves
instead of recalculating the complete window.

7.5. Monotonic Window

Some sliding-window problems require the: maximum element , minimum element, inside every window.
A normal window does not provide an efficient way to know the maximum after the previous maximum leaves.
Example:
Window: [1, 3, 2, 5]
Maximum: 5
When the window moves, repeatedly scanning the entire window to find the maximum costs: O(k) for every window. Total: O(n × k) To avoid this, we use a **monotonic deque**.

7.5.1 Monotonic Deque

A monotonic deque maintains elements in a specific order.
For maximum-window problems:
Decreasing order, For minimum-window problems:, Increasing order

Example for maximum:
Deque: 8 → 6 → 3 → 1
The largest element is always at the front.

For minimum:
Deque:1 → 3 → 6 → 8

The smallest element is always at the front. Usually, the deque stores **indexes**, not values.
This lets us determine whether an element has moved outside the current window.

CodeWithNishchal

7.6 Sliding Window Maximum

Given an array and an integer k, find the **maximum element in every contiguous subarray (window) of size k**.

Example: Array = [1, 3, -1, -3, 5, 3, 6, 7] k = 3

Windows:

[1, 3, -1] → 3, [3, -1, -3] → 3, [-1, -3, 5] → 5, [-3, 5, 3] → 5 , [5, 3, 6] → 6 , [3, 6, 7] → 7

Therefore: Result = [3, 3, 5, 5, 6, 7]

Approach — Sliding Window + Monotonic Deque

A normal approach could scan all k elements of every window to find the maximum, giving O(nk) time. We can optimize this using a **monotonic decreasing deque**. The deque stores **indexes**, not values. The indexes inside the deque are maintained so that their corresponding array values are in decreasing order: arr[deque[0]] >= arr[deque[1]] >= arr[deque[2]] … Because of this, the index at the **front of the deque always represents the maximum element of the current window**.
For every new element at index right:

1. Remove indexes from the front if they are outside the current window.
2. Remove indexes from the back while their values are smaller than or equal to arr[right].
3. Add right to the back of the deque.
4. Once the window reaches size k, deque.peekFirst() gives the index of the maximum element.

```
static int[] maxSlidingWindow(int[] arr, int k) {
    int n = arr.length;   int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    int index = 0;
    for (int right = 0; right < n; right++) {
        // Remove indexes outside the window
        while (!deque.isEmpty()
                && deque.peekFirst() <= right - k) {
            deque.pollFirst();
        }
        // Maintain decreasing order
        while (!deque.isEmpty() && arr[deque.peekLast()] <= arr[right]) {
            deque.pollLast();
        }
        deque.offerLast(right);
        // Window becomes complete
        if (right >= k - 1) {
            result[index++] = arr[deque.peekFirst()];
        }
    }
    return result;
}
```

Complexity: Time = O(n) Space = O(k)

7.7. Fixed vs Variable vs Monotonic Window

Pattern	Window Size	Main Idea	Common Use
Fixed Size	Exactly k	Add new, remove old	Sum/average/count of size k
Variable Size	Changes	Expand & shrink based on condition	Longest/shortest valid range
Monotonic Deque	Fixed or variable	Maintain ordered candidates	Window maximum/minimum

7.8. Sliding Window vs Two Pointers

Sliding Window often uses two pointers, but the ideas are not identical.

Two Pointers

Focus: relationship between two positions

Examples: Pair sum, Palindrome , Partition Remove duplicates

Sliding Window

Focus: continuous range between two boundaries

Example:

left right

↓ ↓

[current window]

The entire interval usually has some maintained state such as: sum, frequency, distinct count

A useful distinction is: Two Pointers → process positions

Sliding Window → maintain a contiguous range

7.9. When Sliding Window Works

Sliding Window is a strong candidate when the problem contains: subarray, substring, consecutive, contiguous, window of size K, longest valid range, shortest valid range, at most K, no more than K, maximum/minimum over consecutive elements

Ask these questions:

1. Is the required answer based on a contiguous range?
2. Can I update the result when one element enters and another leaves?
3. Is there a rule that tells me when to expand or shrink the window?

If yes, Sliding Window is likely useful.

7.10. Important Limitation of Variable Sliding Window

A variable-size sliding window requires a condition that changes in a predictable way when the window expands or shrinks.

For example, with non-negative values: Adding an element cannot decrease the sum. Removing an element cannot increase the sum.

This gives predictable movement. With arbitrary negative numbers: [5, -10, 8, ...]

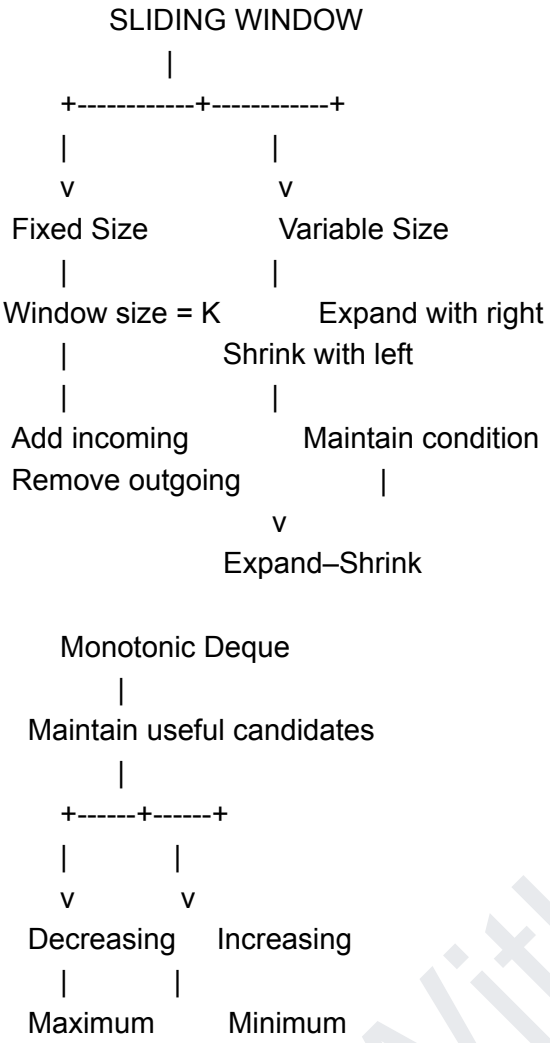
expanding the window can decrease the sum, so logic such as: sum > target → shrink

may no longer safely work.

Therefore never apply sliding window only because the problem contains the word "subarray". The condition must support controlled window movement.

CodeWithNishchal

7.11. Final Pattern Summary



Remember the core logic:
Fixed Window
→ Maintain exactly K elements.

Variable Window
→ Expand, detect condition, shrink when required.
Longest Valid Window
→ Shrink when invalid, then maximize length.

Shortest Valid Window
→ Once valid, keep shrinking and minimize length.

Monotonic Window
→ Maintain only elements that can still become
the maximum or minimum.

Sliding Window is most powerful when a brute-force solution repeatedly processes overlapping contiguous ranges. Instead of rebuilding each range, maintain the current window and update only the elements entering and leaving it.

CodeWithNishchal

8. More Array Patterns

The following patterns are among the most important reusable techniques for array problems:

Prefix Based
├── Prefix Sum
├── Prefix XOR
└── 2D Prefix

Kadane / Subarray
├── Maximum Subarray Sum
├── Maximum Product Subarray
└── Subarray With Given Sum / XOR

Binary Search
├── Binary Search on Index
└── Binary Search on Answer

8.1. Prefix Based Pattern

The prefix technique stores information from the beginning of the array up to every index.
Instead of repeatedly calculating:
arr[0] + arr[1] + ... + arr[i]

we calculate it once and reuse it.
General idea:
Original: [a0, a1, a2, a3, a4]
Prefix: [p0, p1, p2, p3, p4]
where
p[i] = information from index 0 to i
Prefix techniques are useful when many queries ask about information inside ranges.

8.1.1 Prefix Sum

A **prefix sum array** stores cumulative sums.
For:
arr = [2, 4, 1, 3, 5]
Prefix sum:
Index: 0 1 2 3 4
Array: 2 4 1 3 5
Prefix: 2 6 7 10 15
Calculation:
prefix[0] = arr[0]
prefix[i] = prefix[i - 1] + arr[i]
Example:prefix[3] = 2 + 4 + 1 + 3 = 10

CodeWithNishchal

Range Sum

Suppose we need the sum from index L to R.
Without prefix sum:
arr[L] + arr[L+1] + ... + arr[R]
which may take: O(n)

With prefix: sum(L, R) = prefix[R] - prefix[L - 1]
If: L = 0 then: sum(0, R) = prefix[R]
Example: arr = [2, 4, 1, 3, 5]
Find sum from index 1 to 3: 4 + 1 + 3 = 8
Using prefix: prefix[3] - prefix[0] = 10 - 2 = 8

Java Implementation

```
static int[] buildPrefixSum(int[] arr) {
    int[] prefix = new int[arr.length];
    prefix[0] = arr[0];
    for (int i = 1; i < arr.length; i++) {
        prefix[i] = prefix[i - 1] + arr[i];
    }
    return prefix;
}

Range query:
static int rangeSum(int[] prefix, int left, int right) {
    if (left == 0) {
        return prefix[right];
    }
    return prefix[right] - prefix[left - 1];
}
```

How the code works

The first loop builds cumulative totals:
Prefix[i] =
sum of elements from 0 to i
For a range [left, right], prefix[right] contains everything from 0 to right.
Subtracting:
prefix[left - 1]

removes the unwanted part before left.
Complexity: Build Prefix = O(n) Each Range Query = O(1) Extra Space = O(n)

Code Nishchal

8.1.2 Alternative Prefix Representation

Another common representation uses an array of size: n + 1

with: prefix[0] = 0
Then:
int[] prefix = new int[arr.length + 1];

```
for (int i = 0; i < arr.length; i++) {
    prefix[i + 1] = prefix[i] + arr[i];
}
```

Now range sum becomes: int sum = prefix[right + 1] - prefix[left];
This representation is often cleaner because it avoids special handling for left == 0.
Example:
Array: [2, 4, 1, 3]
Prefix: [0, 2, 6, 7, 10]
For indexes 1 to 3: prefix[4] - prefix[1] = 10 - 2 = 8

8.2 Prefix XOR

Prefix XOR works exactly like prefix sum, except cumulative XOR is stored instead of cumulative addition.
For: arr = [5, 2, 7, 3]
Define: prefixXor[i] = arr[0] ^ arr[1] ^ ... ^ arr[i]
where ^ is the XOR operator. The key XOR property is: x ^ x = 0
x ^ 0 = x
Therefore repeated parts cancel.

Range XOR

For range:
[L, R]
the XOR is: prefixXor[R] ^ prefixXor[L - 1]
because:
(a ^ b ^ c ^ d)^ (a ^ b)=c ^ d
Example:Array: [5, 2, 7, 3]
Prefix XOR:
5
5 ^ 2
5 ^ 2 ^ 7
5 ^ 2 ^ 7 ^ 3

To find XOR from index 1 to 3:
prefixXor[3] ^ prefixXor[0]

CodeWithNishchal

Java Implementation

```
static int[] buildPrefixXor(int[] arr) {
    int[] prefixXor = new int[arr.length];
    prefixXor[0] = arr[0];
    for (int i = 1; i < arr.length; i++) {
        prefixXor[i] = prefixXor[i - 1] ^ arr[i];
    }
    return prefixXor;
}
```

Complexity: Build = O(n) Range Query = O(1) Space = O(n)

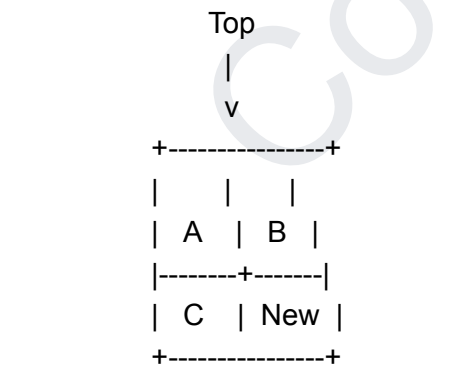
Prefix XOR is especially useful in problems involving:

- XOR of subarrays,
- XOR range queries,
- counting subarrays with a required XOR.

8.3 2D Prefix Sum

2D Prefix Sum extends the same idea to matrices.
Suppose:
Matrix:
1 2 3
4 5 6
7 8 9

A 2D prefix value stores the sum of the rectangle from the top-left corner to the current cell.
Conceptually:
prefix[r][c] = sum of rectangle from (0,0) to (r,c)
A convenient implementation uses a prefix matrix of size: (rows + 1) × (cols + 1)
Formula: prefix[r][c] = matrix[r-1][c-1] + prefix[r-1][c] + prefix[r][c-1] - prefix[r-1][c-1]
The subtraction is necessary because the top-left overlapping area was added twice.
Visual:



Prefix above + Prefix left
contains A twice
So subtract A once.

CodeWithNishchal

Building 2D Prefix

```
static int[][] build2DPrefix(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int[][] prefix = new int[rows + 1][cols + 1];
    for (int r = 1; r <= rows; r++) {
        for (int c = 1; c <= cols; c++) {
            prefix[r][c] =
                    matrix[r - 1][c - 1]
                    + prefix[r - 1][c]
                    + prefix[r][c - 1]
                    - prefix[r - 1][c - 1];
        }
    }
    return prefix;
}
```

Rectangle Sum Query

Suppose we want the rectangle:
(row1, col1) to (row2, col2)
Using the n + 1 style prefix matrix:

```
static int rectangleSum(
        int[][] prefix,
        int row1,
        int col1,
        int row2,
        int col2) {

    return prefix[row2 + 1][col2 + 1]
            - prefix[row1][col2 + 1]
            - prefix[row2 + 1][col1]
            + prefix[row1][col1];
}
```

The idea is: Whole rectangle - area above - area left + area removed twice

Complexity: Build = O(rows × cols)
Each Rectangle Query = O(1)
Space = O(rows × cols)

CodeWithNishchal

8.4. Kadane / Subarray Patterns

Subarray problems deal with **contiguous elements**.
Example: Array: [1, 2, 3, 4]
Subarrays include:
[1]
[1, 2]
[2, 3]
[1, 2, 3]
[3, 4]
...
But not: [1, 3] because 1 and 3 are not contiguous.

8.4.1 Maximum Subarray Sum — Kadane's Algorithm

Given an integer array, find the **contiguous subarray with the maximum possible sum**.
Example: Input = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
The maximum-sum subarray is: [4, -1, 2, 1] Its sum is: 4 + (-1) + 2 + 1 = 6
Therefore: Maximum Subarray Sum = 6

Approach — Kadane's Algorithm

Kadane's Algorithm processes the array from left to right while maintaining two values:
currentSum → maximum subarray sum ending exactly at the current index.
maxSum → maximum subarray sum found anywhere so far.
At every index, we make one decision:
Should we continue the previous subarray? or
Should we start a new subarray from the current element?
Therefore:currentSum = max(arr[i], currentSum + arr[i])
If the previous sum is helpful, we continue it.
If the previous sum is negative and hurts the result, we discard it and start fresh from the current element.
After calculating currentSum, update:
maxSum = max(maxSum, currentSum)
Main idea:
Previous sum helps → Continue the subarray
Previous sum hurts → Start a new subarray

```
static int maxSubarraySum(int[] arr) {
    int currentSum = arr[0];
    int maxSum = arr[0];
    for (int i = 1; i < arr.length; i++) {
        currentSum = Math.max(  arr[i], currentSum + arr[i] );
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

Complexity: Time = O(n) Space = O(1)

Codewithlishchal

8.4.2 Why Initialize With arr[0]?

Avoid: int maxSum = 0; because the array may contain only negative numbers.
Example: [-5, -2, -8] Correct answer: -2
If maxSum started at 0, the result would incorrectly become 0.

8.5 Maximum Product Subarray

Given an integer array, find the **contiguous subarray with the maximum possible product**.
Example: Input = [2, 3, -2, 4]
Possible products include:
[2, 3] → 6, [2, 3, -2] → -12, [-2, 4] → -8. Therefore: Maximum Product = 6

Key Observation

If the current number is negative: max × negative → may become minimum
min × negative → may become maximum
Therefore swap the current maximum and minimum before multiplying.

Approach — Track Maximum and Minimum Product

Use two variables while scanning the array:
maxEndingHere → maximum product of a subarray ending at the current index.
minEndingHere → minimum product of a subarray ending at the current index.
For every element num:

1. If num < 0, swap maxEndingHere and minEndingHere because multiplying by a negative number reverses their roles.
2. Update the maximum product: maxEndingHere = max(num, maxEndingHere × num)
3. Update the minimum product: minEndingHere = min(num, minEndingHere × num)
4. Update the overall answer:
   answer = max(answer, maxEndingHere)

```
static int maxProduct(int[] arr) {
    int maxEndingHere = arr[0];
    int minEndingHere = arr[0];
    int answer = arr[0];
    for (int i = 1; i < arr.length; i++) {
        int num = arr[i];
        if (num < 0) {
            int temp = maxEndingHere;
            maxEndingHere = minEndingHere;
            minEndingHere = temp;
        }
        maxEndingHere = Math.max(  num, maxEndingHere * num );
        minEndingHere = Math.min( num,  minEndingHere * num );
        answer = Math.max(answer, maxEndingHere);
    }
    return answer;
}
```

WithNishchal

8.6 Subarray With Given Sum

Given an integer array and a target value, determine whether there exists a **contiguous subarray whose sum is exactly equal to the target**.
This approach works even when the array contains **negative numbers**.
Example: Array = [1, 4, -2, 3] Target = 5
A valid subarray is: [1, 4] because: 1 + 4 = 5

Approach — Prefix Sum + HashSet

Maintain a running prefix sum while traversing the array.
Let: prefix → sum of elements from index 0 to the current index.
Suppose an earlier prefix sum was: previousPrefix
Then the sum of the subarray between that earlier position and the current index is: prefix - previousPrefix

We want this subarray sum to equal: target So: prefix - previousPrefix = target
Rearranging: previousPrefix = prefix - target
Therefore, at every index:

1. Add the current element to prefix.
2. Calculate: prefix - target
3. Check whether this value has appeared as a prefix sum before.
4. If yes, a subarray with sum target exists.
5. Otherwise, store the current prefix sum and continue.
   Main idea: Current Prefix → Need prefix - target → Seen before? → Subarray found

```
static boolean hasSubarrayWithSum(int[] arr, int target) {
    Set<Integer> prefixSums = new HashSet<>();
    int prefix = 0;
    prefixSums.add(0);
    for (int num : arr) {
        prefix += num;
        if (prefixSums.contains(prefix - target)) {
            return true;
        }
        prefixSums.add(prefix);
    }
    return false;
}
```

Why add 0 initially?
It represents the prefix sum before any element has been processed.
This allows us to detect subarrays beginning at index 0.
Complexity:
Time = O(n) average Space = O(n)

Nishchal

8.7 Count Subarrays With Given Sum

Given an integer array and a target value, count the **number of contiguous subarrays whose sum is exactly equal to the target**.
Example: Array = [1, 1, 1] Target = 2
Valid subarrays:
[1, 1] from index 0 to 1
[1, 1] from index 1 to 2
Therefore: Count = 2

Approach — Prefix Sum + HashMap

For existence, storing previous prefix sums in a HashSet is enough.
But for counting subarrays, we need to know **how many times each prefix sum has appeared**.
So we use: HashMap<PrefixSum, Frequency>
Let: prefix → current prefix sum.
For a subarray to have sum target: prefix - previousPrefix = target
Therefore: previousPrefix = prefix - target
So at every index:

1. Add the current element to prefix.
2. Calculate: prefix - target
3. Check how many times this required prefix has appeared before.
4. Add that frequency to count.
5. Store/update the frequency of the current prefix.
   Main idea: Current Prefix → Need prefix - target → Find its frequency → Add frequency to count → Store current prefix

```
static int countSubarraysWithSum(int[] arr, int target) {
    Map<Integer, Integer> freq = new HashMap<>();
    freq.put(0, 1);
    int prefix = 0;
    int count = 0;
    for (int num : arr) {
        prefix += num;
        count += freq.getOrDefault(prefix - target, 0);
        freq.put( prefix, freq.getOrDefault(prefix, 0) + 1
        );
    }
    return count;
}
```

Why use frequency?
Suppose the required earlier prefix occurred three times.
Then the current index forms: 3 valid subarrays not just one.
Complexity:
Time = O(n) average
Space = O(n)

8.8. Binary Search

Binary Search is a searching technique used when the search space has an **ordered or monotonic property** that allows us to discard half of the remaining possibilities after every step.
Classic case:
Sorted Array = [2, 5, 8, 12, 16, 23, 38]
Target = 16
Instead of checking:
2 → 5 → 8 → 12 → 16
Binary Search checks the middle element and decides which half can be discarded.

```
2   5   8   12   16   23   38
        ↑
       mid
```

If:
arr[mid] == target → Found
arr[mid] < target → Search right half
arr[mid] > target → Search left half
Each step reduces the search space approximately by half:
n → n/2 → n/4 → n/8 → ... → 1
Therefore:
Time = O(log n)
The most important idea is:
Binary Search = Find mid → Use a condition → Discard one half

8.8.1 Binary Search on Index

Given a **sorted array** and a target value, return the index of the target. If the target does not exist, return -1. Example: Array = [2, 5, 8, 12, 16, 23, 38]
Target = 16
Answer: Index = 4

Approach

Maintain two pointers:
left → beginning of the current search range.
right → end of the current search range.
At every step:

1. Find the middle index.
2. If arr[mid] == target, return mid.
3. If arr[mid] < target, the target must be on the right because the array is sorted.
4. If arr[mid] > target, the target must be on the left.
5. Continue until the search range becomes empty.
   Use:
   mid = left + (right - left) / 2
   instead of:
   mid = (left + right) / 2
   because the first form avoids possible integer overflow.

CodeWithNishchal

Visual

Array = [2, 5, 8, 12, 16, 23, 38]
Target = 16

```
L          M              R
↓          ↓              ↓
2   5   8   12   16   23   38
```

arr[mid] = 12
12 < 16 → discard left half

```
              L      M      R
              ↓      ↓      ↓
2   5   8   12   16   23   38
```

arr[mid] = 23
23 > 16→ discard right half

```
              L/R
               ↓
2   5   8   12   16   23   38
```

arr[mid] = 16 → Found

CODE:

```
static int binarySearch(int[] arr, int target) {
    int left = 0;
    int right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) {
            return mid;
        }
        if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

Complexity

Time = O(log n) Space = O(1)

Nishchal

8.9 Common Binary Search Variations

Binary Search is not limited to finding an exact value. Common variations include:
Exact Search, First Occurrence, Last Occurrence, Lower Bound, Upper Bound, Search Insert Position, Search in Rotated Sorted Array, Find Peak Element, Binary Search on Answer

8.9.1 Lower Bound

Lower Bound means: First index where arr[index] >= target
Example: Array = [1, 2, 4, 4, 6, 8] Target = 4
Index: 0 1 2 3 4 5
Array: [1, 2, 4, 4, 6, 8]
↑
Lower Bound
Answer: 2 because: arr[2] = 4 and index 2 is the first position whose value is at least 4.

8.9.2 Upper Bound

Upper Bound means: First index where arr[index] > target Example: Array = [1, 2, 4, 4, 6, 8]
Target = 4, Index: 0 1 2 3 4 5
Array: [1, 2, 4, 4, 6, 8]
↑
Upper Bound
Answer: 4 because: arr[4] = 6 is the first value strictly greater than 4.

8.10 First and Last Occurrence

For a sorted array containing duplicates: [1, 2, 4, 4, 4, 6]
For target 4: First occurrence = 2 Last occurrence = 4
Using bounds: First occurrence = lowerBound(target), Last occurrence = upperBound(target) - 1
But always verify that the target actually exists before using these indexes.

8.11. Binary Search on Answer

Binary Search can also be used when we are **not searching for an element in an array**.
Instead, we search through a range of possible answers. Suppose possible answers are: low ---------- high. For each candidate mid, we ask: Can the problem be solved using this value? If the answer changes monotonically across the search range, Binary Search can locate the boundary. This technique is called: Binary Search on Answer

8.12 Monotonic Condition

Binary Search on Answer requires a monotonic condition.
F F F F F T T T T
↑
First True
or:
T T T T T F F F F
↑
Last True, Example: Capacity: 1 2 3 4 5 6 7 8 Can complete task?
N N N N Y Y Y Y
↑
First valid
If capacity 5 works, then larger capacities such as 6, 7, and 8 also work.
That monotonic property makes Binary Search possible.

CodeWithNishchal

8.13 Four Steps for Binary Search on Answer

Almost every Binary Search on Answer problem can be solved using four steps.

Step 1 — Define the Answer Range

Find: low = smallest possible answer, high = largest possible answer
Example for shipping capacity: low = largest single package high = sum of all package weights because the capacity cannot be smaller than the heaviest package and never needs to exceed the total weight.

Step 2 — Write a Feasibility Function

Create: boolean isPossible(int candidate) It answers: Can the problem be solved using candidate as the answer? Example: Can all packages be shipped within D days using capacity = candidate?

Step 3 — Verify Monotonicity

Make sure the results look like: False False False | True True True or: True True True | False False False. If feasibility changes back and forth, normal Binary Search cannot be used.

Step 4 — Binary Search the Boundary

Depending on the problem, find:First True or: Last True

8.14 Minimum Valid Answer

Suppose:Candidate:
1 2 3 4 5 6 7 8
Possible?
F F F F T T T T
↑
First True
We want the smallest valid answer.

Approach

If: isPossible(mid) == true then mid works, but a smaller value might also work. So: high = mid
If: isPossible(mid) == false then mid and every smaller value are invalid. So: low = mid + 1

8.15 Example — Minimum Shipping Capacity

Given an array weights, where each value represents the weight of a package, and an integer days, find the **minimum ship capacity required to ship all packages within the given number of days**. Packages must be shipped **in the same order**.
Example: Weights = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] Days = 5 Answer: Minimum Capacity = 15
The answer is some ship capacity between:
low = maximum weight and high = sum of all weights Why?
Maximum weight is the minimum possible capacity because every package must fit individually.

CodeWithNishchal

Sum of weights is the maximum required capacity because it can ship everything in one day.
Now apply Binary Search on this capacity range. For every candidate capacity mid:

1. Simulate shipping the packages in order.
2. Keep adding packages to the current day's load while they fit.
3. If adding the next package exceeds mid, start a new day.
4. Count how many days are required. Now: If requiredDays <= days: mid is sufficient. Try a smaller capacity: high = mid. If requiredDays > days: mid is too small.
   Try a larger capacity: low = mid + 1 Continue until: low == high

```
static int shipWithinDays(int[] weights, int days) {
    int low = 0;  int high = 0;
      for (int weight : weights) {
        low = Math.max(low, weight);
        high += weight;
    }
        while (low < high) {

        int mid = low + (high - low) / 2;
        if (isPossible(weights, days, mid)) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }

    return low;
}

static boolean isPossible(int[] weights, int days, int capacity) {
    int requiredDays = 1;   int currentLoad = 0;

    for (int weight : weights) {

        if (currentLoad + weight <= capacity) {
            currentLoad += weight;
        } else {

            requiredDays++; currentLoad = weight;
        }

        if (requiredDays > days) { return false;
        }}
    return true;
}
```

9. Stack

A **Stack** is a linear data structure that follows the **LIFO principle**:
LIFO = Last In, First Out

The element inserted most recently is the first one removed.
Real-life analogy:

```
 Top
  ↓
+-----+
| 40  |  ← inserted last, removed first
+-----+
| 30  |
+-----+
| 20  |
+-----+
| 10  |
+-----+
```

If elements are inserted in this order:
10 → 20 → 30 → 40

they are removed in:
40 → 30 → 20 → 10

Unlike an array, where any index can usually be accessed directly, a stack normally allows operations only at one end called the **top**.

9.1. Core Stack Operations

The fundamental operations are:

Operation	Meaning
push(x)	Insert x at the top
pop()	Remove and return the top element
peek() / top()	Read the top element without removing it
isEmpty()	Check whether the stack is empty
size()	Number of elements currently stored

CodeWithNishchal

Visual:
Initial:

Top
↓
[30]
[20]
[10]

push(40)

Top
↓
[40]
[30]
[20]
[10]

pop()
Returns 40

Top
↓
[30]
[20]
[10]

9.2. Stack Pointer / Top

In an array-based implementation, a variable called top commonly tracks the most recently inserted element.
For: [10, 20, 30, _, _]

we can imagine:

```
Index:  0   1   2   3   4
      +---+---+---+---+---+
      |10 |20 |30 |   |   |
      +---+---+---+---+---+
            ↑
           top
```

Here: top = 2
When pushing:
top++
store value at top
When popping:
read arr[top]
top--

CodeWithNishchal

9.3. Push Operation

Suppose:
Stack:

Top
↓
[30]
[20]
[10]

Perform:
push(40)

Result:
Top
↓
[40]
[30]
[20]
[10]

Only the top changes.
Therefore: Time Complexity = O(1)

9.4. Pop Operation

pop() removes the most recently inserted element.
Before:
Top
↓
[40]
[30]
[20]
[10]

After:
Returned = 40

Top
↓
[30]
[20]
[10]

Time complexity: O(1)

CodeWithNishchal

9.5. Peek Operation

peek() returns the top element without removing it.

Top
↓
[30]
[20]
[10]

peek() → 30

The stack remains unchanged.
Time Complexity = O(1)

9.6. Stack Underflow and Overflow

Underflow

Trying to remove or access an element from an empty stack causes an **underflow condition**.
Stack:

(empty)

pop() ✗
peek() ✗

With Java collections, the exact behavior depends on the method used.

Overflow

Overflow matters mainly in a **fixed-size array implementation**.
Example:
Capacity = 3

[10][20][30]

Trying:
push(40)

cannot succeed unless additional storage is created.
Dynamic collection implementations such as ArrayDeque resize internally, so normal DSA use does not require manual overflow handling.

9.7. Stack Implementation Using Array

A simple fixed-size stack can be implemented manually.

CodeWithNischal

```
class ArrayStack {
    private int[] arr;
    private int top;
    ArrayStack(int capacity) {
        arr = new int[capacity];
        top = -1;
    }
    void push(int value) {
        if (top == arr.length - 1) {
            throw new RuntimeException("Stack Overflow");
        }
        arr[++top] = value;
    }

    int pop() {
        if (top == -1) {
            throw new RuntimeException("Stack Underflow");
        }
        return arr[top--];
    }
    int peek() {
        if (top == -1) {
            throw new RuntimeException("Stack is Empty");
        }

        return arr[top];
    }
    boolean isEmpty() {
        return top == -1;
    }
    int size() {
        return top + 1;
    }
}
```

Logic
Initially: top = -1 means no element exists.
For push: arr[++top] = value;
first increments top, then stores the value.
For pop: return arr[top--];

returns the current top value and then decreases top.

9.8. Stack Implementation Using Linked List

A stack can also be implemented using a linked list.

The head node acts as the top.
Top
↓
[30] → [20] → [10] → null

Push:
new node
  |
  v
[40] → [30] → [20] → [10]
↑
Top

Pop removes the head node.
Because insertion and deletion occur at the beginning:
push = O(1)
pop = O(1)
peek = O(1)

Unlike a fixed array, linked-list storage does not require a predefined capacity, but every node needs an additional reference.

9.9. Stack in Java

Java provides several ways to represent stack behavior.
The preferred modern choice for most DSA problems is:
Deque<Integer> stack = new ArrayDeque<>();

Import:
```
import java.util.ArrayDeque;
import java.util.Deque;
```

Operations:
```
stack.push(10);
stack.push(20);
stack.push(30);

System.out.println(stack.peek()); // 30
stack.pop();
System.out.println(stack.peek()); // 20
```

9.10. Why ArrayDeque Is Preferred Over Stack

Java has an older class:
Stack<Integer> stack = new Stack<>();

CodeWithNishchal

but for normal stack behavior, ArrayDeque is generally preferred.

Feature	ArrayDeque	Stack
Modern API	Yes	Legacy
Implements	Deque	Extends Vector
Synchronization overhead	No	Yes
Common DSA choice	Preferred	Usually avoided
Stack operations	push, pop, peek	push, pop, peek

Use: Deque<Integer> stack = new ArrayDeque<>(); for most interviews and DSA code.

9.11. push, pop, peek vs Deque Methods

For stack-style usage: stack.push(x); stack.pop(); stack.peek();
These operate at the front of the deque.
Equivalent deque-style methods : stack.addFirst(x); stack.removeFirst(); stack.peekFirst();

9.12. Time Complexity of Stack Operations

For a normal stack implementation:

Operation	Time Complexity
push()	O(1)
pop()	O(1)
peek()	O(1)
isEmpty()	O(1)
size()	O(1)
Search arbitrary element	O(n)
Traverse complete stack	O(n)

Space for n stored elements: O(n)
For an array-backed dynamic implementation, an occasional resize may cost O(n), but insertion remains **amortized O(1)**.

CodeWithNishchal

9.13. Balanced Parentheses Pattern

Given a string containing brackets: (), [], {}
Determine whether the brackets are **balanced and correctly matched**.
A string is valid when:

1. Every opening bracket has a matching closing bracket.
2. Brackets close in the correct order.
3. No closing bracket appears without a corresponding opening bracket.

Example: Input = "{[()]}" Output: Valid
Because every opening bracket is closed in the correct reverse order.

Approach — Stack

Use a stack to keep track of opening brackets that have not been closed yet.
While traversing the string:

1. If the character is an opening bracket (, [, {

Push it onto the stack.

2. If the character is a closing bracket:),], }

First check whether the stack is empty.

If it is empty, there is no opening bracket available to match it, so return false.

3. Otherwise, pop the top opening bracket and check whether it matches the current closing bracket. Examples: (must match), [must match], { must match }
4. If the pair does not match, return false.
5. After processing the entire string, the stack must be empty.

If the stack still contains elements, some opening brackets were never closed.

Main idea: Opening bracket → Push Closing bracket → Check top → Match → Pop
End → Stack must be empty

```
static boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char ch : s.toCharArray()) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push(ch);
        } else {
            if (stack.isEmpty()) {
                return false;
            }
            char top = stack.pop();
            if ((ch == ')' && top != '(') || (ch == ']' && top != '[') ||  (ch == '}' && top != '{')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

Complexity: Time = O(n) Space = O(n)

Nishchal

9.14. Expression Handling

Stacks are heavily used in expression problems.
Important expression forms:
Infix: A + B
Prefix: + A B
Postfix: A B +

Stacks are useful for:

- infix to postfix conversion,
- infix to prefix conversion,
- postfix evaluation,
- prefix evaluation,
- operator precedence handling.

Example postfix: 2 3 + 4 *
Evaluation:
2 → push
3 → push
+ :
pop 3
pop 2
2 + 3 = 5
push 5
4 → push
* :
pop 4
pop 5
5 × 4 = 20
Result: 20
The reason a stack fits is that the most recently available operands/operators often need to be processed first.

9.15. Monotonic Stack Pattern

A **Monotonic Stack** maintains elements in either increasing or decreasing order while processing the array.
It is mainly used when a problem asks about the nearest element that is:
greater, smaller, greater-or-equal, smaller-or-equal, to the left or right.
Examples:
Next Greater Element, Next Smaller Element, Previous Greater Element, Previous Smaller Element, Stock Span, Daily Temperatures, Largest Rectangle in Histogram

The important idea is that elements that can no longer be useful are removed permanently.
That is why many monotonic-stack solutions run in:
O(n) instead of $O(n^2)$.

CodeWithNishchal

9.16. Monotonic Increasing vs Decreasing Stack

Monotonic Increasing Stack

Values are maintained from smaller to larger.
Conceptually:
Bottom Top
1 → 3 → 5 → 8

This is commonly useful when searching for **smaller elements**.

Monotonic Decreasing Stack

Values are maintained from larger to smaller.
Bottom Top

9 → 7 → 4 → 2

This is commonly useful when searching for **greater elements**.
A better rule than memorizing names is:

> Decide which elements become useless when a new value arrives, and pop those elements.

9.17. Next Greater Element

Given:
[2, 1, 4, 3]

find the first greater value to the right of each element.
Result:
2 → 4
1 → 4
4 → -1
3 → -1

Brute force:
For every element
search everything to its right

Worst case:
$O(n^2)$

A monotonic stack solves it in:
O(n)

CodeWithNishchal

9.18. Next Greater Element — Right-to-Left Approach

Process from right to left.

For each element:

1. Pop all elements smaller than or equal to the current element.
2. After popping, the stack top is the next greater element.
3. Push the current element.

Code:

```
static int[] nextGreater(int[] arr) {
    int n = arr.length;
    int[] answer = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = n - 1; i >= 0; i--) {
        while (!stack.isEmpty()  && stack.peek() <= arr[i]) {
            stack.pop();
        }
        answer[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(arr[i]);
    }
    return answer;
}
```

Why do we pop smaller elements?

Suppose: current = 5

Stack top values: 2, 3, 4

None of them can be the next greater element for 5.

Also, once 5 is processed, those smaller values become less useful for elements further left because 5 is both: larger and closer. So they are removed permanently.

9.19. Next Smaller Element

The logic is almost identical.

For next smaller element:

pop elements >= current

After popping:

stack top = next smaller element

Template:

```
while (!stack.isEmpty()
       && stack.peek() >= arr[i]) {
   stack.pop();
}
```

This is why understanding the condition is more useful than memorizing separate algorithms.

CodeWithNishchal

9.20. Previous Greater / Previous Smaller

To find elements on the **left**, traverse from left to right.
General direction rule:
Need answer on right
→ Often traverse right to left
Need answer on left
→ Often traverse left to right

Examples:
Previous Greater: pop <= current
Previous Smaller: pop >= current
Then the stack top gives the nearest valid candidate.

9.21. Monotonic Stack Reference Table

Problem	Traversal	Pop Condition
Next Greater	Right → Left	stackTop <= current
Next Smaller	Right → Left	stackTop >= current
Previous Greater	Left → Right	stackTop <= current
Previous Smaller	Left → Right	stackTop >= current

This table assumes strict greater/smaller comparison. For problems involving >= or <=, the pop condition must be adjusted carefully.

9.22. Why Monotonic Stack Is O(n)

A common confusion is seeing:

```
for (...) {
   while (...) {
      stack.pop();
   }
}
```

and assuming:
$O(n^2)$
But each element is:
pushed at most once
popped at most once
For n elements:
At most n pushes
At most n pops

Therefore: O(n + n) = O(n)
This is called **amortized analysis**.

CodeWithNishchal

9.23. Stock Span Pattern

Given an array where each element represents the stock price for a day, find the **span of each day**. The span of a stock price is the number of **consecutive days ending at the current day** for which the price was less than or equal to the current price.

Example: Prices = [100, 80, 60, 70, 60, 75, 85]

Result: [1, 1, 1, 2, 1, 4, 6]

For price 75:

60 <= 75

70 <= 75

60 <= 75

But: 80 > 75

Therefore: Span = 4

Approach — Monotonic Stack / Previous Greater Element

For every index, we need to find the **nearest previous element that is greater than the current price**.

Use a stack that stores **indexes** of stock prices.

The stack maintains prices in **strictly decreasing order** from bottom to top.

For every index i:

1. Remove indexes from the top while: prices[stack.peek()] <= prices[i]
   These prices cannot stop the span because they are smaller than or equal to the current price.
2. After removing them:

- If the stack is empty, there is no greater price on the left.
  Therefore: span[i] = i + 1
- Otherwise, stack.peek() is the nearest previous greater element.
  Therefore: span[i] = i - stack.peek()

3. Push the current index onto the stack.

Main idea:

Remove smaller/equal prices → Find previous greater → Calculate distance → Push current index

Visual

Prices = [100, 80, 60, 70, 60, 75, 85]

For price 75 at index 5:

Stack represents previous useful prices.

60 <= 75 → remove

70 <= 75 → remove

60 <= 75 → remove

80 > 75 → stop

Previous greater index = 1

Span:

5 - 1 = 4

CodeWithNishchal

Code

```
static int[] stockSpan(int[] prices) {
    int n = prices.length;
    int[] span = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        // Remove smaller or equal prices
        while (!stack.isEmpty()  && prices[stack.peek()] <= prices[i]) {
            stack.pop();
        }
        // Calculate span
        if (stack.isEmpty()) {
            span[i] = i + 1;
        } else {
            span[i] = i - stack.peek();
        }
        // Store current index
        stack.push(i);
    }
    return span;
}
```

Time = O(n) Although there is a while loop, every index is pushed once and popped at most once.
Space = O(n) In the worst case, all indexes may remain in the stack.

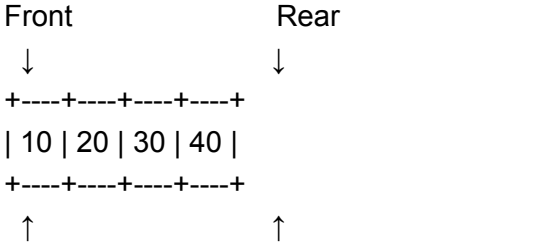
9.24. Overall Complexity Summary

Stack Use	Time	Extra Space
Push	O(1)	—
Pop	O(1)	—
Peek	O(1)	—
Traverse stack	O(n)	—
Balanced brackets	O(n)	O(n)
Expression processing	Usually O(n)	O(n)
Next Greater/Smaller	O(n)	O(n)
Stock Span	O(n)	O(n)
Daily Temperatures style	O(n)	O(n)
Histogram with monotonic stack	O(n)	O(n)

10. Queue

A **Queue** is a linear data structure that follows the **FIFO principle**:
FIFO = First In, First Out

The element inserted first is removed first.



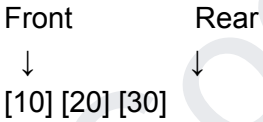
If elements are inserted as: 10 → 20 → 30 → 40
they are removed as: 10 → 20 → 30 → 40
A queue mainly maintains two ends:

- **Front** → element that will be removed next.
- **Rear** → position where new elements are inserted.

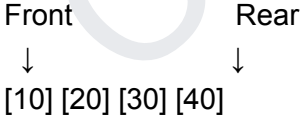
10.1. Core Queue Operations

Operation	Meaning
enqueue(x)	Insert an element at the rear
dequeue()	Remove the front element
peek() / front()	Read the front element without removing it
isEmpty()	Check whether queue is empty
size()	Number of stored elements

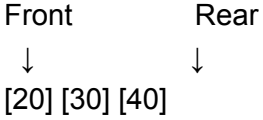
Initial:



enqueue(40)



dequeue()
Returned = 10



For a proper queue implementation:
enqueue = O(1), dequeue = O(1), peek = O(1)

CodeWithNishchal

```
class ArrayQueue {
    private int[] arr;
    private int front;
    private int rear;
    private int size;

    ArrayQueue(int capacity) {
        arr = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }
    // Insert element
    void enqueue(int value) {

        if (isFull()) {
            throw new RuntimeException("Queue Overflow");
        }

        rear++;
        arr[rear] = value;
        size++;
    }
    // Remove front element
    int dequeue() {

        if (isEmpty()) {
            throw new RuntimeException("Queue Underflow");
        }

        int value = arr[front];

        front++;
        size--;

        return value;
    }
    // Return front element
    int peek() {
        if (isEmpty()) {
            throw new RuntimeException("Queue is Empty");
        }

        return arr[front];
    }
```

```
    // Return rear element
    int rear() {
        if (isEmpty()) {
            throw new RuntimeException("Queue is Empty");
        }
        return arr[rear];
    }
    // Check if queue is empty
    boolean isEmpty() {
        return size == 0;
    }
    // Check if queue is full
    boolean isFull() {
        return rear == arr.length - 1;
    }
    // Return number of elements
    int size() {
        return size;
    }
}
```

10.2. Array-Based Queue

A simple queue can be implemented using an array and two indexes: front and rear

Suppose:

```
Index:   0    1    2    3    4
      +----+----+----+----+----+
      | 10 | 20 | 30 |    |    |
      +----+----+----+----+----+
        ↑         ↑
      front      rear
```

One convention is:

front = index of first element

rear = index of last element

After removing 10:

```
Index:   0    1    2    3    4
      +----+----+----+----+----+
      |    | 20 | 30 |    |    |
      +----+----+----+----+----+
           ↑    ↑
         front rear
```

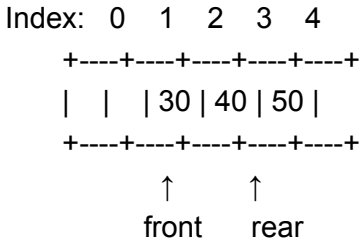
Notice that position 0 is now empty.

This leads to an important problem in a normal linear array queue.

10.3. Limitation of a Linear Array Queue

Suppose capacity is 5.

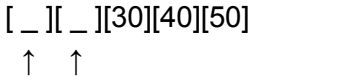
After several operations:



Indexes 0 and 1 are free, but rear has already reached the last index.

A basic linear queue may incorrectly behave as if no more space exists.

Free space exists here:



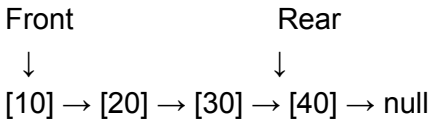
But the rear cannot move further → Shifting all elements toward the beginning would allow reuse, but shifting costs: O(n). The better solution is a **Circular Queue**.

10.4. Queue Using Linked List

A queue can also be implemented with a linked list.

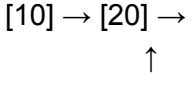
Maintain: front → first node, rear → last node

Example:



Enqueue

Add a node after rear:



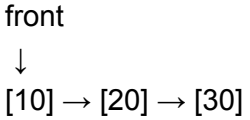
enqueue(40)



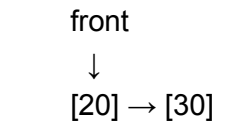
Dequeue

Remove the node at front:

Before:



After:



Both operations take:O(1) provided both front and rear references are maintained.

CodeWithNishchal

10.5. Queue in Java

In Java, the Queue interface is commonly used with ArrayDeque.

```
import java.util.ArrayDeque;
import java.util.Queue;
Queue<Integer> queue = new ArrayDeque<>();
```

Example:

```
queue.offer(10); queue.offer(20); queue.offer(30);
System.out.println(queue.peek()); // 10
queue.poll();

System.out.println(queue.peek()); // 20
```

For most DSA problems:

```
Queue<Integer> queue = new ArrayDeque<>();
```

is a good default choice.

10.6. offer() vs add() and poll() vs remove()

Java's Queue interface provides two styles of methods.

Operation	Exception Style	Special-Value Style
Insert	add()	offer()
Remove	remove()	poll()
Examine front	element()	peek()

10.7. Queue Operation Complexity

For ArrayDeque-based queues:

Operation	Time Complexity
offer() / enqueue	Amortized O(1)
poll() / dequeue	O(1)
peek()	O(1)
isEmpty()	O(1)
size()	O(1)
Search	O(n)
Complete traversal	O(n)

Space for n elements: O(n)

CodeWithNishchal

10.8Circular Queue

A **Circular Queue** is an array-based queue in which the end of the array is logically connected back to the beginning. Instead of: 0 → 1 → 2 → 3 → 4 → stop

we treat indexes as:

0 → 1 → 2 → 3 → 4

↑ |

|_____|

This allows empty positions at the beginning of the array to be reused.

10.9. Circular Movement

Suppose capacity is:5

The next index is calculated as: (index + 1) % capacity

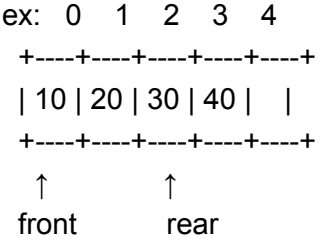
For indexes: 0 → 1, 1 → 2, 2 → 3, 3 → 4, 4 → 0

Because: (4 + 1) % 5 = 5 % 5 = 0. This modulo operation creates the circular behavior.

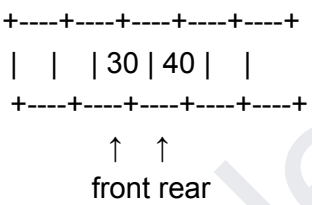
10.10. Circular Queue Visual

Suppose:Capacity = 5

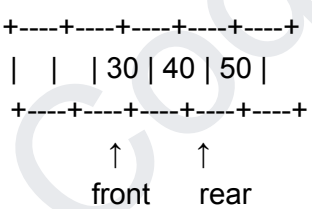
Initially:



After removing 10 and 20:

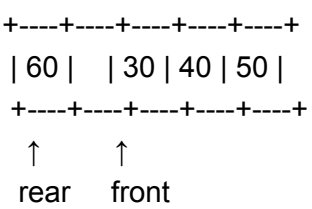


Insert 50:



Insert 60.

Rear wraps around:



Logical queue order is: 30 → 40 → 50 → 60

even though the physical array looks like: [60, _, 30, 40, 50]

CodeWithNishchal

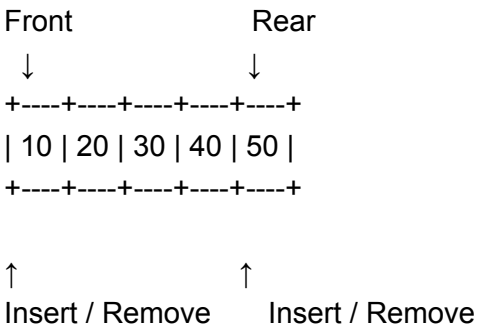
10.11. Circular Queue Implementation in Java

```
class CircularQueue {
    private int[] arr;
    private int front;
    private int rear;
    private int size;
    CircularQueue(int capacity) {
        arr = new int[capacity];
        front = 0;
        rear = 0;
        size = 0;
    }
    boolean enqueue(int value) {
        if (size == arr.length) {
            return false;
        }
        arr[rear] = value;
        rear = (rear + 1) % arr.length;
        size++;
        return true;
    }
    int dequeue() {
        if (size == 0) {  throw new RuntimeException("Queue is empty");
        }
        int value = arr[front];
        front = (front + 1) % arr.length; size--;
        return value;
    }
    int peek() {
        if (size == 0) { throw new RuntimeException("Queue is empty");
        }
        return arr[front];
    }
    boolean isEmpty() {
        return size == 0;
    }
    boolean isFull() {
        return size == arr.length;
    }
    int size() {
        return size;
    }
}
```

10.12Deque

A **Deque** stands for:
Double Ended Queue

Unlike a normal queue, a deque allows insertion and deletion at **both ends**.
DEQUE



Therefore a deque can behave as:
Queue
Stack
or both

depending on which operations are used.

10.13. Core Deque Operations

Operation	Meaning
addFirst(x)	Insert at front
addLast(x)	Insert at rear
removeFirst()	Remove from front
removeLast()	Remove from rear
peekFirst()	Examine front
peekLast()	Examine rear

Example:
Initial:
[20] [30]
addFirst(10) [10] [20] [30]
addLast(40) [10] [20] [30] [40]
removeFirst() [20] [30] [40]
removeLast() [20] [30]

CodeWithNishchal

10.14. Deque in Java

Example:

```
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(20);
deque.offerFirst(10);
deque.offerLast(30);
deque.offerLast(40);
System.out.println(deque.peekFirst()); // 10
System.out.println(deque.peekLast());  // 40
```

10.15. Java Deque Methods

Like Queue, Deque provides exception-style and special-value-style operations.

Action	Exception Method	Special-Value Method
Insert front	addFirst()	offerFirst()
Insert rear	addLast()	offerLast()
Remove front	removeFirst()	pollFirst()
Remove rear	removeLast()	pollLast()
Examine front	getFirst()	peekFirst()
Examine rear	getLast()	peekLast()

10.16. Using Deque as a Queue

Normal FIFO behavior:

```
Deque<Integer> queue = new ArrayDeque<>();

queue.offerLast(10);
queue.offerLast(20);
queue.offerLast(30);

System.out.println(queue.pollFirst()); // 10
```

Visual:

```
remove                insert
  ↓                     ↓
Front                 Rear
[10] [20] [30] [40]
```

CodeWithNishchal

10.17. Using Deque as a Stack

LIFO behavior: Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.pop()); // 30
This is one reason ArrayDeque is extremely useful in Java: it efficiently supports both stack and queue behavior.

10.18. Deque Time Complexity

For ArrayDeque:

Operation	Complexity
Insert front	Amortized O(1)
Insert rear	Amortized O(1)
Remove front	O(1)
Remove rear	O(1)
Peek front	O(1)
Peek rear	O(1)
Search	O(n)
Traversal	O(n)

Space: O(n)

10.19. Queue vs Deque vs Stack

Feature	Queue	Deque	Stack
Principle	FIFO	Both ends	LIFO
Insert	Rear	Front or rear	Top
Delete	Front	Front or rear	Top
Access ends	Front mainly	Both	Top
Java choice	Queue + ArrayDeque	Deque + ArrayDeque	Deque + ArrayDeque
Common use	BFS	Sliding window, both-end processing	DFS, matching, monotonic stack

CodeWithNishchal

10.20. Monotonic Deque Pattern

Just as a stack can be monotonic, a deque can maintain elements in increasing or decreasing order.

A **Monotonic Deque** is especially useful when we need the maximum or minimum of a moving window.

Classic examples: Sliding Window Maximum, Sliding Window Minimum

Suppose: Array = [1, 3, -1, -3, 5] Window size = 3

Windows:

[1, 3, -1] → maximum = 3

[3, -1, -3] → maximum = 3

[-1, -3, 5] → maximum = 5

Scanning every window independently costs approximately:O(n × k)

A monotonic deque reduces it to:O(n)

10.21. Decreasing Deque for Window Maximum

For maximum problems, maintain indexes whose values are in decreasing order:

Front Rear

↓ ↓

[8] [6] [4] [2]

Largest value is always at front.

When a new value arrives:

1. Remove indexes that are outside the window.
2. Remove smaller or equal values from the rear.
3. Insert current index at the rear.
4. Front gives the maximum.

Why remove smaller values?

Suppose:

Deque values: 8, 5, 3

and new value is:6

3 and 5 can never become maximum while 6 remains in the window because 6 is both larger and newer.

So:

8, 5, 3

↑ remove

8, 5

↑ remove

8, 6

CodeWithNishchal

10.22. Sliding Window Maximum Code

Given an integer array arr and a window size k, find the **maximum element in every contiguous subarray of size k**.

Example: Array = [1, 3, -1, -3, 5, 3, 6, 7] k = 3

Windows:

[1, 3, -1] → 3

[3, -1, -3] → 3

[-1, -3, 5] → 5

[-3, 5, 3] → 5

[5, 3, 6] → 6

[3, 6, 7] → 7

Result:

[3, 3, 5, 5, 6, 7]

Approach — Sliding Window + Monotonic Deque

Use a **deque of indexes** to efficiently track the maximum element of the current window.

The deque maintains indexes whose corresponding values are in **decreasing order**.

Therefore: arr[deque.peekFirst()]

always represents the maximum element of the current window.

For every index right:

1. Remove expired indexes from the front. An index is outside the current window when: deque.peekFirst() <= right - k
2. Remove smaller or equal elements from the back.

While: arr[deque.peekLast()] <= arr[right] remove the last index.

These elements are no longer useful because the current element is larger and will remain in the window longer.

3. Add the current index to the back of the deque.
4. Once the window reaches size k, store:

arr[deque.peekFirst()]

as the maximum for that window.

Main idea:

Remove expired indexes → Remove smaller values → Add current index → Front gives maximum

Visual

Window = [1, 3, -1]

Deque values: [3, -1]

Front = 3, Maximum = 3, Next element = -3

Deque: [3, -1, -3]

Maximum = 3, Next element = 5

3 is outside the window → remove

5 > -3 → remove -3

5 > -1 → remove -1

Deque: [5]

Maximum = 5

CodeWithNishchal

CODE:

```
static int[] maxSlidingWindow(int[] arr, int k) {
    int n = arr.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();
    int resultIndex = 0;
    for (int right = 0; right < n; right++) {
        // Remove indexes outside the window
        while (!deque.isEmpty()
                && deque.peekFirst() <= right - k) {
            deque.pollFirst();
        }
        // Maintain decreasing values
        while (!deque.isEmpty()
                && arr[deque.peekLast()] <= arr[right]) {
            deque.pollLast();
        }
        deque.offerLast(right);
        // Record answer once window size reaches k
        if (right >= k - 1) {
            result[resultIndex++] =
                    arr[deque.peekFirst()];
        }
    }
    return result;
}
```

Logic

The deque stores **indexes**, not just values, because we must know when an element leaves the window.

deque.peekFirst()

always contains the index of the largest element in the current window.

Although there are nested while loops, every index enters the deque once and leaves at most once.

Therefore:

Time = O(n)

Space = O(k)

10.23. Increasing Deque for Window Minimum

For minimum problems, reverse the ordering logic.
Maintain:
smallest → ... → largest

When a new value arrives, remove larger or equal values from the rear.
General pop condition:

```
while (!deque.isEmpty()
        && arr[deque.peekLast()] >= arr[right]) {

    deque.pollLast();
}
```

Then:
deque front = current window minimum

10.24. Why Store Indexes in a Monotonic Deque?

Suppose the window size is k.
We need to know whether the element at the front has become too old.
If current index is:
right

then valid indexes satisfy:
index > right - k

So outdated indexes are removed using:
deque.peekFirst() <= right - k

Values alone would not tell us whether an element still belongs to the current window.

10.25. Overall Operation Complexity

Data Structure	Insert Front	Insert Rear	Delete Front	Delete Rear	Peek
Queue	—	O(1)	O(1)	—	O(1)
Circular Queue	—	O(1)	O(1)	—	O(1)
Deque	O(1)*	O(1)*	O(1)	O(1)	O(1)
Stack	—	O(1)	—	O(1)	O(1)

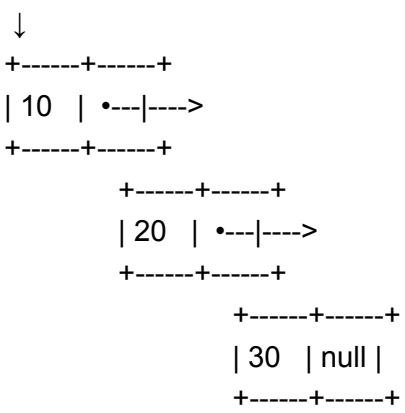
* For ArrayDeque, insertion is amortized O(1) because occasional internal resizing can take O(n).

CodeWithNishchal

11. Linked List

A **Linked List** is a linear data structure made of **nodes**. Unlike an array, elements are not accessed through contiguous indexes. Each node stores some data and one or more references that connect it to other nodes.

For a basic singly linked list:



Each node contains: Data + Link

In Java:

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
    }
}
```

A linked list does not provide direct indexed access like: arr[5]
To reach a node, we normally start from head and follow references.

11.1. Why Linked Lists Are Used

Arrays provide very fast indexed access, but their size is fixed and insertion/deletion in the middle may require shifting many elements.

Linked lists solve a different problem: they allow nodes to be dynamically connected and disconnected.

A linked list is useful when:

- frequent insertion/deletion is required,
- fixed contiguous storage is undesirable,
- the structure should grow dynamically,
- direct random access is not the main requirement.

CodeWithNishchal

Property	Array	Linked List
Memory structure	Indexed contiguous-style storage	Nodes connected by references
Size	Fixed after creation	Dynamic
Random access	O(1)	O(n)
Insert at beginning	O(n) if shifting required	O(1)
Delete beginning	O(n) if shifting required	O(1)
Extra per-element memory	Low	Reference(s) required

11.2. Basic Terminology

Node

A node is the basic unit of a linked list.

```
+---------+---------+
|  Data   |  Next   |
+---------+---------+
```

Head

head points to the first node.

```
head
 ↓
[10] → [20] → [30] → null
```

If:head == null
the list is empty.

Tail

The final node is often called the **tail**.

```
head                tail
 ↓                   ↓
[10] → [20] → [30] → null
```

The tail's next is null in a normal singly linked list.

11.3. Types of Linked Lists

The major types are:

```
Linked List
  |
  +-- Singly Linked List
  |
  +-- Doubly Linked List
  |
  +-- Circular Singly Linked List
  |
  +-- Circular Doubly Linked List
```

CodeWithNishchal

11.4. Singly Linked List

A **Singly Linked List** stores one reference per node:
next
Structure:
head
↓
[10 | •] → [20 | •] → [30 | •] → null
Traversal is only naturally possible in the forward direction.
Java node:

```
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

11.5. Traversing a Singly Linked List

To traverse the list, start from head and repeatedly follow next.

```
static void printList(Node head) {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " ");
        current = current.next;
    }
}
```

Visual:
current
↓
[10] → [20] → [30] → null
current
↓
[10] → [20] → [30] → null
current
↓
[10] → [20] → [30] → null
Complexity:
Time = O(n)
Space = O(1)

CodeWithNishchal

11.6. Insertion in Singly Linked List

The three major cases are: Beginning , End Specific Position

11.6.1 Insert at Beginning

Suppose:

head

↓

[20] → [30] → null

Insert 10.

Create:

newNode = [10]

Point it to current head:

[10] → [20] → [30]

Move head:

head

↓

[10] → [20] → [30] → null

Code:

```
static Node insertFirst(Node head, int value) {
    Node newNode = new Node(value);
    newNode.next = head;
    head = newNode;
    return head;
}
```

Complexity: O(1). No traversal is required.

11.6.2 Insert at End

Before:

head

↓

[10] → [20] → [30] → null

Insert 40.

Find the last node:

[10] → [20] → [30]

↑

current

Then:

current.next = newNode

Result:

[10] → [20] → [30] → [40] → null

Code:

CodeWithNishchal

CODE:

```
static Node insertLast(Node head, int value) {
    Node newNode = new Node(value);
    if (head == null) {
        return newNode;
    }
    Node current = head;
    while (current.next != null) {
        current = current.next;
    }
    current.next = newNode;
    return head;
}
```

Complexity without a tail reference: O(n). If a tail reference is maintained: O(1)

11.6.3 Insert at a Position

Assume a zero-based position.
Example: List: 10 → 20 → 40. Insert 30 at position 2
We must reach the node at: position - 1
Result:10 → 20 → 30 → 40
Pointer changes: Before: 20 --------> 40 and New Node: 30
Step 1:
30.next = 20.next
20 30 → 40
Step 2:
20.next = 30
20 → 30 → 40

```
static Node insertAtPosition(Node head, int value, int position) {
    if (position == 0) {
        return insertFirst(head, value);
    }
    Node current = head;
    for (int i = 0; i < position - 1 && current != null; i++) {
        current = current.next;
    }
    if (current == null) {
        throw new IndexOutOfBoundsException();
    }
    Node newNode = new Node(value);
    newNode.next = current.next;
    current.next = newNode;
    return head;
}
```

Complexity: Traversal = O(n) Link change = O(1) Overall = O(n)

WithNishchal

11.7. Deletion in Singly Linked List

Major cases: Delete first , Delete last, Delete by value, Delete by position

11.7.1 Delete First Node

Before:

head

↓

[10] → [20] → [30]

Move: head = head.next

After:

head

↓

[20] → [30] → null

```
static Node deleteFirst(Node head) {
    if (head == null) {
        return null;
    }
    return head.next;
}
```

Complexity: O(1)

11.7.2 Delete Last Node

Before:

[10] → [20] → [30] → null

We need the **second-last node**:

[10] → [20] → [30]

↑

current

Then: current.next = null

```
static Node deleteLast(Node head) {
    if (head == null || head.next == null) {
        return null;
    }
    Node current = head;
    while (current.next.next != null) {
        current = current.next;
    }
    current.next = null;
    return head;
}
```

Complexity: O(n)

Even with a tail reference, a singly linked list cannot directly move from the tail to the previous node, so deleting the last node generally remains O(n) unless additional structure is maintained.

CodeWithNishchal

11.7.3 Delete by Value

Suppose:
10 → 20 → 30 → 40

Delete 30.
Find the node before 30:
10 → 20 → 30 → 40
↑
current
Then: current.next = current.next.next

Result:
10 → 20 → 40

```
static Node deleteValue(Node head, int value) {
    if (head == null) {
        return null;
    }
    if (head.data == value) {
        return head.next;
    }
Node current = head;
    while (current.next != null && current.next.data != value) {
        current = current.next;
    }
    if (current.next != null) {
        current.next = current.next.next;
    }
    return head;
}
```

Complexity: O(n)

11.7.4 Delete at Position

Example: 10 → 20 → 30 → 40
Delete position 2
Reach position 1:
10 → 20 → 30 → 40
↑
Then bypass node 30:
20.next = 30.next
Result: 10 → 20 → 40
Complexity: O(n)

11.8. Searching in a Linked List

1. Start from the first node: current = head
2. Continue while the current node exists: while (current != null)
3. Compare the current node's data with the target: current.data == target
4. If they are equal: return true Otherwise, move to the next node: current = current.next
6. If we reach: current == null. it means the target was not found. So return: false

```
static boolean search(Node head, int target) {
    Node current = head;
    while (current != null) {
        if (current.data == target) { return true;}
        current = current.next;
    }
    return false;
}
```

Worst case: O(n)

11.9. Accessing an Element by Position

To get the element at index k, traverse k links.

```
static int get(Node head, int index) {
    Node current = head;
    for (int i = 0; i < index; i++) {
        if (current == null) {
            throw new IndexOutOfBoundsException();
        }
        current = current.next;
    }
    if (current == null) {  throw new IndexOutOfBoundsException(); }
    return current.data;
}
```

Complexity: O(n). This is one of the biggest differences from arrays.

11.10. Updating a Node

```
static void set(Node head, int index, int value) {
    Node current = head;
    for (int i = 0; i < index; i++) {
        if (current == null) { throw new IndexOutOfBoundsException();
        }
        current = current.next;
    }
    if (current == null) { throw new IndexOutOfBoundsException(); }
    current.data = value;
}
```

Complexity: O(n). The actual update is O(1), but reaching the node is O(n).

11.11. Reversing a Singly Linked List

Original:
head
↓
10 → 20 → 30 → 40 → null
Reversed:
head
↓
40 → 30 → 20 → 10 → null
Three references are usually used: previous, current, next
Visual:
prev current
↓ ↓
null [10] → [20] → [30]

Save next: next = current.next
Reverse the link:
10 → null

Move pointers:
prev = current
current = next
Repeat.

```
static Node reverse(Node head) {
    Node previous = null;
    Node current = head;
    while (current != null) {
        Node next = current.next;
        current.next = previous;
        previous = current;
        current = next;
    }
    return previous;
}
```

Pointer flow

Before one iteration:
prev current next
↓ ↓ ↓
10 20 30
↑ |
|_____|
After reversing current link:
10 ← 20 30
Complexity: Time = O(n) Space = O(1)

GitHubNishchal

11.12. Finding the Middle Node

A very common linked-list technique uses:
slow pointer → 1 step
fast pointer → 2 steps
Example:
10 → 20 → 30 → 40 → 50
S
F
After movements:
10 → 20 → 30 → 40 → 50
S
F

When fast reaches the end, slow is near the middle.
Code:

```
static Node middle(Node head) {
    Node slow = head;
    Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Complexity: Time = O(n) Space = O(1)

11.13. Detecting a Cycle

A linked list may contain a cycle:
10 → 20 → 30 → 40
↑ |
|_____|
Then traversal never reaches null.
The standard solution is **Floyd's Cycle Detection Algorithm**:
slow → 1 step, fast → 2 steps. If there is a cycle, the pointers eventually meet.

```
static boolean hasCycle(Node head) {
    Node slow = head; Node fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) { return true;
        }}
    return false;
}
```

Complexity: Time = O(n) Space = O(1)

11.14. Complete Singly Linked List Complexity

Operation	Complexity
Access by index	O(n)
Search by value	O(n)
Traverse	O(n)
Insert beginning	O(1)
Insert end without tail	O(n)
Insert end with tail	O(1)
Insert at position	O(n)
Delete beginning	O(1)
Delete end	O(n)
Delete by value	O(n)
Delete at position	O(n)
Reverse	O(n)
Find middle	O(n)
Cycle detection	O(n)

Space for n nodes: O(n)

11.15. Doubly Linked List

A **Doubly Linked List** stores two references in every node: previous, next

Structure:

null ← [10] ⇄ [20] ⇄ [30] → null

↑

head

Node:

```
class DNode {
    int data;
    DNode prev;
    DNode next;
    DNode(int data) {
        this.data = data;
    }
}
```

A doubly linked list can be traversed in both directions.

CodeWithNishchal

11.16. Doubly Linked List Structure

Each node:

```
+--------+------+--------+
|  prev  | data |  next  |
+--------+------+--------+
```

Example:

```
      previous       next
         ←           →
null ← [10] ⇄ [20] ⇄ [30] → null
```

This extra prev reference makes some deletion operations easier, but uses more memory.

11.17. Insert at Beginning — Doubly Linked List

Before:

```
null ← 20 ⇄ 30 → null
     ↑
    head
```

Insert 10.

Required links:

newNode.next = head

head.prev = newNode

head = newNode

Result:

```
null ← 10 ⇄ 20 ⇄ 30 → null
     ↑
    head
```

Code:

```
static DNode insertFirst(DNode head, int value) {
    DNode newNode = new DNode(value);
    newNode.next = head;
    if (head != null) {
        head.prev = newNode;
    }
    return newNode;
}
```

Complexity:

O(1)

---

deWithNishchal

11.18. Insert at End — Doubly Linked List

With only head, reaching the end costs:O(n). With a maintained tail:O(1)

Conceptually:

Before:

null ← 10 ⇄ 20 ⇄ 30 → null

↑

tail

Insert 40:

30.next = 40

40.prev = 30

tail = 40

Result:

null ← 10 ⇄ 20 ⇄ 30 ⇄ 40 → null

↑

tail

11.19. Insert Between Two Nodes

Suppose:

10 ⇄ 20 ⇄ 40

Insert 30 between 20 and 40.

Need four link changes:

30.prev = 20

30.next = 40

20.next = 30

40.prev = 30

Result:

10 ⇄ 20 ⇄ 30 ⇄ 40

This is the key difference from singly linked lists: both directions must remain consistent.

11.20. Delete a Node — Doubly Linked List

Suppose:

10 ⇄ 20 ⇄ 30 ⇄ 40

Delete node 30.

Because 30 already knows both neighbors:

30.prev = 20

30.next = 40

we connect:

20.next = 40

40.prev = 20

Result:

10 ⇄ 20 ⇄ 40

If the target node reference is already known, deletion is: O(1)

If we first need to search for the value: O(n)

CodeWithNishchal

11.21. Traversing Doubly Linked List

Forward:

```
DNode current = head;
while (current != null) {
    System.out.print(current.data + " ");
    current = current.next;
}
Backward, if tail is known:
DNode current = tail;
while (current != null) {
    System.out.print(current.data + " ");
    current = current.prev;
}
```

Both are: O(n)

11.22. Reverse Doubly Linked List

In a doubly linked list, reversing means swapping: next, and, prev for every node.

Conceptually:

Before: null ← 10 ⇄ 20 ⇄ 30 → null

After: null ← 30 ⇄ 20 ⇄ 10 → null

Algorithm

1. Start from the head.
2. For the current node, swap its two pointers: prev ↔ next
3. After swapping, move to the next node using: current = current.prev
   We use prev because after swapping, it points to the original next node.
4. Repeat this for every node until current == null.
5. The last processed node becomes the new head.

```
static DNode reverse(DNode head) {
    DNode current = head;
    DNode newHead = null;
    while (current != null) {
        DNode next = current.next;
        current.next = current.prev;
        current.prev = next;
        newHead = current;
        current = next;
    }
    return newHead;
}
```

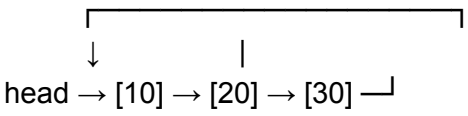
Complexity: O(n)

CodeWithNishchal

11.23. Circular Singly Linked List

In a **Circular Singly Linked List**, the last node does not point to null.
Instead:
tail.next = head

Visual:



There is no null marking the end.
This means normal traversal must stop when we return to the starting node.

11.24. Traversing Circular Singly Linked List

Algorithm

1. If head == null, the list is empty, so stop.
2. Set: current = head
3. Process the current node. For example: print current.data
4. Move to the next node: current = current.next
5. Continue until current becomes equal to head again.
6. Stop when we return to the starting node.

Incorrect:
while (current != null) because current never becomes null.
Correct:

```
static void printCircular(Node head) {
    if (head == null) {
        return;
    }
    Node current = head;
    do {
        System.out.print(current.data + " ");
        current = current.next;
    } while (current != head);
}
```

The do-while is convenient because the head itself must be processed before checking whether we returned to it.

WithNishchal

11.25. Circular Singly Insertion

If a tail reference is maintained:
tail.next = head

Insert at Beginning

Suppose:
head
↓
10 → 20 → 30
↑ |
|_____|
↑
tail
Insert 5.
newNode.next = head
head = newNode
tail.next = head
Result:
5 → 10 → 20 → 30
↑ |
|_____|

Complexity with tail: O(1)

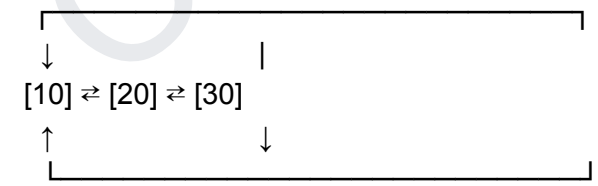
Insert at End

With tail: newNode.next = head
tail.next = newNode
tail = newNode
Also:O(1)

11.26. Circular Doubly Linked List

In a **Circular Doubly Linked List**:
head.prev = tail
tail.next = head

Visual:



Every node has both:
prev , next

and the structure has no null at either end.

CodeWithNishchal

11.27. Circular Doubly Structure

For:
10 ⇄ 20 ⇄ 30
connections include:
10.prev = 30
30.next = 10

So movement is possible indefinitely in either direction.
This structure is useful for cyclic navigation such as:

- playlists,
- round-robin scheduling,
- repeated next/previous navigation.

11.28. Circular vs Non-Circular Linked List

Property	Normal Linked List	Circular Linked List
Last node next	null	head
Natural end marker	null	Return to start
Repeated cycling	Not direct	Natural
Traversal condition	current != null	current != head after start
Common use	General sequences	Cyclic processing

11.29. Complete Type Comparison

Feature	Singly	Doubly	Circular Singly	Circular Doubly
next	Yes	Yes	Yes	Yes
prev	No	Yes	No	Yes
Last points to null	Yes	Yes	No	No
Backward traversal	No	Yes	No	Yes
Circular traversal	No	No	Yes	Yes
Memory/node	Lower	Higher	Lower	Higher
Implementation difficulty	Lowest	Medium	Medium	Highest

CodeWithNishchal

11.30. Linked List Operation Complexity Summary

Singly Linked List

Operation	Without Tail	With Tail
Access by index	O(n)	O(n)
Search	O(n)	O(n)
Insert beginning	O(1)	O(1)
Insert end	O(n)	O(1)
Delete beginning	O(1)	O(1)
Delete end	O(n)	O(n)
Insert after known node	O(1)	O(1)
Delete after known node	O(1)	O(1)
Traverse	O(n)	O(n)

Doubly Linked List

Operation	Complexity
Access/Search	O(n)
Insert beginning	O(1)
Insert end with tail	O(1)
Delete beginning	O(1)
Delete end with tail	O(1)
Insert around known node	O(1)
Delete known node	O(1)
Traverse	O(n)

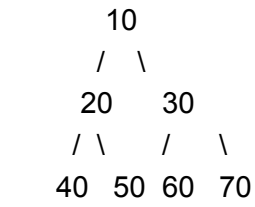
Important distinction:
Finding a node may cost O(n).

Changing links around an already known node is usually O(1).

CodeWithNishchal

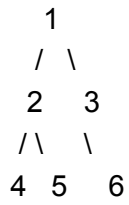
12. Tree

A **Tree** is a non-linear hierarchical data structure made of **nodes connected by edges**. Unlike arrays, stacks, queues, and linked lists, which are primarily linear, trees organize data in parent-child relationships.



Here 10 is the root, 20 and 30 are its children, and 40, 50, 60, 70 are leaf nodes. Trees are widely used for hierarchical data, searching, indexing, expression evaluation, file systems, databases, compilers, routing, and many recursive algorithms.

12.1. Tree Terminology



Term	Meaning	Example
Root	Topmost node	1
Parent	Node directly above another node	2 is parent of 4
Child	Node directly below another node	4 is child of 2
Sibling	Nodes having same parent	4 and 5
Leaf	Node with no children	4, 5, 6
Internal Node	Node with at least one child	1, 2, 3
Edge	Connection between two nodes	1 → 2
Path	Sequence of connected nodes	1 → 2 → 5
Ancestor	Node above another node	1 and 2 are ancestors of 5
Descendant	Node below another node	4 is descendant of 1
Subtree	Tree formed from a node and its descendants	subtree rooted at 2
Depth	Number of edges from root to node	depth of 5 = 2
Height of Node	Longest path in edges from node to a leaf	height of node 2 = 1
Height of Tree	Height of root	here = 2

If height is measured in **nodes** instead of edges, the same tree would have height 3. In DSA, always check which convention a problem uses.

CodeWithNishchal

12.2. Basic Properties

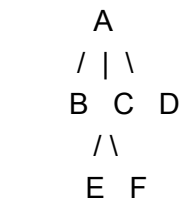
For a tree with n nodes: Number of edges = n - 1
provided it is a connected tree.
A tree:

- has exactly one root,
- has no cycles,
- has exactly one unique path between any two nodes,
- can contain multiple subtrees.

12.3. Important Types of Trees

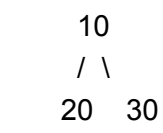
General Tree

A node can have any number of children.



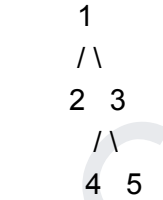
Binary Tree

Each node can have at most two children: left child, right child



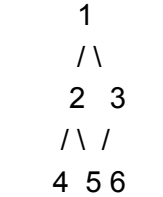
Full Binary Tree

Every node has either:
0 children or 2 children
Example:



Complete Binary Tree

All levels are completely filled except possibly the last level, and the last level is filled from left to right.

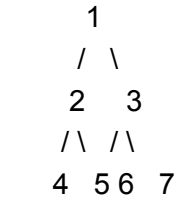


This property is important for **Binary Heap**.

CodeWithNishchal

Perfect Binary Tree

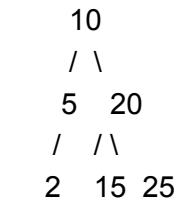
Every internal node has exactly two children and all leaves occur at the same level.



For height h measured in edges: Maximum nodes = 2^(h + 1) - 1

Balanced Binary Tree

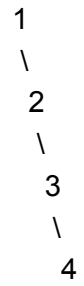
A tree is considered height-balanced when the left and right subtree heights remain reasonably close. In an AVL-style definition, their height difference is at most 1 at every node.



Balanced trees generally keep operations efficient.

Skewed Tree

Every node has only one child.



A skewed binary tree behaves similarly to a linked list and can cause tree operations to degrade to O(n).

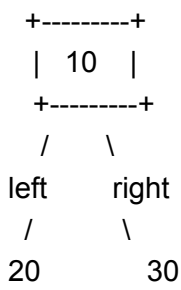
12.4. Binary Tree Representation in Java

A binary tree node commonly contains: data, left reference, right reference

```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;
    TreeNode(int data) {
        this.data = data;
    }
}
```

Visual:

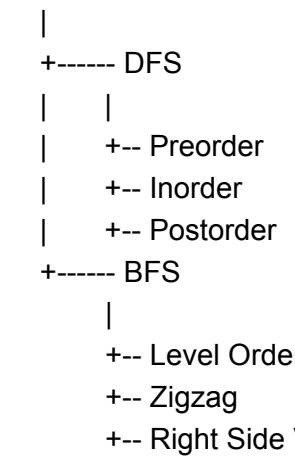
CodeWithNishchal



The root is represented by: TreeNode root;
If: root == null
the tree is empty.

12.5. Tree Traversal

Traversal means visiting every node in a particular order.
Two major approaches are:



12.6. DFS — Depth First Search

DFS explores deeply into one subtree before moving to another.
For a binary tree, the three main DFS traversals are:
Preorder
Inorder
Postorder

Consider:



12.6.1 Preorder Traversal

Order: Root → Left → Right

CodeWithNishchal

For the tree above:
1, 2, 4, 5, 3, 6, 7
Visual:
Visit root
↓
Traverse left subtree
↓
Traverse right subtree
Recursive implementation:

```
static void preorder(TreeNode root) {
    if (root == null) {
        return;
    }
    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
}
```

Why it works
At every node:

1. Process the current node.
2. Recursively process its left subtree.
3. Recursively process its right subtree.

Preorder is useful when the parent must be processed before its children, such as:

- copying a tree,
- serialization-style traversal,
- generating prefix expressions,
- top-down recursive processing.

12.6.2 Inorder Traversal

Order: Left → Root → Right . Result: 4, 2, 5, 1, 6, 3, 7
Implementation:

```
static void inorder(TreeNode root) {
    if (root == null) {
        return;
    }
    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
}
```

Inorder is especially important for a **Binary Search Tree** because it visits BST values in sorted order.

12.6.3 Postorder Traversal

Order:

devWithNishchal

Left → Right → Root

Result:
4, 5, 2, 6, 7, 3, 1
Implementation:

```
static void postorder(TreeNode root) {
    if (root == null) {
        return;
    }
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
}
```

Postorder is useful when children must be processed before the parent, for example:

- calculating subtree height,
- subtree size,
- deleting/freeing tree nodes conceptually,
- checking balance,
- evaluating expression trees.

12.6.4 DFS Order Comparison

Traversal	Order	Common Use
Preorder	Root → Left → Right	Parent before children
Inorder	Left → Root → Right	Sorted order in BST
Postorder	Left → Right → Root	Children before parent

Mnemonic:
Pre → Root comes before subtrees
In → Root is in between
Post → Root comes after subtrees

12.7. DFS Complexity

Every node is visited once. Time = O(n)
Recursive stack space depends on tree height: Space = O(h)
For a balanced tree: h ≈ log n Space = O(log n)
For a skewed tree: h ≈ n Space = O(n)

12.8. Iterative DFS

Recursive DFS implicitly uses the call stack. We can perform the same traversal using an explicit stack.
Example preorder:

CodeWithNishchal

```
static void preorderIterative(TreeNode root) {
    if (root == null) {
        return;
    }
    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        System.out.print(node.data + " ");
        if (node.right != null) {
            stack.push(node.right);
        }
        if (node.left != null) {
            stack.push(node.left);
        }
    }
}
```

Right is pushed first because stack is LIFO, so left gets processed first.
Stack: push right, push left, left comes out first
Time: O(n) Space: O(h) to O(n) depending on traversal and tree structure.

12.9. BFS — Breadth First Search / Level Order

BFS processes the tree **level by level**.
For:

```
       1
      /   \
     2     3
    / \   / \
   4   5 6   7
```

Level order:
1
2 3
4 5 6 7

Result:
1, 2, 3, 4, 5, 6, 7
BFS uses a **Queue**.

12.9.1 Level Order Traversal

Algorithm

CodeWithNishchal

1. If the tree is empty, return.
2. Create a queue and add the root.
3. While the queue is not empty:
    - Remove the front node.
    - Process/print that node.
    - If it has a left child, add it to the queue.
    - If it has a right child, add it to the queue.
4. Repeat until the queue becomes empty.

```
static void levelOrder(TreeNode root) {
    if (root == null) {
        return;
    }
    Queue<TreeNode> queue = new
ArrayDeque<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.data + " ");
        if (node.left != null) {
            queue.offer(node.left);
        }
        if (node.right != null) {
            queue.offer(node.right);
        }
    }
}
```

Flow:
Queue = [1]
Remove 1,
Add 2, 3
Queue = [2, 3]
Remove 2
Add 4, 5
Queue = [3, 4, 5]
Remove 3
Add 6, 7
Queue = [4, 5, 6, 7]

12.9.2 Processing Level by Level

Sometimes the problem needs each level separately.

Algorithm — Process Tree Level by Level

CodeWithNishchal

1. If root == null, return.
2. Create a queue and add the root.
3. While the queue is not empty:
4. Store the number of nodes currently in the queue: levelSize = queue.size()
   This represents the number of nodes in the current level.
5. Run a loop exactly levelSize times.
6. In each iteration:
   - Remove one node from the front.
   - Process/print it.
   - Add its left child if it exists.
   - Add its right child if it exists.
7. After processing levelSize nodes, the current level is finished.
8. The queue now contains nodes of the next level.
9. Repeat until the queue becomes empty.

```
static void levelByLevel(TreeNode root) {
    if (root == null) {
        return;
    }
    Queue<TreeNode> queue = new
ArrayDeque<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll();
            System.out.print(node.data + " ");
            if (node.left != null) {
                queue.offer(node.left);
            }

            if (node.right != null) {
                queue.offer(node.right);
            }
        }
        System.out.println();
    }
}
```

The queue size at the beginning of each outer-loop iteration gives the number of nodes currently belonging to that level.

12.10. Zigzag Level Order

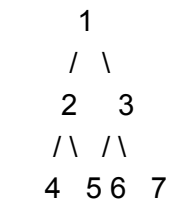
Zigzag traversal alternates direction at each level.
Level 0: Left → Right
Level 1: Right → Left

Level 2: Left → Right
...

Algorithm — Zigzag Level Order Traversal

1. If root == null, return an empty result.
2. Create a queue and add the root node.
3. Keep a direction flag: leftToRight = true
4. While the queue is not empty:
5. Store the number of nodes in the current level: size = queue.size()
6. Create a list to store the current level.
7. Process exactly size nodes.
8. For each node:
    - Remove the node from the queue.
    - If leftToRight == true, add its value at the **end** of the level list.
    - Otherwise, add its value at the **front** of the level list.
    - Add the left child to the queue if it exists.
    - Add the right child to the queue if it exists.
9. After the current level is complete, add the level list to the result.
10. Reverse the direction: leftToRight = !leftToRight
11. Repeat until the queue becomes empty.

Example:



Zigzag:
1
3 2
4 5 6 7

Result:
1, 3, 2, 4, 5, 6, 7

One clean method is BFS with a direction flag.

```
static List<List<Integer>> zigzag(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) {
        return result;
    }
    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);
    boolean leftToRight = true;
    while (!queue.isEmpty()) {
        int size = queue.size();
```

CodeWithNishchal

```
        LinkedList<Integer> level = new LinkedList<>();
        for (int i = 0; i < size; i++) {

            TreeNode node = queue.poll();

            if (leftToRight) {
                level.addLast(node.data);
            } else {
                level.addFirst(node.data);
            }

            if (node.left != null) {
                queue.offer(node.left);
            }

            if (node.right != null) {
                queue.offer(node.right);
            }
        }

        result.add(level);
        leftToRight = !leftToRight;
    }

    return result;
}
```

Time:
O(n)

Extra space:
O(n)

in the worst case because the queue can contain an entire wide level.

12.11. Right Side View

The right side view contains the node visible when looking at the tree from the right.
Example:

```
    1          ← visible
   /  \
  2    3       ← visible
   \    \
    5    4     ← visible
```

Right side view:1, 3, 4

Using BFS, take the last node of every level.

```
static List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) {
        return result;
    }
    Queue<TreeNode> queue = new ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) {
                result.add(node.data);
            }

            if (node.left != null) {
                queue.offer(node.left);
            }
            if (node.right != null) {
                queue.offer(node.right);
            }
        }
    }
    return result;
}
```

Because BFS processes each level separately, the last processed node in that level is the rightmost node.

12.12. DFS vs BFS

Feature	DFS	BFS
Main structure	Stack / Recursion	Queue
Movement	Deep before wide	Level by level
Common forms	Pre/In/Post order	Level order
Space	O(h) recursive	O(width)

Best for	Subtree recursion, structural computations	Levels, shortest path in unweighted structures
Typical tree questions	Height, balance, path, subtree	Level order, views, nearest level

Both normally take: O(n) when all nodes are visited.

12.13. Top-Down Approach

In a **Top-Down** approach, information flows from the parent to the child.

The recursive function receives some state representing what has happened along the path from the root.

General structure:

```
void dfs(TreeNode node, State state) {
    if (node == null) {
        return;
    }
    State updatedState = update(state, node);
    dfs(node.left, updatedState);
    dfs(node.right, updatedState);
}
```

Conceptually:

```
          Root
        state = ...
         /      \
        ↓        ↓
   Left child   Right child
   gets state   gets state
```

Typical top-down state:

- current depth,
- running path sum,
- maximum value seen on path,
- path contents,
- remaining target.

12.13.1 Example — Maximum Depth Using Top-Down

```
static void maxDepthTopDown(TreeNode node,  int depth, int[] answer) {
    if (node == null) {
        return;
    }
    answer[0] = Math.max(answer[0], depth);
    maxDepthTopDown(node.left, depth + 1, answer);
```

CodeWithNishchal

```
    maxDepthTopDown(node.right, depth + 1, answer);
}
```

Call: int[] answer = {0};
maxDepthTopDown(root, 1, answer);
Here:
Parent gives current depth ↓
Child receives depth + 1
Information travels downward.

12.14. Bottom-Up Approach

In a **Bottom-Up** approach, each child subtree solves its own problem and returns information to the parent.

General structure:

```
Result dfs(TreeNode node) {
    if (node == null) {
        return baseResult;
    }
    Result left = dfs(node.left);
    Result right = dfs(node.right);
    return combine(left, right, node);
}
```

Conceptually:

```
      Parent
       ↑   ↑
   result  result
     /      \
 Left       Right
 Subtree    Subtree
```

Bottom-up recursion is especially common for:

- height,
- subtree size,
- balanced tree checking,
- diameter,
- maximum path calculations.

12.15. Top-Down vs Bottom-Up

Feature	Top-Down	Bottom-Up
Information flows	Parent → Child	Child → Parent
Main idea	Carry state downward	Return result upward

Useful for	Path/depth/running state	Height/subtree calculations
Typical DFS order	Often preorder-like	Often postorder-like
Function usually	Accepts extra state	Returns computed state

Recognition:
Need information from ancestors? → Top-Down
Need information from children? → Bottom-Up

12.16. Common Tree Recursive Patterns

Height / Depth

```
return 1 + Math.max(
    height(root.left),
    height(root.right)
);
```

Count Nodes

```
return 1
        + count(root.left)
        + count(root.right);
```

Count Leaves

```
if (root.left == null && root.right == null) {
    return 1;
}
```

Then recursively count leaves in both subtrees.

Search

```
return root.data == target
        || search(root.left, target)
        || search(root.right, target);
```

These patterns show how recursive tree solutions often reduce a large problem into the same problem on smaller subtrees.

12.17. Binary Search Tree — BST

A **Binary Search Tree** is a binary tree with an ordering property.
For every node: All values in left subtree < node, All values in right subtree > node
assuming duplicate values are not allowed.
Example:

```
          50
        /    \
```

CodeWithNishchal

```
      30      70
     /  \    /  \
   20   40  60   80
```

At node 50: Left subtree values < 50, Right subtree values > 50.
The same rule applies recursively to every node.

12.18. Why BST Is Useful

Because values are ordered, we can often discard half of the remaining tree during search.
At node 50, searching for 70: 70 > 50 Therefore the entire left subtree can be ignored.

```
        50
          \
           70
```

This gives efficient average search in a well-balanced BST.

12.19. Search in BST

```
static boolean searchBST(TreeNode root, int target) {
    while (root != null) {
        if (root.data == target) {
            return true;
        }
        if (target < root.data) {
            root = root.left;
        } else {
            root = root.right;
        }
    }
    return false;
}
```

Flow: target == node → found
target < node → go left
target > node → go right
Average in balanced BST:O(log n)
Worst case in skewed BST:O(n)

12.20. BST Insertion

To insert a value:
value < node → left
value > node → right

until a null position is found.
Example: insert 65.

```
        50
```

\
70
/
60

Steps:
65 > 50 → right
65 < 70 → left
65 > 60 → right

Result:
50
\
70
/
60
\
65

Recursive code:

```
static TreeNode insertBST(TreeNode root, int value) {
    if (root == null) {
        return new TreeNode(value);
    }
    if (value < root.data) {
        root.left = insertBST(root.left, value);

    } else if (value > root.data) {
        root.right = insertBST(root.right, value);
    }

    return root;
}
```

If duplicates are allowed, the implementation must define a consistent policy.

12.21. Minimum and Maximum in BST

Because smaller values are always left:
Minimum = leftmost node

```
static TreeNode findMin(TreeNode root) {
    while (root.left != null) {
        root = root.left;
    }
    return root;
}
```

Similarly: Maximum = rightmost node
Time: O(h)

12.22. BST Deletion

Deletion has three important cases.

Case 1: Leaf Node

Delete:

```
    30
   /  \
  20    40
```

If deleting 20:

```
    30
      \
       40
```

Simply remove the leaf reference.

Case 2: Node With One Child

Before:

```
    50
   /
  30
    \
     40
```

Delete 30.
Connect its parent directly to its child:

```
    50
   /
  40
```

Case 3: Node With Two Children

This is the important case.
Example:

```
        50
```

CodeWithNishchal

```
    /   \
   30     70
         / \
        60  80
```

Delete 70. A common method is to replace 70 with its **inorder successor**:
smallest value in right subtree Here: 80
Actually for subtree rooted at 70, the right subtree is rooted at 80, so successor is 80.
Another valid method uses the **inorder predecessor**:
largest value in left subtree

12.22.1 BST Delete Implementation

```
static TreeNode deleteBST(TreeNode root, int key) {
    if (root == null) {
        return null;
    }
    if (key < root.data) {
        root.left = deleteBST(root.left, key);
    } else if (key > root.data) {
        root.right = deleteBST(root.right, key);
    } else {
        // No left child
        if (root.left == null) {
            return root.right;
        }
        // No right child
        if (root.right == null) {
            return root.left;
        }
        // Two children
        TreeNode successor = findMin(root.right);
        root.data = successor.data;
        root.right = deleteBST(root.right, successor.data);
    }
    return root;
}
```

Search for the target using BST ordering. Once found: 0 children → return null
1 child → return the existing child
2 children → copy inorder successor value
→ delete successor from right subtree
The BST ordering remains valid.

12.23. Inorder Traversal of BST

This is one of the most important BST properties.

```
      50
     /   \
```

```
      30      70
     /  \    /  \
   20   40  60   80
```

Inorder traversal gives: 20, 30, 40, 50, 60, 70, 80

Therefore: BST + Inorder → Sorted Order. This fact is used heavily in BST problems.

12.24. BST Complexity

If tree height is h:

Operation	Complexity
Search	O(h)
Insert	O(h)
Delete	O(h)
Minimum	O(h)
Maximum	O(h)
Traversal	O(n)

Balanced BST: h ≈ log n so:Search / Insert / Delete ≈ O(log n) . Skewed BST: h ≈ n so: Search / Insert / Delete = O(n)

12.25. Balanced vs Skewed BST

Balanced:

```
       40
      /  \
     20    60
    / \    / \
   10 30  50 70 Height:O(log n)
```

Skewed:

```
10
  \
   20
     \
      30
        \
         40
```

Height:O(n)

This is why BST does **not automatically guarantee O(log n)**.

12.26. Binary Tree vs BST

Feature	Binary Tree	Binary Search Tree
Max children	2	2

CodeWithNishchal

Ordering rule	None	Left < Root < Right
Search	Usually O(n)	Average O(log n) if balanced
Inorder sorted?	Not necessarily	Yes
Typical purpose	Hierarchical structure	Ordered searching

Every BST is a binary tree, but every binary tree is **not** a BST.

12.27. Tree Operation Complexity Overview

General Binary Tree

Operation	Complexity
DFS traversal	O(n)
BFS traversal	O(n)
Search unsorted tree	O(n)
Count nodes	O(n)
Find height	O(n)
Find leaves	O(n)

BST

Operation	Balanced	Worst / Skewed
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Min / Max	O(log n)	O(n)
Traversal	O(n)	O(n)

CodeWithNishchal

13. Recursion

Recursion is a technique in which a function solves a problem by calling itself on a **smaller version of the same problem** until it reaches a condition where no further recursive call is needed.

A recursive solution usually has two essential parts:

1. Base Case → stops recursion

2. Recursive Case → reduces the problem and calls the function again

Example:

```
static void printNumbers(int n) {
    if (n == 0) {
        return;
    }
    System.out.println(n);
    printNumbers(n - 1);
}
```

printNumbers(3); the calls are:

printNumbers(3)
↓
printNumbers(2)
↓
printNumbers(1)
↓
printNumbers(0)
↓
stop

Output:

3
2
1

The important idea is that every call receives a **smaller problem**: 3 → 2 → 1 → 0 and eventually reaches the base case.

13.1. Base Case and Recursive Case

Consider factorial: 5! = 5 × 4 × 3 × 2 × 1

It can be expressed recursively as: n! = n × (n - 1)! with: 0! = 1

```
static int factorial(int n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Here: Base Case: n == 0, Recursive Case: n * factorial(n - 1)

The base case is not just an optional if. It defines the smallest directly solvable problem and prevents infinite recursion.

13.2. How Recursion Actually Works

Each recursive call creates a separate **stack frame** on the Java call stack.

A stack frame stores information such as:

- Method parameters
- Local variables
- Return location

WithNishchal

- Temporary execution state

```
static void fun(int n) {
    if (n == 0) {
        return;
    }
    System.out.println("Before " + n);
    fun(n - 1);
    System.out.println("After " + n);
}
```

Call: fun(3);
The execution first moves downward:

```
fun(3)
 |
 ├─ print Before 3
 |
 └─ fun(2)
     |
     ├─ print Before 2
     |
     └─ fun(1)
         |
         ├─ print Before 1
         |
         └─ fun(0)
             |
             └─ return
```

Then the suspended calls resume in reverse order:
fun(1) → print After 1
fun(2) → print After 2
fun(3) → print After 3
Output:

```
Before 3
Before 2
Before 1
After 1
After 2
After 3
```

This happens because recursion uses a **LIFO call stack**.

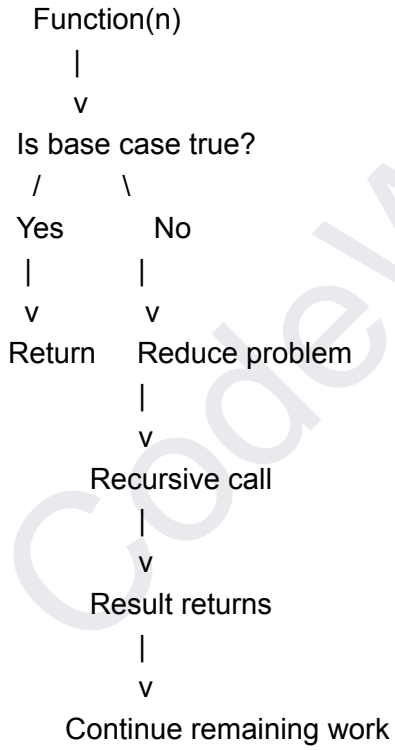
13.3. Call Stack Visual

For: factorial(4); **recursive calls:**
factorial(4)
factorial(3)
factorial(2)

CodeWithNishchal

factorial(1)
factorial(0)
At the deepest point: Call Stack
Top
↓
+----------------+
| factorial(0) |
+----------------+
| factorial(1) |
+----------------+
| factorial(2) |
+----------------+
| factorial(3) |
+----------------+
| factorial(4) |
+----------------+
Now: factorial(0) returns 1
The stack unwinds: factorial(1) = 1 × 1 = 1, factorial(2) = 2 × 1 = 2, factorial(3) = 3 × 2 = 6, factorial(4)= 4 × 6 = 24. So recursion has two phases: Calling Phase → move toward base case
Returning / Unwinding Phase → return results back through previous calls

13.4. Recursion Flow



13.5. Work Before and After Recursive Call

This is one of the most important concepts in recursion.

CodeWithNishchal

```
static void printDescending(int n) {
    if (n == 0) { return;
    }
    System.out.print(n + " ");
    printDescending(n - 1);
}
```

Output: 5 4 3 2 1

The work happens **before recursion**.
Now:

```
static void printAscending(int n) {
    if (n == 0) {
        return;
    }
    printAscending(n - 1);
    System.out.print(n + " ");
}
```

Output: 1 2 3 4 5
Here the printing happens during **stack unwinding**.
Therefore: Work before recursive call → performed while going deeper
Work after recursive call → performed while returning
This distinction becomes extremely important in tree traversal and backtracking.

13.6. Recursive Function With Return Value

Some recursive methods only perform actions: void function(...)
Others return information to the previous call.
Example:

```
static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum(n - 1);
}
```

For: sum(4). expansion:
sum(4) = 4 + sum(3) = 4 + 3 + sum(2) = 4 + 3 + 2 + sum(1) = 4 + 3 + 2 + 1 + sum(0)
= 4 + 3 + 2 + 1 + 0 = 10
The recursive function can therefore be viewed as saying:

> Assume recursion correctly solves the smaller problem; use that result to solve the current problem.

This is one of the best ways to reason about recursive code.

Written With Vishchal

13.7. Three Steps for Writing Recursive Solutions

Step 1 — Define the function's responsibility

Example: factorial(n) means: Return factorial of n. Do not initially worry about how the smaller recursive call works internally.

Step 2 — Identify the base case

factorial(0) = 1

Code:

if (n == 0) { return 1;

}

Step 3 — Reduce to a smaller problem

factorial(n) = n × factorial(n - 1)

Code: return n * factorial(n - 1); The recursive call should always move toward termination.

13.8. Direct Recursion

This is **direct recursion**. When a function directly calls itself:

```
static void fun(int n) {
    if (n == 0) { return;
    }
    fun(n - 1);
}
```

13.9. Indirect Recursion

When one function calls another and eventually the second calls the first:

```
static void funA(int n) {
    if (n <= 0) { return;
    }
    funB(n - 1);
}
static void funB(int n) {
    if (n <= 0) { return;
    }
    funA(n - 1);
}
```

Flow:

funA

↓

funB

↓

funA

↓

...

This is called **indirect or mutual recursion**. It is less common in standard DSA problems but important conceptually.

CodeWithNishchal

13.10. Head Recursion

In **head recursion**, the recursive call happens before the main processing.

```
static void fun(int n) {
    if (n == 0) {
        return;
    }
    fun(n - 1);
    System.out.println(n);
}
```

Output:
1
2
3
...
n
The work is performed during stack unwinding.
Conceptually:
Recursive Call
↓
Remaining Work

13.11. Tail Recursion

In **tail recursion**, the recursive call is the last operation performed.

```
static void fun(int n) {
    if (n == 0) {
        return;
    }
    System.out.println(n);
    fun(n - 1);
}
```

There is no work left after the recursive call returns.
Conceptually:
Work
↓
Recursive Call
↓
Nothing remaining
Some languages optimize tail recursion into iteration, but **Java does not guarantee tail-call optimization**, so deep tail recursion can still cause StackOverflowError.

CodeWithNishchal

13.12. Linear Recursion

A function makes one recursive call per invocation.

```
static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum(n - 1);
}
```

Recursion tree:

```
sum(4)
  |
sum(3)
  |
sum(2)
  |
sum(1)
  |
sum(0)
```

Depth: O(n)

13.13. Multiple / Tree Recursion

A function may make more than one recursive call.

Classic example:

```
static int fibonacci(int n) {
    if (n <= 1) {  return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Recursion tree:

```
                fib(5)
               /      \
          fib(4)      fib(3)
          /   \       /   \
       fib(3) fib(2) fib(2) fib(1)
       / \
     ...
```

This creates a branching recursion tree. Naive Fibonacci repeatedly calculates the same values:

fib(3)

fib(2)

...

many times.

Therefore its time complexity is exponential: O(2^n) approximately

while recursion depth is:O(n). This repeated-work problem motivates **memoization / dynamic programming**.

Code with Nishchal

13.14. Recursion Tree

A recursion tree visualizes how recursive calls expand.
Consider:

```
void solve(int n) {
    if (n == 0) {
        return;
    }
    solve(n - 1);
    solve(n - 1);
}
```

Tree:

```
            solve(n)
            /      \
       solve(n-1)   solve(n-1)
        /   \        /   \
      ...   ...    ...   ...
```

If branching factor is b and depth is d, the total number of calls can roughly behave like:
1 + b + b² + ... + b^d
which is why branching recursion can become very expensive.

13.15. Recursion on Arrays

Recursive traversal of an array can use an index.

```
static void printArray(int[] arr, int index) {
    if (index == arr.length) {
        return;
    }
    System.out.println(arr[index]);
    printArray(arr, index + 1);
}
```

The state is: index
Each recursive call processes one smaller remaining portion:
index 0 → remaining n elements
index 1 → remaining n-1
index 2 → remaining n-2
...

Complexity: Time = O(n), Space = O(n)
because the recursive call stack has depth n.
An iterative loop would use only: O(1)
extra space, so recursion is not automatically better.

CodeWithNishchal

13.16. Recursion on Strings

Example: checking whether a String is a palindrome.

```
static boolean isPalindrome(
        String str,
        int left,
        int right) {
    if (left >= right) {
        return true;
    }
    if (str.charAt(left) != str.charAt(right)) {
        return false;
    }
    return isPalindrome(
            str,
            left + 1,
            right - 1
    );
}
```

Visual:

```
"racecar"
r a c e c a r
↑         ↑
L         R
r == r
  a       a
  ↑       ↑
a == a
    c   c
    ↑   ↑
c == c
      e
      ↑
```

left >= right
→ true

Time: O(n)

Call stack:
O(n)

more precisely roughly O(n/2), which simplifies to O(n).

CodeWithNishchal

13.17. Top-Down Recursion

In top-down recursion, information is passed from parent calls to child calls.
General form:

```
void dfs(Node node, State state) {
    if (node == null) {
        return;
    }
    State nextState = update(state, node);
    dfs(node.left, nextState);
    dfs(node.right, nextState);
```

Examples of downward state:
Current depth, Path sum, Current path , Remaining target
Maximum value seen so far
Visual:

```
        Parent
       state = 5
       /      \
      ↓        ↓
  state = 6   state = 6
```

Think: "What information must my children know?"

13.18. Bottom-Up Recursion

Bottom-up recursion collects information from recursive calls and returns a result upward.
General form:

```
Result solve(Node node) {
    if (node == null) {
        return baseResult;
    }
    Result left = solve(node.left);
    Result right = solve(node.right);
    return combine(left, right);
}
```

Examples:
Tree height
Subtree size
Tree balance
Diameter
Maximum path information
Think:
"What information do I need from my children
to solve the current node?"

WithNishchal

13.19. Binary Search Using Recursion

```
static int binarySearch(
        int[] arr,
        int left,
        int right,
        int target) {
    if (left > right) {
        return -1;
    }
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) {
        return mid;
    }
    if (target < arr[mid]) {
        return binarySearch(
                arr,
                left,
                mid - 1,
                target
        );
    }
    return binarySearch(
            arr,
            mid + 1,
            right,
            target
    );
}
```

Each call halves the search space:
n
↓
n/2
↓
n/4
↓
n/8
...

Therefore: Time = O(log n), Space = O(log n)
The iterative version has the same time but only O(1) extra space.

Code Nishchal

13.20. Recurrence Relation

A **recurrence relation** describes the running time of a recursive algorithm in terms of smaller inputs. Example: fun(n - 1);

with constant work per call gives: T(n) = T(n - 1) + O(1) which results in: O(n)

Binary Search: T(n) = T(n/2) + O(1) results in: O(log n)

Merge Sort: T(n) = 2T(n/2) + O(n) results in: O(n log n)

Naive Fibonacci: T(n) ≈ T(n - 1) + T(n - 2) + O(1) is exponential.

You do not always need to formally solve a recurrence, but understanding the number of calls and recursion depth is essential.

13.21. Time Complexity of Recursive Functions

To analyze recursion, separate two questions:

1. How many function calls are made?
2. How much work does each call perform?

Then estimate:Total Time ≈ Number of Calls × Work per Call

Examples:

Recursion	Calls / Structure	Time
f(n-1)	Linear chain	O(n)
f(n/2)	Halving chain	O(log n)
Two calls on halves	Balanced recursion tree	Often O(n) or O(n log n) depending on extra work
f(n-1) + f(n-2)	Branching tree	Exponential

Do not determine recursive complexity only by looking at the number of recursive statements. Their input sizes and repeated work matter.

13.22. Space Complexity of Recursion

Recursive algorithms use call-stack memory. If maximum recursion depth is: d then call-stack space is typically: O(d)

Algorithm	Maximum Depth	Stack Space
Factorial	n	O(n)
Linear array recursion	n	O(n)
Binary Search	log n	O(log n)
Balanced tree DFS	log n	O(log n)
Skewed tree DFS	n	O(n)

Important: Time complexity depends on **total calls**, while stack space depends on the **maximum number of active calls at one time**. These are not the same thing.

CodeWithNishchal

13.23. Infinite Recursion

A base case alone does not guarantee termination.

```
static void fun(int n) {
    if (n == 0) { return;
    }
    fun(n + 1);
}
```

Starting from: n = 5 produces: 5 → 6 → 7 → 8 → ...

It moves **away** from the base case. A correct recursive solution requires: Base Case + Progress toward Base Case

13.24. Recursion vs Iteration

Many recursive algorithms can also be written using loops or explicit stacks.

Feature	Recursion	Iteration
Control mechanism	Function calls	Loops
Extra call stack	Yes	Usually no
Code for recursive structures	Often cleaner	Can be more verbose
Deep execution	Risk of stack overflow	Usually safer
Trees / DFS	Very natural	Explicit stack required
Simple linear repetition	Often unnecessary	Usually preferred

Example:

Factorial recursively:

```
static int factorial(int n) {
    if (n <= 1) {  return 1;
    }
    return n * factorial(n - 1);
}
```

Iteratively:

```
static int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Both take: O(n) time but: Recursive → O(n) stack space and Iterative → O(1) extra space

Use recursion when the recursive structure makes the solution clearer or naturally matches the problem.

Nishchal

14. Backtracking

Backtracking is a recursive problem-solving technique used when we need to explore multiple possible choices and build a solution step by step.

The main idea is:

Choose

↓

Explore

↓

Undo the choice

↓

Try another choice

It is especially useful when the problem asks to generate or search through possibilities such as:

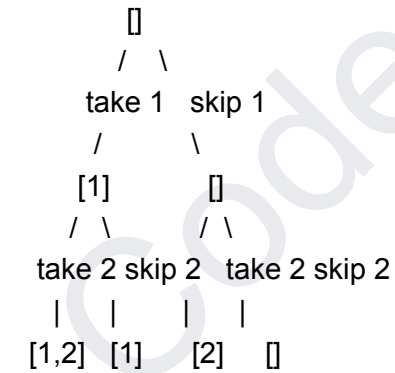
- subsets,
- permutations,
- combinations,
- paths,
- board configurations,
- valid arrangements,
- constraint-based solutions.

Backtracking is essentially **DFS on a decision tree**.

14.1. Decision Tree Idea

Suppose we have: [1, 2]. For every element, we may either: Take it or Skip it

Decision tree:



Backtracking performs DFS through this tree.

14.2. Core Backtracking Structure

Most backtracking problems contain three main parts:

1. Base Case
2. Make a Choice
3. Undo the Choice

CodeWithNishchal

```
void backtrack(...) {
    if (baseCase) {
        // store/process solution
        return;
    }
    for (Choice choice : choices) {
        // choose
        makeChoice(choice);
        // explore
        backtrack(...);
        // undo
        undoChoice(choice);
    }
}
```

The most important line conceptually is: Choose → Recurse → Unchoose

14.3. Why Undo Is Necessary

Suppose: current = [1, 2]. After exploring this branch, we want to try: [1, 3]
Before choosing 3, we must remove 2.

```
[1, 2]
  ↓ undo 2
[1]
  ↓ choose 3
[1, 3]
```

If we do not undo the previous choice, state from one branch incorrectly leaks into another.

14.4. Backtracking vs Normal Recursion

Every backtracking solution uses recursion, but not every recursive solution is backtracking.

Normal Recursion

Usually computes one recursive result.
Example: height(left) height(right) combine

Backtracking

Explores multiple choices and returns to earlier states.

```
choice 1
  ↓
explore
  ↓
undo

choice 2
  ↓
explore
```

The key feature is: state modification + restoration

CodeWithNishchal

14.5. Backtracking vs Brute Force

Backtracking is closely related to brute force because it may still explore many possibilities.

The improvement comes from **pruning**.

Brute force:

Generate everything

Then check validity

Backtracking:

Build partial solution

|

v

Is it already invalid?

|

Yes

|

v

Stop this branch immediately

This avoids exploring impossible branches.

14.6. Pruning

Pruning means stopping recursion when the current partial solution can no longer lead to a valid answer. Example: Suppose target sum is 7. Current path: [3, 5] sum = 8

If all remaining numbers are positive: 8 > 7

there is no reason to continue this branch.

Prune

```
        []
       /  \
     [3]   ...
      |
    [3,5]
      |
  sum = 8 > 7
     X
 branch stopped
```

Good pruning can drastically reduce the practical search space.

14.7. State in Backtracking

A backtracking function usually tracks some combination of:

Current path/ Current index/ Remaining target/ Visited elements/ Board state/ Current sum/ Current position

Example:

List<Integer> path = new ArrayList<>();

The path represents the partial solution currently being built.

CodeWithNishchal

14.8. Subsets Pattern

Given:[1, 2, 3]

generate all subsets.
Result:
[]
[1]
[2]
[3]
[1,2]
[1,3]
[2,3]
[1,2,3]
For each element, there are two choices: include and exclude
Therefore total subsets: 2^n

14.8.1 Subsets Using Backtracking

```
static void generateSubsets(
        int[] nums,
        int index,
        List<Integer> current,
        List<List<Integer>> result) {
    if (index == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }
    // Include
    current.add(nums[index]);
    generateSubsets(
            nums,
            index + 1,
            current,
            result
    );
    // Undo
    current.remove(current.size() - 1);
    // Exclude
    generateSubsets(
            nums,
            index + 1,
            current,
            result
    );
}
```

Logic

At every index: Take nums[index] or Skip nums[index]
When all elements have been considered, the current path is one complete subset.
Important: new ArrayList<>(current) creates a copy.
If we store current directly, later modifications would affect previously stored results.

14.8.2 Alternative Subset Template

A common compact template stores every partial path immediately.

```
static void subsets(
        int[] nums,
        int start,
        List<Integer> current,
        List<List<Integer>> result) {
    result.add(new ArrayList<>(current));
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        subsets(nums, i + 1, current, result);
        current.remove(current.size() - 1);
    }
}
```

Here: start
ensures earlier elements are not chosen again.
Complexity: Time = O(n × 2^n) Space = O(n) recursion/path
The extra n in time comes from copying subsets into the result.

14.9. Permutation Pattern

A permutation changes the **order** of elements. For: [1, 2, 3]
permutations are:
[1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]
[3,1,2]
[3,2,1]
Total: n!
Unlike subsets:
Subset → choose elements
Permutation → choose position/order

CodeWithNishchal

14.9.1 Permutations With used[]

```
static void permutations(
        int[] nums,
        boolean[] used,
        List<Integer> current,
        List<List<Integer>> result) {

    if (current.size() == nums.length) {
        result.add(new ArrayList<>(current));
        return;
    }

    for (int i = 0; i < nums.length; i++) {

        if (used[i]) {
            continue;
        }

        used[i] = true;
        current.add(nums[i]);

        permutations(nums, used, current, result);

        current.remove(current.size() - 1);
        used[i] = false;
    }
}
}
```

Logic

At every position, choose any unused element.
Example:
Current: [1]
Available: 2, 3
Choose 2: [1,2]

After exploring: undo 2
then try: [1,3]
Complexity:
Time ≈ O(n × n!)
Space = O(n)

excluding stored output.

Code Nishchal

14.10. Combination Pattern

A combination chooses a fixed number of elements where order does not matter.

Example:

Choose 2 from [1,2,3,4]

Possible combinations:

[1,2]
[1,3]
[1,4]
[2,3]
[2,4]
[3,4]

[1,2] and [2,1] are the same combination, so only one is generated. The key is: Move forward using start index

---

14.10.1 Combination Template

```
static void combinations(
        int[] nums,
        int start,
        int k,
        List<Integer> current,
        List<List<Integer>> result) {

    if (current.size() == k) {
        result.add(new ArrayList<>(current));
        return;
    }
    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        combinations(
                nums,
                i + 1,
                k,
                current,
                result
        );
        current.remove(current.size() - 1);
    }
}
```

The important difference from permutation is:

Combination: next recursion starts from i + 1

Permutation:

can consider all unused elements

Nishchal

14.11. Combination Sum Pattern

Given an array of candidate numbers and a target value, find all unique combinations whose sum is exactly equal to the target. The same number can be used multiple times.

Example: Candidates = [2, 3, 6, 7] Target = 7
Valid combinations: [2, 2, 3], [7]

The order of elements inside a combination does not matter, so combinations like:
[2, 3, 2] and [3, 2, 2] should not be treated as separate answers.The goal is to generate every valid combination using backtracking.

```
static void combinationSum(
        int[] nums,
        int start,
        int remaining,
        List<Integer> current,
        List<List<Integer>> result) {
    if (remaining == 0) {
        result.add(new ArrayList<>(current));
        return;
    }
    if (remaining < 0) { return;
    }

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);
        combinationSum(
                nums,
                i,
                remaining - nums[i],
                current,
                result
        );
        current.remove(current.size() - 1);
    }
}
```

Why pass i instead of i + 1?
Because:
same element may be reused

If reuse were not allowed:
i + 1

would be used.

14.12. Subset vs Combination vs Permutation

Pattern	Order Matters?	Reuse Usually Allowed?	Main State
Subset	No	No	index/start
Combination	No	Depends	start
Permutation	Yes	No	used[]
Combination Sum	No	Often yes	start + remaining

14.13. Handling Duplicates

Suppose: [1, 2, 2]
Naive backtracking may generate duplicate subsets.
A common solution:

1. Sort the array.
2. Skip duplicate values at the same recursion level.

Arrays.sort(nums);
Then:

```
if (i > start && nums[i] == nums[i - 1]) {
   continue;
}
```

Example:

```
static void uniqueSubsets( int[] nums, int start, List<Integer> current, List<List<Integer>> result)
{
    result.add(new ArrayList<>(current));
    for (int i = start; i < nums.length; i++) {
        if (i > start && nums[i] == nums[i - 1]) {
            continue;
        }
        current.add(nums[i]);
        uniqueSubsets(
                nums,
                i + 1,
                current,
                result
        );
        current.remove(current.size() - 1);
    }
}
```

The condition: i > start
is important because duplicates are skipped only among choices at the **same decision level**.

14.14. Grid Backtracking

Backtracking is also common in matrix and grid problems.
Typical movement: up, down, left, right
A cell is often:
choose → mark visited
explore neighbors
undo → unmark visited
General template:

```
void dfs(int row, int col) {
    if (invalid(row, col)) {
        return;
    }
    visited[row][col] = true;

    dfs(row + 1, col);
    dfs(row - 1, col);
    dfs(row, col + 1);
    dfs(row, col - 1);

    visited[row][col] = false;
}
```

Whether we undo visited depends on the problem. If cells may be reused in different search paths, unmarking is required.

14.15. Word Search Pattern

Given a character board and a word, determine whether the word can be formed by adjacent cells.
Example:
A B C E
S F C S
A D E E
Search:
"ABCCED"
State includes: row, column, word index, visited cells
At each step:
Does board[row][col] match word[index]?
If not: return false
If yes: mark cell
explore neighbors
unmark cell

This is a classic example of:
Choose → Explore → Undo

Codewith

14.16. N-Queens Pattern

The N-Queens problem asks us to place N queens on an N × N chessboard such that **no two queens can attack each other**.

A queen can attack another queen if they are in the same:

- Row
- Column
- Main diagonal
- Anti-diagonal

Example for N = 4:

. Q . .
. . . Q
Q . . .
. . Q .

Here, no two queens share the same row, column, or diagonal. The goal is to find valid arrangements of N queens.

Approach — Backtracking

Place queens **row by row**.

Since we place exactly one queen in each row, we do not need to check row conflicts.

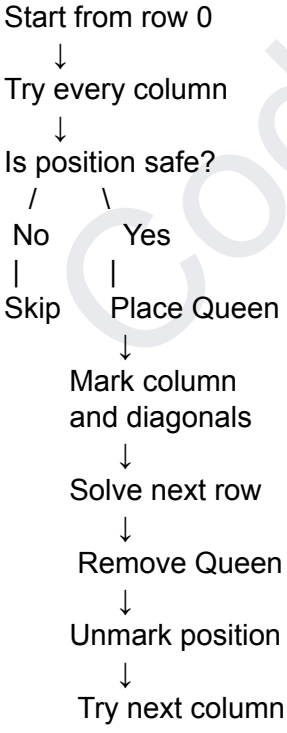
For every row:

1. Try placing a queen in every column.
2. Check whether that column or diagonal is already occupied.
3. If the position is safe:
    - Place the queen.
    - Mark its column and diagonals as occupied.
    - Recursively move to the next row.
4. After recursion returns:
    - Remove the queen.
    - Unmark the column and diagonals.
5. Try the next column.

If: row == n

then queens have been successfully placed in all rows, so a valid arrangement has been found.

Main Flow



CodeWithNishchal

Why Backtracking Works

Suppose we place a queen and later discover that no valid placement is possible in the next rows.
We do not keep that wrong choice.
Instead:
Place queen
↓
Explore
↓
No solution possible
↓
Remove queen
↓
Try another position
This is the core idea of backtracking:
Choose → Explore → Undo

14.16.1 Tracking Conflicts Efficiently

Instead of scanning the whole board every time, use three boolean arrays:
boolean[] columns;
boolean[] diag1;
boolean[] diag2;
They tell us in O(1) time whether a position is safe.

---

1. Column Check

For position: (row, col)
we check: columns[col]
If it is true, another queen already exists in that column.

---

2. Main Diagonal

Cells on the same diagonal \ have the same: row - col
Example:
(0,0) → 0 - 0 = 0
(1,1) → 1 - 1 = 0
(2,2) → 2 - 2 = 0
(3,3) → 3 - 3 = 0
So all these cells belong to the same diagonal.
But: row - col can be negative.
Therefore we shift it by: n - 1
So: int d1 = row - col + n - 1;
This makes every diagonal index non-negative.

---

3. Anti-Diagonal

Cells on the same diagonal / have the same: row + col
Example:
(0,3) → 0 + 3 = 3
(1,2) → 1 + 2 = 3
(2,1) → 2 + 1 = 3
(3,0) → 3 + 0 = 3
Therefore:
int d2 = row + col;

CodeWithNishchal

Valid Position Check

A queen can be placed only if all three are free:

```
if (columns[col] || diag1[d1] || diag2[d2]) {
    continue;
}
```

Meaning:

```
Column occupied?
        OR
Main diagonal occupied?
        OR
Anti-diagonal occupied?
        ↓
Do not place queen
```

14.16.2 Code

```
static void solve(
        int row,
        char[][] board,
        boolean[] columns,
        boolean[] diag1,
        boolean[] diag2) {

    int n = board.length;
    // All rows successfully filled
    if (row == n) {

        // Valid board found
        printBoard(board);
        System.out.println();

        return;
    }

    // Try every column in the current row
    for (int col = 0; col < n; col++) {

        int d1 = row - col + n - 1;
        int d2 = row + col;

        // Position is under attack
        if (columns[col] || diag1[d1] || diag2[d2]) {
            continue;
        }

        // Choose
        board[row][col] = 'Q';
        columns[col] = true;
        diag1[d1] = true;
        diag2[d2] = true;
```

Vishchal

```
        // Explore next row
        solve(row + 1, board, columns, diag1, diag2);

        // Backtrack
        board[row][col] = '.';

        columns[col] = false;
        diag1[d1] = false;
        diag2[d2] = false;
    }
}
```

Helper method:

```
static void printBoard(char[][] board) {

    for (char[] row : board) {

        for (char cell : row) {
            System.out.print(cell + " ");
        }

        System.out.println();
    }
}
```

Calling the algorithm:

```
static void solveNQueens(int n) {

    char[][] board = new char[n][n];

    for (int i = 0; i < n; i++) {
        Arrays.fill(board[i], '.');
    }
    boolean[] columns = new boolean[n];
    boolean[] diag1 = new boolean[2 * n - 1];
    boolean[] diag2 = new boolean[2 * n - 1];
    solve(
            0,
            board,
            columns,
            diag1,
            diag2
    );
}
```

Complexity

Time = O(N!) approximately in the worst case due to backtracking.
Conflict checking is: O(1)
because of the boolean arrays.
Extra space: $O(N^2)$ for the board
plus O(N) for columns, diagonals, and recursion depth.

vishchal

14.17. Backtracking With In-Place Swap

Permutations can also be generated without used[] by swapping elements.

```
static void permute(int[] arr, int index) {
    if (index == arr.length) {
        // process permutation
        return;
    }
    for (int i = index; i < arr.length; i++) {
        swap(arr, index, i);
        permute(arr, index + 1);
        swap(arr, index, i); // undo
    }
}
```

Visual: [1,2,3]
swap index 0 with 1
↓
[2,1,3]
explore
swap back
↓
[1,2,3]
The second swap restores the original state.

14.18. Backtracking Complexity

Backtracking complexity depends on the size of the search tree.
There is no single complexity such as: O(n)
for all backtracking problems.
Typical examples:

Problem	Approximate Search Space
Subsets	2^n
Permutations	n!
Choose k elements	C(n,k)
N-Queens	Roughly factorial-scale search
Grid path search	Often exponential

General form: Time ≈ Number of states explored × work per state
Recursion-space complexity is usually: O(depth of decision tree)
For subsets/permutations: O(n)
excluding result storage.

CodeWithNishchal

14.19. Output Complexity

When a problem asks to generate all solutions, the output itself may be exponential.
Example: n elements → 2^n subsets
Therefore no algorithm can generate every subset in O(n) time.
This is important when analyzing backtracking algorithms:
Large complexity is sometimes unavoidable
because the number of valid answers is itself large.

14.20. Backtracking vs DFS

Backtracking usually uses DFS, but the concepts are not identical.

DFS

Means:
Explore depth-first

Backtracking

Means:
Explore depth-first + modify state + undo state + try alternate choices
So: Backtracking is commonly implemented using DFS.

14.21. Backtracking vs Dynamic Programming

These two techniques are often confused.

Backtracking

Explores possibilities.
Try choice
Explore
Undo

Dynamic Programming

Avoids solving repeated states again.
State
→ compute once
→ store result
→ reuse
If a recursive backtracking problem repeatedly reaches the exact same state, memoization may sometimes convert it into a DP-style solution.

14.22. Backtracking vs Greedy

Greedy:
Make the locally best choice
and usually never undo it
Backtracking:
Try a choice
If it fails, undo it
Try another choice Therefore: Greedy → commit Backtracking → explore alternatives

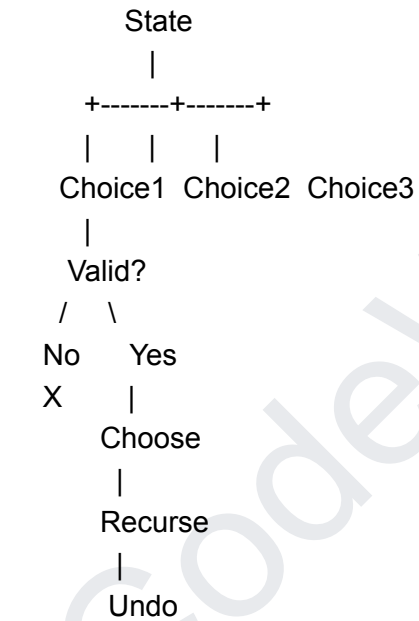
CodeWithNishchal

14.23. Standard Backtracking Template

A practical template to remember:

```
void backtrack(State state) {
    if (solutionFound(state)) {
        saveSolution(state);
        return;
    }
    for (Choice choice : possibleChoices(state)) {
        if (!isValid(choice, state)) {
            continue;
        }
        makeChoice(state, choice);
        backtrack(state);
        undoChoice(state, choice);
    }
}
```

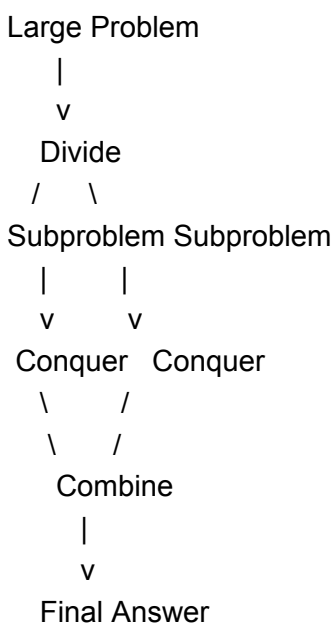
Conceptually:



CodeWithNishchal

15. Divide and Conquer

Divide and Conquer is an algorithm-design technique in which a large problem is broken into smaller subproblems of the same type, those subproblems are solved independently, and their results are combined to form the final answer. The core flow is:



The three essential stages are:

Divide → Break the problem into smaller subproblems

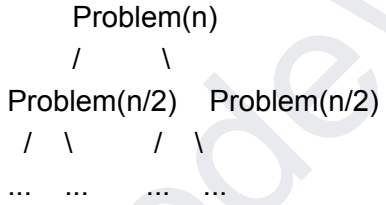
Conquer → Solve those subproblems, usually recursively

Combine → Merge their results into the original solution

A base case handles problems small enough to solve directly.

15.1. General Structure

Suppose a problem of size n is divided into two problems of roughly size n/2.




```
Result solve(Problem problem) {
    if (smallEnough(problem)) {
        return directSolution(problem);
    }
    Problem left = divideLeft(problem);
    Problem right = divideRight(problem);
    Result leftResult = solve(left);
    Result rightResult = solve(right);
    return combine(leftResult, rightResult);
}
```

Not every Divide and Conquer algorithm requires a difficult combine phase. In **Binary Search**, for example, only one half is recursively explored, so combination is essentially trivial.

CodeWithNishchal

15.2. Base Case

Every recursive Divide and Conquer algorithm must eventually reach a problem that can be solved directly.

Example in Merge Sort:

Array of size 1 → already sorted

Example in Binary Search:

Search range becomes empty → target does not exist

Without a proper base case, recursion would continue indefinitely.

15.3. Divide Phase

The original problem is divided into smaller subproblems.

Examples:

Merge Sort

→ divide array into left and right halves

Binary Search → compare middle element and keep one half

Quick Sort → partition elements around a pivot

A good division should make the subproblems significantly smaller so recursion progresses toward the base case.

15.4. Conquer Phase

Each smaller subproblem is solved.

If it is still large:

apply Divide and Conquer again

If it has reached the base case:

solve directly

Example:

[8, 3, 5, 1]

divide

[8, 3] [5, 1]

divide divide

[8] [3] [5] [1]

base cases reached

15.5. Combine Phase

After smaller problems are solved, their answers may need to be combined.

In Merge Sort:

Sorted left = [3, 8]

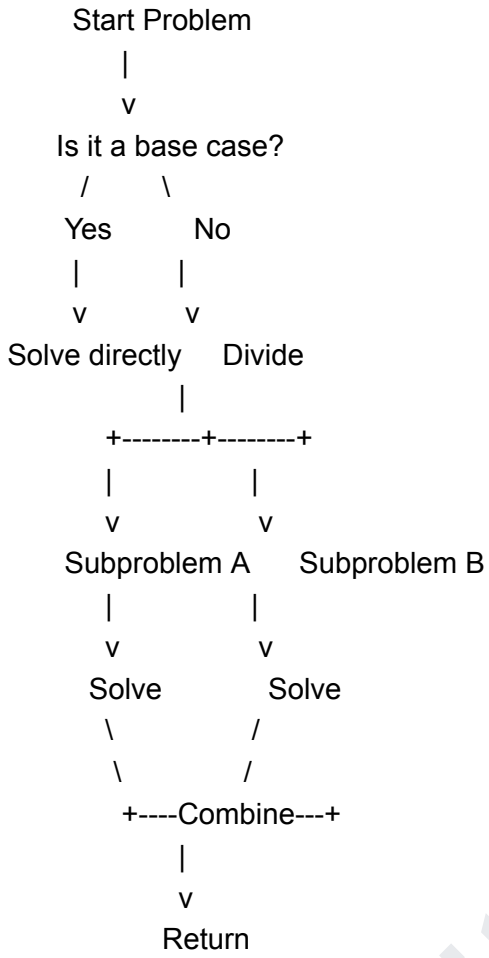
Sorted right = [1, 5]

Combine: [1, 3, 5, 8]

The cost of this combined step often has a major effect on overall complexity.

CodeWithNishchal

15.6. Complete Divide and Conquer Flow



15.7. Divide and Conquer vs Recursion

The two concepts are related but not identical.

Recursion is a programming technique where a function calls itself.

Divide and Conquer is an algorithm-design strategy.

A Divide and Conquer algorithm is often implemented recursively, but recursion can also be used for many other purposes.

Recursion
├── Divide and Conquer
├── Tree DFS
├── Backtracking
├── Dynamic Programming
└── Mathematical recursion

Example:
factorial(n - 1)

uses recursion but is not normally considered Divide and Conquer because it does not divide the problem into multiple independent portions.

CodeWithNishchal

15.8. Binary Search — Divide and Conquer

Binary Search searches a sorted array by repeatedly eliminating half of the search space.

Example:

Array:

[2, 5, 8, 12, 16, 23, 38]

Target = 23

First:

```
2   5   8   12   16   23   38
          ↑
         mid
```

Since:

23 > 12

discard the entire left half.

```
16   23   38
     ↑
```

Continue until the target is found.

Recursive Implementation

```
static int binarySearch(
        int[] arr,
        int left,
        int right,
        int target) {
    if (left > right) { return -1;
    }
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) { return mid;
    }
    if (target < arr[mid]) {
        return binarySearch(
                arr,
                left,
                mid - 1,
                target
        );
    }
    return binarySearch(
            arr,
            mid + 1,
            right,
            target
    );
}
```

Vishchal

Divide and Conquer View

Divide → Split search space around mid

Conquer → Search only the relevant half

Combine → No real merge required

Each recursive call reduces:

n

↓

n/2

↓

n/4

↓

n/8

Therefore: Time = O(log n) Space = O(log n) recursive stack

The iterative version uses O(1) extra space.

15.9. Merge Sort

Given an unsorted array, sort it in ascending order using the **Merge Sort** algorithm.

Merge Sort follows the **Divide and Conquer** technique:

Divide → Solve smaller parts → Merge them

Example: Input = [8, 3, 5, 1, 7, 4, 6, 2]

Output: [1, 2, 3, 4, 5, 6, 7, 8]

Approach — Divide and Conquer

Merge Sort works in two main phases:

1. Divide

Repeatedly divide the array into two halves until every part contains only one element.

A single element is already considered sorted.

Example:

```
[8 3 5 1 7 4 6 2]
       /              \
 [8 3 5 1]           [7 4 6 2]
   /     \             /     \
[8 3]   [5 1]       [7 4]   [6 2]
 / \     / \         / \     / \
[8][3] [5][1]       [7][4] [6][2]
```

2. Merge

Now combine the smaller sorted parts.

While merging two sorted sections:

1. Compare the current element of both sections.
2. Copy the smaller element into a temporary array.
3. Move the pointer of the section from which the element was taken.
4. Continue until one section finishes.
5. Copy any remaining elements.
6. Copy the merged result back into the original array.

CodeWithNishchal

Example:
[8] [3]
↓
[3, 8]

[5] [1]
↓
[1, 5]

[3, 8] + [1, 5]
↓
[1, 3, 5, 8]
Eventually:
[1, 2, 3, 4, 5, 6, 7, 8]

Algorithm

mergeSort()

1. If the section contains one or zero elements, return.
2. Find the middle index.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.

Short flow:
Find middle
↓
Sort left half
↓
Sort right half
↓
Merge both sorted halves

merge()

Suppose: Left = [2, 6, 8] and Right = [1, 4, 7]
Use two pointers:
i → left section
j → right section
Compare:
2 vs 1 → take 1
2 vs 4 → take 2
6 vs 4 → take 4
6 vs 7 → take 6
8 vs 7 → take 7
Left remaining → take 8
Result:[1, 2, 4, 6, 7, 8]
Main idea:
Compare → Take smaller → Move pointer → Repeat

CodeWithNishchal

Code

```
static void mergeSort(int[] arr, int left, int right) {
    if (left >= right) {
        return;
    }
    int mid = left + (right - left) / 2;
    // Sort left half
    mergeSort(arr, left, mid);
    // Sort right half
    mergeSort(arr, mid + 1, right);
    // Merge both sorted halves
    merge(arr, left, mid, right);
}
static void merge(
        int[] arr,
        int left,
        int mid,
        int right) {
    int[] temp = new int[right - left + 1];
    int i = left;
    int j = mid + 1;
    int k = 0;
    // Compare elements from both halves
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    // Copy remaining left-half elements
    while (i <= mid) {
        temp[k++] = arr[i++];
    }
    // Copy remaining right-half elements
    while (j <= right) {
        temp[k++] = arr[j++];
    }
    // Copy merged result back to original array
    for (int x = 0; x < temp.length; x++) {
        arr[left + x] = temp[x];
    }
}
```

Vishchal

Calling Merge Sort:
mergeSort(arr, 0, arr.length - 1);

How the Code Works

mergeSort() keeps dividing the array until only single-element sections remain.
Then merge() combines those sections in sorted order.
For every merge:
Left sorted section
+
Right sorted section
↓
Compare their front elements
↓
Take smaller element
↓
Move corresponding pointer
↓
Continue until both sections finish

Why Merge Sort Is O(n log n)

The array is divided into halves repeatedly.
Number of division levels: $\log_2 n$
At every level, all elements are processed during merging.
So work per level is: O(n)
Number of levels: O(log n)
Therefore: O(n) × O(log n) Time = O(n log n)

Complexity

Best Case = O(n log n)
Average Case = O(n log n)
Worst Case = O(n log n)
Space = O(n)
The extra space is required for the temporary array used during merging.

Easy Memory

Merge Sort
Divide array into halves
↓
Continue until single elements
↓
Merge sorted halves
↓
Compare smaller elements
↓
Build sorted array
Pattern: Divide → Recursively Sort → Merge

CodeWithNishchal

15.10. Quick Sort

Given an unsorted array, sort it in ascending order using the **Quick Sort** algorithm.

Quick Sort follows the **Divide and Conquer** approach.

Instead of dividing the array exactly into equal halves, it selects a **pivot** and rearranges the array so that:

Smaller/Equal Values | Pivot | Larger Values

Example: Input = [7, 2, 9, 4, 3, 8]

Output: [2, 3, 4, 7, 8, 9]

After partitioning, the pivot reaches its correct sorted position. Then the left and right parts are sorted recursively.

Approach — Divide and Conquer + Partitioning

Quick Sort works in three main steps:

1. Choose a pivot.

In Lomuto Partitioning, the last element is usually selected:

pivot = arr[high]

2. Partition the array.

Move all values smaller than or equal to the pivot to the left side and larger values to the right side.

After partitioning:

Smaller/Equal | Pivot | Larger

The pivot is now in its final sorted position.

3. Recursively sort the left and right parts.

Sort:

low ... pivotIndex - 1 and: pivotIndex + 1 ... high

The pivot itself does not need to be processed again.

Main idea: Choose Pivot → Partition → Place Pivot Correctly → Sort Left → Sort Right

Algorithm — Quick Sort

1. If low >= high, return because the current part has at most one element.
2. Partition the current section of the array.
3. Store the returned pivot position: pivotIndex
4. Recursively sort the left side: low to pivotIndex - 1
5. Recursively sort the right side: pivotIndex + 1 to high

Short flow:

Choose pivot

↓

Partition array

↓

Pivot reaches correct position

↓

Sort left side

↓

Sort right side

CodeWithNishchal

Algorithm — Lomuto Partition

Use:

pivot = arr[high]

Maintain two indexes:

i → end of the region containing values <= pivot

j → scans the array

Initially:

i = low - 1

For every j from low to high - 1:

1. Check:

arr[j] <= pivot

2. If true:
    - Increment i
    - Swap arr[i] and arr[j]
3. Continue until all elements are checked.
4. Finally, swap:

arr[i + 1] and arr[high]

5. Return:

i + 1

This is the final position of the pivot.

Partition Visual

During traversal:

<= Pivot | > Pivot | Unprocessed | Pivot

```
        ↑       ↑
        i       j
```

After partitioning:

<= Pivot | Pivot | > Pivot

Example

Consider:

Array = [7, 2, 9, 4, 3, 8]

Pivot:

8

Process each element:

7 <= 8 → keep in left region

2 <= 8 → keep in left region

9 > 8 → leave in right region

4 <= 8 → move to left region

3 <= 8 → move to left region

Before placing pivot:

[7, 2, 4, 3, 9, 8]

Now place 8 after all smaller values:

[7, 2, 4, 3] | 8 | [9]

Now 8 is in its final sorted position.

Then Quick Sort recursively sorts:

[7, 2, 4, 3] and [9]

CodeWithNishchal

```
static void quickSort(int[] arr, int low, int high) {
    if (low >= high) {
        return;
    }
    int pivotIndex = partition(arr, low, high);
    quickSort(arr, low, pivotIndex - 1);
    quickSort(arr, pivotIndex + 1, high);
}
static int partition(int[] arr, int low, int high)
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}
```

Calling Quick Sort: quickSort(arr, 0, arr.length - 1);

Complexity

Best / Average Case

If the pivot divides the array into reasonably balanced parts: Time = O(n log n)

Worst Case

If the pivot repeatedly creates partitions of size: 0 and n - 1

then: Time = $O(n^2)$ Example:

Using the last element as pivot on an already sorted array can cause this worst case.

Case	Complexity
Best Time	O(n log n)
Average Time	O(n log n)
Worst Time	$O(n^2)$
Average Recursion Space	O(log n)
Worst Recursion Space	O(n)

Quick Sort does not require a separate O(n) temporary array like Merge Sort, but recursion still uses stack space.

CodeWithVishchal

15.11. Finding Maximum Using Divide and Conquer

Given an integer array, find the **maximum element** using the **Divide and Conquer** approach.
Example: Array = [7, 2, 9, 4, 6]
Output: Maximum = 9

Algorithm — Divide and Conquer

1. If the current part contains only one element: left == right

return that element because it is automatically the maximum of that part.

2. Find the middle index: mid = left + (right - left) / 2
3. Recursively find the maximum element in the left half: left ... mid
4. Recursively find the maximum element in the right half: mid + 1 ... right
5. Compare both results: Math.max(leftMax, rightMax)
6. Return the larger value.

Short Flow

Split array into two halves
↓
Find maximum in left half
↓
Find maximum in right half
↓
Compare both maximums
↓
Return larger value

Example:

[7, 2, 9, 4, 6]
↓ Split
[7, 2, 9] [4, 6]
↓ ↓
leftMax = 9 rightMax = 6
↓ Compare
max(9, 6)
↓
9

```
static int findMax(int[] arr, int left, int right) {
    if (left == right) {
        return arr[left];
    }
    int mid = left + (right - left) / 2;
    int leftMax = findMax(arr, left, mid);
    int rightMax = findMax(arr, mid + 1, right);
    return Math.max(leftMax, rightMax);
}
```

deWithNishchal

15.12. Recurrence Relations

Divide and Conquer algorithms are commonly analyzed using a **recurrence relation**.
A general recurrence looks like:

T(n) = aT(n/b) + f(n)
where: a → number of recursive subproblems

n/b → size of each subproblem

f(n)
→ work done outside recursive calls
such as dividing or combining

Examples:

Binary Search

T(n) = T(n/2) + O(1)
Result: O(log n)

Merge Sort

T(n) = 2T(n/2) + O(n)
Result: O(n log n)

Balanced Quick Sort

Approximately: T(n) = 2T(n/2) + O(n)
Result: O(n log n)

15.13. Recursion Tree Analysis

A recursion tree helps visualize work at each level.
For Merge Sort:

```
          n
        /   \
      n/2    n/2
     /  \    /  \
   n/4 n/4 n/4 n/4
   ...
```

Work per level:
Level 0: n
Level 1: n/2 + n/2 = n
Level 2: 4 × n/4 = n
There are: log n
levels, giving: O(n log n)
This is often the easiest way to understand Divide and Conquer complexity.

CodeWithNishchal

15.14. Divide and Conquer vs Dynamic Programming

Both break problems into smaller subproblems, but their relationship between subproblems is different.

Divide and Conquer

Subproblems are usually **independent**.

Problem
/ \
A B

A and B are solved separately.

Examples:
Merge Sort
Quick Sort
Binary Search

Dynamic Programming

Subproblems often **overlap**.

```
      f(5)
      /  \
   f(4)   f(3)
    |
   f(3)  ← repeated state
```

DP stores previously computed results to avoid recomputation.

Feature	Divide & Conquer	Dynamic Programming
Subproblems	Usually independent	Often overlapping
Result reuse	Usually unnecessary	Essential
Storage	Often recursion only	Memo/table commonly used
Example	Merge Sort	Fibonacci DP

15.15. Divide and Conquer vs Backtracking

Divide and Conquer

Breaks a problem into smaller subproblems whose results contribute to the final answer.
Divide, Solve, Combine

Backtracking

Explores alternative choices and may undo them. Choose, Explore, Undo, Try next
Example:Merge Sort → Divide and Conquer
N-Queens → Backtracking

CodeWithNishchal

15,16. Divide and Conquer vs Greedy

Divide and Conquer

Solves smaller subproblems and combines their results.

Greedy

Makes the best-looking local choice and usually commits to it.

Divide & Conquer

→ solve multiple smaller problems

Greedy

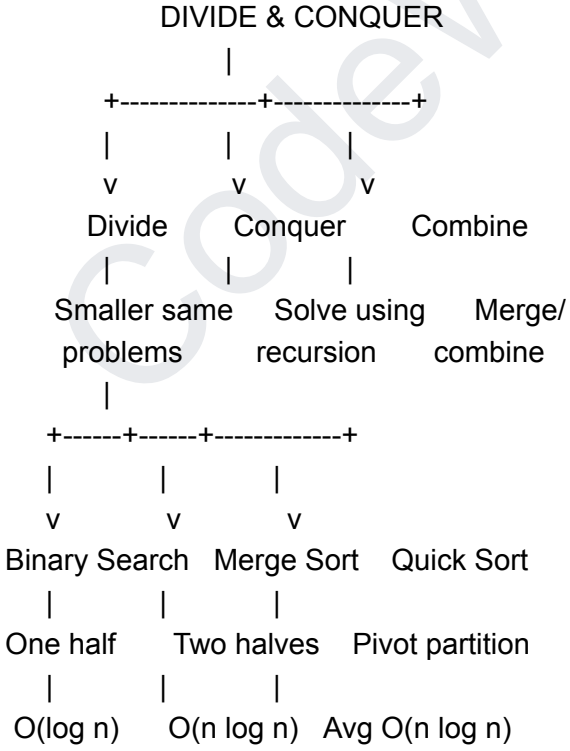
→ choose one locally best action

The two strategies solve fundamentally different classes of problems.

15.17. Complexity Comparison

Algorithm	Divide	Combine / Extra Work	Time
Binary Search	Keep one half	O(1)	O(log n)
Merge Sort	Two halves	O(n) merge	O(n log n)
Quick Sort average	Partition around pivot	O(n) partition	O(n log n)
Quick Sort worst	Highly unbalanced	O(n) partition	$O(n^2)$
Find Maximum	Two halves	O(1) comparison per combine	O(n)

15.18. Final Pattern Map



CodeWithNishchal

16. Heap

A **Heap** is a specialized tree-based data structure that follows the **complete binary tree** property and maintains an ordering rule between parent and child nodes.

The two main types are:

Min Heap

→ Parent <= Children

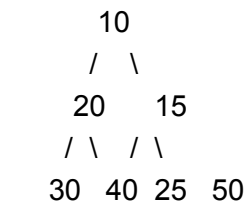
→ Smallest element is at root

Max Heap

→ Parent >= Children

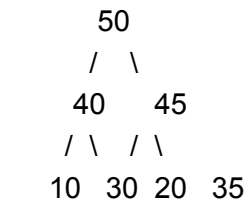
→ Largest element is at root

Example Min Heap:



Every parent is smaller than or equal to its children.

Example Max Heap:



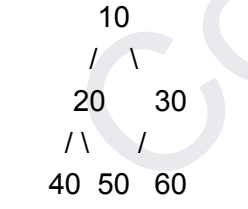
Every parent is greater than or equal to its children. A heap does **not** mean that the entire structure is sorted. Only the parent-child heap property is guaranteed.

16.1. Complete Binary Tree Property

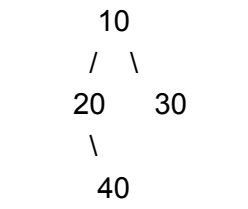
A heap must be a **complete binary tree**:

- every level is completely filled except possibly the last
- The last level is filled from left to right.

Valid:



Invalid heap shape:

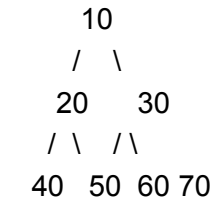


because a position on the left was skipped. The complete-tree property allows heaps to be stored efficiently inside an array.

CodeWithNishchal

16.2. Array Representation of Heap

Consider:



Array representation:

Index: 0 1 2 3 4 5 6

10	20	30	40	50	60	70

For a node at index i:

Left Child = 2*i + 1

Right Child = 2*i + 2

Parent = (i - 1) / 2

Example:

i = 1

Left = 3

Right = 4

arr[1] = 20

arr[3] = 40

arr[4] = 50

This representation avoids explicit left and right references.

16.3. Min Heap vs Max Heap

Feature	Min Heap	Max Heap
Root	Minimum	Maximum
Parent rule	Parent ≤ children	Parent ≥ children
Fastest retrieval	Minimum	Maximum
Common use	K largest, shortest item, merge	K smallest, highest priority
Root access	O(1)	O(1)

CodeWithNishchal

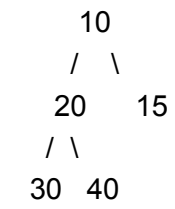
16.4. Core Heap Operations

The important heap operations are:
Insert, Peek root, Remove root , Heapify , Build Heap
Complexities:

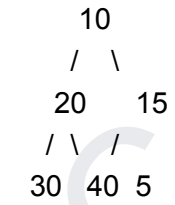
Operation	Time
Peek min/max	O(1)
Insert	O(log n)
Remove root	O(log n)
Heapify one node	O(log n)
Build heap	O(n)
Search arbitrary value	O(n)

16.5. Insertion — Heapify Up

Suppose we have a Min Heap:

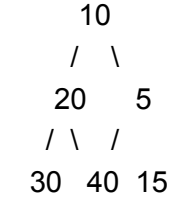


Insert 5.
Because a heap is complete, insert at the next available position:



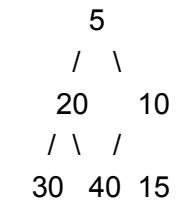
The Min Heap property is violated because:
5 < 15

Swap with parent:



CodeWithNishchal

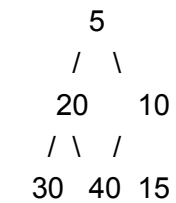
Still:
5 < 10
Swap again:



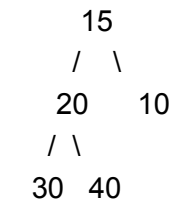
Heap restored.
This process is called: Heapify Up/ Bubble Up/ Percolate Up
Maximum movement is from a leaf to the root.
Heap height: O(log n)
Therefore insertion: O(log n)

16.6. Remove Root — Heapify Down

In a Min Heap, deletion usually removes the minimum element at the root.
Example:

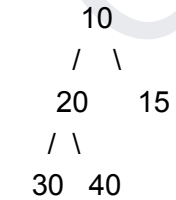


Remove 5.
Move the last element 15 to the root:



Heap property is violated:
15 > 10

Swap with the smaller child:



Heap restored.
This is: Heapify Down/ Sift Down/ Percolate Down
Again, maximum traversal is one root-to-leaf path: O(log n)

CodeWithNishchal

16.7. Heapify

Heapify means restoring the heap property from a node downward.
For a Min Heap:

1. Compare node with its children.
2. Find the smallest.
3. If a child is smaller than the parent, swap.
4. Continue from the swapped child.

Example:

```
      40
     /  \
    20   10
```

Smallest is 10.
Swap:

```
      10
     /  \
    20   40
```

Complexity: O(log n)

16.8. Building a Heap

Given an arbitrary array: [10, 5, 20, 2, 4, 8]
we can convert it into a heap by applying heapify from the last non-leaf node upward.
Last non-leaf index: n / 2 - 1
Conceptually:

```
for i = n/2 - 1 down to 0
   heapify(i)
```

Although each heapify can cost O(log n), building a heap this way is: O(n) not O(n log n).
This happens because most nodes are near the bottom and require little or no movement.

16.9. Manual Min Heap Implementation

```
class MinHeap {
    private int[] heap;
    private int size;
    MinHeap(int capacity) {
        heap = new int[capacity];
    }
    void add(int value) {
        heap[size] = value;
        int index = size;
        size++;

        while (index > 0) {
```

CodeWithNishchal

```
        int parent = (index - 1) / 2;
        if (heap[parent] <= heap[index]) {
            break;
        }
        swap(parent, index);
        index = parent;
    }
}

int peek() {
    if (size == 0) {
        throw new RuntimeException("Heap is empty");
    }
    return heap[0];
}

int remove() {

    if (size == 0) {
        throw new RuntimeException("Heap is empty");
    }

    int min = heap[0];

    heap[0] = heap[size - 1];
    size--;

    heapifyDown(0);

    return min;
}
private void heapifyDown(int index) {
    while (true) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int smallest = index;
        if (left < size && heap[left] < heap[smallest]) {
            smallest = left;
        }
        if (right < size
                && heap[right] < heap[smallest]) {
            smallest = right;
        }
        if (smallest == index) {
            break;
```

```
            }
            swap(index, smallest);
            index = smallest;
        }
    }

    private void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }
}
```

The main ideas are:

Insertion

→ add at end

→ heapify upward

Removal

→ replace root with last element

→ heapify downward

16.10. PriorityQueue in Java

Java provides heap behavior through: PriorityQueue

Import: import java.util.PriorityQueue;

By default: PriorityQueue<Integer> heap = new PriorityQueue<>(); creates a **Min Heap**.

Example: PriorityQueue<Integer> heap = new PriorityQueue<>();

heap.offer(30);

heap.offer(10);

heap.offer(20);

System.out.println(heap.peek()); // 10

System.out.println(heap.poll()); // 10

System.out.println(heap.peek()); // 20

Important methods:

Method	Meaning
offer(x)	Insert
add(x)	Insert
peek()	Read root
poll()	Remove root
size()	Number of elements
isEmpty()	Check empty

CodeWithNishchal

16.11. Max Heap in Java

Java's PriorityQueue is a Min Heap by default.
A Max Heap can be created using:
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

Example:

```
maxHeap.offer(10);
maxHeap.offer(40);
maxHeap.offer(20);
System.out.println(maxHeap.peek());
```

Output:40
For custom objects, use a comparator.
Example:

```
PriorityQueue<int[]> heap =
    new PriorityQueue<>(
        (a, b) -> Integer.compare(a[1], b[1])
    );
```

This orders arrays according to element at index 1.

16.12. Heap Sort

Given an unsorted array, sort it in ascending order using the **Heap Sort** algorithm.
Heap Sort uses a **Max Heap** for ascending order.
In a Max Heap:
Parent >= Children
So the largest element is always present at the root.
Example:
Input = [7, 2, 9, 4, 3, 8]
Output: [2, 3, 4, 7, 8, 9]

Approach — Build Max Heap + Repeatedly Remove Maximum

Heap Sort works in two main phases.

1. Build a Max Heap

Convert the array into a Max Heap. After building the heap, the maximum element is at index 0.
Example: Array: [9, 7, 8, 2, 4, 3]
Max Heap:

```
      9
    /   \
   7     8
  / \   /
 2   4 3
```

CodeWithNishchal

2. Move Maximum to Its Correct Position

The root contains the maximum element.

So:

1. Swap the root with the last element of the unsorted part.
2. Reduce the heap size by one.
3. Heapify the root again.
4. Repeat until only one element remains.

Example:

Max Heap: [9, 7, 8, 2, 4, 3]

Swap root with last: [3, 7, 8, 2, 4 | 9]

9 is now in its final position.

Heapify remaining part: [8, 7, 3, 2, 4 | 9]

Continue the same process until the entire array is sorted.

Main idea:

Build Max Heap → Swap Maximum to End → Reduce Heap Size → Heapify → Repeat

Algorithm

Step 1 — Build Max Heap

1. Start from the last non-leaf node: n / 2 - 1
2. Move backward toward index 0.
3. Call heapify() on each node.

After this step, the complete array becomes a Max Heap.

Step 2 — Sort the Array

For end = n - 1 down to 1:

1. Swap: arr[0] and arr[end]
2. The maximum element is now fixed at index end.
3. Reduce the heap size to: end
4. Call heapify() on index 0.
5. Repeat until the array is sorted.

Heapify Algorithm

heapify() restores the Max Heap property.

For a node at index i:

Left Child = 2 * i + 1

Right Child = 2 * i + 2

Steps:

1. Assume the current node is the largest.
2. Compare it with the left child.
3. Compare it with the right child.
4. Find the largest among:

Current Node, Left Child, Right Child

5. If one of the children is larger:
    - Swap it with the current node.
    - Recursively heapify the affected subtree.
6. Stop when the Max Heap property is satisfied.

CodeWithNishchal

```
static void heapSort(int[] arr) {
    int n = arr.length;
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    // Move maximum elements to the end
    for (int end = n - 1; end > 0; end--) {
        // Swap root with last unsorted element
        int temp = arr[0];
        arr[0] = arr[end];
        arr[end] = temp;

        // Restore heap property
        heapify(arr, end, 0);
    }
}

static void heapify(int[] arr, int heapSize, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    // Check left child
    if (left < heapSize && arr[left] > arr[largest]) {
        largest = left;
    }
    // Check right child
    if (right < heapSize && arr[right] > arr[largest]) {
        largest = right;
    }

    // If current node is not the largest
    if (largest != i) {

        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Heapify affected subtree
        heapify(arr, heapSize, largest);
    }
}
```

Calling Heap Sort:

```
heapSort(arr);
```

Vishchal

Complexity

Building the Max Heap: O(n)
Then we perform about n removals. Each heapify operation takes: O(log n)
Therefore: Time = O(n log n)
for best, average, and worst cases. Extra space: O(log n) with this recursive heapify() because of the recursion stack. If heapify() is written iteratively: Space = O(1)
Heap Sort is generally **not stable**.

16.13. Heap Pattern Recognition

Heap should come to mind when a problem asks repeatedly for:
Minimum, Maximum, Top K. Kth smallest/largest, Closest K, Highest priority, Lowest cost
Earliest ending time, Merge smallest current candidate
A useful question is:
"Do I repeatedly need the best element
among a changing collection?"

If yes, a heap is often a strong choice.

16.14. Top K Pattern

Suppose we need the **K largest elements** from n elements.
A simple approach is sorting: O(n log n)

A better heap-based solution can maintain only k elements.
For K largest elements:
Use Min Heap of size K

Why Min Heap?
Because among the current K largest elements, we want the **smallest one easily accessible** so that it can be removed when a larger candidate appears.

16.15. K Largest Elements

Given an integer array and an integer k, find the **k largest elements**.
Example: Array = [3, 2, 1, 5, 6, 4] K = 2
Output: [5, 6]

Algorithm — Min Heap of Size K

1. Create a Min Heap.
2. Traverse every element.
3. Add the current element to the heap.
4. If heap size becomes greater than k, remove the smallest element.
5. After processing all elements, the heap contains the k largest elements.

Main idea: Add element → Size > k? Remove minimum → Keep only k largest

CodeWithNishchal

Code:

```
static List<Integer> kLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return new ArrayList<>(minHeap);
}
```

Complexity: Time = O(n log k) Space = O(k)

16.16. Kth Largest Element

Given an integer array and an integer k, find the **kth largest element** in the array.

Example: Array = [3, 2, 1, 5, 6, 4]

K = 2 Output: 5

Algorithm — Min Heap of Size K

1. Create a Min Heap.
2. Traverse every element.
3. Add the current element to the heap.
4. If heap size becomes greater than k, remove the smallest element.
5. After processing all elements, the heap contains the k largest elements.
6. The heap root is the smallest among those k elements, so it is the **kth largest element**.

Main idea: Keep k largest elements → Heap root = kth largest

Code

```
static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return minHeap.peek();
}
```

Complexity

Time = O(n log k)

Space = O(k)

WithNishchal

16.17. Kth Smallest Element

For Kth smallest, reverse the logic.
Use: Max Heap of size K

because we want the largest among the current K smallest values available at the root.
K largest → Min Heap size K
K smallest → Max Heap size K
This is an important rule.

16.18. K Closest Points

Suppose each point is: (x, y)
Distance from origin is: sqrt($x^2 + y^2$)
Because square root preserves ordering, we can compare: $x^2 + y^2$
instead.To find K closest points: Use Max Heap of size K
Why Max Heap?
Among the K closest points currently stored, the **farthest** should be easy to remove when a closer point appears.
Code:

```
static int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> maxHeap =
        new PriorityQueue<>(
            (a, b) -> Integer.compare(
                b[0] * b[0] + b[1] * b[1],
                a[0] * a[0] + a[1] * a[1]
            )
        );

    for (int[] point : points) {
        maxHeap.offer(point);
        if (maxHeap.size() > k) {
            maxHeap.poll();
        }
    }
    int[][] result = new int[k][2];
    for (int i = 0; i < k; i++) {
        result[i] = maxHeap.poll();
    }
    return result;
}
```

Complexity:Time = O(n log k) Space = O(k)
For very large coordinates, use long for squared distance to avoid integer overflow.

16.19. Top-K Pattern Summary

Problem	Heap
K largest	Min Heap of size K
Kth largest	Min Heap of size K
K smallest	Max Heap of size K
Kth smallest	Max Heap of size K
K closest	Usually Max Heap of size K
K most frequent	Heap ordered by frequency

The main idea:Keep only K useful candidates.
Root = easiest candidate to discard.

16.20. Greedy + Heap Pattern

A heap is frequently combined with **Greedy Algorithms**.
Greedy decides:
"What is the best available choice right now?"

Heap provides that choice efficiently:
minimum available
maximum available
earliest finishing
highest frequency
lowest cost

General pattern:

```
Candidates
   |
   v
Priority Queue
   |
   v
Take best candidate
   |
   v
Update state
   |
   v
Repeat
```

CodeWithNishchal

16.21. Task Scheduler

Given a list of tasks and a cooldown value n, find the **minimum number of time intervals required to complete all tasks**. The same task must have at least n intervals between two executions. Example: Tasks = [A, A, A, B, B, B] n = 2
One valid schedule: A B idle A B idle A B Answer: 8

Algorithm — Max Heap + Cooldown Queue

1. Count the frequency of each task.
2. Store all frequencies in a **Max Heap**.
3. Use a queue to store tasks that are currently in cooldown.
4. At every time interval:
    - Move tasks whose cooldown has finished back into the heap.
    - Take the task with the highest remaining frequency from the heap.
    - Execute it and decrease its frequency.
    - If it still has occurrences left, put it into the cooldown queue.
5. If no task is available, the current interval becomes idle.
6. Continue until both the heap and cooldown queue are empty.

```
static int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    for (char task : tasks) { freq[task - 'A']++;
    }
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
    for (int count : freq) {
        if (count > 0) { maxHeap.offer(count);
        }
    }
    // {remaining frequency, available time}
    Queue<int[]> cooldown = new ArrayDeque<>();
    int time = 0;
    while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
        // Return cooled-down tasks to heap
        while (!cooldown.isEmpty() && cooldown.peek()[1] <= time) {
            maxHeap.offer(cooldown.poll()[0]);
        }
        if (!maxHeap.isEmpty()) {
            int count = maxHeap.poll() - 1;
            if (count > 0) {
                cooldown.offer(  new int[]{count, time + n + 1}
                ); } }
        time++;
    }
    return time;
}
```

Time = O(T log 26) ≈ O(T) where T includes task and idle intervals. Space = O(26) ≈ O(1) for uppercase English tasks.

16.22. Meeting Rooms

Given a list of meeting intervals: [start, end]

find the **minimum number of meeting rooms required** so that no overlapping meetings use the same room.

Example: Meetings = [[0,30], [5,10], [15,20]]

Output: 2

Because [0,30] overlaps with both other meetings.

Algorithm — Sorting + Min Heap

1. Sort all meetings by their start time.
2. Use a **Min Heap** to store meeting end times.
3. For each meeting:
    - Check the earliest ending meeting using heap.peek().
    - If:
      earliestEnd <= currentStart
      then that room is free, so remove that end time and reuse the room.
    - Add the current meeting's end time to the heap.
4. After all meetings are processed, the heap size gives the number of rooms needed.

Main idea: Sort by start time → Reuse earliest available room → Otherwise use new room

```
static int minMeetingRooms(int[][] meetings) {
    Arrays.sort(
        meetings,
        Comparator.comparingInt(a -> a[0])
    );
    PriorityQueue<Integer> endTimes =  new PriorityQueue<>();
    for (int[] meeting : meetings) {
        if (!endTimes.isEmpty()
                && endTimes.peek() <= meeting[0]) {
            endTimes.poll();
        }
        endTimes.offer(meeting[1]);
    }
    return endTimes.size();
}
```

Complexity:
Sorting = O(n log n)
Heap   = O(n log n)

Overall = O(n log n)

Ujjwal Vishchal

16.23. Reorganize String

Given:
"aab"

rearrange characters so that no two adjacent characters are equal.
Possible result: "aba"
The greedy idea: Always use the currently most frequent available character.
Use a Max Heap based on character frequency.
But after using a character, it should not be immediately reused.
General flow:
Max Heap
|
take highest frequency char
|
append it
|
hold previous char temporarily
|
put previous back when allowed
Simplified structure:

```
class CharFreq {
    char ch;
    int freq;

    CharFreq(char ch, int freq) {
        this.ch = ch;
        this.freq = freq;
    }
}

Then maintain:
PriorityQueue<CharFreq> maxHeap =
    new PriorityQueue<>(
        (a, b) -> Integer.compare(b.freq, a.freq)
    );
```

The key pattern is:
Pick highest-frequency candidate
→ temporarily block it
→ use next best candidate
→ return previous candidate later

If at some point no valid candidate exists while characters remain, reorganization is impossible.

16.24. Huffman Encoding

Huffman Encoding is a greedy compression algorithm.
Suppose characters have frequencies:
A → 5
B → 9
C → 12
D → 13
E → 16
F → 45
The algorithm repeatedly combines the two smallest frequencies.
Therefore use: Min Heap

Steps:
1. Insert all frequencies.
2. Remove two smallest.
3. Combine them.
4. Insert combined frequency back.
5. Repeat until one tree remains.
Example:
5 + 9 = 14
Heap now contains:
12, 13, 14, 16, 45
12 + 13 = 25
...
The repeated rule is: Always combine two minimum-frequency nodes.
That greedy choice produces an optimal prefix code.
Complexity for n distinct symbols: O(n log n)
because heap operations are repeated until one node remains.

16.25. Greedy + Heap Summary

Problem	Heap Strategy
Task Scheduler	Max Heap by remaining frequency
Meeting Rooms	Min Heap by ending time
Reorganize String	Max Heap by character frequency
Huffman Encoding	Min Heap by frequency
Minimum cost merging	Min Heap
Scheduling by priority	Depends on priority direction

The heap should be ordered by:
the value needed for the next greedy choice

CodeWithNishchal

16.26. K-Way Merge

K-Way Merge is used when we have multiple sorted sequences and need to merge them efficiently.
Example:
List 1: [1, 4, 7]
List 2: [2, 5, 8]
List 3: [3, 6, 9]

Instead of comparing every element from every list, maintain only the smallest current candidate from each list.
Use: Min Heap

Initial heap:
1 from List 1
2 from List 2
3 from List 3
Remove smallest:v1
Then insert the next element from the same list:v4
Heap:2, 3, 4
Continue: 1, 2, 3, 4, 5, 6, 7, 8, 9

16.27. K-Way Merge Structure

Each heap entry should know:
value
which list it came from
index inside that list

Example class:

```
class Entry {
    int value;
    int listIndex;
    int elementIndex;

    Entry(int value, int listIndex, int elementIndex) {
        this.value = value;
        this.listIndex = listIndex;
        this.elementIndex = elementIndex;
    }
}
```

LeWithNishchal

16.28. Merge K Sorted Arrays

```
static List<Integer> mergeKSortedArrays( int[][] arrays) {
    PriorityQueue<Entry> minHeap =
        new PriorityQueue<>(
            Comparator.comparingInt(e -> e.value)
        );
    for (int i = 0; i < arrays.length; i++) {
        if (arrays[i].length > 0) {
            minHeap.offer(
                new Entry(arrays[i][0], i, 0)
            );
        }
    }
    List<Integer> result = new ArrayList<>();
    while (!minHeap.isEmpty()) {
        Entry current = minHeap.poll();
        result.add(current.value);
        int nextIndex = current.elementIndex + 1;
        if (nextIndex
                < arrays[current.listIndex].length) {
            minHeap.offer(
                new Entry(
                    arrays[current.listIndex][nextIndex],
                    current.listIndex,
                    nextIndex
                )
            );
        }
    }
    return result;
}
```

Logic

The heap contains at most one current candidate from each list. After removing the smallest candidate: only the next value from that same list needs to be inserted

because all other lists already have their smallest unprocessed candidate in the heap.
If:
N = total number of elements
K = number of sorted lists

then: Heap size <= K and complexity is: Time = O(N log K) Space = O(K) excluding output.

16.29. Merge K Sorted Linked Lists

The exact same pattern works for linked lists.
Initial heap:
head of List 1
head of List 2
...
head of List K
Repeatedly: Remove smallest node, Append it to result, Insert removedNode.next
Complexity: O(N log K). This is one of the most important applications of K-way merge.

16.30. Heap vs Balanced BST

Both can support ordered data, but with different strengths.

Feature	Heap	Balanced BST
Min/Max root access	O(1)	O(log n) or direct with suitable API
Insert	O(log n)	O(log n)
Remove min/max	O(log n)	O(log n)
Search arbitrary value	O(n)	O(log n)
Full sorted traversal	Not natural	Natural
Main use	Priority	Ordered search

Heap is best when only the **highest/lowest priority** matters.

16.31. Overall Complexity Table

Operation / Pattern	Time	Extra Space
Peek root	O(1)	—
Insert	O(log n)	—
Remove root	O(log n)	—
Heapify	O(log n)	—
Build Heap	O(n)	—
Heap Sort	O(n log n)	O(1) auxiliary aside from implementation details
Kth Largest	O(n log k)	O(k)
K Closest	O(n log k)	O(k)
Meeting Rooms	O(n log n)	O(n) worst case
Huffman	O(n log n)	O(n)
K-Way Merge	O(N log K)	O(K)

CodeWithNishchal

17. Graphs

A **Graph** is a non-linear data structure consisting of:

Vertices / Nodes + Edges / Connections

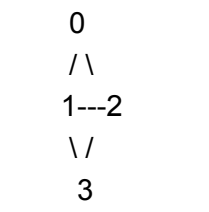
Formally:

G = (V, E)

V = set of vertices

E = set of edges

Example:



Vertices: V = {0, 1, 2, 3}

Edges: E = {(0,1), (0,2), (1,2), (1,3), (2,3)}

Graphs are used for networks, roads, social connections, dependencies, routing, scheduling, recommendation systems, computer networks, maps, and many other relationship-based problems.

17.1. Important Graph Terminology

Term	Meaning
Vertex	A node in the graph
Edge	Connection between two vertices
Adjacent Vertices	Vertices directly connected by an edge
Degree	Number of edges connected to a vertex
In-degree	Number of incoming edges in a directed graph
Out-degree	Number of outgoing edges
Path	Sequence of connected vertices
Cycle	Path that returns to its starting vertex
Connected Graph	Every vertex is reachable from every other vertex in an undirected graph
Connected Component	Maximal connected part of a graph
Weight	Cost/distance associated with an edge
Source	Starting vertex
Destination	Target vertex

CodeWithNishchal

17.2. Types of Graphs

Undirected Graph

Edges work in both directions.

0 ----- 1

0 ↔ 1

If: 0 -- 1 then both are neighbors of each other.

Directed Graph

Edges have direction.

0 ----> 1

This means: 0 can go to 1 but not necessarily: 1 can go to 0

Weighted Graph

Edges contain a weight: A ----5---- B The weight may represent: distance/ cost/ time/ capacity

---

Unweighted Graph

Every edge is treated equally. A ---- B

For shortest-path purposes, each edge can be considered weight 1.

17.3. Graph Representation

The two most common representations are:

- Adjacency Matrix
- Adjacency List

17.3.1 Adjacency Matrix

For V vertices, create a: V × V matrix.

Example:

0 -- 1

| |

2 -- 3

Matrix:

```
   0  1  2  3
   ------------
0 | 0  1  1  0
1 | 1  0  0  1
2 | 1  0  0  1
3 | 0  1  1  0
```

If: matrix[u][v] = 1 an edge exists. For weighted graphs: matrix[u][v] = weight

CodeWithNishchal

Complexity

Operation	Complexity
Check edge	O(1)
Add edge	O(1)
Remove edge	O(1)
Find all neighbors	O(V)
Space	O(V^2)

Adjacency matrix is useful for dense graphs or when edge lookup must be extremely fast.

17.3.2 Adjacency List

Each vertex stores its neighbors.
0 → 1, 2
1 → 0, 3
2 → 0, 3
3 → 1, 2

```
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) {
    graph.add(new ArrayList<>());
}
graph.get(0).add(1);
graph.get(1).add(0);

For an undirected edge:
graph.get(u).add(v);
graph.get(v).add(u);
For directed:
graph.get(u).add(v);
```

Complexity

Operation	Complexity
Add edge	O(1) amortized
Traverse neighbors	O(degree)
Space	O(V + E)

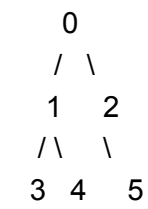
Adjacency List is usually preferred in DSA because most real graph problems are sparse.

17.4. Graph Traversal

The two fundamental traversal techniques are: BFS and DFS
Both require a visited structure because graphs may contain: cycles/ multiple paths to same node
Without visited, traversal may repeat indefinitely.

17.5. BFS — Breadth First Search

BFS explores nodes **level by level**. It uses: Queue



Starting from 0:
Level 0: 0
Level 1: 1, 2
Level 2: 3, 4, 5
Traversal: 0, 1, 2, 3, 4, 5

BFS Flow

Start Node
|
Mark visited
|
Add to Queue
|
v
Remove front
|
Process node
|
Add unvisited neighbors
|
Repeat until queue empty

```
static void bfs(
        List<List<Integer>> graph,
        int start) {
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> queue = new ArrayDeque<>();
    visited[start] = true;
    queue.offer(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.print(node + " ");
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }}}}
```

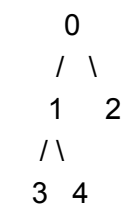
Important:
Mark visited when inserting into the queue not when removing it.
This prevents the same node from being inserted multiple times.

---

ithNishchal

17.6. DFS — Depth First Search

DFS explores as deep as possible before returning.
It can use:
Recursion or Explicit Stack
Example:



Possible DFS: 0 → 1 → 3 → 4 → 2 The exact order can vary depending on adjacency-list order.

Recursive DFS

```
static void dfs(
        int node,
        List<List<Integer>> graph,
        boolean[] visited) {
    visited[node] = true;
    System.out.print(node + " ");
    for (int neighbor : graph.get(node)) {

        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}
```

Call: boolean[] visited = new boolean[n];
dfs(0, graph, visited);

BFS vs DFS

Feature	BFS	DFS
Main structure	Queue	Stack / Recursion
Exploration	Level by level	Deep first
Unweighted shortest path	Excellent	Not guaranteed
Components	Yes	Yes
Cycle detection	Yes	Yes
Topological sort	Kahn's	DFS version
Space	Can hold wide level	Recursion depth / stack

For adjacency lists:

BFS Time = O(V + E)
DFS Time = O(V + E)

because every vertex and edge is processed only a constant number of times.

17.7. Disconnected Graphs

Starting BFS/DFS from one vertex only explores its connected component.
Example:
0 -- 1 -- 2
3 -- 4
Starting from 0 will never reach 3 or 4.
Therefore:

```
for (int i = 0; i < n; i++) {

    if (!visited[i]) {
        dfs(i, graph, visited);
    }
}
```

This also allows counting connected components:
Every new DFS/BFS start = one new component

17.8. Cycle Detection in Undirected Graph

In an undirected graph:

```
0 -- 1
 \  /
   2
```

there is a cycle: 0 → 1 → 2 → 0

When traversing an undirected graph, seeing an already visited neighbor does **not automatically mean cycle**, because it may simply be the node we came from.
Therefore we track:
parent

17.8.1 DFS Cycle Detection — Undirected

Rule:
If neighbor is visited
AND neighbor != parent
→ cycle exists

CodeWithNishchal

```
static boolean hasCycleUndirected(
        int node,
        int parent,
        List<List<Integer>> graph,
        boolean[] visited) {
    visited[node] = true;
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            if (hasCycleUndirected(
                    neighbor,
                    node,
                    graph,
                    visited)) {
                return true;
            }
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}
```

For disconnected graphs, call it from every unvisited vertex.
Complexity: Time = O(V + E) Space = O(V)

17.8.2 BFS Cycle Detection — Undirected

Store: (node, parent) in the queue.
Conceptually: Visit neighbor. If unvisited → add neighbor with current as parent
If already visited and not parent → cycle This has the same: O(V + E) time complexity.

17.9. Cycle Detection in Directed Graph

Directed graphs require different logic.
Example:
0 → 1 → 2
  ↑ |
  |___|

Cycle: 1 → 2 → 1
A simple visited[] is not enough because an edge to an already completed node does not necessarily indicate a cycle.
We need to know whether a node is still in the **current DFS path**.
Use: visited[], pathVisited[]

17.9.1 DFS Cycle Detection — Directed

Rules:
Unvisited neighbor → DFS
Neighbor currently in recursion path → cycle
Neighbor visited but no longer in recursion path → safe

CODE:

```
static boolean hasCycleDirected(
        int node,
        List<List<Integer>> graph,
        boolean[] visited,
        boolean[] pathVisited) {

     visited[node] = true;
    pathVisited[node] = true;

    for (int neighbor : graph.get(node)) {

        if (!visited[neighbor]) {

            if (hasCycleDirected(
                    neighbor,
                    graph,
                    visited,
                    pathVisited)) {
                return true;
            }

        } else if (pathVisited[neighbor]) {
            return true;

        }
    }

    pathVisited[node] = false;
    return false;
}
```

Why remove from pathVisited?
Because when DFS finishes processing a node, it is no longer part of the active recursion path.

17.10. Topological Sort

Given a **directed graph**, find a linear ordering of its vertices such that for every directed edge:

u → v

vertex u appears before vertex v. This ordering is called a **Topological Order**.

Example: Study Basics → Learn DSA → Interview

A valid topological order is: Study Basics → Learn DSA → Interview

Topological ordering is possible only for a: Directed Acyclic Graph (DAG)

If the graph contains a directed cycle:

A → B → C

↑ |

└──────┘

no topological ordering exists because every node in the cycle depends on another node in the same cycle. There can be **multiple valid topological orders** for the same DAG.

17.11. Kahn's Algorithm — BFS Topological Sort

Kahn's Algorithm finds a topological ordering using: In-degree + Queue

In-Degree

The **in-degree** of a vertex is the number of incoming edges to that vertex.

Example:

0 → 2

1 → 2

2 → 3

In-degrees:

0 → 0

1 → 0

2 → 2

3 → 1

Nodes with: in-degree = 0 have no remaining dependencies, so they can be processed first.

Algorithm — Kahn's Algorithm

1. Calculate the in-degree of every vertex.
2. Add all vertices having: in-degree = 0 to a queue.
3. While the queue is not empty:
    - Remove a node from the queue.
    - Add it to the topological order.
    - Visit all of its outgoing neighbors.
    - Decrease each neighbor's in-degree by 1.
    - If any neighbor's in-degree becomes 0, add it to the queue.
4. Continue until the queue becomes empty.
5. If all V vertices were processed, a valid topological order exists.

CodeWithNishchal

Short Flow

Calculate in-degrees

↓

Add all 0-in-degree nodes to queue

↓

Remove node from queue

↓

Add node to topological order

↓

Decrease in-degree of its neighbors

↓

Neighbor becomes 0?

↓

Add neighbor to queue

↓

Repeat

Example

Graph:

0 → 2

1 → 2

2 → 3

Initially:

Indegree:

0 = 0

1 = 0

2 = 2

3 = 1

Queue = [0, 1]

Process 0: indegree[2] = 1

Process 1: indegree[2] = 0 Add 2. Queue = [2]

Process 2: indegree[3] = 0 Add 3.

Final order: 0, 1, 2, 3. Another valid order could also exist depending on queue ordering.

Code

```
static List<Integer> topoSortKahn(
        List<List<Integer>> graph) {
    int n = graph.size();
    int[] indegree = new int[n];
    // Calculate in-degree
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) {
            indegree[v]++;
```

CodeWithNishchal

```
        }
    }
    Queue<Integer> queue = new ArrayDeque<>();
    // Add all nodes with in-degree 0
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            queue.offer(i);
        }
    }
    List<Integer> order = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.get(node)) {
            indegree[neighbor]--;
            if (indegree[neighbor] == 0) {
                queue.offer(neighbor);
            }
        }
    }
    return order;
}
```

Complexity

Time = O(V + E) Every vertex is processed once and every edge is examined once.
Space = O(V) for the in-degree array, queue, and result.

17.12. Cycle Detection Using Kahn's Algorithm

Kahn's Algorithm can also detect a cycle in a directed graph.
Given a directed graph, determine whether it contains a **directed cycle**.
The key observation is: Nodes inside a cycle can never reduce all their dependencies to zero.
Therefore, they never enter the queue.

Algorithm

1. Run Kahn's Algorithm normally.
2. Count how many vertices are processed.
3. Compare the processed count with the total number of vertices V.

If: processed == V then: No cycle exists and the graph is a DAG.
If: processed < V then: Cycle exists because some vertices could never reach in-degree 0.

Main Idea

Processed all vertices?
/ \
Yes No
| |
DAG Cycle

Code

```
static boolean hasCycleKahn(
        List<List<Integer>> graph) {
    int n = graph.size();
    int[] indegree = new int[n];
    for (int u = 0; u < n; u++) {
        for (int v : graph.get(u)) {
            indegree[v]++;
        }
    }
    Queue<Integer> queue = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            queue.offer(i);
        }
    }
    int processed = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        processed++;
        for (int neighbor : graph.get(node)) {
            indegree[neighbor]--;
            if (indegree[neighbor] == 0) {
                queue.offer(neighbor);
            }
        }
    }
    return processed < n;
}
```

Time = O(V + E)
Space = O(V)

17.13. DFS-Based Topological Sort

Topological sorting can also be performed using **Depth First Search**.

The key idea is:

A node should be added to the topological order only **after all vertices reachable from it have been processed**.

Therefore, we use:

DFS Postorder

and store nodes in a stack.

Algorithm — DFS Topological Sort

1. Create a visited array.
2. For every unvisited vertex, start DFS.
3. During DFS:
    - Mark the current node as visited.
    - Recursively visit all unvisited neighbors.
    - After all neighbors are completely processed, push the current node onto the stack.
4. After DFS finishes for all vertices, pop elements from the stack.
5. The popped sequence gives the topological ordering.

Short Flow

Visit node

↓

Visit all neighbors

↓

Neighbors completely processed

↓

Push current node onto stack

↓

Repeat

↓

Pop stack for topological order

Why Push After Processing Neighbors?

Suppose:

A → B

A must appear before B in the final topological order.

DFS from A first visits B.

B finishes first and is pushed onto the stack.

Then A is pushed.

Stack:

Top

A

B

When popped: A → B

which gives the correct dependency order.

CodeWithNishchal

Code — DFS Topological Sort

```
static List<Integer> topoSortDFS(
        List<List<Integer>> graph) {
    int n = graph.size();
    boolean[] visited = new boolean[n];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {

        if (!visited[i]) {
            topoDfs(i, graph, visited, stack);
        }
    }

    List<Integer> order = new ArrayList<>();
    while (!stack.isEmpty()) {
        order.add(stack.pop());
    }
    return order;
}

static void topoDfs(
        int node,
        List<List<Integer>> graph,
        boolean[] visited,
        Deque<Integer> stack) {

    visited[node] = true;
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            topoDfs(
                    neighbor,
                    graph,
                    visited,
                    stack
            );
        }
    }
    // Postorder
    stack.push(node);
}
```

This version assumes that the input graph is already known to be a DAG.

17.13.1 DFS Topological Sort With Cycle Detection

If the graph is not guaranteed to be a DAG, we should detect cycles while performing DFS.

Use three states:

0 → Unvisited

1 → Currently in DFS path

2 → Completely processed

If during DFS we reach a node whose state is:

1

then we have found a **back edge**, which means a directed cycle exists.

Algorithm

1. Start DFS from every unvisited node.
2. Mark the current node as: state[node] = 1
3. Visit every neighbor.
4. If a neighbor has: state[neighbor] == 1 a cycle exists.
5. If a neighbor is unvisited, recursively process it.
6. After all neighbors are processed: state[node] = 2
7. Push the node onto the stack.
8. If no cycle is found, pop the stack to obtain the topological order.

Code

```
static List<Integer> topoSortDFSWithCycleCheck(
        List<List<Integer>> graph) {
    int n = graph.size();
    int[] state = new int[n];
    Deque<Integer> stack = new ArrayDeque<>();\
    for (int i = 0; i < n; i++) {
        if (state[i] == 0) {
            if (hasCycleDFS(
                    i,
                    graph,
                    state,
                    stack)) {
                return new ArrayList<>();
            }
        }
    }
    List<Integer> order = new ArrayList<>();
    while (!stack.isEmpty()) {
        order.add(stack.pop());
    }
    return order;
}
```

Nishchal

```
static boolean hasCycleDFS(
        int node,
        List<List<Integer>> graph,
        int[] state,
        Deque<Integer> stack) {

    state[node] = 1;

    for (int neighbor : graph.get(node)) {

        if (state[neighbor] == 1) {
            return true;
        }

        if (state[neighbor] == 0) {

            if (hasCycleDFS(
                    neighbor,
                    graph,
                    state,
                    stack)) {

                return true;
            }
        }
    }

    state[node] = 2;

    stack.push(node);

    return false;
}
```

If the returned list is empty because a cycle was detected, no valid topological ordering exists.

Complexity

Time = O(V + E)

Space = O(V)

The recursion stack can contain up to V vertices in the worst case.

Easy Memory

Kahn's Algorithm:

In-degree → Queue → Remove dependency → In-degree becomes 0 → Process

DFS Topological Sort:

DFS → Process neighbors → Push node after finishing → Reverse using stack

Topological Sort is valid only when the graph is a: DAG — Directed Acyclic Graph

17.14. Kahn vs DFS Topological Sort

Feature	Kahn	DFS Topological
Technique	BFS	DFS
Main state	In-degree	Visited + stack
Cycle detection	Count processed nodes	Recursion-path state
Time	O(V+E)	O(V+E)
Main intuition	Remove dependencies	Finish dependencies first

17.15. Shortest Path Overview

The correct shortest-path algorithm depends heavily on edge weights.

Graph	Recommended Algorithm
Unweighted	BFS
All weights 1	BFS
Weights 0 or 1	0-1 BFS
Non-negative weighted	Dijkstra
Negative edges allowed	Bellman-Ford
All-pairs shortest path	Floyd-Warshall
DAG weighted graph	Topological-order relaxation

Choosing the correct algorithm is more important than memorizing code.

17.16. Unweighted Shortest Path — BFS

BFS finds shortest path in an unweighted graph because it explores nodes by number of edges:
distance 0
distance 1
distance 2
...
Maintain:
int[] distance = new int[n];
Arrays.fill(distance, -1);
When discovering: distance[neighbor] = distance[node] + 1
First discovery is guaranteed to have the smallest number of edges. Complexity: O(V + E)

CodeWithNishchal

17.17. Weighted Graph Representation

For weighted graphs, each adjacency entry needs: neighbor, weight

```
class Edge {
    int to;
    int weight;
    Edge(int to, int weight) {
        this.to = to;
        this.weight = weight;
    }
}
```

Graph: List<List<Edge>> graph;

17.18. Dijkstra's Algorithm

Dijkstra finds shortest paths from one source in graphs with: non-negative edge weights
It must **not** be used with negative edge weights.
Core idea: Always process the currently known closest unprocessed vertex.
Use: Min Heap / PriorityQueue

Relaxation

Suppose: distance[u] = 5
edge u → v has weight 3
Possible distance to v: 5 + 3 = 8
If: 8 < distance[v] update: distance[v] = 8
This process is called: Edge Relaxation

17.18.1 Dijkstra Flow

dist[source] = 0
others = infinity
Put source in Min Heap
    |
    v
Take node with minimum distance
    |
    v
Relax all outgoing edges
    |
If distance improves→ push updated pair into heap
    |
Repeat

CodeWithNishchal

```
static int[] dijkstra(
        List<List<Edge>> graph,
        int source) {
    int n = graph.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    PriorityQueue<int[]> pq =
        new PriorityQueue<>(
            Comparator.comparingInt(a -> a[1])
        );
    dist[source] = 0;
    pq.offer(new int[]{source, 0});
    while (!pq.isEmpty()) {
        int[] current = pq.poll();
        int node = current[0];
        int currentDist = current[1];
        if (currentDist != dist[node]) {
            continue;
        }
        for (Edge edge : graph.get(node)) {
            if (dist[node] != Integer.MAX_VALUE
                    && dist[node] + edge.weight
                    < dist[edge.to]) {

                dist[edge.to] =
                        dist[node] + edge.weight;

                pq.offer(
                    new int[]{
                        edge.to,
                        dist[edge.to]
                    }
                );
            }}}
    return dist;
}
```

The stale-entry check: if (currentDist != dist[node]) continue;
ignores older heap entries after a better distance has already been found.
Complexity with adjacency list + binary heap: O((V + E) log V)
commonly written: O(E log V)
for connected sparse graphs.

17.19. Why Dijkstra Fails With Negative Edges

Dijkstra assumes that once the smallest-distance node is selected, a later path cannot improve it.
Negative edges break this assumption.
Example:
A --2--> B
A --5--> C
C --(-10)--> B
Initially:
distance(B) = 2
but path:
A → C → B = 5 - 10 = -5
is better.
Therefore use: Bellman-Ford
when negative weights may exist.

17.20. Bellman-Ford Algorithm

Bellman-Ford finds shortest paths from one source even when edges can be negative.
It can also detect:
Negative Weight Cycles

Main idea:
Relax every edge repeatedly.

For V vertices, shortest simple paths contain at most:
V - 1 edges

Therefore relax all edges:
V - 1 times

Bellman-Ford Code

Represent edges:

```
class WEdge {
    int from;
    int to;
    int weight;

    WEdge(int from, int to, int weight) {
        this.from = from;
        this.to = to;
        this.weight = weight;
    }
}
```

CodeWithNishchal

Algorithm — Bellman-Ford

1. Create a distance array dist[].
2. Initialize all distances as infinity: dist[i] = ∞
3. Set the source distance to: dist[source] = 0
4. Repeat the following process exactly V - 1 times:
    - Traverse every edge.
    - For an edge:
      u → v with weight w
    - Check whether reaching v through u gives a shorter distance:
      dist[u] + w < dist[v]
    - If yes, update:
      dist[v] = dist[u] + w
5. If during one full iteration no distance changes, stop early.
6. After V - 1 relaxations, dist[] contains the shortest distances from the source.

```
static int[] bellmanFord(
        int vertices,
        List<WEdge> edges,
        int source) {
    int[] dist = new int[vertices];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[source] = 0;
    for (int i = 1; i <= vertices - 1; i++) {

        boolean changed = false;

        for (WEdge edge : edges) {

            if (dist[edge.from] != Integer.MAX_VALUE && dist[edge.from] + edge.weight < dist[edge.to]) {
                dist[edge.to] = dist[edge.from] + edge.weight;
                changed = true;
            }
        }
        if (!changed) {
            break;
        }
    }

    return dist;
}
```

Complexity:
O(VE)

17.21. Negative Cycle Detection

After V - 1 relaxation rounds, perform one additional pass.
If any distance can still improve:
negative cycle reachable from source exists
Why?
A shortest simple path cannot contain more than V - 1 edges.
If improvement still occurs, some cycle is being repeatedly exploited to reduce total cost.

17.22. Floyd-Warshall Algorithm

Floyd-Warshall computes shortest paths between: every pair of vertices
It uses Dynamic Programming. Maintain: dist[i][j]
representing the shortest known distance from i to j.
Core update:

```
dist[i][j] =
min(
   dist[i][j],
   dist[i][k] + dist[k][j]
)
```

Meaning: Is going through k better than current direct/best route?

Floyd-Warshall Code

```
static void floydWarshall(int[][] dist) {
    int n = dist.length;
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != Integer.MAX_VALUE
                        && dist[k][j] != Integer.MAX_VALUE) {
                    dist[i][j] =
                        Math.min(
                            dist[i][j],
                            dist[i][k] + dist[k][j]
                        );
                }
            }
        }
    }
}
```

Complexity:
Time = $O(V^3)$ Space = $O(V^2)$
It supports negative edges but no meaningful finite shortest paths through negative cycles.
A negative cycle can be detected when after processing:
dist[i][i] < 0 for some vertex i.

17.23. Dijkstra vs Bellman-Ford vs Floyd-Warshall

Algorithm	Negative Edges	Negative Cycle Detection	Source Type	Time
Dijkstra	No	No	Single source	O((V+E) log V)
Bellman-Ford	Yes	Yes	Single source	O(VE)
Floyd-Warshall	Yes	Can detect	All pairs	$O(V^3)$
BFS	Only unweighted	—	Single source	O(V+E)
0-1 BFS	Weights only 0/1	—	Single source	O(V+E)

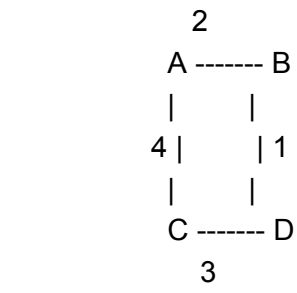
17.24. Minimum Spanning Tree

A **Spanning Tree** of a connected undirected graph:

- includes every vertex,
- has exactly V - 1 edges,
- contains no cycle.

A **Minimum Spanning Tree (MST)** is a spanning tree with minimum possible total edge weight.

Example:



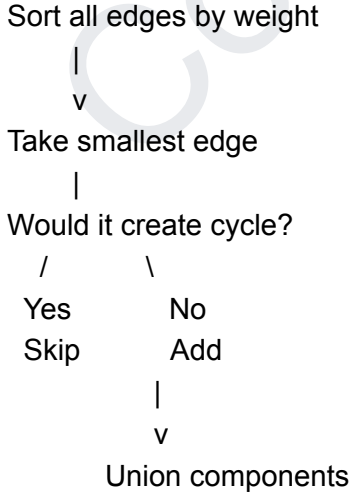
An MST selects enough edges to connect all vertices with minimum cost and no cycles.

Shortest Path Tree != Minimum Spanning Tree

The shortest path minimizes distances from a source. MST minimizes the total cost of connecting the entire graph.

17.25. Kruskal's Algorithm

Kruskal builds the MST by repeatedly choosing the smallest-weight edge that does **not create a cycle**. Main tools: Sorting + Disjoint Set Union (DSU)



CodeWithNishchal

Kruskal Code

```
static int kruskal(
        int n,
        List<WEdge> edges) {
    edges.sort(
        Comparator.comparingInt(e -> e.weight)
    );
    DSU dsu = new DSU(n);
    int totalWeight = 0;
    int edgesUsed = 0;
    for (WEdge edge : edges) {
        if (dsu.find(edge.from)
                != dsu.find(edge.to)) {
            dsu.union(edge.from, edge.to);
            totalWeight += edge.weight;
            edgesUsed++;
            if (edgesUsed == n - 1) {
                break;
            }
        }
    }
    return totalWeight;
}
```

Complexity is dominated by sorting: O(E log E)
DSU operations are almost constant amortized time.

17.26. Prim's Algorithm

Prim grows the MST from a starting vertex.
Instead of globally sorting all edges, it repeatedly chooses: minimum-weight edge connecting the current MST to an unvisited vertex
Use: Min Heap
Conceptual flow:
Start at any node
    |
Add all outgoing edges
    |
Take cheapest edge
    |
If destination unvisited
→ add vertex to MST
→ add its edges
    |
Repeat

Prim Code

```
static int prim(
        List<List<Edge>> graph) {
    int n = graph.size();
    boolean[] visited = new boolean[n];
    PriorityQueue<int[]> pq =
        new PriorityQueue<>(
            Comparator.comparingInt(a -> a[1])
        );
    pq.offer(new int[]{0, 0});
    int totalWeight = 0;
    int visitedCount = 0;
    while (!pq.isEmpty()) {
        int[] current = pq.poll();
        int node = current[0];
        int weight = current[1];
        if (visited[node]) {
            continue;
        }
        visited[node] = true;
        visitedCount++;
        totalWeight += weight;

        for (Edge edge : graph.get(node)) {

            if (!visited[edge.to]) {
                pq.offer(
                    new int[]{
                        edge.to,
                        edge.weight
                    }
                );
            }
        }
    }

    if (visitedCount != n) {
        throw new IllegalArgumentException(
            "Graph is disconnected"
        );
    }

    return totalWeight;
}
```

Complexity with adjacency list and heap: O(E log V)

17.27. Kruskal vs Prim

Feature	Kruskal	Prim
Main idea	Pick globally smallest safe edge	Grow from current tree
Main structure	DSU	Min Heap
Needs sorted edges	Yes	No global sort
Good for	Sparse edge-list graphs	Adjacency-list graphs
Time	O(E log E)	O(E log V)
Cycle handling	DSU	Visited vertices

Both produce an MST for a connected weighted undirected graph.

17.28. Union-Find / Disjoint Set Union

DSU maintains groups of connected elements.
It supports two major operations:
find(x) → Which component contains x?
union(a, b) → Merge components containing a and b
Example:
{0,1,2}
{3,4}
{5}
DSU can quickly determine whether: two vertices are already connected

---

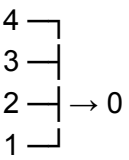
17.29. DSU Internal Structure

Each element has a parent. Initially:
0 → 0
1 → 1
2 → 2
3 → 3
Every node is its own component.
After unions:
0
|
1
|
2
the representative/root identifies the component.

CodeWithNishchal

17.30. Path Compression

Basic find() may follow many parents. Path compression directly connects visited nodes to the root.
Before: 4 → 3 → 2 → 1 → 0
After:



This makes future operations extremely fast.

17.31. Union by Rank / Size

When merging two components, attach the smaller/shallower tree under the larger/deeper one. This prevents DSU trees from becoming unnecessarily tall.

DSU Implementation

```
class DSU {
    int[] parent;
    int[] size;
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }}
    int find(int x) { if (parent[x] != x) { parent[x] = find(parent[x]);  }
        return parent[x];
    }
    boolean union(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
        if (rootA == rootB) {  return false;
        }
        if (size[rootA] < size[rootB]) {
            int temp = rootA;
            rootA = rootB;
            rootB = temp;
        }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        return true;
    }}
```

With path compression + union by size/rank: Amortized complexity ≈ O(α(n))
where α(n) is the inverse Ackermann function and grows so slowly that it behaves almost like constant time in practice.

17.32. Cycle Detection in Undirected Graph Using DSU

For every edge: (u, v)

check: find(u) == find(v)?

If yes: u and v are already connected. Adding this edge creates a cycle.

Otherwise: union(u, v)

Example:

0 -- 1

| |

2 ---

Process:

0-1 → union

0-2 → union

1-2 → already same component

→ cycle

This is especially useful when edges are provided as an edge list.

17.33. Bipartite Graph

A graph is **bipartite** if vertices can be divided into two groups such that every edge connects vertices from different groups.

Equivalent view: Can we color graph using only 2 colors such that adjacent vertices have different colors?

Example:

A ---- B

| |

D ---- C

Possible coloring:

A, C → Color 0

B, D → Color 1

17.34. Bipartite Check Using BFS

Maintain: color[node] = -1 for uncolored. Then assign: 0 or 1

For every edge: neighbor must receive opposite color

```
static boolean isBipartite(
        List<List<Integer>> graph) {
    int n = graph.size();
    int[] color = new int[n];
    Arrays.fill(color, -1);
    for (int start = 0; start < n; start++) {
        if (color[start] != -1) {
            continue;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        color[start] = 0;
        queue.offer(start);
```

CodeWithNishchal

```
        while (!queue.isEmpty()) {
            int node = queue.poll();
            for (int neighbor : graph.get(node)) {
                if (color[neighbor] == -1) {
                    color[neighbor] =
                            1 - color[node];
                    queue.offer(neighbor);

                } else if (
                    color[neighbor] == color[node]) {

                    return false;
                }}}}
    return true;
}
```

Complexity: O(V + E). An undirected graph is bipartite iff it contains **no odd-length cycle**.

17.35. Multi-Source BFS

Normal BFS begins from one source. Multi-source BFS begins from: multiple sources simultaneously

Example: Find distance of every cell/ from nearest hospital/fire/zero/rotten orange/etc.

Instead of running BFS from every cell, add all sources into the queue first.

Sources:

S . . S

. . . .

. S . .

Initial queue: [S1, S2, S3]. All sources have: distance = 0 Then BFS expands them together.

Multi-Source BFS Template

```
Queue<Integer> queue = new ArrayDeque<>();
for (each source) {
    distance[source] = 0;
    queue.offer(source);
}
while (!queue.isEmpty()) {
    int node = queue.poll();
    for (int neighbor : graph.get(node)) {
        if (distance[neighbor] == -1) {
            distance[neighbor] =
                    distance[node] + 1;
            queue.offer(neighbor);
}}}
```

This computes distance to the **nearest source** in an unweighted graph.

Complexity:O(V + E)

not: O(numberOfSources × (V+E))

WithNishchal

17.36. 0-1 BFS

0-1 BFS is designed for graphs where every edge weight is only:0 or 1
Instead of a priority queue, use: Deque
Rule:
Weight 0 → add to front
Weight 1 → add to back
Why? A zero-cost edge should be processed immediately because it does not increase distance.

0-1 BFS Flow

Current node
  |
Relax edge
  |
  +-- weight 0 → offerFirst()
  |
  +-- weight 1 → offerLast()

```
static int[] zeroOneBfs(
        List<List<Edge>> graph,
        int source) {
    int n = graph.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Deque<Integer> deque = new ArrayDeque<>();
    dist[source] = 0;
    deque.offerFirst(source);
    while (!deque.isEmpty()) {
        int node = deque.pollFirst();
        for (Edge edge : graph.get(node)) {
            int newDist = dist[node] + edge.weight;
            if (newDist < dist[edge.to]) {
                dist[edge.to] = newDist;

                if (edge.weight == 0) {
                    deque.offerFirst(edge.to);
                } else {
                    deque.offerLast(edge.to);
                }
            }
        }
    }
    return dist;
}
```

Complexity: O(V + E)
for 0/1-weight graphs. It is preferable to Dijkstra in this special case.

17.37. Shortest Path in a DAG

An important graph pattern is often missed.

If the graph is a weighted DAG:

1. Perform topological sort.
2. Process vertices in topological order.
3. Relax outgoing edges once.

Because every dependency is processed before later nodes, no repeated heap processing is needed.

Complexity: O(V + E)

This even supports negative edge weights as long as the graph remains acyclic.

17.38. Strongly Connected Components

Another important directed-graph concept:

A **Strongly Connected Component (SCC)** is a maximal set of vertices where every vertex can reach every other vertex.

Example:

0 → 1 → 2

↑ |

|_____|

All three belong to one SCC.

Common algorithms: Kosaraju's Algorithm, Tarjan's Algorithm

These are useful for:

- dependency analysis,
- cycle grouping,
- component compression,
- directed connectivity.

Typical complexity: O(V + E)

This can be studied separately as an advanced graph pattern.

17.39. Bridges and Articulation Points

Another important advanced category:

Bridge

An edge whose removal increases the number of connected components.

Articulation Point

A vertex whose removal disconnects the graph.

These are found using DFS concepts such as:

discovery time

low-link value

Tarjan-style algorithms solve them in: O(V + E)

They are important but are usually treated after basic graph traversal, shortest paths, MST, and DSU.

CodeWithNishchal

17.40. Graph Algorithm Selection Guide

Problem	Technique
Traverse graph	BFS / DFS
Count connected components	BFS / DFS / DSU
Shortest unweighted path	BFS
Nearest among many sources	Multi-source BFS
Shortest path with 0/1 weights	0-1 BFS
Shortest non-negative weighted path	Dijkstra
Negative edges	Bellman-Ford
Detect negative cycle	Bellman-Ford
All-pairs shortest paths	Floyd-Warshall
Directed dependency ordering	Topological Sort
Directed cycle	DFS path state / Kahn
Undirected cycle	DFS/BFS parent / DSU
Minimum spanning tree	Prim / Kruskal
Dynamic connectivity	DSU
Two-color graph	Bipartite BFS/DFS
Weighted DAG shortest path	Topological relaxation

17.41. Important Complexity Table

Algorithm	Time	Typical Space
BFS	O(V+E)	O(V)
DFS	O(V+E)	O(V)
Undirected Cycle DFS	O(V+E)	O(V)
Directed Cycle DFS	O(V+E)	O(V)
Kahn Topological Sort	O(V+E)	O(V)
DFS Topological Sort	O(V+E)	O(V)
Unweighted Shortest Path	O(V+E)	O(V)
Dijkstra	O((V+E) log V)	O(V+E)
Bellman-Ford	O(VE)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Prim	O(E log V)	O(V+E)
Kruskal	O(E log E)	O(V+E)
DSU operation	O(α(V)) amortized	O(V)
Bipartite Check	O(V+E)	O(V)
Multi-source BFS	O(V+E)	O(V)
0-1 BFS	O(V+E)	O(V)
DAG Shortest Path	O(V+E)	O(V)

CodewithNishchal

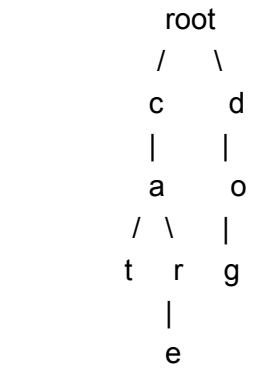
15. Trie

A **Trie** (also called a **Prefix Tree**) is a tree-based data structure designed for efficiently storing and searching **strings, words, and prefixes**.

Unlike a HashSet or HashMap, which treats a complete String as one key, a Trie stores Strings **character by character**, allowing words with common prefixes to share the same path.

Suppose we store: cat, car, care, dog

Trie structure:



More explicitly:

root
├── c
│ └── a
│ ├── t → word ends: "cat"
│ └── r → word ends: "car"
│ └── e → word ends: "care"
└── d
└── o
└── g → word ends: "dog"

The paths: root → c → a

are shared by: cat, car, care

This **prefix sharing** is the central idea of a Trie.

15.1. Why Trie Is Useful

Suppose we store: apple, app, application, apply. All four words begin with: app

A Trie stores that common prefix only once:

root
|
a
|
p
|
p
/ \
l (word "app" ends here)
|
e / i / y ...

CodeWithNishchal

Trie is particularly useful when problems involve:

- prefix search,
- dictionary lookup,
- autocomplete,
- search suggestions,
- spell checking,
- word matching,
- storing large collections of related Strings,
- longest common prefix,
- counting words with a given prefix.

15.2. Trie Node Structure

Each Trie node usually stores: Children + Information about whether a word ends here
For lowercase English letters:

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    boolean isEndOfWord;
}
```

Each index represents:
0 → a
1 → b
2 → c
...
25 → z
Character mapping: int index = ch - 'a'; Example: 'c' - 'a' = 2
So: children[2] represents the child corresponding to 'c'.

15.3. Why isEndOfWord Is Necessary

Consider storing: app apple
root
|
a
|
p
|
p ← "app" ends here
|
l
|
e ← "apple" ends here
If we only stored paths, we could not tell whether: app is an actual word or only a prefix of: apple
Therefore each node commonly has: boolean isEndOfWord; For the node representing the last p: isEndOfWord = true
For intermediate prefix nodes: isEndOfWord = false unless a word also ends there.

CodeWithNishchal

15.4. Trie Root

A Trie normally begins with an empty root node.

root
|
c
|
a
|
t

The root itself does not represent a character. It represents: starting point of all stored words

```
class Trie {
    private final TrieNode root;
    Trie() {
        root = new TrieNode();
    }
}
```

15.5. Insert Operation

To insert: "cat" , start from the root and process each character. c → a → t. For every character:

1. Find its corresponding child.
2. If a child does not exist, create it.
3. Move to that child.
4. After the final character, mark isEndOfWord = true.

Initially: root, Insert: cat

root
|
c
|
a
|
t*

* means: word ends here

Now insert: car

c already exists.

a already exists.

Create r.

```
      root
        |
        c
        |
        a
      /   \
     t*    r*
```

Common prefix: ca is reused.

CodeWithNishchal

```
public void insert(String word) {
    TrieNode current = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (current.children[index] == null) {
            current.children[index] = new
TrieNode();
        }
        current = current.children[index];
    }

    current.isEndOfWord = true;
}
```

current starts at root. For every character:
Does the required edge exist?
|
No → create node
|
v
Move current to child
After processing all characters: current.isEndOfWord = true; marks that complete word as stored.

15.6. Search Complete Word

Suppose Trie contains: cat, car, care
Search:car
Follow: **root** → **c** → **a** → **r**
If every character exists **and** the final node has: isEndOfWord == true then the word exists.

```
public boolean search(String word) {
    TrieNode current = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (current.children[index] == null) {
            return false;
        }
        current = current.children[index];
    }
    return current.isEndOfWord;
}
```

Important distinction: If Trie contains: apple then searching: app
may successfully reach the node representing p, but: isEndOfWord == false
unless "app" itself was inserted.
Therefore: search("app") = false
if only "apple" was inserted.

Nishchal

15.7. Prefix Search

Prefix search asks:
"Does any stored word start with this prefix?"
Suppose Trie contains: apple, application, apply
Then:
startsWith("app") → true
startsWith("appl") → true
startsWith("bat") → false
Unlike complete-word search, we do **not** require: isEndOfWord == true at the final prefix node.

```
public boolean startsWith(String prefix) {
    TrieNode current = root;
    for (char ch : prefix.toCharArray()) {
        int index = ch - 'a';
        if (current.children[index] == null) {
            return false;
        }
        current = current.children[index];
    }
    return true;
}
```

15.8. Search vs Prefix Search

Operation	Requirement at Final Node
search(word)	Path exists + isEndOfWord == true
startsWith(prefix)	Path only needs to exist

Example Trie:
apple

Query	Result
search("apple")	true
search("app")	false
startsWith("app")	true
startsWith("apple")	true
startsWith("bat")	false

Code With Nishchal

15.9. Complete Basic Trie Implementation

The following implementation assumes only lowercase English letters a-z.

```
class Trie {
    static class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEndOfWord;
    }
    private final TrieNode root;
    public Trie() { root = new TrieNode();
    }
    public void insert(String word) {
        TrieNode current = root;
        for (char ch : word.toCharArray()) {

            int index = ch - 'a';

            if (current.children[index] == null) {
                current.children[index] =
                        new TrieNode();
            }
            current = current.children[index];
        }
        current.isEndOfWord = true;
    }
    public boolean search(String word) {
        TrieNode current = root;
        for (char ch : word.toCharArray()) {
            int index = ch - 'a';
            if (current.children[index] == null) {
                return false;
            }
            current = current.children[index];
        }
        return current.isEndOfWord;
    }
    public boolean startsWith(String prefix) {
        TrieNode current = root;
        for (char ch : prefix.toCharArray()) {
            int index = ch - 'a';
            if (current.children[index] == null) { return false;
            }
            current = current.children[index];
        }
        return true;
    }}
```

Usage:
Trie trie = new Trie();
trie.insert("apple");
trie.insert("app");

System.out.println(trie.search("apple")); // true
System.out.println(trie.search("app")); // true
System.out.println(trie.search("ap")); // false
System.out.println(trie.startsWith("ap")); // true

15.10. Time Complexity

Let: L = length of the word/prefix
For an array-based child representation:

Operation	Time
Insert	O(L)
Search word	O(L)
Prefix search	O(L)
Delete	O(L) plus cleanup
Traverse all Trie nodes	O(total nodes)

The algorithm follows at most one child per character. Therefore search does **not directly depend on the number of stored words**.

15.11. Space Complexity

Suppose total number of characters inserted across all words is:T
In the worst case, when little prefix sharing occurs:Number of Trie nodes = O(T)
However, with: TrieNode[] children = new TrieNode[26];
every node reserves space for 26 references even if only one or two are used.
Therefore Tries can consume substantial memory.

15.12. Array Children vs HashMap Children

There are two common ways to store children.

Fixed Array

TrieNode[] children = new TrieNode[26];
Advantages:

- direct child access, simple, fast, predictable O(1) character lookup.

Disadvantages:

- wastes memory when most child positions are empty,
- tied to a known alphabet.

CodeWithNishchal

HashMap

```
Map<Character, TrieNode> children =
        new HashMap<>();

Node:
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEndOfWord;
}
```

Advantages:

- stores only existing children,
- supports larger/flexible character sets.

Disadvantages:

- hashing overhead,
- more object overhead,
- average rather than strict constant-time child lookup.

Comparison:

Feature	Array	HashMap
Child lookup	O(1)	Average O(1)
Memory for sparse nodes	Higher	Often better
Alphabet	Fixed	Flexible
Implementation	Simple	Flexible
Common lowercase DSA use	Excellent	Also valid

For problems explicitly restricted to: 'a' to 'z'
a 26-element array is very common.

15.13. Deleting a Word

Deletion is more subtle than simply changing:
isEndOfWord = false;

Consider: car, care
Trie: c → a → r* → e*
Delete: car
We must not remove the r node because "care" still needs it.
Correct result: c → a → r → e*
Only: r.isEndOfWord becomes false.

CodeWithNishchal

15.14. When Can a Trie Node Be Removed?

A node can be removed only when:

1. It is not the end of another word.
2. It has no children.

Example: cat, car. Deleting "cat" allows removing node t because: t has no children and no remaining word ends there. But nodes: c, a must remain because "car" uses them.

Recursive Delete

```
public void delete(String word) {
    delete(root, word, 0);
}
private boolean delete(
        TrieNode node,
        String word,
        int depth) {
    if (depth == word.length()) {
        if (!node.isEndOfWord) { return false;
        }
        node.isEndOfWord = false;
        return hasNoChildren(node);
    }
    int index = word.charAt(depth) - 'a';
    TrieNode child = node.children[index];
    if (child == null) {
        return false;
    }
    boolean shouldDeleteChild =
            delete(child, word, depth + 1);
    if (shouldDeleteChild) {
        node.children[index] = null;
    }
    return !node.isEndOfWord
            && hasNoChildren(node);
}
private boolean hasNoChildren(TrieNode node) {
    for (TrieNode child : node.children) {
        if (child != null) {
            return false;
        }}
    return true;
}
```

The recursive call first reaches the end of the word, then removes unnecessary nodes while recursion unwinds.

15.15. Counting Words With a Prefix

A useful Trie extension is storing a counter in every node.

```
class TrieNode {
    TrieNode[] children = new TrieNode[26];
    int prefixCount;
    int wordCount;
}
```

Suppose we insert: apple, app, apply. The prefix: app is used by all three words.
Therefore the node representing the second p can store: prefixCount = 3

Insert With Counts

```
public void insert(String word) {
    TrieNode current = root;
    for (char ch : word.toCharArray()) {
        int index = ch - 'a';
        if (current.children[index] == null) {
            current.children[index] = new TrieNode();
        }
        current = current.children[index];
        current.prefixCount++;
    }
    current.wordCount++;
}
```

Now: wordCount → number of times exact word was inserted
prefixCount → number of inserted words passing through node. This is useful when duplicate words are allowed.

15.16. Count Words Starting With Prefix

```
public int countPrefix(String prefix) {
    TrieNode current = root;
    for (char ch : prefix.toCharArray()) {
        int index = ch - 'a';
        if (current.children[index] == null) {
            return 0;
        }
        current = current.children[index];
    }
    return current.prefixCount;
}
```

Example:
Words: app, apple, application, banana
countPrefix("app") → 3 . Time: O(length of prefix)

15.17. Autocomplete / Search Suggestions

Trie is naturally suited for autocomplete.

Suppose words are: car, card, care, careful, cat, dog

Input prefix: "car"

First locate the prefix node: root → c → a → r

Then perform DFS from that node.

Possible suggestions: car, card care careful

Flow:

Find Prefix Node

|

v

"car"

|

v

DFS through subtree

|

v

Collect every node where isEndOfWord = true

Complexity depends on:prefix length + number of Trie nodes explored for suggestions

A Trie makes reaching the prefix efficient; generating many suggestions naturally costs time proportional to the output explored.

15.18. Longest Common Prefix Using Trie

Suppose: flower, flow, flight

After insertion:

root

|

f

|

l

/ \

o i

The common path continues only while: current node has exactly one child AND current node is not the end of a word

Common path: f → l

Therefore: Longest Common Prefix = "fl"

Trie can solve this, although for a one-time longest-common-prefix problem a simple direct String comparison may use less memory.

CodeWithNishchal

15.19. Trie vs HashSet

Suppose a dictionary contains many Strings. A HashSet<String> provides excellent exact-word lookup: search exact word → average O(L)

because computing the String hash itself depends on its characters, though Java Strings cache their hash after computation. However, prefix search is different. With HashSet: "Does any word start with app?"

may require checking many stored words. With Trie: root → a → p → p directly determines whether that prefix exists.

Comparison:

Feature	Trie	HashSet
Exact word search	O(L)	Average O(L) considering key hashing/comparison
Prefix search	O(P)	Not naturally efficient
Autocomplete	Excellent	Poor without extra structure
Memory	Often high	Usually lower
Prefix sharing	Yes	No
Ordering traversal	Possible	Not inherent

15.20. Trie vs BST

Feature	Trie	BST
Key organization	Character path	Key comparison
Search String	O(L)	Depends on tree height and String comparisons
Prefix queries	Excellent	Less direct
Memory	Often larger	Usually smaller per stored key
Main use	String/prefix operations	Ordered keys

Trie complexity depends primarily on:
key length

rather than directly on:
number of stored keys

CodeWithNishchal

15.21. Trie Traversal

A Trie can be traversed using DFS.
If children are processed from: a → z
then stored lowercase words can be generated in lexicographical order.
Example: Trie words: app, apple, bat, ball
DFS in alphabetic child order can produce: app, apple, ball, bat
The current path can be maintained using: StringBuilder
During DFS: append character, recurse, delete last character
This resembles a backtracking pattern.

15.22. Word Dictionary With Wildcard

A common Trie pattern extends search so that:
'.'
matches any character.
Example: Stored, bad, dad, mad
Queries:
"bad" → true
".ad" → true
"b.." → true

For normal character: follow one child
For wildcard: try every existing child
This turns search into DFS/backtracking.
Worst-case complexity may become exponential when many wildcard positions are present because multiple Trie branches must be explored.

15.23. Word Search + Trie

Trie becomes very powerful when searching **many dictionary words simultaneously** in a character grid. Without Trie:
For every word → run board DFS separately
With Trie:
Build Trie of all words
|
Start DFS from board cells
|
Move through Trie at same time
|
If Trie path stops
→ prune board search immediately
This combines: Trie + DFS + Backtracking + Pruning
The Trie prevents exploring board prefixes that cannot lead to any dictionary word.

CodeWithNishchal

15.24. Binary Trie / Bitwise Trie

Trie is not limited to characters. A **Binary Trie** stores bits:0, 1

For integers, each level represents one bit. Example conceptually:

```
        root
       /    \
      0      1
     / \    / \
    0   1  0   1
```

It is commonly used in problems involving: Maximum XOR, Minimum XOR, XOR pairs, Bit-prefix queries. For a 32-bit integer, Trie depth is typically fixed around: 32 levels

This is an important Trie variant but conceptually uses the same principle:

key represented as a sequence

→ store sequence path in a tree

15.25. Prefix Matching Pattern

One of the most important Trie patterns is: Find the longest stored prefix, matching a given String.

Stored: a, app, apple

Input: application

Follow: a → p → p → l ...

Whenever: isEndOfWord == true, remember the most recent valid prefix.

This pattern appears in:

- dictionary replacement,
- prefix routing,
- word segmentation helpers,
- IP/prefix-style matching concepts.

15.26. Trie Complexity Summary

Let: L = word length, P = prefix length, T = total number of characters inserted

Operation	Time
Insert	O(L)
Exact Search	O(L)
Prefix Search	O(P)
Delete	O(L) plus child checks/cleanup
Count prefix	O(P)
Find prefix node	O(P)
Autocomplete	O(P + output/subtree explored)
Traverse all words	O(total Trie nodes + output)

Space: O(total created Trie nodes). Worst-case node count: O(T)

CodeWithNishchal

15.27. Common Trie Patterns

Trie
|
+-- Exact Word Lookup
|
+-- Prefix Lookup
|
+-- Prefix Counting
|
+-- Autocomplete
|
+-- Longest Common Prefix
|
+-- Dictionary + Wildcard Search
|
+-- Grid Word Search
|
+-- Binary Trie / XOR Problems

15.28. Common Mistakes

1. Confusing Prefix With Complete Word

A path existing does not necessarily mean a complete word exists.
apple inserted

path "app" exists
but: search("app")
is false unless app itself is marked as a word.

---

2. Forgetting isEndOfWord

Without an end marker, there is no reliable way to distinguish:
complete word
from: prefix only

---

3. Assuming children[26] Supports Every Character

This implementation works only for the alphabet it was designed for.
For: uppercase, digits, symbolsm, Unicode

either expand the representation or use a map.

4. Claiming Trie Is Always Better Than HashMap

Trie is excellent for prefix-based problems, but can have much larger memory overhead.
For exact lookup only: HashSet / HashMap
may be simpler and more memory-efficient.

CodeWithNishchal

16. Dynamic Programming

Dynamic Programming (DP) is an algorithmic technique used when a problem can be divided into smaller subproblems and the **same subproblems are solved repeatedly**.

Instead of recalculating the same result again and again, DP:

Solve a state once

↓

Store its answer

↓

Reuse it whenever needed again

The central idea is:

Recursion + Repeated Subproblems + Store Results = Dynamic Programming

DP is not simply "using an array." The real skill is deciding:

1. What does a state represent?
2. What choices/transitions are possible?
3. What are the base cases?
4. In what order should states be calculated?
5. What is the final state we need?

Once these five parts are clear, the code usually becomes straightforward.

16.1. Why Dynamic Programming Is Needed

Consider Fibonacci: **F(n) = F(n - 1) + F(n - 2)**

Naive recursion:

```
static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

For: fib(5) . the recursion tree contains repeated states:

```
                fib(5)
               /      \
          fib(4)       fib(3)
         /    \       /    \
      fib(3)  fib(2)  fib(2) fib(1)
      /   \
   fib(2) fib(1)
```

Notice:

fib(3)

fib(2)

are calculated multiple times.

Naive recursion: Time ≈ O(2^n). But there are only: n + 1

different states: fib(0), fib(1), ..., fib(n)

If every state is calculated once: Time = O(n). That is the basic power of DP.

CodeWithLishchal

16.2. Two Properties Behind DP

DP usually applies when the problem has:

Overlapping Subproblems

The same smaller state appears repeatedly.

Example:

fib(5)

↓

fib(3) appears multiple times

Without storing results: calculate again, calculate again, calculate again

With DP: calculate once, store, reuse

Optimal Substructure

The solution to a larger problem can be constructed using solutions to smaller subproblems.

Example:

Minimum cost to reach i = minimum(cost to reach i-1, cost to reach i-2) + current cost

Many shortest path, knapsack, sequence, grid, and partition problems have this property.

16.3. The Most Important DP Concept — State

A **state** represents one unique subproblem. Example: dp[i]

might mean: Maximum money that can be robbed from houses 0...i or: Number of ways to reach step i or: Minimum cost to reach index i

The meaning depends entirely on the problem. Before writing DP code, write: dp[...] = ? in plain English. Example: dp[i] = maximum answer considering elements from index 0 to i

If you cannot clearly explain what dp[i] means, the DP has not yet been properly defined.

16.4. State Dimensions

The number of parameters needed to uniquely describe a subproblem usually determines the DP dimensions.

1D State dp[i]

State depends on one changing parameter.

Examples: Fibonacci/ Climbing Stairs/ House Robber/ Maximum Sum

2D State

dp[i][j]

Two parameters determine the subproblem.

Examples: Grid path/ LCS/ Knapsack/ Edit Distance

3D or Higher

dp[i][j][k]

Used when three independent pieces of information are required.

Examples: index/ remaining operations/ current state. But higher-dimensional DP can consume enormous memory, so always ask whether some dimensions can be removed.

CodeWithNishchal

16.5. Transition

The **transition** explains how the answer for one state is obtained from smaller states.
Example Fibonacci: dp[i] = dp[i - 1] + dp[i - 2]
Climbing Stairs: Ways to reach i = ways from i-1 + ways from i-2
House Robber: dp[i] = max(dp[i - 1], nums[i] + dp[i - 2])
The transition is usually the mathematical heart of the DP solution.

16.6. Base Cases

Base cases define the smallest states whose answers are already known.
Example Fibonacci: dp[0] = 0, dp[1] = 1
Climbing stairs: dp[0] = 1, dp[1] = 1
The correct base case depends on the exact meaning of the state.
A large number of DP bugs come from: Correct transition + Incorrect initialization

16.7. DP Workflow

For almost every DP problem, follow this order:
1. Start from recursion / choices.
2. Identify changing parameters.
3. Define the DP state.
4. Write recurrence / transition.
5. Write base cases.
6. Memoize recursion.
7. Convert to tabulation if useful.
8. Determine answer location.
9. Analyze complexity.
10. Optimize memory if possible.

Visual:

Problem
↓
Choices
↓
Recursive relation
↓
Repeated states?
↓
Define state
↓
Memoization
↓
Tabulation
↓
Space optimization

This is much easier than trying to invent a DP table directly.

CodeWithNishchal

16.8. Memoization — Top-Down DP

Memoization means: Recursion + Cache
We start with recursive logic and save every computed state.

Fibonacci:

```
static int fib(int n, int[] dp) {
    if (n <= 1) {
        return n;
    }

    if (dp[n] != -1) {
        return dp[n];
    }
    dp[n] = fib(n - 1, dp) + fib(n - 2, dp);
    return dp[n];
}
```

Initialization:

```
int[] dp = new int[n + 1];
Arrays.fill(dp, -1);
int answer = fib(n, dp);
```

Flow:

```
fib(5)
  |
Is dp[5] stored?
  |
 No
  |
calculate recursively
  |
store dp[5]
```

Later:

```
fib(5)
  |
dp[5] already exists
  |
return immediately
```

Each state is evaluated only once.

CodeWithNishchal

16.9. Tabulation — Bottom-Up DP

Tabulation removes recursion and calculates states in an order where required previous states are already available.

Fibonacci:

```
static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    int[] dp = new int[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Flow:

dp[0], dp[1]
↓
dp[2]
↓
dp[3]
↓
dp[4]
↓
dp[n]

16.10. Memoization vs Tabulation

Feature	Memoization	Tabulation
Direction	Top-down	Bottom-up
Uses recursion	Yes	No
Calculates states	Only visited states	Usually all states
Stack overhead	Yes	No
Easier from recursion	Usually	Requires dependency order
Stack overflow risk	Possible	No recursive stack
Space optimization	Sometimes harder	Often easier

A useful learning strategy:

Recursive solution → Memoization/ → Tabulation/ → Space Optimization

CodeWithNishchal

16.11. Space Optimization

Suppose: dp[i] = dp[i-1] + dp[i-2]

To calculate dp[i], we do not need the complete DP array.
We only need: previous, previousPrevious
Therefore:

```
static int fib(int n) {
    if (n <= 1) {
        return n;
    }
    int prev2 = 0;
    int prev1 = 1;
    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Complexity changes:
Before: Time = O(n) Space = O(n)
After: Time = O(n) Space = O(1)
General rule: If current DP state depends only on a few previous states, full history may not be needed.

16.12. 1D Dynamic Programming

1D DP uses: dp[i]
where one parameter usually represents: index/ position/ amount/ length
Important 1D patterns include: Linear DP/ Take / Skip DP/ Prefix-like DP/ State-machine DP

16.13. Linear DP — Climbing Stairs

You are climbing a staircase with n steps.
At each move, you can climb either: 1 step or 2 steps
Find the **number of distinct ways** to reach the top. Example: n = 4
Possible ways:
1 + 1 + 1 + 1
1 + 1 + 2
1 + 2 + 1
2 + 1 + 1
2 + 2
Therefore: Answer = 5

CodeWithNishchal

Algorithm — Linear Dynamic Programming

To reach stair i, the last move must come from either: i - 1 using a 1-step move or i - 2 using a 2-step move

Therefore: dp[i] = dp[i - 1] + dp[i - 2]

Steps:

1. Create a DP array of size n + 1.
2. Set base cases: dp[0] = 1, dp[1] = 1
3. For every stair from 2 to n: dp[i] = dp[i - 1] + dp[i - 2]
4. Return: dp[n]

Main idea: Ways to reach current stair = ways from previous stair + ways from two stairs before

Code

```
static int climbStairs(int n) {
    if (n <= 1) { return 1;
    }
    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Time = O(n) Space = O(n)

The space can be optimized to O(1) because only the previous two values are needed.

16.14. Decision DP — House Robber

You are given an array nums where nums[i] represents the money available in the ith house.

You cannot rob two adjacent houses.

Find the **maximum amount of money** you can rob.

Example: nums = [2, 7, 9, 3, 1] Answer: 12

One optimal choice is: 2 + 9 + 1 = 12

Algorithm — Take or Skip DP

At every house i, there are two choices:

1. Skip the current house. Then the best answer remains: dp[i - 1]
2. Rob the current house.

Then we cannot rob the previous house, so: nums[i] + dp[i - 2]

Therefore: dp[i] = max(dp[i - 1], nums[i] + dp[i - 2])

CodeWithNishchal

Steps:

1. Set: dp[0] = nums[0]
2. For the second house: dp[1] = max(nums[0], nums[1])
3. For every house from index 2 onward:
    - Calculate the value if we skip it.
    - Calculate the value if we take it.
    - Store the maximum.
4. Return the last DP value.

Main idea:
Skip current house → dp[i - 1]
Take current house → nums[i] + dp[i - 2]

Choose the better one

```
static int rob(int[] nums) {

    if (nums.length == 1) {
        return nums[0];
    }

    int[] dp = new int[nums.length];

    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {

        dp[i] = Math.max(
                dp[i - 1],
                nums[i] + dp[i - 2]
        );
    }

return dp[nums.length - 1];
}
```

Complexity

Time = O(n) Space = O(n)

This is a common DP pattern: Take vs Skip

Nishchal

16.15. Decision DP Pattern

General form: answer(state) =
best(
    choose option 1,
    choose option 2,
    ...
)
Examples: Take / Skip, Buy / Don't Buy, Include / Exclude, Cut / Don't Cut, Match / Don't Match
Decision DP is one of the broadest DP categories.

16.16. 2D Dynamic Programming

2D DP uses: dp[i][j]
because the answer depends on two variables.
Common interpretations: row + column/ index1 + index2/ item + capacity/ left + right
Major examples: Grid DP, LCS, Edit Distance, Knapsack, Interval DP

16.17. Grid DP

A grid DP state normally represents: dp[row][col]
Examples: Number of ways to reach cell/ Minimum cost to reach cell/ Maximum reward to reach cell. Suppose movement is allowed: Right, Down
Then to reach: (r, c). we can arrive from:(r - 1, c) or (r, c - 1) This directly creates the transition.

16.18. Unique Paths

Given a grid with rows rows and cols columns, count the number of unique paths from the **top-left cell** to the **bottom-right cell**.

You can move only: Right or Down

Example:

S . .

. . .

. . E

Algorithm — Grid DP

Define: dp[r][c], as the number of ways to reach cell (r, c). To reach any cell, we can only come from: Top → dp[r - 1][c] or Left → dp[r][c - 1]. Therefore: dp[r][c] = dp[r - 1][c] + dp[r][c - 1]

Steps:

1. Create a rows × cols DP table.
2. Fill the first column with 1 because there is only one way to move straight down.
3. Fill the first row with 1 because there is only one way to move straight right.
4. For every remaining cell: dp[r][c] = top + left
5. Return: dp[rows - 1][cols - 1]

Main idea:

Ways to current cell = Ways from top + Ways from left

CodeWithNishchal

```
static int uniquePaths(int rows, int cols) {
    int[][] dp = new int[rows][cols];
    for (int r = 0; r < rows; r++) {
        dp[r][0] = 1;
    }
    for (int c = 0; c < cols; c++) {
        dp[0][c] = 1;
    for (int r = 1; r < rows; r++) {
        for (int c = 1; c < cols; c++) {
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
        }
    }
    return dp[rows - 1][cols - 1];
}
```

Time = O(rows × cols) Space = O(rows × cols)

16.19. Minimum Path Sum

Given a grid where each cell contains a cost, find the **minimum path sum** from the top-left cell to the bottom-right cell.

You can move only: Right or Down

Example:

1 3 1

1 5 1

4 2 1

The minimum path sum is: 7

One minimum-cost path is: 1 → 3 → 1 → 1 → 1

Algorithm — Grid DP

Define: dp[r][c]

as the **minimum cost required to reach cell (r, c)**. To reach any cell, we can only come from:

Top → dp[r - 1][c] or Left → dp[r][c - 1]

So we choose the cheaper path and add the current cell cost:

dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])

Steps:

1. Set the starting cell: dp[0][0] = grid[0][0]
2. Fill the first row because it can only be reached from the left.
3. Fill the first column because it can only be reached from above.
4. For every remaining cell:
    - Take the minimum of top and left.
    - Add the current cell value.
5. Return:
   dp[rows - 1][cols - 1]

Main idea:

Minimum cost to current cell = Current cost + Minimum(top, left)

CodeWithNishchal

```
static int minPathSum(int[][] grid) {
    int rows = grid.length;
    int cols = grid[0].length;
    int[][] dp = new int[rows][cols];

    dp[0][0] = grid[0][0];

    // First row
    for (int c = 1; c < cols; c++) {
        dp[0][c] = dp[0][c - 1] + grid[0][c];
    }

    // First column
    for (int r = 1; r < rows; r++) {
        dp[r][0] = dp[r - 1][0] + grid[r][0];
    }
    // Remaining cells
    for (int r = 1; r < rows; r++) {
        for (int c = 1; c < cols; c++) {
            dp[r][c] =
                    grid[r][c]
                    + Math.min(
                        dp[r - 1][c],
                        dp[r][c - 1]
                    );
        }
    }
    return dp[rows - 1][cols - 1];
}
```

Time = O(rows × cols) Space = O(rows × cols)

Common DP idea:

Counting problems → Add possibilities

Optimization problems → Choose min/max possibility

16.20. Grid DP Recognition

Think of grid DP when:

Movement follows restricted directions + same cells/states can be reached through multiple previous paths

Common transitions: Top, Left, Diagonal, Previous row, Neighboring columns

Be careful: If movement allows arbitrary revisiting or cycles, a simple grid DP may no longer work directly. It may instead become a graph problem.

16.21. Knapsack Pattern

Knapsack is one of the most important DP families.

General idea: Items + Capacity / Target + Choose or skip items

State commonly looks like: dp[index][capacity]

meaning: Best answer using items from index onward with remaining capacity

---

16.22. 0/1 Knapsack

Given n items, where each item has: weight[i] → weight of the item
value[i] → value of the item and a knapsack with maximum capacity W, find the **maximum total value** that can be obtained without exceeding the capacity.

Each item can be used: 0 times or 1 time

So for every item, we have two choices: Take it or Skip it

Algorithm — 0/1 Knapsack Memoization

Define: solve(index, capacity)

as the maximum value we can obtain using items from index onward with the given remaining capacity.

For every item: **Skip**

Do not take the current item: skip = solve(index + 1, capacity)

Take If: weights[index] <= capacity then take the item:

take = values[index] + solve(index + 1, capacity - weights[index])

Then choose the better option: max(take, skip)

Store the result in DP so the same state is not calculated again.

Main idea: For each item → Take or Skip → Store best answer

Code — Memoization

```
static int knapsack(
        int index,
        int capacity,
        int[] weights,
        int[] values,
        int[][] dp) {
    if (index == weights.length) {  return 0;
    }
    if (dp[index][capacity] != -1) {
        return dp[index][capacity];
    }
    // Skip current item
    int skip = knapsack(
            index + 1,
            capacity,
            weights,
```

WithNishchal

```
            values,
            dp
        );
        int take = 0;
        // Take current item if it fits
        if (weights[index] <= capacity) {

            take = values[index]
                    + knapsack(
                        index + 1,
                        capacity - weights[index],
                        weights,
                        values,
                        dp
                    );
        }
        return dp[index][capacity]  = Math.max(take, skip);
    }
```

Time = O(n × capacity) Space = O(n × capacity)

16.23. 0/1 Knapsack — Tabulation

Algorithm

Define: dp[i][c]

as the maximum value obtainable using the **first i items** with capacity c.

For every item and capacity:

Skip

Do not take the current item: skip = dp[i - 1][c]

TakeIf the current item fits: take = value[i - 1] + dp[i - 1][c - weight[i - 1]]

Then: dp[i][c] = max(skip, take)

Steps:

1. Create a DP table of size:

(n + 1) × (capacity + 1)

2. Initialize row 0 with 0 because with no items, maximum value is 0.
3. For every item:
    - Try every capacity.
    - Calculate skip.
    - Calculate take if the item fits.
    - Store the maximum.
4. Return: dp[n][capacity]

Main idea:

Use first i items → Take current or Skip current → Store maximum

CodeWithNishchal

Code — Tabulation

```
static int knapsackTabulation(
        int[] weights,
        int[] values,
        int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];
    for (int i = 1; i <= n; i++) {
        for (int c = 0; c <= capacity; c++) {
            int skip = dp[i - 1][c];
            int take = 0;
            if (weights[i - 1] <= c) {
                take = values[i - 1]
                        + dp[i - 1][
                            c - weights[i - 1]
                        ];
            }
            dp[i][c] = Math.max(take, skip);
        }
    }
    return dp[n][capacity];
}
```

Complexity

Time = O(n × capacity) Space = O(n × capacity)

Core pattern: Take vs Skip

16.24. Knapsack Variants

Problem	Core Question
0/1 Knapsack	Max value under capacity
Subset Sum	Can subset produce target?
Equal Partition	Can array split into equal sums?
Count Subsets	Number of subsets reaching target
Target Sum	Assign signs to reach target
Unbounded Knapsack	Items may be reused
Coin Change	Min coins / number of ways

CodeVidyashchal

16.25. Subset Sum

Given an integer array and a target value, determine whether there exists a **subset whose sum is exactly equal to the target**. Each element can be used at most once.
Example: Array = [3, 2, 7, 1] Target = 6 A valid subset is: [3, 2, 1]. Therefore: Answer = true

Algorithm — Take or Skip DP

For every element, we have two choices: Take or Skip. Define: solve(index, remaining)

as whether it is possible to form the remaining target using elements from index onward.

Skip

Do not use the current element: skip = solve(index + 1, remaining)

Take

If the current element can fit into the remaining target: take = solve(index + 1, remaining - nums[index])

Then: answer = take || skip. Base cases: remaining == 0 → true because the target has been formed. index == nums.length → false because no elements remain.

Main idea: For each element → Take or Skip → If any choice forms target, return true

```
static boolean subsetSum(
        int index,
        int remaining,
        int[] nums,
        Boolean[][] dp) {
    if (remaining == 0) { return true;
    }
    if (index == nums.length) { return false;
    }
    if (dp[index][remaining] != null) { return dp[index][remaining];
    }
    boolean skip = subsetSum(
            index + 1,
            remaining,
            nums,
            dp
    );
    boolean take = false;
    if (nums[index] <= remaining) {
        take = subsetSum( index + 1, remaining - nums[index], nums, dp);
    }
    return dp[index][remaining] = take || skip;
}
```

Time = O(n × target) Space = O(n × target) Pattern: Take or Skip

Subset Sum is structurally similar to 0/1 Knapsack.

16.26. Equal Sum Partition

Suppose total array sum is: S

To split into equal parts: Each subset must sum to S / 2

Therefore: If S is odd → impossible

Else → solve Subset Sum with target S/2

This demonstrates an important interview skill: Many "new" DP problems are transformations of an existing DP pattern.

16.27. Unbounded Knapsack

Unlike 0/1 Knapsack, an item may be reused.

Main difference: 0/1:

Take item → move to next item

Unbounded: Take item → may remain at same item

Recursive transition: take = value[i] + solve(i, capacity - weight[i]) not: solve(i + 1, ...)

This small index difference completely changes the problem.

16.28. Coin Change

Coin Change is an **Unbounded Knapsack pattern** because each coin can be used multiple times.

There are two common versions: **Minimum Coins** and **Number of Ways**.

16.28.1 Minimum Coins

Given an array of coin denominations and a target amount, find the **minimum number of coins required** to make exactly that amount. Each coin can be used any number of times.

Example: coins = [1, 2, 5], amount = 11 One optimal solution is: 5 + 5 + 1

Therefore: Minimum Coins = 3. If the amount cannot be formed, return -1.

Algorithm — Minimum Coin DP

Define dp[a] as the minimum number of coins required to create amount a.

Initialize: dp[0] = 0 because zero coins are needed to create amount 0.

For every amount from 1 to amount, try every coin. If the coin can be used (coin <= a) and the remaining amount is reachable, then: dp[a] = min(dp[a], 1 + dp[a - coin])

The 1 represents using the current coin.

Main idea: For each amount → Try every coin → Take minimum coins

```
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];  Arrays.fill(dp, amount + 1); dp[0] = 0;
    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin <= a) {
                dp[a] = Math.min( dp[a],  1 + dp[a - coin] );}}}
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Time = O(n × amount) Space = O(amount)

CodeWithNishchal

16.28.2 Number of Ways

Given coin denominations and a target amount, find the **number of different combinations** that can make exactly that amount.

Each coin can be reused any number of times.

Example: coins = [1, 2, 5], amount = 5

Valid combinations include: [5], [2,2,1], [2,1,1,1], [1,1,1,1,1]

Therefore: Number of Ways = 4

Different orders of the same coins are **not counted separately**.

Algorithm — Count Combinations DP

Define dp[a] as the number of combinations that can create amount a.

Initialize: dp[0] = 1 because there is one way to create amount 0: choose no coins.

Process each coin first. For each coin, update amounts from coin to amount: dp[a] += dp[a - coin]

This means every way of creating a - coin can form a by adding the current coin.

The loop order is important:

Coins outside → Amount inside → Counts combinations

This prevents combinations such as [1,2,2] and [2,1,2] from being counted separately.

Main idea: Process one coin → Update all reachable amounts → Add number of previous ways

Code

```
static int coinChangeWays(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    dp[0] = 1;
    for (int coin : coins) {
        for (int a = coin; a <= amount; a++) {
            dp[a] += dp[a - coin];
        }
    }
    return dp[amount];
}
```

Time = O(n × amount) Space = O(amount)

Key Difference

Minimum Coins → Use min()

Number of Ways → Use addition

Coin outside, amount inside → Combinations

Amount outside, coin inside → Can count different orders as permutations

16.29. Sequence DP

Sequence DP works on: arrays/ strings/ ordered sequences

Important problems:

- Longest Common Subsequence
- Longest Increasing Subsequence
- Edit Distance
- Longest Palindromic Subsequence
- Distinct Subsequences
- The state often contains:
- one index or two indexes

16.30. Longest Common Subsequence — LCS

Given two strings, find the length of their **longest common subsequence**.

A subsequence keeps the relative order of characters but does not need to be contiguous.

Example: text1 = "abcde", text2 = "ace". LCS = "ace", so Length = 3

Algorithm — 2D DP

- Define dp[i][j] as the LCS length between the first i characters of text1 and the first j characters of text2.
- If the current characters match: text1[i - 1] == text2[j - 1], then include that character: dp[i][j] = 1 + dp[i - 1][j - 1].
- If they do not match, skip one character from either string and take the better result: dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]).

Main idea: Characters match → Take both | Characters differ → Skip one and take maximum

Code

```
static int lcs(String a, String b) {
    int n = a.length();
    int m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.max(
                        dp[i - 1][j],
                        dp[i][j - 1]
                );}}}
    return dp[n][m];
}
```

Time = O(n × m)

Space = O(n × m)

Nischal

16.31. Longest Increasing Subsequence — LIS

Given an integer array, find the length of the **longest strictly increasing subsequence**.

Example: nums = [10, 9, 2, 5, 3, 7, 101, 18]

One LIS is: [2, 3, 7, 18]

Therefore: Length = 4

Algorithm — DP

Define dp[i] as the length of the longest increasing subsequence ending exactly at index i.

Initially, every element itself forms a subsequence of length 1, so set all dp[i] = 1.

For every index i, check all previous indexes j < i. If nums[j] < nums[i], then nums[i] can extend the subsequence ending at j. Update: dp[i] = max(dp[i], dp[j] + 1).

The final answer is the maximum value in the dp array.

Main idea: For each element → Check smaller previous elements → Extend the best subsequence

```
static int lis(int[] nums) {
    int n = nums.length;
    if (n == 0) {
        return 0;
    }
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    int answer = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(
                        dp[i],
                        dp[j] + 1
                );
            }
        }
        answer = Math.max(answer, dp[i]);
    }
    return answer;
}
```

Complexity

Time = O(n²)

Space = O(n)

An optimized LIS solution using Binary Search runs in O(n log n), but this DP version clearly shows the state transition.

16.32. Edit Distance

Given two strings A and B, find the minimum number of operations required to convert A into B.

Allowed operations are: Insert, Delete, and Replace.

Example: A = "horse", B = "ros"

Minimum operations = 3

Algorithm — 2D Decision DP

Define dp[i][j] as the minimum operations required to convert the first i characters of A into the first j characters of B.

Base cases: dp[i][0] = i because converting i characters into an empty string requires i deletions. dp[0][j] = j because converting an empty string into j characters requires j insertions.

If the current characters match: A[i - 1] == B[j - 1], then no new operation is needed: dp[i][j] = dp[i - 1][j - 1].

Otherwise choose the minimum among: Delete → dp[i - 1][j], Insert → dp[i][j - 1], Replace → dp[i - 1][j - 1]. So: dp[i][j] = 1 + min(delete, insert, replace).

Main idea: Characters match → No operation | Different → Try Insert, Delete, Replace

```
static int editDistance(String a, String b) {
    int n = a.length();
    int m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 0; i <= n; i++) {
        dp[i][0] = i;
    }
    for (int j = 0; j <= m; j++) {
        dp[0][j] = j;
    }
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                int delete = dp[i - 1][j];
                int insert = dp[i][j - 1];
                int replace = dp[i - 1][j - 1];
                dp[i][j] = 1 + Math.min(
                        delete,
                        Math.min(insert, replace)
                );}}}
    return dp[n][m];
}
```

Time = O(n × m) Space = O(n × m)

16.33. Longest Palindromic Subsequence

Given a string, find the length of its **longest subsequence that is also a palindrome**.

Example: s = "bbbab"

One longest palindromic subsequence is "bbbb".

Therefore: Length = 4

Algorithm — Transform Into LCS

The Longest Palindromic Subsequence problem can be converted into an LCS problem.

Steps: Reverse the original string, then find the LCS between the original string and the reversed string.

For example: Original = "bbbab", Reverse = "babbb".

Then: Longest Palindromic Subsequence = LCS(original, reverse).

Main idea: Reverse string → Find LCS(original, reversed) → Result is LPS length

```
static int longestPalindromicSubsequence(String s) {
    String reversed = new StringBuilder(s).reverse().toString();
    return lcs(s, reversed);
}
static int lcs(String a, String b) {
    int n = a.length();
    int m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] =1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = Math.max(
                        dp[i - 1][j],
                        dp[i][j - 1]
                );}}}
    return dp[n][m];
}
```

Complexity

Time = O(n²)

Space = O(n²)

Pattern: Problem Transformation → Reverse String → Apply LCS

16.34. Partition DP

In **Partition DP**, we divide a sequence/range at different possible positions and choose the best partition.

General structure: solve(start, end) = best over every possible cut

Transition often looks like: for cut = start ... end

answer =

best(

  solve(left part)

  +

  solve(right part)

  +

  cost of current partition

)

Common examples:

- Matrix Chain Multiplication
- Palindrome Partitioning
- Burst Balloons
- Optimal cutting problems

16.35. Matrix Chain Multiplication

Given a sequence of matrices, find the **minimum number of scalar multiplications required to multiply all matrices together**.

The order of matrices cannot change, but we can change the **parenthesization**.

Example: A1 = 10 × 20, A2 = 20 × 30, A3 = 30 × 40. Two possible orders: (A1 × A2) × A3

Cost = (10×20×30) + (10×30×40) = 6000 + 12000 = 18000

A1 × (A2 × A3)

Cost = (20×30×40) + (10×20×40) = 24000 + 8000 = 32000

Therefore: Minimum Cost = 18000

Algorithm — Partition DP

Suppose dimensions are stored as: dims = [10, 20, 30, 40] Then matrix Ai has dimensions: dims[i - 1] × dims[i] Define dp[i][j] as the minimum cost to multiply matrices from Ai to Aj.

Steps:

1. If i == j, only one matrix exists, so no multiplication is required. Return 0.
2. Try every possible partition point k between i and j.
3. Split the matrices into: i ... k and k + 1 ... j.
4. Find the minimum cost of the left part: solve(i, k).
5. Find the minimum cost of the right part: solve(k + 1, j).
6. Cost to multiply the two resulting matrices is: dims[i - 1] × dims[k] × dims[j].
7. Total cost = leftCost + rightCost + multiplicationCost.
8. Try every k and store the minimum cost.

CodeWithNishchal

Code — Memoization

```
static int matrixChain(
        int i,
        int j,
        int[] dims,
        int[][] dp) {
    // Only one matrix
    if (i == j) {
        return 0;
    }
    if (dp[i][j] != -1) {
        return dp[i][j];
    }

    int minCost = Integer.MAX_VALUE;
    // Try every partition
    for (int k = i; k < j; k++) {

        int leftCost =  matrixChain(i, k, dims, dp);
        int rightCost = matrixChain(k + 1, j, dims, dp);
        int multiplyCost = dims[i - 1] * dims[k] * dims[j];
        int totalCost =  leftCost + rightCost + multiplyCost;
        minCost = Math.min(minCost, totalCost);
    }
    return dp[i][j] = minCost;
}
Calling it:
int[] dims = {10, 20, 30, 40};
int n = dims.length;
int[][] dp = new int[n][n];
for (int[] row : dp) {
    Arrays.fill(row, -1);
}
int answer = matrixChain(1, n - 1, dims, dp);
```

Complexity

Time = $O(n^3)$

Space = $O(n^2)$

Pattern: Range DP → Try every partition → Solve left and right → Combine → Take minimum

16.36. Interval DP

Interval DP operates on contiguous ranges: [left, right]
State: dp[left][right]
Examples:

- Burst Balloons
- Palindrome interval problems
- Optimal game strategy
- Matrix Chain Multiplication
- Minimum cost to merge/cut ranges

A common recurrence: dp[left][right] = best over k inside [left,right]
where k may represent: first operation/ last operation/ partition position

16.37. Interval DP Filling Order

Suppose: dp[left][right]
depends on smaller intervals.
Then tabulation usually processes: short intervals first → longer intervals later
Example:
length = 1
length = 2
length = 3
...
Pseudo-structure:

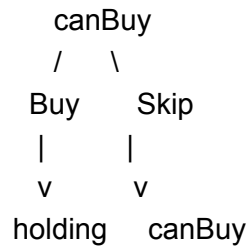
```
for (int len = 1; len <= n; len++) {
    for (int left = 0;
         left + len - 1 < n;
         left++) {
        int right = left + len - 1;
        // calculate dp[left][right]
    }
}
```

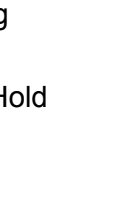
This ensures the required smaller ranges have already been solved.

16.38. DP on Stocks / State Machine DP

Some DP problems use a small logical state rather than just an index.
Example stock trading: index + holding/not holding + remaining transactions
State may be: dp[index][canBuy]
At each day: If allowed to buy: Buy or Skip
If holding: Sell or Hold
This is sometimes called: State Machine DP
because the solution moves between logical states.

16.39. State Transition View

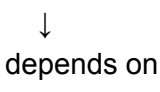




This way of thinking is useful when DP includes: boolean states/ transactions/ cooldown/ previous action

16.40. DP as a DAG

An extremely useful conceptual view:
Every DP state = graph node
Transition between states = directed edge
Because valid DP dependencies do not form cycles, the state graph behaves like a: DAG
Example:



Memoization performs: DFS through this state DAG
Tabulation performs: states in dependency/topological order
This explains why memoization and tabulation solve the same recurrence in different directions.

16.41. Bitmask DP

Bitmask DP is used when the state needs to represent a **subset of a small set**.
Suppose: n <= 20
A subset can be represented using an integer mask.
Example for 4 elements: mask = 0101
means: element 0 selected, element 1 not selected, element 2 selected, element 3 not selected
Check whether element i exists: (mask & (1 << i)) != 0
Add element: mask | (1 << i)
Remove: mask & ~(1 << i)

CodeWithNishchal

16.42. Bitmask DP State

A classic form: dp[mask][last]

meaning: Best answer after using the set represented by mask and currently ending at last
This appears in problems such as:

- Travelling Salesman Problem
- Assignment problems
- Visit-all-nodes variants
- Small-set scheduling
- Number of subsets: 2^n

Therefore typical complexity: O(2^n × n) or: O(2^n × n²) depending on transitions.
Bitmask DP is practical only when n is relatively small.

16.43. Travelling Salesman DP

Given n cities and the travel cost between every pair of cities, find the **minimum cost to start from one city, visit every city exactly once, and return to the starting city**.

A brute-force solution tries all possible city orders, which takes about O(n!).

Using **Bitmask DP**, we store which cities have already been visited and reduce the complexity to about O(2^n × n²).

Algorithm — Bitmask DP

Define dp[mask][city] as the minimum cost to visit all cities represented by mask and currently be at city.

Here, each bit of mask represents whether a city has been visited.

Example: mask = 1011 means cities 0, 1, and 3 are visited.

Steps:

1. Start from city 0 and mark it visited: mask = 1 << 0.
2. From the current city, try every city next.
3. If next is already visited, skip it.
4. Otherwise, mark it visited using: newMask = mask | (1 << next).
5. Add the travel cost from city to next and recursively solve the remaining cities.
6. Take the minimum cost among all possible next cities.
7. When all cities are visited, return the cost of travelling back to city 0.

Main idea: Current city + visited mask → Try every unvisited city → Mark visited → Recurse → Take minimum

```
static int tsp(
        int mask,
        int city,
        int[][] cost,
        int[][] dp) {
    int n = cost.length;
```

CodeWithNishchal

```
    // All cities visited
    if (mask == (1 << n) - 1) {
        return cost[city][0];
    }
    if (dp[mask][city] != -1) {
        return dp[mask][city];
    }
    int minCost = Integer.MAX_VALUE;

    for (int next = 0; next < n; next++) {

        // Skip already visited city
        if ((mask & (1 << next)) != 0) {
            continue;
        }
        int newMask = mask | (1 << next);
        int currentCost =
                cost[city][next]
                + tsp(
                    newMask,
                    next,
                    cost,
                    dp
                );

        minCost = Math.min(
                minCost,
                currentCost
        );
    }
    return dp[mask][city] = minCost;
}
```

Time = O(2^n × n²) Space = O(2^n × n)

Pattern: Bitmask DP → State = visited cities + current city → Try every unvisited city → Take minimum

16.44. Digit DP

Digit DP counts or optimizes numbers in a range while satisfying digit-related constraints.

Examples:

Count numbers <= N

with digit sum = K

Count numbers without repeated digits

Count numbers that do not contain digit 7

Count numbers with exactly K occurrences of some digit

Instead of iterating through every number, process the number digit by digit.

16.45. Digit DP Core State

A typical state is: dp[position][state][tight][started]
The important concepts are:

Position

Which digit are we currently choosing?

Tight

Are previous digits exactly equal to N's prefix?
If: tight = true
the current digit cannot exceed the corresponding digit of N.
If: tight = false
we can generally choose: 0 ... 9

Started

Tracks whether we have started placing non-leading-zero digits.
This is important when: 000123
should represent: 123
rather than a six-digit number.

Additional State

May track: digit sum/ remainder/ number of occurrences/ mask of used digits
Digit DP is mainly useful when: N is extremely large but has relatively few digits

---

16.46. Digit DP Range Trick

To count valid numbers in: [L, R]
calculate: count(R) - count(L - 1)
where: count(X)
means number of valid integers: 0 ... X
This is a standard Digit DP pattern.

16.47. DP on Trees

Trees naturally support DP because every subtree is a smaller independent structure.
A common state: dp[node]
represents information about the subtree rooted at node.
Typical problems:

- Subtree size
- Maximum independent set
- Tree diameter
- Tree matching
- Minimum cameras
- Include/exclude node

Tree DP is usually performed with: DFS + Bottom-up combination

---

CodeWithNishchal

16.48. Simple Tree DP — Subtree Size

Given a tree, find the **size of the subtree rooted at every node**.

The subtree size of a node means the total number of nodes contained in that node's subtree, including the node itself.

Example: If node 2 has children 4 and 5, then: subtreeSize[2] = 3 because the subtree contains: 2, 4, 5

Algorithm — DFS + Tree DP

Define dp[node] as the number of nodes in the subtree rooted at node.

Steps:

1. Start DFS from the root.
2. Initialize: dp[node] = 1 because every subtree contains the node itself.
3. Traverse all children of the current node.
4. Skip the parent so that DFS does not move back upward.
5. Recursively calculate the subtree size of each child.
6. Add each child's subtree size to the current node: dp[node] += dp[child]
7. Return dp[node].

Main idea: Start with 1 for current node → Calculate each child subtree → Add all child sizes

```
static int dfs(
        int node,
        int parent,
        List<List<Integer>> tree,
        int[] dp) {
    dp[node] = 1;
    for (int child : tree.get(node)) {
        if (child == parent) {
            continue;
        }
        dp[node] += dfs(
                child,
                node,
                tree,
                dp
        );
    }
    return dp[node];
}
```

Calling it:

int[] dp = new int[n];

dfs(0, -1, tree, dp);

Complexity

Time = O(n) Each node is processed once. Space = O(n) for the DP array and recursion stack.

Pattern: DFS → Solve child subtrees → Combine child answers into parent

16.49. Include / Exclude Tree DP

A common tree DP uses two states:
dp[node][0] → best answer if node is excluded
Dp[node][1] → best answer if node is included

Example: maximum-weight independent set on a tree. If current node is included: children cannot be included
If current node is excluded: each child may be included or excluded
Transition: include(node) = value[node] + sum(exclude(child))
exclude(node) = sum(max(include(child), exclude(child)))
This is essentially House Robber generalized from an array to a tree.

16.50. Rerooting DP

An advanced Tree DP pattern is **rerooting**.
First compute answers relative to one root.
Then transfer information so every node can be considered the root.
Typical use: Sum of distances from every node/ Best value when rooted at every vertex
Usually involves:
First DFS → child-to-parent information
Second DFS → parent-to-child information
This is an advanced extension of top-down + bottom-up DP.

16.51. Memoization Complexity

Memoization complexity is usually: Number of unique states × work per state
Example: state = (index, capacity)
Possible states: n × capacity
If each state does: O(1)
work: Time = O(n × capacity)
This is one of the easiest and most reliable ways to analyze DP.

16.52. How to Count DP States

Examples: **dp[i]**
i has n possible values. States = O(n)

dp[i][j]
i has n values
j has m values
States = O(nm)

dp[mask][i]
mask has 2^n possibilities
i has n possibilities
States = O(2^n × n)
Then multiply by the number of transitions examined per state.

CodeWithNishchal

16.53. Time Complexity Examples

DP	States	Work/State	Time
Fibonacci	n	O(1)	O(n)
Grid DP	n×m	O(1)	O(nm)
0/1 Knapsack	n×W	O(1)	O(nW)
LCS	n×m	O(1)	O(nm)
LIS basic	n	O(n)	O(n²)
Interval DP	O(n²)	O(n) cuts	O(n³)
Bitmask DP	2^n×n	often O(n)	O(2^n n²)

16.54. Space Optimization in 2D DP

Suppose: dp[row][col]
only depends on: previous row, current row
Then instead of: O(rows × cols)
memory, use: O(cols) or: two rows
Example: previous[], current[]
After finishing a row: previous = current
Some problems can be optimized even further to one array.

16.55. DP Table Filling Order

The correct filling order is determined by dependencies.
If: dp[i] depends on dp[i-1]
calculate left to right.
If: dp[i][j] depends on dp[i-1][j] and dp[i][j-1], process top-left toward bottom-right.
If: dp[left][right] depends on smaller intervals, process shorter intervals first.
Never memorize an iteration order without understanding: Which states must already exist before computing the current state?

16.56. Pull DP vs Push DP

Two equivalent ways to think about transitions:

Pull DP

Current state asks:
"Which previous states can reach me?"
Example: dp[i] = dp[i-1] + dp[i-2]

Push DP

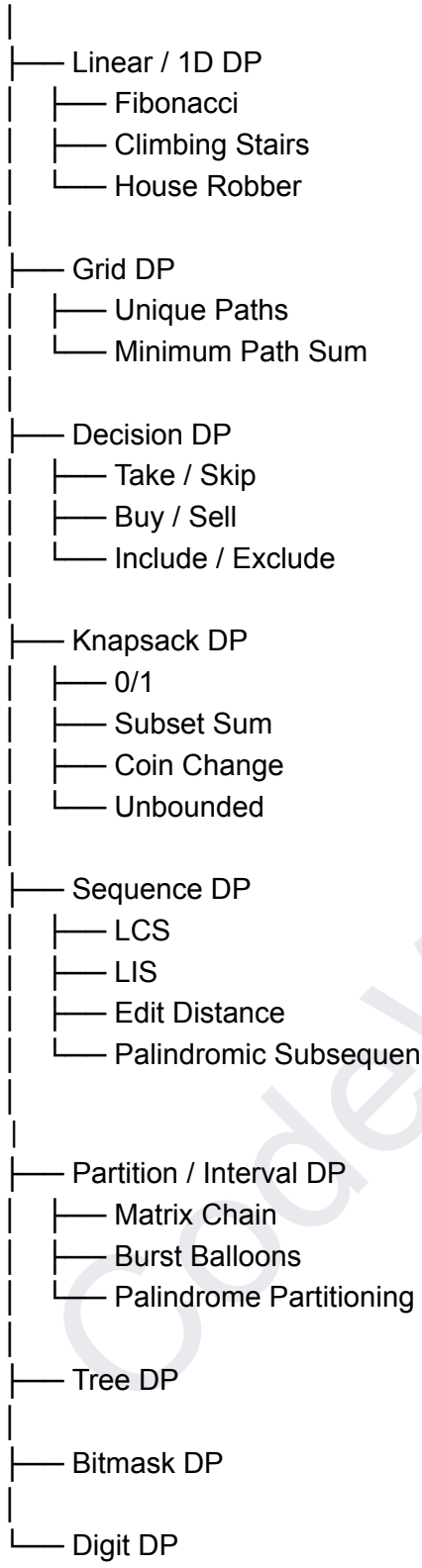
The current state sends its answer to future states.
Example: dp[i+1] += dp[i]
dp[i+2] += dp[i]
Both can solve the same problem.
Pull DP is often easier to reason about, while push DP can be natural for graph-like state transitions.

CodeWithNishchal

16.57. DP Pattern Classification

A useful practical classification is:

Dynamic Programming



CodeWithNishchal

16.58. How to Recognize a DP Problem

Strong signals include: Maximum / Minimum , Number of ways, Can we achieve…, Longest /, Shortest, Choose / Skip, Partition, Subsequence, Repeated decisions, Limited capacity

But these words alone do not guarantee DP.

Ask:

1. Can the problem be expressed recursively?
2. Do different recursive paths reach the same state?
3. Can a state be described using a small number of parameters?
4. Does the answer for a state depend only on smaller states?

If yes, DP is likely.

16.59. DP and Greedy

Do not use DP automatically when choices exist. Greedy works when a locally optimal choice can be proven to lead to a globally optimal answer.

DP is needed when: different choices must be explored because local best choice may not be globally best. Greedy: Choose one, Never reconsider DP: Evaluate alternatives, Store best state result

16.60. DP and Backtracking

Backtracking may explore: all possible choices

DP avoids repeatedly solving identical states.

Example:

Backtracking: same (index, remaining) reached repeatedly

Memoized DP: solve it once

Conceptually:

Backtracking

+

Repeated states

↓

Memoization

↓

Dynamic Programming

Not every backtracking problem can benefit from DP, because the full state may include the complete path/visited configuration.

16.61. DP and Divide & Conquer

Both solve smaller problems.

Divide & Conquer

Subproblems are generally: independent

Dynamic Programming

Subproblems: overlap

Difference:

Independent subproblems → Divide & Conquer

Repeated subproblems → DP

CodeWithNishchal

16.62. Common DP Mistakes

1. Undefined State Meaning

Writing:int[][] dp; without being able to explain dp[i][j] causes confusion immediately.
Always define the state first.

---

2. Wrong Base Case

Base cases must match the state definition. A correct recurrence with incorrect initialization still produces incorrect answers.

---

3. Memoizing Incomplete State

Suppose result depends on: index ,capacity, but cache stores only: dp[index]
Different capacities would incorrectly reuse the same answer.
Every variable that changes the answer must be represented in the state.

---

5. Optimizing Space Too Early

First build a correct recurrence. Then:
Memoization
→ Tabulation
→ Space optimization
Trying to derive a compressed DP immediately often causes mistakes.

16.63. DP Complexity Rule

The most important complexity formula is:
Time ≈ Number of DP states × Transitions checked per state
Space: DP table size + recursion stack if memoized
This rule works for nearly every DP problem.

16.64. Core Pattern Summary Table

Pattern	Typical State	Main Transition
Linear DP	dp[i]	Previous positions
Grid DP	dp[r][c]	Neighboring cells
Decision DP	dp[state]	Choose best option
Knapsack	dp[i][capacity]	Take / Skip
Sequence DP	dp[i][j]	Match / Skip
LIS-style	dp[i]	Previous compatible indexes
Partition DP	dp[l][r]	Try every partition
Interval DP	dp[l][r]	Smaller intervals
Tree DP	dp[node][state]	Combine children
Bitmask DP	dp[mask][state]	Add unused element
Digit DP	dp[pos][state][tight]	Try valid digit

CodeWithNishchal

17. Greedy Algorithms

A **Greedy Algorithm** builds a solution step by step by making the **best-looking choice at the current moment**, without reconsidering previous choices.

The basic idea is:

Current State
    |
Find best available choice
    |
    v
Make that choice
    |
    v
Update state
    |
    v
Repeat

Unlike Dynamic Programming or Backtracking, greedy algorithms usually do **not explore every alternative**.

Example: Need to schedule maximum number of meetings.

Greedy choice: Always select the meeting that finishes earliest.

Why? Because finishing earlier leaves the largest possible remaining time for future meetings.

That local decision can be proven to lead to a globally optimal answer.

17.1. Greedy Thinking

A greedy solution usually contains:

1. Candidates
2. A rule for choosing the best candidate
3. A validity condition
4. An update to the current state
5. No reconsideration of previous choices

Generic structure:

```
sortOrPrioritize(candidates);
for (Candidate candidate : candidates) {
    if (isValid(candidate)) {
        choose(candidate);
        updateState();
    }
}
```

The hardest part is usually **not coding**. The difficult part is deciding: "What should be greedy about?"

Examples: Earliest finishing interval / Highest profit/ Lowest cost/ Largest reachable position/ Smallest frequencies/ Best ratio

CodeWithNishchal

17.2. Greedy vs Dynamic Programming

Both may involve making choices, but they handle them differently.

Greedy

Choose what looks best now

↓

Commit

↓

Never reconsider

Dynamic Programming

Consider multiple possible decisions

↓

Compute their results

↓

Choose best final answer

Comparison:

Feature	Greedy	DP
Choices explored	Usually one	Multiple
Previous choices reconsidered	No	Indirectly yes
State table	Usually no	Usually yes
Speed	Often very fast	Usually more expensive
Correctness	Requires greedy-choice property	Requires recurrence
Example	Activity Selection	0/1 Knapsack

Important: A problem having "minimum" or "maximum" does NOT automatically mean greedy.

17.3. When Greedy Works

Greedy solutions generally rely on two properties.

Greedy Choice Property

A globally optimal solution can begin with the greedy choice.

Example: Activity Selection

→ one optimal solution exists that starts with the earliest-finishing activity

Optimal Substructure

After making the greedy choice, the remaining problem is still an optimal problem of the same type.

Choose activity A

↓

Remaining compatible activities

↓

Solve the same problem again

CodeWithNishchal

17.4. How Greedy Correctness Is Usually Proved

Greedy algorithms should not be accepted simply because they "seem right."

Common proof techniques include:

Exchange Argument

Show that if an optimal solution does not use the greedy choice, we can replace one of its choices with the greedy choice without making the solution worse.

Staying Ahead

Show that after every step, the greedy solution is at least as good as any alternative.

Structural Argument

Show that the problem naturally forces the locally optimal decision.

For interview problems, you usually do not need a formal mathematical proof, but you should understand **why the greedy rule cannot hurt future choices**.

17.5. Sorting Is Central to Greedy

Many greedy algorithms begin with sorting.

But the important question is:

Sort by what?

Possible rules: End time, Start time, Profit, Deadline, Weight, Ratio,Cost, Distance

Different sorting criteria create completely different algorithms.

This is one of the most important greedy skills.

17.6. Interval Greedy Pattern

Given a set of intervals represented as: [start, end]

select intervals according to the problem goal, such as maximizing non-overlapping intervals, scheduling the maximum number of activities, or removing the minimum number of overlapping intervals.

Example: Intervals = [[1,3], [2,4], [3,5], [6,8]]

A common greedy strategy is to prefer the interval that **finishes earliest**.

Algorithm — Sort by End Time

1. Sort all intervals by their ending time.
2. Select the first interval.
3. Store its ending time as lastEnd.
4. Traverse the remaining intervals.
5. If the current interval starts at or after lastEnd, select it.
6. Update lastEnd to the current interval's ending time.
7. Continue until all intervals are processed.

Why this works: choosing the interval that finishes earliest leaves the maximum remaining space for future intervals.

Main idea: Sort by end time → Pick earliest finishing interval → Skip overlaps → Repeat

CodeWithNishchal

Code — Maximum Non-Overlapping Intervals

```
static int maxNonOverlapping(int[][] intervals) {
    Arrays.sort(
            intervals,
            Comparator.comparingInt(a -> a[1])
    );
    int count = 0;
    int lastEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {
            count++;
            lastEnd = interval[1];
        }
    }
    return count;
}
```

Time = O(n log n), Space = O(1) extra, apart from sorting overhead.

17.7. Activity Selection

Given a list of activities represented as [start, end], select the **maximum number of non-overlapping activities** that can be performed by one person.

Example: [[1,2], [3,4], [0,6], [5,7], [8,9]]

One optimal selection is: [1,2] → [3,4] → [5,7] → [8,9]

Therefore: Maximum Activities = 4

Algorithm — Greedy by Ending Time

1. Sort all activities by their ending time.
2. Keep lastEnd as the ending time of the last selected activity.
3. Traverse the sorted activities.
4. If activity.start >= lastEnd, select the activity because it does not overlap.
5. Update lastEnd = activity.end.
6. Otherwise, skip the activity.
7. Continue until all activities are processed.

The greedy choice is to always select the compatible activity that **finishes earliest**, because it leaves the most time available for future activities.

Main idea: Sort by end time → Select compatible activity → Update lastEnd → Skip overlaps

```
static int maxActivities(int[][] activities) {
    Arrays.sort(
            activities,
            Comparator.comparingInt(a -> a[1])
    );
    int count = 0;
    int lastEnd = Integer.MIN_VALUE;
    for (int[] activity : activities) {
        if (activity[0] >= lastEnd) {
            count++;
            lastEnd = activity[1];
        }}
    return count;
}
```

Time = O(n log n) because of sorting. Space = O(1) extra, apart from sorting overhead.

17.8. Why Earliest Finish Works

Suppose two compatible choices are: A ends at 4, B ends at 8
Choosing: A
leaves time after 4.
Choosing: B
leaves time only after 8.
So choosing the earliest finish cannot reduce future scheduling possibilities.
This is the core exchange argument behind many interval greedy problems.

17.9. Non-Overlapping Intervals

Given a list of intervals [start, end], find the **maximum number of non-overlapping intervals** that can be selected. Example: [[1,2], [2,3], [3,4], [1,3]]

One valid maximum selection is: [1,2] → [2,3] → [3,4]

Therefore: Maximum Non-Overlapping Intervals = 3

Algorithm — Greedy by Ending Time

1. Sort all intervals by their ending time.
2. Maintain lastEnd, the end time of the last selected interval.
3. Traverse the sorted intervals.
4. If currentStart >= lastEnd, select the interval and update lastEnd = currentEnd.
5. Otherwise, skip the interval because it overlaps with the previously selected interval.
6. Continue until all intervals are processed.

Choosing the interval that **finishes earliest** leaves the most room for future intervals.

Main idea: Sort by end time → Accept non-overlapping interval → Update lastEnd → Skip overlapping interval

CodeWithNishal

```
static int maxNonOverlapping(int[][] intervals) {
    Arrays.sort(
            intervals,
            Comparator.comparingInt(a -> a[1]));
    int count = 0;
    int lastEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {
            count++;
            lastEnd = interval[1];
        }}
    return count;
}
```

Time = O(n log n), Space = O(1) extra, apart from sorting overhead.

17.10. Minimum Removals to Make Intervals Non-Overlapping

Given a list of intervals [start, end], find the **minimum number of intervals that must be removed so that the remaining intervals do not overlap**.

Example: [[1,2], [2,3], [3,4], [1,3]]

We can keep: [1,2] → [2,3] → [3,4]. Therefore: Minimum Removals = 1

Algorithm — Greedy by Ending Time

1. Sort all intervals by their ending time.
2. Maintain lastEnd, the ending time of the last interval we decided to keep.
3. Traverse the intervals in sorted order.
4. If currentStart >= lastEnd, keep the interval and update lastEnd = currentEnd.
5. Otherwise, skip the interval because it overlaps with an already selected interval.
6. Count the maximum number of intervals that can be kept.
7. Calculate: Minimum Removals = Total Intervals - Intervals Kept.

Choosing intervals that finish earlier leaves more space for future intervals.

Main idea: Sort by end time → Keep maximum non-overlapping intervals → Removals = n - keep

```
static int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(
            intervals,
            Comparator.comparingInt(a -> a[1]) );
    int keep = 0;
    int lastEnd = Integer.MIN_VALUE;
    for (int[] interval : intervals) {
        if (interval[0] >= lastEnd) {
            keep++;
            lastEnd = interval[1];}}
    return intervals.length - keep; }
```

Time = O(n log n),Space = O(1) extra, apart from sorting overhead.

NewWithNishchal

17.11. Alternative Strategy — Keep Smaller End

Another way is to sort intervals by **start time**.

When two intervals overlap, keep the interval that has the **smaller ending time** because it leaves more room for future intervals.

Example: A = [1,10], B = [2,4]

These intervals overlap, so keep [2,4] because it ends earlier.

Algorithm

1. Sort intervals by start time.
2. Keep track of the end of the previous interval.
3. If the current interval does not overlap, move forward normally.
4. If an overlap occurs, increment the removal count.
5. Keep the interval with the smaller end by updating: prevEnd = min(prevEnd, currentEnd).
6. Continue until all intervals are processed.

Main idea: Overlap → Remove one interval → Keep the one with smaller end

```
static int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(
            intervals,
            Comparator.comparingInt(a -> a[0])
    );
    int removals = 0;
    int prevEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        int currentStart = intervals[i][0];
        int currentEnd = intervals[i][1];
        if (currentStart < prevEnd) {
            removals++;
            prevEnd = Math.min(
                    prevEnd,
                    currentEnd
            );
        } else {
            prevEnd = currentEnd;
        }
    }
    return removals;
}
```

Both approaches use the same greedy principle: **prefer the interval that finishes earlier**.

17.12. Interval Greedy Recognition

When the goal is: maximize number of non-overlapping intervals
think: Earliest end time
When the goal is: remove minimum intervals
think: Keep maximum non-overlapping
When overlap decisions occur: prefer smaller ending point

17.13. Scheduling Greedy

Scheduling problems often contain: jobs, deadlines, durations, profits, resources
The key is to determine what should be prioritized.
Common rules include: Earliest deadline, Highest profit, Shortest duration, Earliest completion
The correct rule depends on the objective.

17.14. Job Sequencing With Deadlines and Profit

Given n jobs where each job takes exactly **1 unit of time** and has a deadline and profit, schedule jobs to **maximize the total profit**. A job earns profit only if it is completed on or before its deadline.

Job	Deadline	Profit
A	2	100
B	1	19
C	2	27
D	1	25
E	3	15

An optimal schedule is: C → A → E

Total Profit: 27 + 100 + 15 = 142

Algorithm — Highest Profit + Latest Available Slot

1. Sort all jobs in **descending order of profit**.
2. Find the maximum deadline D and create D available time slots.
3. Process jobs from highest profit to lowest profit.
4. For each job, search backward from its deadline to find the **latest free slot**.
5. If a free slot exists, schedule the job there and add its profit.
6. If no slot is available before its deadline, skip the job.
7. Continue until all jobs are processed.

Why use the latest available slot? Placing a job as late as possible keeps earlier slots free for jobs with smaller deadlines.

Main idea: Higher profit first → Place as late as possible → Preserve earlier slots

CodeWithNishchal

Example Flow

Jobs sorted by profit: A(100) → C(27) → D(25) → B(19) → E(15)
Initial slots: [_ , _ , _]
A, deadline 2: [_ , A , _]
C, deadline 2: [C , A , _]
D, deadline 1: slot 1 already occupied → Skip
B, deadline 1: slot 1 already occupied → Skip
E, deadline 3: [C , A , E] . Total Profit: 142

```
class Job {
    int deadline;
    int profit;
    Job(int deadline, int profit) {
        this.deadline = deadline;
        this.profit = profit;
    }
}
static int maxProfit(Job[] jobs) {
    Arrays.sort(
            jobs,
            (a, b) -> Integer.compare(b.profit, a.profit) );
    int maxDeadline = 0;
    for (Job job : jobs) {
        maxDeadline = Math.max(
                maxDeadline,
                job.deadline
        );
    }
    boolean[] used = new boolean[maxDeadline + 1];
    int totalProfit = 0;
    for (Job job : jobs) {
        for (int slot = job.deadline;
             slot >= 1;
             slot--) {
            if (!used[slot]) {
                used[slot] = true;
                totalProfit += job.profit;
                break;
            }}}
    return totalProfit;
}
```

Sorting takes O(n log n). For each job, we may scan up to D slots, where D is the maximum deadline. Time = O(n log n + nD), Space = O(D) The slot search can be optimized using **Disjoint Set Union (DSU)**.

17.15. Deadline Scheduling Insight

The greedy strategy has two important rules:
Profit decides priority → Higher-profit jobs are considered first.
Deadline decides placement → Schedule each selected job in the latest possible free slot.
Easy memory: Highest Profit First → Latest Valid Slot

17.16. Resource Allocation Greedy

Some interval problems do not ask which intervals to select. Instead, they ask for the **minimum number of resources needed to handle all overlapping events**.
Common examples: Meeting Rooms, Minimum Railway Platforms, CPU Scheduling, and other resource-allocation problems.

General Algorithm

1. Sort events according to their start time or separately sort start/end times depending on the problem.
2. Track when currently used resources become free.
3. When a new event starts, reuse a resource if one is available.
4. If every resource is busy, allocate a new resource.
5. Track the maximum number of resources used at the same time.

Main idea: Overlapping events → Need multiple resources | Finished event → Reuse resource
This differs from interval selection: Interval Selection → Choose some intervals, while Resource Allocation → Keep all intervals and count required resources.

17.17. Meeting Rooms

Given a list of meetings represented as [start, end], find the **minimum number of meeting rooms required** so that all meetings can take place without conflict.
Example: Meetings = [[0,30], [5,10], [15,20]]
Meeting [0,30] overlaps with [5,10] and [15,20], so at least 2 rooms are required.
Therefore: Minimum Rooms = 2

Algorithm — Sorting + Min Heap

1. Sort all meetings by their start time.
2. Use a **Min Heap** to store the end times of rooms currently allocated.
3. For each meeting, check the earliest ending room using heap.peek().
4. If heap.peek() <= currentStart, that room is free, so remove its old end time using heap.poll().
5. Add the current meeting's end time to the heap.
6. If no room is free, no element is removed, so the heap size increases and a new room is effectively allocated.
7. After processing all meetings, the heap size represents the minimum number of rooms required.

CodeWithNishchal

Why Min Heap? heap.peek() always gives the room that becomes available earliest.
Main idea: Sort by start time → Check earliest ending room → Reuse if free → Otherwise allocate new room

```
static int minMeetingRooms(int[][] meetings) {
    if (meetings.length == 0) {
        return 0;
    }
    Arrays.sort(
            meetings,
            Comparator.comparingInt(a -> a[0])
    );
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int[] meeting : meetings) {
        // Reuse earliest available room
        if (!heap.isEmpty() && heap.peek() <= meeting[0]) {
            heap.poll();
        }
        // Store current meeting's end time
        heap.offer(meeting[1]);
    }
    return heap.size();
}
```

Time = O(n log n) because of sorting and heap operations.
Space = O(n) in the worst case when all meetings overlap.
Pattern: Sort by Start Time → Min Heap of End Times → Reuse Earliest Available Room

---

17.28. Why Meeting Rooms Is Greedy + Heap

Greedy choice:If any room is reusable, reuse the one finishing earliest.
Heap allows us to efficiently find: minimum end time
So the combined pattern is: Greedy + Priority Queue

17.29. Minimum Platforms

Given arrival and departure times of trains, find the **minimum number of railway platforms required** so that no train has to wait.
Example:
Arrival = [900, 940, 950, 1100]
Departure = [910, 1120, 1130, 1200]
If multiple trains are present at the station at the same time, each train requires a separate platform.

Algorithm — Sorting + Two Pointers

1. Sort the arrival array.
2. Sort the departure array.
3. Use pointer i for the next arrival and pointer j for the next departure.
4. Maintain current, the number of platforms currently occupied.
5. If the next train arrives before the earliest departure, a new platform is required: current++ and move i.
6. Otherwise, a train has departed, so one platform becomes free: current-- and move j.
7. After every arrival, update the maximum number of occupied platforms.
8. Continue until all arrivals or departures are processed.

```
static int minPlatforms(
        int[] arrival,
        int[] departure) {
    Arrays.sort(arrival);
    Arrays.sort(departure);
    int i = 0;
    int j = 0;

    int current = 0;
    int answer = 0;

    while (i < arrival.length
            && j < departure.length) {

        if (arrival[i] < departure[j]) {
            current++;
            answer = Math.max(answer, current);
            i++;
        } else {
            current--;
            j++;
        }
    }
    return answer;
}
```

Equality Case

The comparison depends on the problem statement. If a platform can be reused when: arrival == departure use: arrival[i] < departure[j]
If an arriving train still requires a separate platform at the exact departure time, use: arrival[i] <= departure[j]

Time = O(n log n) because both arrays are sorted.
Space = O(1) extra, apart from sorting overhead.

Pattern: Sort Arrivals + Departures → Two Pointers → Track Current Overlap → Maximum Overlap = Minimum Platforms

17.20. Meeting Rooms vs Activity Selection

These two look similar but solve different goals.

Activity Selection

Choose maximum compatible intervals. Greedy:sort by end time

Meeting Rooms

Keep all intervals. Find maximum simultaneous overlap. Greedy: sort by start + track earliest ending active interval Do not confuse them.

17.21. Jump Game Pattern

Jump Game problems ask whether or how efficiently we can reach the last index when: nums[i] = maximum jump length from index i. The key greedy idea is to track the **farthest reachable index**.

17.22. Jump Game I

Given an array nums, where nums[i] represents the maximum distance we can jump from index i, determine whether the last index is reachable. Example: nums = [2, 3, 1, 1, 4]
From index 0, we can reach index 1 or 2. From index 1, we can reach up to index 4.
Therefore: Answer = true

Algorithm — Greedy Farthest Reach

1. Initialize farthest = 0.
2. Traverse the array from left to right.
3. If i > farthest, the current index itself is unreachable, so return false.
4. Otherwise update: farthest = max(farthest, i + nums[i]).
5. If farthest >= nums.length - 1, the last index is reachable, so return true.
6. Continue until the traversal finishes.

```
static boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) { return false;
        }
        farthest = Math.max(
                farthest,
                i + nums[i]
        );
        if (farthest >= nums.length - 1) { return true;
        }}
    return true;
}
```

Time = O(n) Space = O(1)

17.23. Jump Game I Visual

Example: nums = [2, 3, 1, 1, 4]
Start at index 0: farthest = max(0, 0 + 2) = 2. Now index 1 is reachable because: 1 <= 2
Update: farthest = max(2, 1 + 3) = 4
Last index is 4, so it is reachable.The greedy state keeps only one important piece of information: farthest reachable index

17.24. Jump Game II

Given an array nums, where nums[i] represents the maximum jump length from index i, find the **minimum number of jumps required to reach the last index**.
Assume the last index is reachable. Example: nums = [2, 3, 1, 1, 4]
One optimal path is: 0 → 1 → 4. Therefore: Minimum Jumps = 2

Algorithm — Greedy BFS-Like Range

Maintain three values: jumps → number of jumps taken so far
currentEnd → farthest index reachable using the current number of jumps
farthest → farthest index reachable using one more jump

Steps:

1. Initialize jumps = 0, currentEnd = 0, and farthest = 0.
2. Traverse from index 0 to nums.length - 2.
3. At each index, update: farthest = max(farthest, i + nums[i]).
4. When i == currentEnd, we have finished scanning all positions reachable with the current number of jumps.
5. Take one jump: jumps++.
6. Expand the next reachable range: currentEnd = farthest.
7. Continue until the last index is covered.

```
static int jump(int[] nums) {
    int jumps = 0; int currentEnd = 0; int farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(
                farthest,
                i + nums[i]
        );
        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
```

Nishchal

Intuition

Suppose the current number of jumps allows us to reach: 0 ... currentEnd
While scanning all indexes in this range, compute how far one additional jump could take us.
That best next boundary is stored in: farthest
When we finish the current range: i == currentEnd
we commit one jump and expand the range to: farthest
This behaves like **BFS levels**, but without using an explicit queue.

Time = O(n), Space = O(1)
Pattern: Jump Game I → Track farthest reachable index | Jump Game II → Track current range and next farthest range

17.25. Huffman Encoding

Huffman Encoding is a classic greedy compression algorithm.
Each character has a frequency.
Example:
A → 5
B → 9
C → 12
D → 13
E → 16
F → 45
Greedy rule: Repeatedly combine the two smallest frequencies.
Why? Low-frequency symbols can safely be placed deeper in the coding tree, while high-frequency symbols should remain closer to the root.

17.26. Huffman Algorithm

Use a: Min Heap

Steps:
1. Add all frequencies to heap.
2. Remove two smallest.
3. Combine: merged = first + second
4. Add merged value back.
5. Repeat until one node remains.
Example: 5 + 9 = 14
Heap becomes: 12, 13, 14, 16, 45
Then: 12 + 13 = 25
Continue until one tree remains.

Complexity:
O(n log n) for n distinct symbols.

CodeWithNishchal

17.27. Minimum Merge Cost Pattern

Huffman is part of a broader greedy pattern:
Repeatedly merge two smallest values.
Example: Files:2, 3, 4
If merge cost equals total size:
Option 1:2 + 3 = 5
5 + 4 = 9
Total cost =5 + 9 = 14

Bad order: 3 + 4 = 7
7 + 2 = 9
Total =7 + 9 =16

So smallest-first gives smaller total cost.

17.28. Minimum Merge Cost Code

```
static int minimumMergeCost(int[] values) {
    PriorityQueue<Integer> heap =
            new PriorityQueue<>();
    for (int value : values) {
        heap.offer(value);
    }

    int totalCost = 0;
    while (heap.size() > 1) {
        int first = heap.poll();
        int second = heap.poll();

        int merged = first + second;
        totalCost += merged;
        heap.offer(merged);
    }
    return totalCost;
}
```

Complexity:O(n log n)

Applications include:
Huffman Coding
Merge ropes
Combine files
Minimum connection/merge cost

17.29. Fractional Knapsack — Important Greedy Pattern

This is another fundamental greedy problem.
Each item has: value, weight
Fractions of items are allowed.
Goal: maximize value under capacity
Greedy rule: Take highest value/weight ratio first.
Ratio: value / weight
Why?Each unit of capacity should produce as much value as possible.

17.30. Fractional Knapsack vs 0/1 Knapsack

This comparison is extremely important.

Fractional Knapsack
Items can be split. Greedy works.
Sort by: value / weight descending

0/1 Knapsack
Item must be taken completely or skipped.
Greedy by ratio does NOT always work.
Usually solved with:Dynamic Programming
So:

Problem	Technique
Fractional Knapsack	Greedy
0/1 Knapsack	DP

17.31. Gas Station Pattern

Given two arrays:
gas[i] → fuel available at station i
cost[i] → fuel required to travel from station i to station i + 1
find the starting gas station from which we can complete one full circular trip.Example:
gas = [1, 2, 3, 4, 5]
cost = [3, 4, 5, 1, 2]
Output: Start Index = 3

Algorithm — Greedy

1. Maintain totalGas and totalCost.
2. If totalGas < totalCost, completing the full circle is impossible, so return -1.
3. Maintain currentFuel for the current starting station.
4. Traverse each station and update: currentFuel += gas[i] - cost[i].
5. If currentFuel < 0, the current start cannot work.
6. Also, no station between the current start and i can work, so set: start = i + 1.
7. Reset currentFuel = 0 and continue.
8. After the scan, return start.

CodeWithNishchal

Why can we skip all stations from the old start to i? If starting from the old start could not reach past i, then starting somewhere inside that failed segment gives even less accumulated fuel before reaching i.

Main idea: Check total fuel → Scan stations → Fuel becomes negative? → Move start to i + 1

```
static int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0;
    int totalCost = 0;
    int currentFuel = 0;
    int start = 0;
    for (int i = 0; i < gas.length; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
        currentFuel += gas[i] - cost[i];
        if (currentFuel < 0) {
            start = i + 1;
            currentFuel = 0;
        }
    }
    if (totalGas < totalCost) {
        return -1;
    }
    return start;
}
```

Time = O(n), Space = O(1)
Pattern: Running balance becomes negative → Eliminate current range → Start from next position

17.32. Greedy + Heap

Many greedy problems need the best candidate repeatedly from a changing set.
Then use: Greedy decision + Heap
Examples: Huffman, Meeting Rooms, Task Scheduler, Reorganize String, Minimum Merge Cost
Heap gives: minimum/ maximum/ earliest finishing/ highest frequency efficiently.

17.33. Greedy + Sorting

Another very common form:
Sort candidates
↓
Scan once
↓
Accept/Reject greedily
Examples:
Activity Selection, Non-overlapping Intervals, Fractional Knapsack, Job Scheduling, Minimum Removals
Typical complexity: O(n log n) because sorting dominates.

17.34. Greedy + Two Pointers

Sometimes sorting is followed by two pointers.

Examples: Minimum Platforms, Boats to Save People, Pairing problems, Resource assignment

General idea:

Sort

↓

Compare smallest/largest

↓

Pair or allocate

↓

Move pointers

17.35. Common Greedy Patterns

Greedy
│
├── Interval Greedy
│ ├── Activity Selection
│ ├── Non-overlapping Intervals
│ └── Minimum Removals
│
├── Scheduling
│ ├── Deadlines
│ ├── Profit Selection
│ └── Job Sequencing
│
├── Resource Allocation
│ ├── Meeting Rooms
│ └── Minimum Platforms
│
├── Reachability
│ ├── Jump Game I
│ └── Jump Game II
│
├── Smallest-First Merge
│ ├── Huffman
│ └── Minimum Merge Cost
│
├── Ratio Greedy
│ └── Fractional Knapsack
│
└── Greedy + Heap
├── Scheduling
├── Frequency
└── Resource Problems

CodeWithNishchal

17.36. Choosing the Correct Greedy Rule

A useful recognition table:

Goal	Common Greedy Choice
Max non-overlapping intervals	Earliest finish
Minimum interval removals	Keep earliest finish
Maximum profit with deadlines	Highest profit first
Place job before deadline	Latest available slot
Minimum rooms/platforms	Earliest finishing active interval
Reachability	Farthest reachable point
Minimum jumps	Farthest reach per current layer
Minimum merge cost	Two smallest values
Huffman	Two smallest frequencies
Fractional Knapsack	Highest value/weight
Dynamic priority	Heap

These are patterns, not universal rules. Always verify the objective.

17.37. How to Recognize Greedy Problems

Strong signals include: maximum number of... / minimum number of... / earliest... / latest... /minimum cost... /maximum profit... /schedule... / interval... / deadline... / resource...

But ask:

1. Can I make one locally best decision?
2. Once I make that decision, do I ever need to reconsider it?
3. Can I prove that this choice does not hurt the optimal answer?
4. Can sorting expose the greedy order?
5. Do I repeatedly need min/max?→ maybe heap

If yes, greedy may be appropriate.

17.38. When Greedy Fails

Greedy fails when the locally best decision can block a better global solution.

Example: 0/1 Knapsack

Choosing the best value/weight ratio does not always produce the optimal solution because whole items cannot be split.

Another common warning: If future choices depend heavily on combinations of earlier choices, DP or Backtracking may be necessary. Greedy requires proof, not just intuition.

CodeWithNishchal

17.39. Common Mistakes

1. Sorting by the Wrong Field

Activity Selection requires: end time not necessarily: start time
Sorting criteria often determine whether the greedy method is correct.

2. Using Greedy for 0/1 Knapsack

Ratio greedy works for: Fractional Knapsack not general: 0/1 Knapsack

3. Wrong Overlap Condition

Depending on the problem: start >= previousEnd may be compatible.
But some problems may treat touching boundaries differently. Always follow the exact statement.

17.40. Complexity Summary

Pattern	Typical Complexity
Activity Selection	O(n log n)
Non-overlapping Intervals	O(n log n)
Minimum Removals	O(n log n)
Job Sequencing simple	O(n log n + nD)
Meeting Rooms	O(n log n)
Minimum Platforms	O(n log n)
Jump Game I	O(n)
Jump Game II	O(n)
Huffman	O(n log n)
Minimum Merge Cost	O(n log n)
Fractional Knapsack	O(n log n)

The most common reason for:O(n log n) is: sorting or heap operations

17.41. Greedy vs Other Techniques

Technique	Core Idea
Greedy	Make best safe choice now
DP	Compare multiple states/choices
Backtracking	Explore and undo choices
Divide & Conquer	Split independent subproblems
Heap	Data structure for repeatedly retrieving best priority
Binary Search	Eliminate search space based on monotonicity

CodeWithNishchal

18. Bit Manipulation

Bit Manipulation means directly working with the binary representation of integers using bitwise operators. Computers store integers as bits: 0/ 1. Example: Decimal 10

Binary: 1010, For a 32-bit Java int: 00000000 00000000 00000000 00001010

Bit manipulation is useful for:

- Checking flags
- Setting/unsetting states
- XOR problems
- Power-of-two checks
- Counting set bits
- Subsets using bitmasks
- Compact state representation
- Bitmask DP
- Optimization problems

The main skill is understanding **what each bit represents**.

18.1. Binary Basics

Every binary position represents a power of 2.

Position: 3 2 1 0

Value: 8 4 2 1

| | | |

Binary: 1 0 1 1

Therefore: 1011 = 8 + 2 + 1 = 11

For a bit at position i: Value = 2^i, In code: 1 << i represents:2^i for positions that fit within the type.

18.2. Bit Indexing

Bits are normally numbered from right to left starting at 0.

Example: Number = 13

Binary = 1101

Bit Index: 3 2 1 0

1 1 0 1

So:bit 0 = 1, bit 1 = 0, bit 2 = 1, bit 3 = 1. The rightmost bit is called the: Least Significant Bit LSB. The leftmost relevant bit is the: Most Significant Bit MSB

18.3. Bitwise Operators in Java

Operator	Name
&	AND
`	`
^	XOR
~	NOT / Complement
<<	Left Shift
>>	Signed Right Shift
>>>	Unsigned Right Shift

CodeWithNishchal

18.4. Bitwise AND — &

AND returns 1 only when both corresponding bits are 1.

Truth table:

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Example:

10 = 1010

12 = 1100

```
    1010
AND 1100
------------
    1000
```

Result: 8

Java:

```
int result = 10 & 12;
```

AND is commonly used to: check a bit / clear selected bits/ apply masks

18.5. Bitwise OR — |

OR returns 1 if at least one corresponding bit is 1.

Truth table:

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Example:

10 = 1010

12 = 1100

```
   1010
OR 1100
-----------
   1110
```

Result: 14 OR is commonly used to: set a bit/ combine flags

CodeWithNishchal

18.6. Bitwise XOR — ^

XOR returns 1 when the two bits are different.

Truth table:

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Example:

10 = 1010

12 = 1100

```
    1010
XOR 1100
------------
    0110
```

Result:6. XOR is extremely important in DSA.

18.7. Important XOR Properties

Memorize these:

x ^ 0 = x

x ^ x = 0

Also: a ^ b = b ^ a and: (a ^ b) ^ c = a ^ (b ^ c)

Therefore: 2 ^ 5 ^ 2 = (2 ^ 2) ^ 5 = 0 ^ 5 = 5

This cancellation property is the foundation of many XOR problems.

18.8. Bitwise NOT — ~

NOT flips every bit:

0 → 1

1 → 0

Example conceptually:

00001010

↓ ~

11110101

Java integers use **two's complement signed representation**, so:

System.out.println(~10);

produces:-11

Important identity: ~x = -(x + 1)

for Java's two's-complement integer representation.

Therefore: ~10 = -11

CodeWithNishchal

18.9. Left Shift — <<

Left shift moves bits to the left.
Example:
5 = 0101
5 << 1
1010 = 10
Generally: x << k is equivalent to multiplying by:2^k
when no relevant overflow occurs.
Example:
7 << 2
7 × 4 = 28
Visual:
00000111
<< 2

00011100

18.10. Signed Right Shift — >>

Right shift moves bits to the right.
Example: 12 = 1100
12 >> 1 = 0110 = 6
For non-negative numbers: x >> k is similar to integer division by: 2^k
For negative values, >> preserves the sign by filling new left bits with 1.
This is called: Arithmetic Right Shift

18.11. Unsigned Right Shift — >>>

Java also provides: >>>
Unlike >>, it fills new left positions with:0
Example for a negative int: int x = -8;
System.out.println(x >> 1);
System.out.println(x >>> 1);
These produce very different results because:
>> preserves sign
>>> inserts zero bits
>>> is useful when treating an int as an unsigned 32-bit bit pattern.

18.12. Java Shift Important Detail

For Java int, only the lower 5 bits of the shift distance are used.
So effectively: shift amount is modulo 32
For long: shift amount is modulo 64
Also, use: 1L << i instead of: 1 << i
when working with bit positions belonging to a long.

CodeWithNishchal

18.13. Bit Mask

A **mask** is a binary value created so specific bit positions can be checked or modified.
For bit i: int mask = 1 << i;
Example: i = 3
1 << 3 = 00001000
That mask has only bit 3 set. This single idea powers most basic bit operations.

18.14. Check Whether the ith Bit Is Set

Use: (n & (1 << i)) != 0
Example: n = 10
Binary: 1010
Check bit 1:
mask:
0010

1010
0010

0010
Non-zero result means the bit is set.

```
static boolean isBitSet(int n, int i) {
    return (n & (1 << i)) != 0;
}
```

18.15. Set the ith Bit

To force bit i to become 1: n | (1 << i)
Example: n = 8
1000
Set bit 1: 0010
1000
OR
0010

1010
Result: 10

```
static int setBit(int n, int i) {
    return n | (1 << i);
}
```

CodeWithNishchal

18.16. Clear the ith Bit

To force bit i to 0: n & ~(1 << i)
Example: n = 14: 1110
Clear bit 2: mask: 0100
~mask: ...1011
ANDing clears only that selected bit.

```
static int clearBit(int n, int i) {
    return n & ~(1 << i);
}
```

18.17. Toggle the ith Bit

Toggle means: 0 → 1, 1 → 0
Use XOR: n ^ (1 << i)

```
static int toggleBit(int n, int i) {
    return n ^ (1 << i);
}
```

Example: 1010 toggle bit 1
1010
XOR
0010

1000

18.18. Update a Bit

If bit should become 1: setBit(n, i)
If bit should become 0: clearBit(n, i)
A combined formula for value bit where bit is 0 or 1:
n = (n & ~(1 << i)) | (bit << i);
First: clear position i, then: insert desired value

18.19. Basic Bit Operations Summary

Operation	Expression
Check bit i	(n & (1 << i)) != 0
Set bit i	n | (1 << i)
Clear bit i	n & ~(1 << i)
Toggle bit i	n ^ (1 << i)
Mask for bit i	1 << i

These are fundamental formulas worth remembering.

CodeWithNishchal

18.20. Odd or Even Using Bits

The least significant bit determines parity.
Even: LSB = 0, Odd: LSB = 1
Therefore: (n & 1) == 0 means even.

```
static boolean isOdd(int n) {
    return (n & 1) == 1;
}
```

Examples:
10 = 1010 → even
11 = 1011 → odd

18.21. Multiply / Divide by Powers of Two

For suitable values: n << k ≈ n × 2^k
and for non-negative integers: n >> k ≈ n / 2^k
Example: 6 << 2 = 24 because: 6 × 4 = 24
However, in general Java code, shifts should not blindly replace normal arithmetic because: overflow, negative-number behavior, readability may matter.

18.22. Power of Two

Powers of two have exactly one set bit.
Examples:
1 = 0001
2 = 0010
4 = 0100
8 = 1000
16 = 10000
For a positive power of two: n & (n - 1) = 0
Example:
8   = 1000
7   = 0111
1000
0111

0000

```
static boolean isPowerOfTwo(int n) {
    return n > 0
            && (n & (n - 1)) == 0;
}
```

The: n > 0
condition is essential because 0 is not a power of two.

CodeWithNishchal

18.23. Why n & (n - 1) Works

Consider: n = 12
1100
Then: n - 1 = 11
1011
AND:
1100
1011

1000

The operation: n & (n - 1) removes the **rightmost set bit**. This is one of the most important bit tricks.

18.24. Remove Rightmost Set Bit

Formula: n = n & (n - 1);
Example: n = 12
1100
After once: 1000
After again: 0000
Each operation removes exactly one 1. This gives an efficient bit-counting technique.

18.25. Count Set Bits — Simple Method

A set bit is a bit equal to 1.
Example: 13 = 1101
Number of set bits: 3

Basic approach:

```
static int countSetBits(int n) {
    int count = 0;
    while (n != 0) {
        count += n & 1;
        n >>>= 1;
    }
    return count;
}
```

Using >>> is useful if the method should correctly consume all bits of a negative Java int.

LeWithNishchal

18.26. Brian Kernighan's Algorithm

Use: n & (n - 1) to repeatedly remove the rightmost set bit.

```
static int countSetBits(int n) {
    int count = 0;
    while (n != 0) {
        n = n & (n - 1);
        count++;
    }
    return count;
}
```

If the number contains k set bits: Time = O(k)
instead of checking all 32 bit positions.
Example: n = 13\
1101
↓
1100
↓
1000
↓
0000
count = 3
Java also provides: Integer.bitCount(n);

18.27. Isolate the Rightmost Set Bit

A very useful identity: n & -n

isolates the least significant set bit.
Example: n = 12
1100
Result: 0100because the rightmost set bit of 12 represents: 4
Formula: int lowbit = n & -n;

---

28. Why n & -n Works

Java uses two's complement. For: n = 12
00001100
Two's complement -12: 11110100
AND:
00001100
11110100

00000100
Only the rightmost set bit remains.

CodeWithNishchal

18.29. Single Number Pattern

Suppose every number appears exactly twice except one.
Example: [4, 1, 2, 1, 2]
XOR all values: 4 ^ 1 ^ 2 ^ 1 ^ 2
Rearrange: 4 ^ (1 ^ 1) ^ (2 ^ 2)
Since: x ^ x = 0
result: 4

```
static int singleNumber(int[] nums) {
    int xor = 0;
    for (int num : nums) {
        xor ^= num;
    }
    return xor;
}
```

Complexity: Time = O(n) , Space = O(1)

18.30. Missing Number Using XOR

Suppose an array contains n distinct values from: 0 ... n with exactly one missing.
Example: [3, 0, 1]
Expected values: 0, 1, 2, 3
XOR all expected indices and all actual values.
Everything present cancels.

```
static int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) {
        xor ^= i;
        xor ^= nums[i];
    }
    return xor;
}
```

Complexity: O(n) time, O(1) space

18.31. Two Unique Numbers

Suppose every number appears twice except **two numbers**, which appear once.
Example: [1, 2, 1, 3, 2, 5]
Unique: 3, 5
First XOR all: xor = 3 ^ 5
Since the two numbers differ somewhere, xor contains at least one set bit.
Isolate one differing bit: int diffBit = xor & -xor;
Now divide numbers into two groups:
Group 1: bit is 0
Group 2: bit is 1
Pairs go into the same group and cancel. The two unique numbers go into different groups.

18.31.1 Two Unique Numbers Code

```
static int[] twoSingleNumbers(int[] nums) {
    int xor = 0;
    for (int num : nums) {
        xor ^= num;
    }
    int diffBit = xor & -xor;
    int a = 0;
    int b = 0;
    for (int num : nums) {
        if ((num & diffBit) == 0) {
            a ^= num;
        } else {
            b ^= num;
        }
    }
    return new int[]{a, b};
}
```

Complexity: Time = O(n), Space = O(1)

18.32. Number Appearing Once When Others Appear Three Times

Suppose: [2, 2, 3, 2] Answer: 3
Simple bit-counting strategy: For every bit position: count how many numbers have that bit set
Since all repeated numbers occur three times: count % 3 removes their contribution.
Whatever remains belongs to the unique number.

```
static int singleNumberThree(int[] nums) {
    int answer = 0;
    for (int bit = 0; bit < 32; bit++) {
        int count = 0;
        for (int num : nums) {
            if ((num & (1 << bit)) != 0) {
                count++;
            }
        }
        if (count % 3 != 0) {
            answer |= (1 << bit);
        }
    }
    return answer;
}
```

Time: O(32n) = O(n)

With Nishchal

18.33. Generate All Subsets Using Bitmask

For: [1, 2, 3] there are: 2^3 = 8
subsets. Use a 3-bit mask:
000 → [], 001 → [1], 010 → [2], 011 → [1,2], 100 → [3], 101 → [1,3], 110 → [2,3], 111 → [1,2,3]
Each bit answers: Is element i selected?

18.33.1 Subset Code

```
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    int n = nums.length;
    int totalMasks = 1 << n;
    for (int mask = 0;
         mask < totalMasks;
         mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                subset.add(nums[i]);
            }
        }
        result.add(subset);
    }
    return result;
}
```

Complexity: Number of subsets = 2^n
Checking n bits each: Time = O(n × 2^n)

18.34. Bitmask as a Set

A bitmask can represent a set when the universe is small.
Suppose:
Elements = {0,1,2,3,4}
Mask: 10101
represents: {0,2,4}
Operations:

Add element i
mask |= (1 << i);

Remove element i
mask &= ~(1 << i);

Check element
(mask & (1 << i)) != 0

Toggle
mask ^= (1 << i);

Codewithshchal

18.35. Union, Intersection, Difference With Masks

If sets are represented by masks: A, B
then:

Union
A | B

Intersection
A & B

Symmetric Difference
A ^ B

Remove all elements of B from A
A & ~B
Bitwise operators behave naturally like set operations.

18.36. Check Whether One Mask Is a Subset of Another

Suppose: sub mask
To check whether every set bit of sub is also present in mask:(sub & mask) == sub
Example:
mask = 11101
sub = 10100
AND:10100
equals sub→ yes

18.37. Enumerating Submasks

For advanced bitmask problems, all non-zero submasks of mask can be generated using:

```
for (int sub = mask;
     sub > 0;
     sub = (sub - 1) & mask) {
    // process sub
}
```

Example: mask = 1011
This visits only subsets contained in the original mask.
This pattern is especially useful in: Bitmask DP/ Partitioning subsets/ Subset convolution-style problems

Across all masks, enumerating all submasks has typical complexity: O(3^n)
because every bit can effectively be: outside mask / inside mask but not submask/ inside both

---

WithNischal

18.38. Count Set Bits for All Numbers

Sometimes we need: number of set bits for every value 0...n
A useful DP relation: bits[i] = bits[i >> 1] + (i & 1)
Why? i >> 1
removes the last bit, and: i & 1 tells whether the removed bit was 1.

```
static int[] countBits(int n) {
    int[] bits = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        bits[i] = bits[i >> 1] + (i & 1);
    }
    return bits;
}
```

Another relation: bits[i] = bits[i & (i - 1)] + 1 because i & (i - 1) removes one set bit.

18.39. Hamming Distance

The **Hamming Distance** between two integers is the number of bit positions where they differ.
Example:
x = 1 → 0001
y = 4 → 0100
Different positions: 2
XOR identifies differing bits:
0001
0100

0101
Then count set bits.

```
static int hammingDistance(int x, int y) {
    return Integer.bitCount(x ^ y);
}
```

18.40. Hamming Weight

Hamming weight means: number of set bits So: 13 = 1101
has Hamming weight: 3
In Java: Integer.bitCount(13); returns: 3

18.41. Clear the Rightmost Set Bit

Already seen: n & (n - 1)
Example:
10110000
minus 1: 10101111
AND: 10100000
One rightmost 1 disappears.

Code With Nishchal

18.42. Set the Rightmost Unset Bit

A useful trick for suitable non-negative values:n | (n + 1)

Example:n = 10: 1010

n + 1:1011 OR: 1011

The rightmost 0 involved in the increment becomes set. This trick is less commonly needed than n & (n-1), so understand it rather than blindly memorizing it.

18.43. Clear All Bits From LSB Through Position i

To clear bits: i, i-1, ..., 0 create a mask whose lower i+1 bits are zero.

One approach: int mask = ~((1 << (i + 1)) - 1); int result = n & mask;

Example: n: 11101101 clear bits 0..3 result: 11100000

Be careful around shifts near bit 31; using long can avoid certain overflow issues when constructing large masks.

18.44. Extract Lower k Bits

To keep only the lowest k bits: n & ((1 << k) - 1)

Example: k = 4

1 << 4: 10000

minus 1: 01111

Mask: 00001111

For k = 32 with int, use special handling because Java masks the shift distance and 1 << 32 does not behave as mathematical 2^32.

18.45. Range Bitwise AND

Classic problem: Find AND of every integer from left to right.

Key observation: Only the common binary prefix survives.

Example: 5 = 101, 6 = 110, 7 = 111, AND: 100 = 4

Approach:

Shift both numbers right until they become equal.

```
static int rangeBitwiseAnd(
        int left,
        int right) {
    int shifts = 0;
    while (left != right) {
        left >>= 1;
        right >>= 1;
        shifts++;
    }
    return left << shifts;
}
```

The differing suffix contains transitions that force those bits to zero.

adeWithNishchal

18.46. XOR From 1 to n

XOR values from: 1 ^ 2 ^ 3 ^ ... ^ n
follow a pattern every 4 numbers.
For: n % 4, results are:

n % 4	XOR 1...n
0	n
1	1
2	n + 1
3	0

```
static int xorOneToN(int n) {
    switch (n & 3) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}
```

Then XOR over range: L ... R
can be found as:xor(1...R) ^ xor(1...(L-1)) because earlier values cancel.

18.47. Java Signed Integer Representation

Java integer types are signed.
For int: 32 bits
Range: -2^31 to 2^31 - 1
The highest bit is part of the signed two's-complement representation.
For example: Integer.MIN_VALUE = 10000000 00000000 00000000 00000000

18.47.1 << 31 Is Negative

Because bit 31 is the sign bit of a Java int:
int x = 1 << 31;
produces:Integer.MIN_VALUE which is negative.
This does not mean the bit operation failed. The bit pattern is correct; Java is simply interpreting it as a signed integer. For wider masks, use: 1L << i with long.

18.48. Operator Precedence and Parentheses

Bitwise expressions can become confusing.
Prefer: if ((n & (1 << i)) != 0)
instead of trying to rely on remembering Java precedence rules.
Parentheses make the intended mask operation explicit and prevent subtle mistakes.

CodeWithNishchal

18.49. Important Built-In Java Methods

1. Integer.bitCount(x): Count set bits
2. Integer.numberOfLeadingZeros(x): Count leading zero bits.
3. Integer.numberOfTrailingZeros(x): Count zeros after the least significant set bit.
4. Integer.highestOneBit(x): Returns value containing only the highest set bit.
5. Integer.lowestOneBit(x) Returns value containing only the lowest set bit.

18.50. Operation Complexity

Basic bitwise operations are treated as: O(1) for fixed-width primitive integers.

Operation	Complexity
AND / OR / XOR	O(1)
Shift	O(1)
Check/Set/Clear bit	O(1)
n & (n-1)	O(1)
Count set bits with Kernighan	O(k)
Generate all subsets	O(n × 2^n)
Single Number	O(n)
Missing Number	O(n)
Two Unique Numbers	O(n)

k = number of set bits.

18.51. Core Formula Table

Goal	Formula
Mask bit i	1 << i
Check bit	(n & (1 << i)) != 0
Set bit	n | (1 << i)
Clear bit	n & ~(1 << i)
Toggle bit	n ^ (1 << i)
Odd/even	n & 1
Remove lowest set bit	n & (n - 1)
Isolate lowest set bit	n & -n
Power of 2	n > 0 && (n & (n-1)) == 0
Union masks	A | B
Intersection masks	z
Symmetric difference	A ^ B
Add element to mask	mask | (1 << i)
Remove element	mask & ~(1 << i)

CodeWithNishchal

19. Range Structures

Range-query data structures are used when an array must support operations such as:
Query a range/ Update elements/ Repeat many times

Example: Array: [2, 4, 1, 7, 3, 6]
Queries:
sum(1, 4)
minimum(0, 3)
maximum(2, 5)

Updates:
arr[3] = 10, add 5 to arr[2], add 3 to every element in [1,4]
A normal array can answer a single range query by scanning: O(n)
but if there are many queries and updates, this becomes expensive.
Two important structures are:
Range Structures
|
+-- Segment Tree
|
+-- Fenwick Tree / Binary Indexed Tree

19.1. Why Range Structures Are Needed

Suppose: n = 100000, q = 100000
and each query asks: sum(left, right)
Naive scanning: O(n) per query
Total: O(nq)
which can be too slow.
A range data structure preprocesses information so that queries and updates become much faster.

19.2. Prefix Sum vs Range Structures

For an immutable array, prefix sum is excellent.
Define: prefix[i] = sum of elements from 0 to i
Then: rangeSum(l, r) = prefix[r] - prefix[l-1]
Query: O(1)
But if one element changes, many prefix values may need recomputation: Update = O(n)
This is where Segment Tree and Fenwick Tree become useful.

Structure	Query	Point Update
Normal Array	O(n)	O(1)
Prefix Sum	O(1)	O(n)
Fenwick Tree	O(log n)	O(log n)
Segment Tree	O(log n)	O(log n)

CodeWithNishchal

19.3. SEGMENT TREE

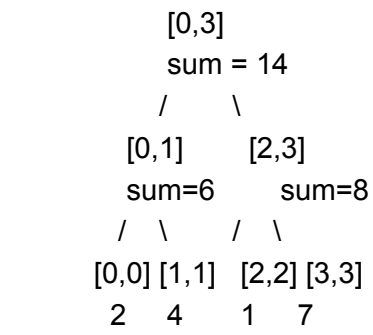
A **Segment Tree** is a binary tree in which every node stores information about a segment/range of the array.

Example array: [2, 4, 1, 7]

Root represents: [0,3]

Its children represent: [0,1] , [2,3]

Then: [0,0] [1,1] [2,2] [3,3]



Every internal node combines information from its children.

For sum: tree[node] = tree[leftChild] + tree[rightChild]

19.3.1 What Can a Segment Tree Store?

A Segment Tree is not limited to sums.

A node can represent:

- Range Sum
- Range Minimum
- Range Maximum
- Range GCD
- Range XOR
- Frequency/count information

The important requirement is that the parent answer can be computed by combining child answers.

Examples:

Sum: left + right

Minimum: min(left, right)

Maximum: max(left, right)

GCD: gcd(left, right)

XOR: left ^ right

19.3.2 Segment Tree Structure

For an array range: [left, right]

calculate: mid = left + (right - left) / 2

Children: Left child: [left, mid]

Right child: [mid + 1, right]

The splitting continues until: left == right

which represents one array element.

CodeWithNishchal

19.3.3. Array Representation of Segment Tree

Like a heap, a Segment Tree can be stored in an array.

Using node index: node

children are commonly: leftChild = 2 * node, rightChild = 2 * node + 1 when using 1-based tree indexing.

Example:

tree[1]

```
     |
 +----+----+
 |         |
```

tree[2] tree[3]

A safe allocation commonly used is: long[] tree = new long[4 * n]; Why 4*n? It is a convenient safe upper bound for the recursive binary-tree representation.

19.3.4. Building a Segment Tree

Given: arr = [2, 4, 1, 7] Build recursively. Base case: left == right Then: tree[node] = arr[left]

Otherwise:

1. Build left child.
2. Build right child.
3. Combine results.

```
class SegmentTree {
    private final long[] tree;
    private final int n;
    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new long[4 * n];
        build(arr, 1, 0, n - 1);
    }
    private void build( int[] arr,  int node, int left, int right) {
        if (left == right) {
            tree[node] = arr[left];
            return;
        }

        int mid = left + (right - left) / 2;
        build( arr, node * 2, left, mid
        );
        build( arr, node * 2 + 1, mid + 1, right
        );
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }
}
```

Build complexity: O(n)

because the Segment Tree contains only a linear number of nodes.

Nishchal

19.3.5. Range Query

Suppose the query is: sum(queryLeft, queryRight)
At every Segment Tree node, compare: node segment vs query segment
There are three cases.

19.3.5.1. Case 1 — Complete Overlap

Suppose current node represents: [2,3] and query is: [1,3]
The entire node range lies inside the query.
Query:
[1 -------- 3]
Node:
[2 ---- 3]
Return stored result directly: return tree[node]; No need to go deeper.

19.3.5.2 Case 2 — No Overlap

Suppose node: [0,1], query: [3,4]
There is no common index. Node: [0---1]
Query: [3---4]
For a sum tree, return:0because zero is the identity for addition.
For minimum tree, identity would usually be: +Infinity
For maximum: -Infinity

19.3.5.3 Case 3 — Partial Overlap

Node:[0 ---------- 5]
Query:
[2 --- 4]
The node is partly inside the query.
Therefore: query left child + query right child and combine their answers.

19.3.6 Segment Tree Range Sum Query

```
public long query(int queryLeft, int queryRight) {
    return query(1,0, n - 1, queryLeft, queryRight
    );
}
Recursive method:
private long query( int node, int left, int right,int queryLeft, int queryRight)
{
    // No overlap
    if (right < queryLeft || left > queryRight) {

        return 0;
    }
```

devWithNishchal

```
    // Complete overlap
    if (queryLeft <= left
            && right <= queryRight) {

        return tree[node];
    }

    // Partial overlap
    int mid = left + (right - left) / 2;

    long leftResult =
            query(
                node * 2,
                left,
                mid,
                queryLeft,
                queryRight
            );

    long rightResult =
            query(
                node * 2 + 1,
                mid + 1,
                right,
                queryLeft,
                queryRight
            );

    return leftResult + rightResult;
}
```

Complexity: O(log n)

for standard range queries on a Segment Tree.

19.3.6 Why Query Is O(log n)

The Segment Tree height is:O(log n)

A range query does not explore every node.
Large completely-covered segments are returned immediately.
Example:query [2,6]
may be answered using only a few Segment Tree nodes rather than visiting every array element.
The number of relevant boundary paths is logarithmic, giving:O(log n)
for common associative range queries.

19.3.7 Point Update

Suppose: arr[2] = 10
Only Segment Tree nodes whose ranges contain index 2 need updating.
Example:

```
[0,3]
  |
[2,3]
  |
[2,2]
```

Only one root-to-leaf path changes.

19.3.8 Point Update Flow

Find leaf representing index
|
v
Update leaf
|
v
Return upward
|
v
Recalculate every ancestor

```
public void update(int index, int value) {
    update(
        1,
        0,
        n - 1,
        index,
        value
    );
}
private void update(
        int node,
        int left,
        int right,
        int index,
        int value) {
    if (left == right) {
        tree[node] = value;
        return;
    }
    int mid = left + (right - left) / 2;
    if (index <= mid) {
```

Nishchal

```
        update( node * 2,  left, mid, index, value
        );
    } else {
        update(
            node * 2 + 1,
            mid + 1,
            right,
            index,
            value
        );
    }
    tree[node]
            = tree[node * 2]
            + tree[node * 2 + 1];
}
```

Complexity: O(log n)

19.3.9 Complete Basic Segment Tree

```
class SegmentTree {
    private final long[] tree;
    private final int n;
    SegmentTree(int[] arr) {
        n = arr.length;
        tree = new long[4 * n];
        build(arr, 1, 0, n - 1);
    }
    private void build(
            int[] arr,
            int node,
            int left,
            int right) {
        if (left == right) {
            tree[node] = arr[left];
            return;
        }
        int mid = left + (right - left) / 2;
        build(arr, node * 2, left, mid);
        build(
            arr,
            node * 2 + 1,
            mid + 1,
            right
        );
```

```
    tree[node]
            = tree[node * 2]
            + tree[node * 2 + 1];
}

public long query(int ql, int qr) {

    return query(
        1,
        0,
        n - 1,
        ql,
        qr
    );
}

private long query(
        int node,
        int left,
        int right,
        int ql,
        int qr) {

    if (right < ql || left > qr) {
        return 0;
    }

    if (ql <= left && right <= qr) {
        return tree[node];
    }

    int mid = left + (right - left) / 2;

    return query(
                node * 2,
                left,
                mid,
                ql,
                qr
            )
            +
            query(
                node * 2 + 1,
                mid + 1,
```

```
                right,
                ql,
                qr
            );
    }

    public void update(int index, int value) {

        update(
            1,
            0,
            n - 1,
            index,
            value
        );
    }

    private void update(
            int node,
            int left,
            int right,
            int index,
            int value) {

        if (left == right) {
            tree[node] = value;
            return;
        }

        int mid = left + (right - left) / 2;

        if (index <= mid) {

            update(
                node * 2,
                left,
                mid,
                index,
                value
            );

        } else {

            update(
                node * 2 + 1,
```

```
                mid + 1,
                right,
                index,
                value
            );
        }

        tree[node]
                = tree[node * 2]
                + tree[node * 2 + 1];
    }
}
```

19.3.10. Segment Tree for Minimum Query

The tree structure stays the same. Only the combine rule changes.
Instead of:

```
tree[node]
        = tree[node * 2]
        + tree[node * 2 + 1];
```

use:

```
tree[node]
        = Math.min(
            tree[node * 2],
            tree[node * 2 + 1]
        );
```

For no-overlap: return Long.MAX_VALUE;
because: min(x, +∞) = x
The structure is reusable; the operation determines how nodes combine.

19.3.11. Range Update Problem

Point update is easy: change one position
But suppose we repeatedly perform: Add 5 to every element from index 2 to 10000
A naive Segment Tree approach could update each element separately.
If range size is k: O(k log n)
That defeats much of the purpose.
This is where: Lazy Propagation is used.

CodeWithNishchal

19.3.12 Lazy Propagation

Lazy Propagation allows a Segment Tree to perform range updates efficiently by delaying child updates until those children are actually needed.
Main idea: If an entire node range is covered by the update, update the node immediately
but postpone updating all its descendants.
The postponed update is stored in: lazy[node]

19.3.13 Why It Is Called Lazy

Suppose: Update: add 5 to range [0,7]
and the root itself represents: [0,7]
Without lazy propagation: visit every leaf
With lazy propagation: tree[root] += 5 * 8
lazy[root] += 5
Stop.
Children are not immediately changed.

```
          [0,7]
          +5 lazy
        /       \
   unchanged    unchanged
    for now      for now
```

When a child is later needed:push the pending update downward

19.3.14 Lazy Propagation State

For range addition + range sum:
tree[node] = correct sum for this entire node range
lazy[node] = amount still waiting to be applied to its children
If node represents: [left, right]
with: length = right - left + 1 and we add: value to every element, its total sum increases by: value × length Therefore: tree[node] += value * (right - left + 1);

19.3.15 Push Operation

Before descending into children, propagate pending work. Suppose: lazy[node] = 5.
Then both children must also eventually receive +5.
For sum tree:

```
private void push(
        int node,
        int left,
        int right) {
    if (lazy[node] == 0
            || left == right) {
        return;
    }
```

CodeWithNishchal

```
    int mid = left + (right - left) / 2;
    int leftChild = node * 2;
    int rightChild = node * 2 + 1;
    tree[leftChild] += lazy[node] * (mid - left + 1);
    tree[rightChild] += lazy[node] * (right - mid);
    lazy[leftChild] += lazy[node];
    lazy[rightChild] += lazy[node];
    lazy[node] = 0;
}
```

After pushing: parent pending value = 0 because the responsibility has moved to its children.

19.3.16 Range Update With Lazy Propagation

Again use the three overlap cases.

No Overlap

return

Complete Overlap

Update current node:
tree[node] += value × segmentLength
Store: lazy[node] += value
Do not descend.

Partial Overlap

Push existing lazy information first.
Then recurse into both children.
Finally: tree[node] = tree[leftChild] + tree[rightChild]

19.3.17 Lazy Range Addition Code

```
class LazySegmentTree {
    private final long[] tree;
    private final long[] lazy;
    private final int n;
    LazySegmentTree(int[] arr) {
        n = arr.length;
        tree = new long[4 * n];
        lazy = new long[4 * n];
        build(arr, 1, 0, n - 1);
    }
    private void build(
            int[] arr,
            int node,
            int left,
            int right) {
```

WithNishchal

```
        if (left == right) {
            tree[node] = arr[left];
            return;
        }
        int mid = left + (right - left) / 2;
        build(arr, node * 2, left, mid);
        build(
            arr,
            node * 2 + 1,
            mid + 1,
            right
        );

        tree[node]
                = tree[node * 2]
                + tree[node * 2 + 1];
    }
```

Range update:

```
    public void rangeAdd(
            int ql,
            int qr,
            long value) {
        rangeAdd( 1, 0, n - 1, ql, qr,value );
    }
    private void rangeAdd( int node, int left, int right, int ql, int qr, long value) {
        // No overlap
        if (right < ql || left > qr) {  return;
        }
        // Complete overlap
        if (ql <= left && right <= qr) {
            tree[node] += value * (right - left + 1);
            lazy[node] += value;
            return;
        }
        push(node, left, right);
        int mid = left + (right - left) / 2;

        rangeAdd( node * 2,  left, mid, ql, qr, value);
        rangeAdd( node * 2 + 1, mid + 1, right, ql, qr, value );
        tree[node]= tree[node * 2]+ tree[node * 2 + 1];
    }
```

19.3.18 Lazy Query Code

Before descending during a partial query: push pending lazy value

```
public long rangeSum(int ql, int qr) {
        return rangeSum( 1, 0,n - 1, ql, qr
        );}
    private long rangeSum( int node, int left, int right, int ql, int qr) {
        if (right < ql || left > qr) { return 0;
        }
        if (ql <= left && right <= qr) { return tree[node];
        }
        push(node, left, right);
        int mid = left + (right - left) / 2;
        return rangeSum( node * 2, left, mid, ql, qr ) + rangeSum(  node * 2 + 1,  mid + 1, right, ql, qr );
    }
```

19.2.18 Complete Lazy Push Method

```
private void push( int node, int left, int right) {
        if (lazy[node] == 0 || left == right) {
            return;
        }
        int mid = left + (right - left) / 2;
        int leftChild = node * 2;
        int rightChild = node * 2 + 1;
        long value = lazy[node];
        tree[leftChild] += value * (mid - left + 1);
        tree[rightChild] += value * (right - mid);
        lazy[leftChild] += value;
        lazy[rightChild] += value;
        lazy[node] = 0;
    }
}
```

19.3.19 Lazy Propagation Complexity

Operation	Complexity
Build	O(n)
Range Query	O(log n)
Range Update	O(log n)
Space	O(n)

More precisely, standard lazy operations are logarithmic up to a small constant multiple of tree height. Without lazy propagation, updating a large range could require: O(n) or many point updates.

19.4 FENWICK TREE

A **Fenwick Tree**, also called a **Binary Indexed Tree (BIT)**, is a compact data structure primarily designed for: Prefix Queries + Point Updates

Most commonly: Prefix Sum. It supports: update(index, delta), prefixSum(index) in: O(log n) time.

19.4.1 Main Fenwick Tree Idea

Suppose array is:

Index: 1 2 3 4 5 6 7 8

Value: 2 4 1 7 3 6 5 8

Fenwick Tree does not store each element directly. Each Fenwick index stores the sum of a specific range ending at that index. The range size is determined by: lowest set bit of the index.

19.4.2. Lowbit

The key Fenwick operation is: i & -i. This isolates the lowest set bit.

1 = 0001 lowbit = 1

2 = 0010 lowbit = 2

3 = 0011 lowbit = 1

4 = 0100 lowbit = 4

6 = 0110 lowbit = 2

8 = 1000 lowbit = 8

Fenwick Tree uses: i & -i to decide how indexes move.

19.4.3. What Does Fenwick Index Store?

For index i, Fenwick Tree stores: sum of: [i - lowbit(i) + 1 ... i]

Example: i = 6, 6 in binary: 0110, lowbit(6) = 2

Therefore: tree[6] stores: sum of indexes [5,6]

Index	Lowbit	Stored Range
1	1	[1,1]
2	2	[1,2]
3	1	[3,3]
4	4	[1,4]
5	1	[5,5]
6	2	[5,6]
7	1	[7,7]
8	8	[1,8]

This overlapping structure lets prefix sums be reconstructed quickly.

Code With Nishchal

19.4.4. Why Fenwick Usually Uses 1-Based Indexing

Fenwick movement uses: i & -i
At: i = 0
lowbit is: 0
which would prevent index movement. Therefore Fenwick Tree normally stores array index: 0 at Fenwick index: 1
So: array index i → Fenwick index i + 1

19.4.5. Fenwick Prefix Sum Query

Suppose we want: sum from index 1 through i
Repeatedly: Add tree[i]
Move: i -= i & -i
Example: prefixSum(7)
Binary: 7 = 0111
Flow: i = 7, use tree[7]
i = 7 - 1 = 6 use tree[6]
i = 6 - 2 = 4 use tree[4]
i = 4 - 4 = 0 stop
Thus: tree[7] + tree[6] + tree[4]
covers:
[7,7]
[5,6]
[1,4]
which together form: [1,7]

19.4.6. Prefix Query Code

```
long prefixSum(int index) {
    long sum = 0;
    while (index > 0) {
        sum += tree[index];
        index -= index & -index;
    }
    return sum;
}
```

Each iteration clears one set bit from the index.
Therefore the number of iterations is at most:
O(log n)

WithNishchal

19.4.7. Fenwick Point Update

Suppose we add: delta to array position i.
Every Fenwick range containing index i must be updated.
Movement: i += i & -i
Example: update index 3
Flow:
3
↓
4
↓
8
↓
...
Each of these Fenwick entries represents a range containing index 3.

19.4.8. Fenwick Update Code

```
void add(int index, long delta) {
    while (index <= n) {
        tree[index] += delta;
        index += index & -index;
    }
}
```

Complexity: O(log n)

19.4.9. Fenwick Tree Complete Implementation

This version exposes **0-based indexes** to the user but internally uses 1-based indexing.

```
class FenwickTree {
    private final long[] tree;
    private final int n;
    FenwickTree(int n) {
        this.n = n;
        tree = new long[n + 1];
    }

    void add(int index, long delta) {
        index++;
        while (index <= n) {
            tree[index] += delta;
            index += index & -index;
        }
    }
```

```
    long prefixSum(int index) {
        index++;
        long sum = 0;
        while (index > 0) {
            sum += tree[index];
            index -= index & -index;
        }
        return sum;
    }

    long rangeSum(int left, int right) {
        if (left == 0) {
            return prefixSum(right);
        }
        return prefixSum(right)
                - prefixSum(left - 1);
    }
}
```

19.4.10. Build Fenwick Tree From Array

Simple build:

```
FenwickTree fenwick = new FenwickTree(arr.length);
for (int i = 0; i < arr.length; i++) {
    fenwick.add(i, arr[i]);
}
```

Complexity: O(n log n)
There is also an O(n) Fenwick build technique, but for most interview implementations, repeated add() is easier to understand unless the problem specifically requires optimal construction.

19.4.11. Range Sum With Fenwick Tree

Fenwick directly calculates: prefixSum(i) = sum[0...i]
Therefore: rangeSum(left, right) = prefixSum(right) - prefixSum(left - 1)
Example: Array: 2 4 1 7 3
prefixSum(4) = 17
prefixSum(1) = 6
Then: sum(2,4) = 17 - 6 = 11 which is: 1 + 7 + 3

CodeWithNischal

19.4.12. Fenwick Tree Point Assignment

Fenwick update naturally performs: add delta not direct assignment.
Suppose: arr[index] currently = 5 and we want: arr[index] = 9
Then: delta = 9 - 5 = 4
Perform: fenwick.add(index, 4);
Therefore if assignment updates are needed, maintain the original/current array too.
Example: long delta = newValue - arr[index];
arr[index] = newValue;
fenwick.add(index, delta);

19.4.13. Fenwick Tree Complexity

Operation	Complexity
Prefix Sum	O(log n)
Range Sum	O(log n)
Point Add	O(log n)
Point Assignment with stored array	O(log n)
Simple Build	O(n log n)
Optimized Build	O(n)
Space	O(n)

Fenwick Tree generally has a smaller constant factor and simpler code than Segment Tree.

19.4.14. Why Fenwick Tree Works

The important binary idea is: i & -i
which gives the size of the block represented by Fenwick index i.
For query: i -= lowbit(i) moves to the next earlier disjoint prefix block.
For update: i += lowbit(i) moves to the next larger block containing that position.
Query movement:
13
↓
12
↓
8
↓
0
Update movement:
5
↓
6
↓
8
↓
16

CodeWithNishchal

19.5.15. Segment Tree vs Fenwick Tree

Feature	Segment Tree	Fenwick Tree
Structure	Binary tree of segments	Binary-indexed prefix blocks
Point Update	O(log n)	O(log n)
Range Sum	O(log n)	O(log n)
Prefix Sum	O(log n)	O(log n)
Range Min/Max	Natural	Not generally supported in the same flexible way
Range Updates	Lazy propagation	Possible with advanced BIT techniques
Implementation	More complex	Simpler
Memory	Around 4n	n+1

19.4.16. When to Use Fenwick Tree

Fenwick is excellent when the problem mainly requires: Point updates + Prefix sums or Range sums derived from prefixes

If the required operation naturally has an inverse, such as: rangeSum(l,r) = prefix(r) - prefix(l-1), Fenwick is especially convenient.

19.4.17. When to Use Segment Tree

Segment Tree is preferred when you need more general operations such as:

Range minimum/ Range maximum/ Range GCD/ Range XOR/ Complex node information/ Range updates/ Lazy propagation **Think: Need highly flexible range aggregation?→ Segment Tree**

19.4.18. Segment Tree Node as Information

A Segment Tree node can store more than one number.

```
class Node {
    long sum;
    long max;
    long min;
}
```

Parent can combine:

sum = left.sum + right.sum, max = max(left.max, right.max), min = min(left.min, right.min)

This flexibility is one of the biggest advantages of Segment Trees.

1.5. Complexity Comparison

Structure	Build	Query	Point Update	Range Update	Space
Prefix Sum	O(n)	O(1)	O(n)	Poor	O(n)
Fenwick Tree	O(n) optimized / O(n log n) simple	O(log n)	O(log n)	Advanced variants	O(n)
Segment Tree	O(n)	O(log n)	O(log n)	O(log n) with lazy	O(n)

CodeWithNishchal

20. Sorting Algorithms

Sorting means arranging elements in a particular order, usually:

Ascending:

[5, 2, 8, 1, 3]
↓
[1, 2, 3, 5, 8]

Descending:

[5, 2, 8, 1, 3]
↓
[8, 5, 3, 2, 1]

Sorting is important because many problems become easier after data is ordered.

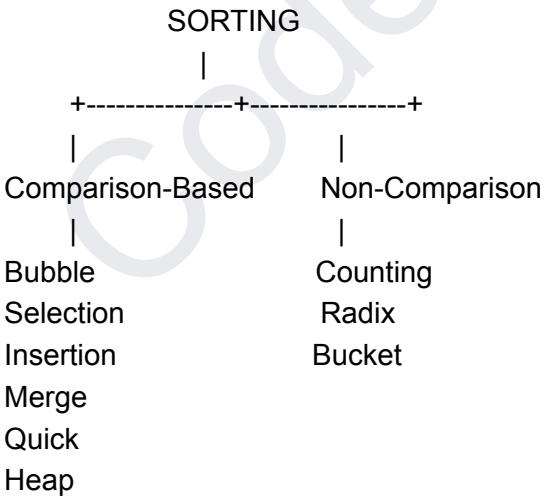
Examples: Binary Search, Two Pointers, Greedy Problems, Interval Problems, Duplicate Detection, Merging, Ranking, Searching, Median / Kth Element

The important sorting algorithms are:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort
- Radix Sort
- Bucket Sort

They are not just different implementations of the same idea. Each one sorts using a fundamentally different strategy.

Main Classification



Comparison-Based Sorting

These algorithms determine order by comparing elements: a < b, a > b

CodeWithNishchal

Examples: Bubble, Selection, Insertion, Merge, Quick, Heap
For arbitrary data, comparison-based sorting has a lower bound of approximately: Ω(n log n) for general sorting in the comparison model.

Non-Comparison Sorting

These algorithms use information about the actual values rather than comparing every pair.
Examples:Counting Sort, Radix Sort, Bucket Sort
They can sometimes achieve near-linear time but require special assumptions such as:
Small integer range, Fixed number of digits, Suitable value distribution

---

Important Sorting Properties

Before studying individual algorithms, understand these properties.

Stable Sorting

A sorting algorithm is **stable** if equal-valued elements keep their original relative order.
Example:
Original:
(80, A)
(70, B)
(80, C)

Sort by marks:
(70, B)
(80, A)
(80, C)

A was before C, and remains before C.
Stable sorting matters when records contain more than one field.

In-Place Sorting

An in-place algorithm uses very little additional memory.
Typically: O(1) extra array space.
Examples:
Bubble, Selection, Insertion, Heap, Quick partitioning
Merge Sort normally needs extra memory for arrays.

Adaptive Sorting

An adaptive algorithm benefits when input is already or nearly sorted.
Example: [1, 2, 3, 5, 4, 6]
Insertion Sort performs very little work here.
Adaptive examples:Insertion Sort, Optimized Bubble Sort

CodeWithNishchal

20.1 BUBBLE SORT

Given an unsorted array, sort it in ascending order using the **Bubble Sort** algorithm.
Example: Input = [5, 3, 8, 4, 2] Output: [2, 3, 4, 5, 8]

Core Idea

Bubble Sort repeatedly compares **adjacent elements**. If the left element is greater than the right element, swap them. arr[j] > arr[j + 1] → Swap

During one complete pass, larger elements keep moving toward the right. Therefore, after each pass, the **largest element in the current unsorted portion reaches its correct position at the end**. **Main idea: Compare Adjacent Elements → Swap if Wrong Order → Largest Bubbles to End → Repeat**

How Bubble Sort Works

Consider: [5, 3, 8, 4, 2]

Pass 1

Compare 5 and 3 → Swap → [3, 5, 8, 4, 2]
Compare 5 and 8 → No Swap → [3, 5, 8, 4, 2]
Compare 8 and 4 → Swap → [3, 5, 4, 8, 2]
Compare 8 and 2 → Swap → [3, 5, 4, 2, 8]
After Pass 1:[3, 5, 4, 2 | 8]
8 is now in its final position.

Pass 2

Only process the unsorted portion:[3, 5, 4, 2]
Compare 3 and 5 → No Swap
Compare 5 and 4 → Swap → [3, 4, 5, 2, 8]
Compare 5 and 2 → Swap → [3, 4, 2, 5, 8]
After Pass 2: [3, 4, 2 | 5, 8]
Now the last two elements are fixed.

Bubble Sort Invariant

After every pass, one more largest element reaches its final position.
Pass 1 → Largest element fixed
Pass 2 → Two largest elements fixed
Pass 3 → Three largest elements fixed
So the unsorted portion becomes smaller after every pass.

Algorithm — Bubble Sort

1. Run passes from 0 to n - 2.
2. During each pass, compare adjacent elements.
3. If arr[j] > arr[j + 1], swap them.
4. After one pass, the largest element of the unsorted portion is fixed at the end.
5. Therefore, the next pass does not need to check the already sorted suffix.
6. Repeat until the array is sorted.

CodeWithNishchal

```
static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int pass = 0; pass < n - 1; pass++) {
        for (int j = 0; j < n - 1 - pass; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }}}}
```

Why n - 1 - pass?

After every pass, one element is already placed correctly at the end.So: Pass 0 → last 1 element fixed, Pass 1 → last 2 elements fixed, Pass 2 → last 3 elements fixed

Therefore, there is no need to compare elements in the already sorted suffix.

Optimized Bubble Sort:

If an entire pass completes without performing any swap, the array is already sorted.

Use a boolean variable: swapped = false, Whenever a swap occurs: swapped = true

After the pass, if: swapped == false, stop immediately.

Algorithm — Optimized Bubble Sort

1. Start a new pass and set swapped = false.
2. Compare adjacent elements in the unsorted portion.
3. Swap whenever arr[j] > arr[j + 1] and set swapped = true.
4. After the pass, check swapped.
5. If no swap occurred, the array is already sorted, so stop.
6. Otherwise, continue with the next pass.

```
static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int pass = 0; pass < n - 1; pass++) {
        boolean swapped = false;
        for (int j = 0; j < n - 1 - pass; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) { break; }}}
```

Best Case = O(n) with optimized Bubble Sort when the array is already sorted.

Average Case = O(n^2), Worst Case = O(n^2), Space = O(1)

Stable = Yes — equal elements keep their relative order.

In-Place = Yes — only constant extra memory is used.

Adaptive = Yes, when optimized — it can stop early for sorted or nearly sorted input.

WithNishchal

20.2 SELECTION SORT

Question Description

Given an unsorted array, sort it in ascending order using the **Selection Sort** algorithm.

Example: Input = [7, 4, 5, 2, 9] Output: [2, 4, 5, 7, 9]

Core Idea

Selection Sort divides the array into two parts: Sorted Prefix | Unsorted Suffix

In every pass, find the **smallest element in the unsorted part** and place it at the beginning of that part. After each pass, one element reaches its final sorted position.

Main idea: Find Minimum → Place at Correct Position → Grow Sorted Prefix → Repeat

How Selection Sort Works

Consider: [7, 4, 5, 2, 9]

Pass 1

Sorted part: []

Unsorted part: [7, 4, 5, 2, 9]

Minimum element = 2

Swap 2 with the first unsorted element 7: [2 | 4, 5, 7, 9], Now 2 is permanently fixed.

Pass 2

Sorted part: [2], Unsorted part: [4, 5, 7, 9]

Minimum element = 4. It is already in the correct position: [2, 4 | 5, 7, 9]

Pass 3

Minimum of [5, 7, 9] is 5. [2, 4, 5 | 7, 9]

Continue until: [2, 4, 5, 7, 9]

Selection Sort Invariant

After each pass, the sorted prefix contains the smallest elements in their correct order.

Pass 1 → smallest element fixed

Pass 2 → two smallest elements fixed

Pass 3 → three smallest elements fixed

So the sorted portion grows from **left to right**.

Algorithm — Selection Sort

1. Start from index i = 0.
2. Assume the current element is the minimum: minIndex = i.
3. Scan the remaining unsorted portion from i + 1 to n - 1.
4. If a smaller element is found, update minIndex.
5. After scanning the complete unsorted portion, swap arr[i] with arr[minIndex].
6. Now arr[i] is in its final sorted position.
7. Move i forward and repeat.

Short flow:

Select Position → Find Minimum in Remaining Array → Swap Minimum into Position → Repeat

CodeWithNishchal

```
static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        // Find minimum element in unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        // Place minimum at index i
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
```

How the Code Works

i represents the position that needs to be filled next. minIndex stores the index of the smallest element found in the unsorted portion.

The inner loop searches: i + 1 ... n - 1 and updates minIndex whenever a smaller element is found. After the search finishes, only one swap is required: arr[i] ↔ arr[minIndex]. Then index i becomes part of the sorted prefix.

Complexity

Selection Sort always scans the remaining unsorted portion.

Number of comparisons: (n - 1) + (n - 2) + ... + 1 = n(n - 1) / 2

Therefore:

- Best Case = O(n²)
- Average Case = O(n²)
- Worst Case = O(n²)
- Space = O(1)

Even if the array is already sorted, Selection Sort still searches for the minimum during every pass.

Properties

Stable = No, usually — swapping may change the relative order of equal elements.

In-Place = Yes — only constant extra memory is required.

Adaptive = No — already sorted input does not significantly reduce the number of comparisons.

Swaps = O(n) — at most one swap is performed per pass.

This low number of swaps is one advantage of Selection Sort when **writes are expensive**.

Easy Memory

Find Minimum → Swap with First Unsorted Position → Grow Sorted Prefix → Repeat

Pattern: Selection of Minimum + One Swap per Pass

CodeWithNishchal

20.3 INSERTION SORT

Given an unsorted array, sort it in ascending order using the **Insertion Sort** algorithm.
Example: Input = [5, 3, 4, 1]
Output: [1, 3, 4, 5]

Core Idea

Insertion Sort divides the array into: Sorted Prefix | Unprocessed Elements

At every step, take the next element from the unprocessed part and insert it into its correct position inside the sorted prefix. It is similar to arranging playing cards in your hand: pick one card and place it at the correct position among the cards already arranged.

Main idea: Pick Next Element → Shift Larger Elements Right → Insert Key → Grow Sorted Prefix

How Insertion Sort Works

Consider: [5, 3, 4, 1], Initially: [5] | 3, 4, 1
The first element is already sorted.

Step 1 — Insert 3

Take:
key = 3
Sorted prefix:[5]
Compare 5 with 3. Since 5 > 3, shift 5 one position to the right: [5, 5, 4, 1]
Insert 3 into the empty position: [3, 5 | 4, 1]

Step 2 — Insert 4

Take: key = 4
Sorted prefix: [3, 5]
Compare backward: 5 > 4 → shift 5
[3, 5, 5, 1]
Now compare:3 < 4
Stop shifting and insert 4: [3, 4, 5 | 1]

Step 3 — Insert 1

Take:key = 1
Sorted prefix: [3, 4, 5]
Shift all larger elements right:
5 → right, 4 → right, 3 → right, Insert 1 at the beginning: [1, 3, 4, 5]

Insertion Sort Invariant

Before processing index i: arr[0 ... i - 1] is already sorted. After inserting arr[i] into its correct position: arr[0 ... i] becomes sorted. So the sorted prefix grows from left to right.

CodeWithNishchal

Algorithm — Insertion Sort

1. Start from index 1 because the first element alone is already sorted.
2. Store the current element as key.
3. Set j = i - 1 to start checking the sorted prefix from right to left.
4. While arr[j] > key, shift arr[j] one position to the right.
5. Continue moving backward until the correct position for key is found.
6. Insert key at arr[j + 1].
7. Repeat for every remaining element.

Short flow:
Pick Key → Compare Backward → Shift Larger Elements → Insert Key → Repeat

```
static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        // Shift larger elements to the right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        // Insert key at correct position
        arr[j + 1] = key;
    }}
```

Why Shifting Instead of Swapping?

Suppose the sorted prefix is: [2, 4, 5, 7] and we need to insert: 3

Instead of repeatedly swapping 3 with every larger element, Insertion Sort shifts the larger elements: 7 → right, 5 → right, 4 → right

Then places 3 directly into the empty position. Result: [2, 3, 4, 5, 7]

Complexity

For an already sorted array, each element requires only a quick comparison and no shifting. Best Case = O(n). For a reverse-sorted array, every new element must move across the entire sorted prefix. Average Case = O(n²), Worst Case = O(n²), Space = O(1)

Properties

Stable = Yes — equal elements preserve their relative order because elements are shifted only when arr[j] > key. **In-Place = Yes — only a few extra variables are used.**
Adaptive = Yes — it performs very well when the array is already or nearly sorted.

Easy Memory

Pick Key → Shift Larger Elements Right → Insert Key at Correct Position → Repeat
Pattern: Grow a Sorted Prefix by Inserting One Element at a Time

CodeWith Nishchal

20.4 MERGE SORT

Question Description

Given an unsorted array, sort it in ascending order using the **Merge Sort** algorithm.
Example: Input = [8, 3, 5, 1, 7, 4]
Output: [1, 3, 4, 5, 7, 8]

Core Idea

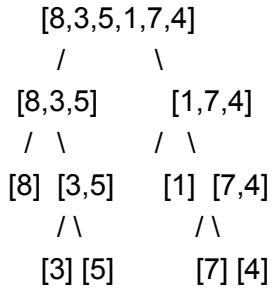
Merge Sort follows the **Divide and Conquer** approach:
Divide → Conquer → Combine

nstead of sorting the complete array directly, divide it into smaller halves until each part contains only one element.
A single-element array is already sorted.
Then merge the smaller sorted parts back together in sorted order.
Main idea: Split into halves → Sort both halves → Merge sorted halves

How Merge Sort Works

Consider: [8, 3, 5, 1, 7, 4]

Divide



At the bottom, every section contains one element, so each section is already sorted.

Merge Upward

Now combine the sorted sections:
[3] + [5] → [3,5]
[8] + [3,5] → [3,5,8]
[7] + [4] → [4,7]
[1] + [4,7] → [1,4,7]
Finally: [3,5,8] + [1,4,7] → [1,3,4,5,7,8]

Algorithm — Merge Sort

1. If the current range contains one or zero elements, return because it is already sorted.
2. Find the middle index.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Continue until the complete array becomes sorted.

Short flow: Find Mid → Sort Left Half → Sort Right Half → Merge Both Halves

CodeWithNishchal

Merge Algorithm

Suppose:

Left = [2, 5, 8]

Right = [1, 4, 7]

Use two pointers: i for the left section and j for the right section.

Steps:

1. Compare Left[i] and Right[j].
2. Copy the smaller value into a temporary array.
3. Move the pointer of the selected value.
4. Repeat until one section is finished.
5. Copy any remaining elements from the unfinished section.
6. Copy the temporary array back into the original array.

2 vs 1 → take 1, 2 vs 4 → take 2, 5 vs 4 → take 4, 5 vs 7 → take 5, 8 vs 7 → take 7

Remaining value → 8. Result: [1, 2, 4, 5, 7, 8]

```
static void mergeSort(
        int[] arr,
        int left,
        int right) {
    // Base case
    if (left >= right) {
        return;
    }
    int mid = left + (right - left) / 2;
    // Sort left half
    mergeSort(arr, left, mid);

    // Sort right half
    mergeSort(arr, mid + 1, right);

    // Merge sorted halves
    merge(arr, left, mid, right);
}
static void merge(
        int[] arr,
        int left,
        int mid,
        int right) {

    int[] temp = new int[right - left + 1];

    int i = left;
    int j = mid + 1;
    int k = 0;
```

Nishchal

```
    // Compare both sorted halves
    while (i <= mid && j <= right) {

        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    // Copy remaining left elements
    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    // Copy remaining right elements
    while (j <= right) {
        temp[k++] = arr[j++];
    }

    // Copy merged result back
    for (int x = 0; x < temp.length; x++) {
        arr[left + x] = temp[x];
    }
}
```

Calling Merge Sort: mergeSort(arr, 0, arr.length - 1);

Merge Sort State

The recursive state is: [left, right] which means: Sort the array from index left to index right
Base case: left >= right means the current portion has at most one element and is already sorted.

Why Merge Sort Is O(n log n)

Each recursive level divides the array approximately in half: n → n/2 → n/4 → ... → 1

Therefore, the number of recursive levels is: O(log n)

At each level, merging processes a total of approximately n elements. So: O(n) work per level × O(log n) levels = O(n log n)

Complexity

- Best Case = O(n log n)
- Average Case = O(n log n)
- Worst Case = O(n log n)
- Space = O(n) for temporary merge storage, with an additional O(log n) recursion stack.

Properties

- Stable = Yes — using <= when merging preserves the relative order of equal elements.
- In-Place = No for the standard array implementation because a temporary array is used.
- Adaptive = No in its standard form.
- Performance = Predictable because best, average, and worst cases are all O(n log n).

CodeWithNishchal

20.5 QUICK SORT

Given an unsorted array, sort it in ascending order using the **Quick Sort** algorithm.
Example: Input = [7, 2, 9, 4, 3] Output: [2, 3, 4, 7, 9]

Core Idea

Quick Sort follows the **Divide and Conquer** approach. It chooses a **pivot** and rearranges the current portion of the array so that:
Values <= Pivot | Pivot | Values > Pivot.
This process is called **partitioning**.

After partitioning, the pivot reaches its **final sorted position**. Then Quick Sort recursively sorts the elements on the left and right sides of the pivot.

Main idea: Choose Pivot → Partition → Pivot Fixed → Sort Left → Sort Right

How Quick Sort Works

Consider: [7, 2, 9, 4, 3]. Choose the last element as pivot: Pivot = 3
After partitioning, one possible state is: [2 | 3 | 9, 4, 7]. Now 3 is permanently in its correct sorted position.
Next, recursively sort: Left = [2] ,Right = [9, 4, 7] Continue until every portion contains at most one element.

Algorithm — Quick Sort

1. If low >= high, return because the current portion has at most one element.
2. Choose a pivot and partition the array.
3. partition() returns the final index of the pivot.
4. Recursively sort the left portion: low ... pivotIndex - 1.
5. Recursively sort the right portion: pivotIndex + 1 ... high.
6. Continue until the complete array is sorted.

Short flow: Choose Pivot → Partition → Sort Left → Sort Right → Repeat

Lomuto Partition

A common Quick Sort implementation uses **Lomuto Partitioning**.

Choose: pivot = arr[high] Maintain two indexes:

i → boundary of values <= pivot, j → scans the current range

Initially: i = low - 1 **Conceptually:**

| <= Pivot | > Pivot | Unprocessed | Pivot |

↑ ↑

i j

CodeWithNishchal

Algorithm — Lomuto Partition

1. Choose the last element as the pivot: pivot = arr[high].
2. Set i = low - 1.
3. Move j from low to high - 1.
4. If arr[j] <= pivot, increment i and swap arr[i] with arr[j].
5. If arr[j] > pivot, do nothing and continue scanning.
6. After the scan, swap arr[i + 1] with the pivot.
7. Return i + 1 as the pivot's final position.

Partition Example

Array: [4, 2, 7, 3, 6], Pivot: 6

Start: i = -1, Scan 4: 4 <= 6 → increment i, swap if needed. [4, 2, 7, 3, 6]

Scan 2: 2 <= 6 → increment i. [4, 2, 7, 3, 6]

Scan 7: 7 > 6 → do nothing.

Scan 3: 3 <= 6 → increment i and swap 7 with 3. [4, 2, 3, 7, 6]

Finally, place pivot after the smaller region: [4, 2, 3, 6, 7]. Now 6 is in its final sorted position.

```
static void quickSort( int[] arr, int low, int high) {
    if (low >= high) {
       return;
    }

int pivotIndex = partition(arr, low, high);
     quickSort( arr, low, pivotIndex - 1 );
    quickSort( arr, pivotIndex + 1, high);
}

static int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; int i = low - 1;
    for (int j = low; j < high; j++) {

        if (arr[j] <= pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
}

// Place pivot in its final position
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
```

Nishchal

```
    arr[high] = temp;
    return i + 1;
}
```

Calling Quick Sort: quickSort(arr, 0, arr.length - 1);

Quick Sort State

The recursive state is: [low, high] meaning: Sort the portion from index low to high

Base case: low >= high, means the current portion has zero or one element and is already sorted.

Complexity

If partitions are reasonably balanced, recursion depth is about log n, and each level performs O(n) partition work.

Therefore: Best Case = O(n log n) ,Average Case = O(n log n)

If the pivot repeatedly becomes the smallest or largest element, partitions become highly unbalanced: n → n-1 → n-2 → ...

Therefore:

- Worst Case = $O(n^2)$
- Recursion space:
- Average = O(log n)
- Worst = O(n)

Why Pivot Selection Matters

A good pivot produces roughly balanced partitions, while a bad pivot repeatedly produces one very large partition and one very small partition.

Common pivot choices: First Element, Last Element, Random Element, Median-of-Three. Randomized pivot selection reduces the chance of repeatedly getting poor partitions.

Properties

- Stable = No, usually
- In-Place = Mostly Yes — partitioning uses constant extra array space, but recursion uses stack memory.
- Adaptive = No in the standard implementation.
- Cache Friendly = Yes — Quick Sort often performs well in practice because it works mainly within the same array.
- When many duplicate values are present, **3-Way Quick Sort** can be useful: < Pivot | = Pivot | > Pivot

Easy Memory

Choose Pivot → Move Smaller Values Left → Place Pivot Correctly → Sort Left → Sort Right

Pattern: Pivot + Partition + Divide and Conquer

CodeWithNishchal

20.6HEAP SORT

Given an unsorted array, sort it in ascending order using the **Heap Sort** algorithm.

Example: Input = [4, 10, 3, 5, 1]

Output: [1, 3, 4, 5, 10]

Core Idea

Heap Sort uses a **Max Heap** for ascending sorting. In a Max Heap: Parent >= Children

Therefore, the largest element is always stored at the root: arr[0]

Heap Sort repeatedly moves this maximum element to the end of the array.

Main idea: Build Max Heap → Move Maximum to End → Reduce Heap Size → Heapify → Repeat

How Heap Sort Works

Consider: [4, 10, 3, 5, 1]

First build a Max Heap:

```
      10
     /  \
    5    3
   / \
  4   1
```

Array representation: [10, 5, 3, 4, 1]

The largest element is: 10

Swap the root with the last element: [1, 5, 3, 4 | 10]

Now 10 is permanently in its correct sorted position.

Heapify the remaining heap: [5, 4, 3, 1 | 10]

Next, move 5 to its final position: [1, 4, 3 | 5, 10]

Heapify again: [4, 1, 3 | 5, 10]

Continue until the complete array is sorted.

Heap Sort State

At every stage, the array is divided into: Heap Region | Sorted Suffix

Example: [5, 4, 3, 1 | 10] then: [4, 1, 3 | 5, 10]

The sorted suffix grows from **right to left**.

Algorithm — Heap Sort

1. Convert the complete array into a Max Heap.
2. The root arr[0] now contains the largest element.
3. Swap the root with the last element of the current heap.
4. Reduce the heap size because the last element is now fixed.
5. Call heapify() on the root to restore the Max Heap property.
6. Repeat until only one element remains in the heap.

Short flow:

Build Max Heap → Swap Root with End → Shrink Heap → Heapify Root → Repeat

CodeWithNishchal

Algorithm — Heapify

For a node at index i: Left Child = 2 × i + 1, Right Child = 2 × i + 2

Steps:

1. Assume the current node is the largest.
2. Compare it with the left child.
3. Compare it with the right child.
4. Find the largest among the current node and both children.
5. If one of the children is larger, swap it with the current node.
6. Continue heapifying from the child's position.
7. Stop when the current node is already larger than both children.

Main idea: Compare Parent with Children → Move Larger Child Up → Continue Downward

```
static void heapSort(int[] arr) {
    int n = arr.length;
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--) { heapify(arr, n, i);
    }
    // Move maximum elements to the end
    for (int end = n - 1; end > 0; end--) {
        // Swap root with last element of heap
        int temp = arr[0];
        arr[0] = arr[end];
        arr[end] = temp;
        // Restore heap in remaining unsorted part
        heapify(arr, end, 0);
    }
}
static void heapify(
        int[] arr,
        int heapSize,
        int index) {
    while (true) {
        int largest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        if (left < heapSize && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < heapSize && arr[right] > arr[largest]) {
            largest = right;
```

```
        }
        // Heap property already satisfied
        if (largest == index) {  break;
        }
        // Swap with larger child
        int temp = arr[index];
        arr[index] = arr[largest];
        arr[largest] = temp;
        // Continue downward
        index = largest;
    }}
```

Calling Heap Sort:

heapSort(arr);

Why Start Building Heap From n / 2 - 1?

All nodes from index: n / 2 to: n - 1 are leaf nodes.

Leaf nodes already satisfy the heap property because they have no children. Therefore, heap construction starts from the **last non-leaf node**: n / 2 - 1 and moves backward to the root.

Why Building a Heap Is O(n)?

It may look like: n nodes × O(log n) heapify = O(n log n)

but most nodes are near the bottom of the heap and require very little movement.

Only a small number of nodes are near the top and can move through many levels.

Because of this distribution, bottom-up heap construction takes: O(n)

After building the heap, we perform approximately n extractions, and each heapify can take: O(log n)

Therefore: Overall Time = O(n log n)

Complexity

- Build Heap = O(n)
- Best Case = O(n log n)
- Average Case = O(n log n)
- Worst Case = O(n log n)
- Space = O(1) with this iterative heapify() implementation.

Properties

- Stable = No — swapping distant elements can change the relative order of equal values.
- In-Place = Yes — sorting happens inside the original array.
- Adaptive = No — already sorted input does not significantly reduce the work.
- Guaranteed O(n log n) = Yes
- Heap Sort gives guaranteed O(n log n) time without requiring the O(n) temporary array used by standard Merge Sort.

Easy Memory

Build Max Heap → Largest at Root → Swap with End → Shrink Heap → Heapify → Repeat
Pattern: Heap Construction + Repeated Maximum Extraction

Code With Nishchal

20.7 COUNTING SORT

Given an array of integers, sort it in ascending order using the **Counting Sort** algorithm.

Counting Sort is useful when the values belong to a **reasonably small integer range**.

Example: Input = [4, 2, 2, 8, 3, 3, 1]

Output: [1, 2, 2, 3, 3, 4, 8]

Core Idea

Unlike comparison-based algorithms, Counting Sort does not repeatedly compare elements.

Instead, it counts how many times each value occurs.

Example: arr = [4, 2, 2, 8, 3, 3, 1]

Frequency table:

Value : 0 1 2 3 4 5 6 7 8

Count : 0 1 2 2 1 0 0 0 1

This means:

count[1] = 1

count[2] = 2

count[3] = 2

count[4] = 1

count[8] = 1

Now reconstruct the array using these frequencies: 1, 2, 2, 3, 3, 4, 8

Main idea: Count Frequencies → Traverse Count Array → Reconstruct Sorted Array

Counting Sort State

The important state is: count[value]

which stores: number of occurrences of value

For example: count[3] = 2 means: 3 appears two times

Unlike Bubble Sort or Selection Sort, Counting Sort does not maintain a sorted and unsorted portion of the original array.

Algorithm — Counting Sort for Non-Negative Integers

1. If the array is empty, return.
2. Find the maximum value in the array.
3. Create a frequency array of size max + 1.
4. Traverse the input and increment count[value] for each element.
5. Traverse the count array from smallest value to largest.
6. For each value, write it into the original array as many times as its frequency.
7. Continue until all values are reconstructed.

Short flow: Find Maximum → Create Count Array → Count Values → Reconstruct Array

CodeWithNishchal

Code — Non-Negative Values

```
static void countingSort(int[] arr) {
    if (arr.length == 0) {
        return;
    }
    int max = arr[0];
    // Find maximum value
    for (int value : arr) {
        max = Math.max(max, value);
    }
    int[] count = new int[max + 1];
    // Count frequencies
    for (int value : arr) {
        count[value]++;
    }
    int index = 0;
    // Reconstruct sorted array
    for (int value = 0;
         value < count.length;
         value++) {
        while (count[value] > 0) {
            arr[index++] = value;
            count[value]--;
        }
    }
}
```

Handling Negative Values

The previous implementation works only when values are non-negative because array indexes cannot be negative.

Suppose: [-5, -2, -5, 3]

Find: min = -5, max = 3

Instead of directly using the value as an index, shift every value by min: index = value - min

Mapping:

-5 → 0

-2 → 3

3 → 8

Required count-array size: max - min + 1

Algorithm — Counting Sort with Negative Values

1. Find the minimum and maximum values.
2. Calculate the range: max - min + 1.
3. Create a count array of that size.
4. Store each frequency using: count[value - min]++.
5. Traverse the count array.
6. Convert each count index back to its original value using: value = index + min.
7. Reconstruct the original array.

Code — Supporting Negative Values

```
static void countingSort(int[] arr) {
    if (arr.length == 0) {
        return;
    }

    int min = arr[0];
    int max = arr[0];
    for (int value : arr) {
        min = Math.min(min, value);
        max = Math.max(max, value);
    }

    int[] count = new int[max - min + 1];
    // Count frequencies
    for (int value : arr) {
        count[value - min]++;
    }

    int index = 0;
    // Reconstruct sorted array
    for (int i = 0; i < count.length; i++) {
        while (count[i] > 0) {
            arr[index++] = i + min;
            count[i]--;
        }
    }
}
```

20.8 Stable Counting Sort

The simple reconstruction version is sufficient when sorting plain integer values.

However, when Counting Sort is used to sort **records with keys**, or as a subroutine inside **Radix Sort**, stability becomes important.

A stable Counting Sort uses cumulative counts instead of directly reconstructing values.

Stable Counting Sort Algorithm

1. Count the frequency of every value.
2. Convert the frequency array into a **prefix-sum array**.
3. After prefix sums, count[value] tells how many elements are <= value, which determines the final position of that value.
4. Traverse the original array from **right to left**.
5. Place each element at count[value] - 1 in an output array.
6. Decrement count[value].
7. Copy the output array back into the original array.

Why right to left? Processing equal elements from right to left while decrementing their positions preserves their original relative order.

Main idea:

Frequency → Prefix Sum → Place from Right to Left → Stable Output

Code — Stable Counting Sort for Non-Negative Integers

```
static void stableCountingSort(int[] arr) {
    if (arr.length == 0) {
        return;
    }
    int max = arr[0];
    for (int value : arr) {
        max = Math.max(max, value);
    }
    int[] count = new int[max + 1];
    // Frequency
    for (int value : arr) {
        count[value]++;
    }
    // Prefix sums
    for (int i = 1; i < count.length; i++) {
        count[i] += count[i - 1];
    }
    int[] output = new int[arr.length];

    // Traverse right to left for stability
```

Nishchal

```
    for (int i = arr.length - 1; i >= 0; i--) {
        int value = arr[i];
        output[count[value] - 1] = value;
        count[value]--;
    }
    // Copy back
    System.arraycopy(
            output,
            0,
            arr,
            0,
            arr.length
    );
}
```

Complexity

Let: n = number of elements, k = size of the value range

Then: Time = O(n + k), Space = O(k) for basic Counting Sort.

For the stable version: Space = O(n + k) because an additional output array is required.

When Counting Sort Is Efficient

Counting Sort works very well when: k is small relative to n.

Example: n = 1000

Values range from: 0 to 100. A small count array is practical.

It becomes inefficient when the range is extremely large.

Example: n = 100

Values range from: 0 to 1,000,000,000, Creating such a large count array would waste enormous memory.

Properties

- Comparison-Based = No
- Stable = Basic reconstruction version is not inherently useful for preserving record order; prefix-position version is stable
- In-Place = No in the strict sense because it requires an additional count array.
- Adaptive = No
- Best Use = Integers with a small value range

Easy Memory

Count Values → Use Frequencies → Reconstruct in Sorted Order

Stable version:

Count → Prefix Sum → Traverse Right to Left → Place into Output

Pattern: Frequency Counting + Direct Positioning

CodeWithNishchal

20.9 RADIX SORT

Given an array of **non-negative integers**, sort it in ascending order using the **Radix Sort** algorithm.
Example: Input = [170, 45, 75, 90, 802, 24, 2, 66]. Output: [2, 24, 45, 66, 75, 90, 170, 802]

Core Idea

Radix Sort is a **non-comparison sorting algorithm** that sorts numbers digit by digit.
For decimal numbers, digits are processed as: Ones → Tens → Hundreds → Thousands → …
In **LSD Radix Sort**, processing starts from the least significant digit and moves toward the most significant digit. Each digit is sorted using a **stable sorting algorithm**, commonly Counting Sort.

Main idea: Sort by Ones → Sort by Tens → Sort by Hundreds → ... → Fully Sorted

How Radix Sort Works

Consider: [170, 45, 75, 90, 802, 24, 2, 66]

Pass 1 — Ones Digit

Number : 170 45 75 90 802 24 2 66

Ones : 0 5 5 0 2 4 2 6

After stable sorting by ones digit: [170, 90, 802, 2, 24, 45, 75, 66]

Pass 2 — Tens Digit

Number : 170 90 802 2 24 45 75 66

Tens : 7 9 0 0 2 4 7 6

After stable sorting by tens digit: [802, 2, 24, 45, 66, 170, 75, 90]

Pass 3 — Hundreds Digit

Number : 802 2 24 45 66 170 75 90

Hundreds : 8 0 0 0 0 1 0 0

After stable sorting: [2, 24, 45, 66, 75, 90, 170, 802]. Now all digit positions have been processed, so the array is sorted.

Radix Sort State

The important state is: exp which represents the digit position currently being processed.

exp = 1 → ones, exp = 10 → tens, exp = 100 → hundreds, exp = 1000 → thousands

Digit extraction: digit = (value / exp) % 10

Example: value = 472, exp = 1 → digit = 2, exp = 10 → digit = 7, exp = 100 → digit = 4

Algorithm — Radix Sort

1. If the array is empty, return.
2. Find the maximum value to determine how many digit positions must be processed.
3. Start with exp = 1 for the ones digit.
4. Perform a **stable Counting Sort** using only the digit at the current exp.
5. Multiply exp by 10 to move to the next digit position.
6. Repeat while the maximum number still has a digit at that position.
7. After all digit positions are processed, the array is sorted.

Short flow: Find Maximum → Sort by Current Digit → Move to Next Digit → Repeat

CodeWithNishchal

Code — Radix Sort

```
static void radixSort(int[] arr) {
    if (arr.length == 0) { return; }
    int max = arr[0];
    // Find maximum value
    for (int value : arr) {
        max = Math.max(max, value);
    }
    // Process ones, tens, hundreds, ...
    for (long exp = 1;
         max / exp > 0;
         exp *= 10) {
        countingSortByDigit(arr, exp);
    }}
```

Algorithm — Stable Counting Sort by Digit

For every digit position:

1. Create count[10] because decimal digits range from 0 to 9.
2. Extract the current digit of every number.
3. Count how many times each digit occurs.
4. Convert the frequency array into prefix sums.
5. Traverse the original array from right to left.
6. Place each value into its correct position in the output array.
7. Decrement the corresponding count.
8. Copy the output array back into the original array.

Main idea: Count Digits → Prefix Sum → Place Right-to-Left → Copy Back

Code — Counting Sort by Digit

```
static void countingSortByDigit(
        int[] arr,
        long exp) {
    int n = arr.length;
    int[] output = new int[n];
    int[] count = new int[10];
    // Count current digits
    for (int value : arr) {
        int digit =(int) ((value / exp) % 10);
        count[digit]++;
    }
```

devWithNishchal

```
    // Convert to prefix positions
    for (int i = 1; i < 10; i++) {
        count[i] += count[i - 1];
    }
    // Build output from right to left
    // to preserve stability
    for (int i = n - 1; i >= 0; i--) {
        int digit = (int) ((arr[i] / exp) % 10);
        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
    // Copy sorted digit-pass result back
    System.arraycopy( output, 0, arr, 0, n );
}
```

Why Stability Matters

Radix Sort depends on the ordering created by previous digit passes.

Suppose two numbers have the same tens digit. During the tens-digit pass, their relative order from the earlier ones-digit pass must remain unchanged.

If the digit sort were unstable, the ordering established by less significant digits could be destroyed.

Therefore: LSD Radix Sort requires a stable sorting algorithm for every digit pass.

This is why **stable Counting Sort** is commonly used.

Complexity

Let: n = number of elements, d = number of digit positions, b = base

Each digit pass requires: O(n + b)

Therefore: Time = O(d(n + b))

For decimal numbers: b = 10 so: Time ≈ O(d × n) ,Space: O(n + b)

For decimal digits: Space = O(n) because b = 10 is constant.

Properties

- Comparison-Based = No
- Stable = Yes when the inner digit sort is stable.
- In-Place = No in this implementation because an output array is required.
- Adaptive = No
- Best Use = Fixed-width integers or keys with a limited number of digits
- The standard implementation above assumes **non-negative integers**. Supporting negative values requires additional handling.

CodeWithNishchal

20.10 BUCKET SORT

Question Description

Given a collection of values, sort them using the **Bucket Sort** algorithm by distributing the elements into different value ranges called **buckets**.

Bucket Sort works especially well when the input values are **approximately uniformly distributed** over a known range.

Example: Input = [0.42, 0.32, 0.23, 0.52, 0.25] Output: [0.23, 0.25, 0.32, 0.42, 0.52]

Core Idea

Bucket Sort divides the complete value range into several smaller ranges called **buckets**.

The algorithm then performs three main operations: Distribute → Sort Buckets → Concatenate

Each value is placed into the bucket corresponding to its range. The values inside every bucket are sorted separately, and finally all buckets are joined from the smallest range to the largest range.

Main idea: Find Bucket → Insert Value → Sort Each Bucket → Concatenate Buckets

How Bucket Sort Works

Consider:

[0.42, 0.32, 0.23, 0.52, 0.25]

Suppose buckets represent:

Bucket 0 → [0.0, 0.1)

Bucket 1 → [0.1, 0.2)

Bucket 2 → [0.2, 0.3)

Bucket 3 → [0.3, 0.4)

Bucket 4 → [0.4, 0.5)

Bucket 5 → [0.5, 0.6)

Distribution

Place each value into its corresponding bucket:

Bucket 0 → []

Bucket 1 → []

Bucket 2 → [0.23, 0.25]

Bucket 3 → [0.32]

Bucket 4 → [0.42]

Bucket 5 → [0.52]

Sort Individual Buckets

Each bucket is sorted independently:

Bucket 2 → [0.23, 0.25]

Bucket 3 → [0.32]

Bucket 4 → [0.42]

Bucket 5 → [0.52]

Concatenate

Read the buckets from left to right: [0.23, 0.25, 0.32, 0.42, 0.52]

CodeWithNishchal

Bucket Sort State

Unlike Counting Sort, a bucket does not simply store the frequency of one exact value.

A bucket can contain **multiple values belonging to the same range**.

For example: Bucket 2 → [0.23, 0.25, 0.27] These elements must still be sorted inside the bucket.

Conceptually: Value → Determine Bucket → Insert into Bucket → Sort Bucket → Concatenate

Algorithm — Bucket Sort

For values in the range: [0, 1)

1. Create n empty buckets, where n is the number of elements.
2. For each value, determine its bucket using: bucketIndex = (int)(value * n).
3. Insert the value into the corresponding bucket.
4. Sort each bucket individually.
5. Traverse the buckets from index 0 to n - 1.
6. Copy all values from the buckets back into the original array.
7. The resulting array is sorted.

Why value * n Determines the Bucket

For values in: 0 <= value < 1 and n buckets: bucketIndex = (int)(value * n)

Example with n = 5:

0.12 × 5 = 0.60 → Bucket 0

0.25 × 5 = 1.25 → Bucket 1

0.42 × 5 = 2.10 → Bucket 2

0.79 × 5 = 3.95 → Bucket 3

0.95 × 5 = 4.75 → Bucket 4

The integer conversion removes the decimal part and gives the bucket index.

```
static void bucketSort(double[] arr) {
    int n = arr.length;
    if (n == 0) { return; }
    List<List<Double>> buckets = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        buckets.add(new ArrayList<>());
    }
    // Distribute values into buckets
    for (double value : arr) {
        int bucketIndex = (int) (value * n);
        buckets.get(bucketIndex).add(value);
    }
    // Sort each bucket
    for (List<Double> bucket : buckets) {
        Collections.sort(bucket);
    }
```

WithNishchal

```
// Concatenate buckets
int index = 0;
for (List<Double> bucket : buckets) {
    for (double value : bucket) {
        arr[index++] = value;
    }}}
```

This implementation assumes: 0 <= value < 1. A value equal to 1.0 would produce index n, which is outside the valid bucket range.

Why Distribution Matters

Bucket Sort performs well when elements are spread approximately evenly among the buckets.

Good distribution:

Bucket 0 → few elements

Bucket 1 → few elements

Bucket 2 → few elements

Bucket 3 → few elements

Each bucket contains only a small number of values, so sorting individual buckets is cheap.

Poor distribution:

Bucket 0 → []

Bucket 1 → []

Bucket 2 → [almost every element]

Bucket 3 → []

Now one bucket contains most of the input, so most of the work is performed by the inner sorting algorithm. Therefore, Bucket Sort performance depends heavily on: **Value Distribution + Bucket Design + Inner Sorting Algorithm**

Complexity

Let: n = number of elements, k = number of buckets

Distribution of elements O(n). Creating/traversing buckets: O(k)

Under a good, roughly uniform distribution, Bucket Sort is commonly treated as:

Average / Expected = O(n + k) When k is proportional to n: Average ≈ O(n)

However, the exact complexity also depends on how each bucket is sorted. If every element falls into one bucket, the bucket sorter dominates the runtime. With a quadratic inner sorting algorithm such as Insertion Sort:

Worst Case = $O(n^2)$ With a comparison sort such as Collections.sort(), the worst-case sorting work is instead approximately: O(n log n)

Space: O(n + k) because the buckets collectively store the input elements.

Properties

- Comparison-Based = Not primarily — bucket placement uses the value itself, although sorting inside each bucket may use comparisons.
- **Stable** = Depends on the bucket implementation and inner sorting algorithm.
- **In-Place** = No — additional buckets are required. **Adaptive** = Depends on the inner sorting algorithm. **Best Use** = Values distributed fairly evenly across a known range.

CodeWithNishchal

Bubble vs Selection vs Insertion

Property	Bubble	Selection	Insertion
Main action	Adjacent swaps	Select minimum	Insert key
Sorted side	Right	Left	Left
Best time	O(n) optimized	O(n²)	O(n)
Worst time	O(n²)	O(n²)	O(n²)
Stable	Yes	Usually No	Yes
Adaptive	Yes optimized	No	Yes
Writes	Many	Few	Moderate

Merge vs Quick vs Heap

Feature	Merge	Quick	Heap
Strategy	Divide + Merge	Partition	Heap
Avg	O(n log n)	O(n log n)	O(n log n)
Worst	O(n log n)	O(n²)	O(n log n)
Stable	Yes	Usually No	No
Extra array memory	O(n)	Low	O(1)
Practical array speed	Good	Often excellent	Often slower
Guaranteed bound	Yes	No	Yes

Complexity Summary

Algorithm	Best	Average	Worst	Extra Space
Bubble	O(n)	O(n²)	O(n²)	O(1)
Selection	O(n²)	O(n²)	O(n²)	O(1)
Insertion	O(n)	O(n²)	O(n²)	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick	O(n log n)	O(n log n)	O(n²)	Avg O(log n) stack
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)
Radix	O(d(n+k))	Same	Same	O(n+k)
Bucket	Near O(n+k) average	Depends	Can degrade	O(n+k)

Follow CodeWithNishchal on social media:

- **Instagram:** https://www.instagram.com/codewithnishchal/
- **LinkedIn:** https://www.linkedin.com/in/nishchal-muradia/
- **YouTube:** https://www.youtube.com/@CodeWithNishchal
- **Watch My Latest Time Complexity Series:** Time Complexity
- **Join My Community for Tech discussions :** CodeN' community

CodeWithNishchal